

START HERE — Project APEX Engineering Apprenticeship

The route from beginner builder to independent world-model researcher

OpenAI

August 2026

Table of Contents

1 What this package is

This package is designed to correct a common failure in technical textbooks: naming concepts without constructing understanding. The primary teaching core does not ask you to memorize GRUs, state-space models, RSSMs, Dreamer, JEPA or Airflow. It creates the problem that each idea solves, makes you build the simplest previous solution, exposes its failure, and only then introduces the stronger mechanism.

The endpoint is a working Project APEX V1: an F1 telemetry world-simulation engine that can use synthetic evidence offline and canonical data produced by FastF1 or OpenF1 adapters, train deterministic and latent sequence models, generate counterfactual rollouts, evaluate them by horizon, register artifacts and serve an interactive race-engineering interface.

The deeper endpoint is independence. You should be able to read a world-model paper, reconstruct its mechanism, identify its assumptions, implement a small reproduction, design a matched APEX ablation, and reject the idea when evidence does not support it.

2 The correct reading and building order

2.1 Stage 1 — Teaching core

Open textbook/output/Project_APEX_Engineering_Apprenticeship_Teaching_Core.pdf.

Work through Chapters 1–20 in order. Every chapter has a runnable lab. Do not read the solutions section before writing your prediction. When a chapter names an APEX source path, open that file only after understanding the smaller lab.

The core sequence is:

Simulator before neural network.

State, action, context and partial observability.

Units, forces and integration.

Sampling, aliasing and alignment.

Canonical contracts and quality gates.

FastF1/OpenF1 source adapters.
Windows, splits, normalization and leakage.
Statistics, baselines and uncertainty.
Tensors, autograd and training.
One-step transitions and autoregressive rollout.
GRU memory.
Classical state-space models.
Selective SSMs and Mamba concepts.
Autoencoders and variational latent state.
RSSMs and Dreamer.
JEPA, LeWorldModel and fine-tuning.
Evaluation, ablation and OOD tests.
MPC and CEM planning.
Pipelines, Airflow, registry and monitoring.
Complete APEX V1 and research roadmap.

2.2 Stage 2 — Executable lab manual

Use `Project_APEX_Executable_Lab_Manual.pdf` beside the `labs/` folder. The manual contains exact execution commands, expected output captured from the packaged solution, debugging instructions and extension work.

A lab is complete only when:

- you predicted output before running;
- you can explain every axis and unit;
- you recomputed one result by hand;
- you injected the listed defect;
- your regression test fails on the defect;
- your repair passes the test;
- you implemented the extension independently.

2.3 Stage 3 — Ten-project ladder

Use `Project_APEX_Ten_Project_Instructor_Ladder.pdf` and build Projects 1–9 before treating the production engine as your own work.

The projects are not copy exercises. Hide or rename the supplied implementation, write acceptance tests, make your own attempt, then compare. Each project ends with an Architecture Decision Record.

2.4 Stage 4 — Production debugging casebook

Use `Project_APEX_Production_Debugging_Casebook.pdf` with `debugging_cases/`.

For each incident, stop before the root-cause section. Preserve a failing artifact, identify the earliest broken contract, implement a probe, repair the cause, and add a test. The cases cover units, axes, leakage, temporal joins, normalizers, recurrent state, eval mode, gradients, SSM stability, KL collapse, planner exploitation, artifact lineage, Airflow payloads and UI false precision.

2.5 Stage 5 — Research reading and design

Use `research_reading/Project_APEX_Research_Reading_and_Design_Guide.pdf` while reading primary papers. Draw training and imagination graphs separately. Mark gradient and stop-gradient paths. Reproduce the smallest mechanism before adapting it.

2.6 Stage 6 — Field workbook

The workbook is not optional. Use it for predictions, manual calculations, ablation manifests, incident reports, architecture decisions, paper maps and independent-build checkpoints.

2.7 Stage 7 — Annotated source companion

The source companion contains line-numbered production code explanations. Use it after attempting to trace the code yourself. It is a verification tool, not a substitute for reading source.

2.8 Stage 8 — Production capstone

Follow `apex_engine/START_HERE.md`, run tests, execute a fresh run, inspect artifacts, launch the UI and reproduce one scenario outside the UI. Then replace one module with your own implementation under the same tests and evaluation.

3 Recommended 24-week apprenticeship

3.1 Weeks 1-4: causal foundations

Complete Chapters 1-5, Labs 1-4 and Projects 1-3. Spend more time on units, time, contracts and causal slicing than on model code. These layers determine whether later metrics mean anything.

Deliverables:

- one-dimensional integrator;
- causal lap sandbox;
- canonical schema;
- validation suite;
- numerical-method decision record.

3.2 Weeks 5-8: evidence and statistical controls

Complete Chapters 6-10, Labs 5-10 and Projects 4-5. Ingest or mock both public data sources, create session splits, build baselines and trace a full training step.

Deliverables:

- adapter fixtures;
- leakage audit;
- persistence/linear baselines;
- residual report;
- instrumented training trace.

3.3 Weeks 9-12: deterministic sequence dynamics

Complete Chapters 11-13, Labs 11-14 and Projects 6-7. Build GRU and SSM models under matched budgets. Do not move on until you can explain hidden-state lifetime and stability.

Deliverables:

GRU horizon curves;
selective-memory diagnostic;
hidden-norm plot;
matched architecture decision.

3.4 Weeks 13–16: latent world models

Complete Chapters 14–16, Labs 15–17 and Project 8. Build deterministic compression, VAE sampling and RSSM prior/posterior paths. Read DreamerV3 and I-JEPA using the research guide.

Deliverables:

reconstruction and future probes;
KL diagnostics;
posterior versus prior rollout comparison;
representation-collapse test.

3.5 Weeks 17–19: planning and evaluation

Complete Chapters 17–18, Lab 18 and Project 9. Build a CEM planner and deliberately give it a model/reward loophole. Define promotion and abstention rules before comparing models.

Deliverables:

full evaluation matrix;
planner-exploitation incident report;
uncertainty/OOD constraint;
planning decision record.

3.6 Weeks 20–22: production engineering

Complete Chapter 19, Labs 19–20 and all debugging cases. Run the local pipeline twice, simulate stage failure, inspect the registry and compile the Airflow DAG.

Deliverables:

immutable run artifacts;
resumable failure test;
lineage report;
monitoring specification.

3.7 Weeks 23–24: APEX capstone and independent research

Complete Chapter 20 and Project 10. Launch the UI, reproduce one plotted point from source frames, run a canonical-source experiment and write an original architecture proposal.

Deliverables:

fresh verified APEX run;
UI-to-model provenance trace;
model card with trusted horizon;
original research hypothesis and ablation plan.

4 Environment setup

```
cd Project_APEX_Engineering_Apprenticeship
python -m venv .venv
source .venv/bin/activate
```

```
cd apex_engine
pip install -e .
pytest -q
```

Run one fast reference:

```
apexsim run \
  --config configs/fast.yaml \
  --run-id my_apprenticeship_reference
```

Launch the UI:

```
apexsim ui \
  --run-dir artifacts/runs/my_apprenticeship_reference \
  --config configs/fast.yaml
```

For a lab:

```
cd ../labs/10_training_loop
python solution.py
```

5 Evidence standards

Do not claim a model is better because its architecture is newer. Every comparison should preserve:
canonical dataset and version;

split identifiers;

normalization;

history and horizon;

training budget;

seed set;

evaluation implementation;

parameter/latency context;

physical guardrails.

Do not claim a system is production ready because it has an API or Dockerfile. Production evidence includes reproducible artifacts, retries, lineage, monitoring, rollback, security boundaries and product-level limitations.

Do not claim a model is Dreamer because it contains an RSSM. Dreamer includes world-model learning, reward/continuation prediction and behaviour learning from imagination.

Do not claim a toy selective recurrent cell is Mamba. Use it to understand input-dependent memory, then evaluate the complete official architecture when scale and sequence length justify it.

6 Mastery checkpoints

You are ready to progress when you can answer and demonstrate:

What exactly is the state, action, context and target?
Which input values are available at prediction time?
What is the split unit and why?
Where are normalization statistics fitted and stored?
What does each tensor axis represent?
What changes after backward and after optimizer step?
How is predicted state fed into the next rollout step?
When is hidden state reset?
Why is an SSM stable or unstable?
What differs between RSSM prior and posterior?
How do you detect collapse?
What baseline must a new model beat?
What horizon is trusted and why?
What can a scenario user change?
How do artifacts trace back to data/config/code?
What failure motivates your proposed architecture?

7 Final independent test

Start an empty repository. Rebuild a smaller APEX using only:
canonical schema;
acceptance tests;
system architecture diagram;
evaluation requirements.

Do not copy implementation. When complete, compare with the supplied repository and write a postmortem: decisions you made differently, defects you avoided or introduced, and one research question created by the experience.

The curriculum succeeds when the supplied code stops being the authority and becomes one implementation you can critique.

Project APEX Engineering Apprenticeship

Hands-on world models, state-space models, F1 telemetry and production simulation engineering

OpenAI

August 2026

Table of Contents

How to use this apprenticeship

This is not a catalogue of topics. Every chapter begins with a failure or engineering need, makes you predict an outcome, builds the smallest working system, inspects its internal state, breaks it deliberately, repairs it with a regression test, compares alternatives, and then transfers the idea into Project APEX.

Use a split screen: the book on one side and the repository on the other. Type the code. Do not merely read it. Before every output, write down what you expect. When your expectation is wrong, that gap is the lesson.

The teaching loop

Problem → Prediction → Small build → Visible trace → Deliberate failure → Diagnosis → Repair → Comparison → APEX integration → Independent challenge

Repository map

labs/ contains small isolated experiments.

projects/01...09/ contains progressively larger builds.

apex_engine/ is the production-structured capstone.

debugging_cases/ contains failures that should be investigated before reading the explanation.

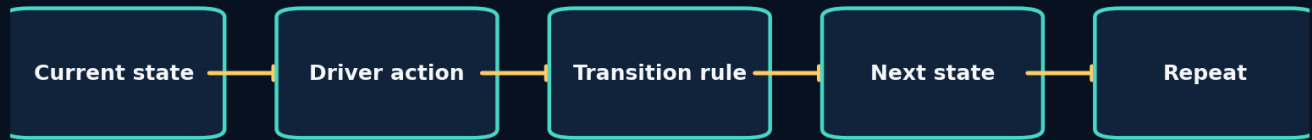
source_code_companion/ contains line-numbered explanations of the production source.

field_workbook/ is where you record predictions, experiment plans, and architecture decisions.

1. Build a Simulator Before You Build a Neural Network

Instructor objective: Understand simulation as repeated state transition, then build and inspect the smallest causal driving world.

A simulator is a transition repeated in time



The first world model can be a hand-written equation. Learning comes later.

1. Build a Simulator Before You Build a Neural Network

The problem that earns this chapter

You want a model that can imagine an F1 car several seconds into the future. It is tempting to open PyTorch immediately. That would hide the central question: **what does it mean for the world to move forward by one step?** Before learning anything from data, we need a transparent transition rule whose mistakes we can see.

Predict before reading

A car is moving at 20 m/s. During the next 0.2 seconds, its acceleration is 3 m/s². Predict the next velocity and the distance travelled if we use the simplest Euler update. Then decide which quantity should be updated first.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A simulator stores a description of the world, receives an intervention, applies a transition, and returns a new description. In our first world, the state is only velocity and position. The action is a throttle value. The transition is an equation. Repeating the equation produces a rollout.

This hand-written simulator is already a world model in the broad engineering sense: it predicts a future state from a present state and an action. It is not a *learned* world model. That difference matters. A learned model replaces part of the transition rule with parameters estimated from evidence. The interface should remain stable: state in, action in, next state out.

Starting here gives us an oracle for later debugging. If a neural network predicts that stronger braking increases velocity, we can compare it with the toy causal rule and ask whether the data contract, target alignment, or learned relationship is broken.

Vocabulary that now has a job

Concept: State - Meaning in plain language: The minimum information carried from one step to the next. - **Role inside APEX:** Speed, progress, tyre state and other dynamic variables.

Concept: Action - Meaning in plain language: A controllable input applied during a transition. - **Role inside APEX:** Throttle, brake, steering and gear choices.

Concept: Transition - Meaning in plain language: The rule that maps current state and action to next state. - **Role inside APEX:** Hand-written physics, a GRU, an SSM or an RSSM.

Concept: Rollout - Meaning in plain language: Repeatedly applying the transition to create a future trajectory. - **Role inside APEX:** Imagining several seconds of future telemetry.

Worked example: calculate it by hand

Euler integration uses

$$v_{t+1} = v_t + a_t \Delta t$$

and

$$x_{t+1} = x_t + v_t \Delta t.$$

With $v_t = 20$, $a_t = 3$, and $\Delta t = 0.2$:

Velocity change: $3 \times 0.2 = 0.6$ m/s.

Next velocity: $20 + 0.6 = 20.6$ m/s.

Distance using the current velocity: $20 \times 0.2 = 4.0$ m.

Next position, if starting at zero: 4.0 m.

A semi-implicit variant updates velocity first and then position, producing $20.6 \times 0.2 = 4.12$ m. Neither is universally “the answer”; the numerical method is an implementation choice whose error shrinks with smaller time steps.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/02_vectorized_kinematics

What we are about to build

A tiny vectorized integrator that converts an acceleration sequence into velocity and position. You will see that a rollout is just a loop with state carried between iterations.

Runnable implementation

```
import numpy as np
```

```
dt = 0.2
```

```
time = np.arange(0, 5, dt)
```

```
speed = 20 + 4*np.sin(time)
```

```
accel = np.gradient(speed, dt)
```

```
reconstructed = speed[0] + np.cumsum(accel)*dt
```

```
print("first accelerations:", accel[:5].round(3))
```

```
print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

Observed output from the packaged solution

```
first accelerations: [3.973 3.894 3.66 3.279 2.768]
```

```
max reconstruction error: 0.7946773231802453
```

Read the important lines like English

Code: `dt = 0.1` - **What the line is doing:** Define how much simulated time one transition represents. -

What to inspect: Changing `dt` changes both numerical error and sequence length.

Code: `velocity[i] = velocity[i-1] + acceleration[i-1] * dt` - **What the line is doing:** Apply one causal state transition. - **What to inspect:** The action at step $i-1$ affects state i , not state $i-1$.

Code: `position[i] = position[i-1] + velocity[i-1] * dt` - **What the line is doing:** Integrate velocity into position. - **What to inspect:** This implementation uses explicit Euler.

State and tensor trace

Trace one step with `velocity[0]=10`, `acceleration[0]=2`, and `dt=0.1`:

before: `velocity = 10.0 m/s`, `position = 0.0 m`

action: `acceleration = 2.0 m/s2`

change: $\Delta v = 2.0 \times 0.1 = 0.2 \text{ m/s}$

after: `velocity = 10.2 m/s`

position change = $10.0 \times 0.1 = 1.0 \text{ m}$

after: `position = 1.0 m`

The array is merely a stored history of repeated state updates.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change `dt` from 0.1 to 1.0 without changing the acceleration profile. Compare the trajectory. Then update position with the newly computed velocity and compare again.

Diagnose from the earliest failed contract

If the two trajectories disagree, the code is not necessarily broken. The earliest changed contract is the numerical integration scheme and time resolution. Ask whether your result converges as `dt` becomes smaller.

Repair and lock the repair with a test

Add a test that simulates constant acceleration for one second at several time steps. Assert that final velocity approaches the analytical value $v_0 + a \cdot t$. Document the position integrator you chose.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Explicit Euler - Choose it when: You need the simplest transparent baseline and small time steps. - **Do not choose it when:** Long unstable rollouts or stiff dynamics.

Implementation: Semi-implicit Euler - Choose it when: Velocity drives position and you want slightly better stability at similar cost. - **Do not choose it when:** You need high-order accuracy.

Implementation: Runge-Kutta - Choose it when: The hand-written dynamics are smooth and numerical accuracy matters. - **Do not choose it when:** The transition is learned from discrete telemetry where solver precision is not the main bottleneck.

Implementation: Learned transition - Choose it when: Important dynamics are unknown but repeated observations are available. - **Do not choose it when:** You have not defined state, action, timing or evaluation.

For APEX, the toy simulator uses simple integration because its purpose is causal education and synthetic data generation. The learned model then predicts transitions on a fixed telemetry grid. We do not pretend the neural network is a high-fidelity rigid-body solver.

Transfer the lesson into Project APEX

The production engine keeps the same conceptual interface: a history encodes the current world, future controls supply actions, and the model emits future states. The complexity grows, but the causal contract learned here should remain visible.

Repository path to inspect

```
projects/01_car_integrator/main.py
apex_engine/src/apexsim/data/synthetic.py
apex_engine/src/apexsim/simulation.py
```

Connection to research

Dreamer-style systems are sophisticated because they learn transition dynamics in a latent state, but they still depend on this repeated transition idea. If you cannot explain the toy rollout, an RSSM will only hide confusion behind distributions.

Check your understanding before continuing

Why is a one-step transition different from a rollout?

What changes when dt is halved?

Why should actions at time t usually predict state at time $t+1$ rather than state at time t ?

Solutions and reasoning

1. A one-step transition consumes one state/action pair; a rollout feeds predictions back repeatedly, so errors change later inputs. 2. There are twice as many transitions for the same physical duration, usually reducing integration error but increasing compute and sequence length. 3. The state at time t was observed before or at the moment the action is applied; using the same-time target can leak the answer or teach no causal delay.

Independent build challenge

Implement both explicit and semi-implicit Euler. Plot their position error against the analytical solution for five time steps. Write an ADR choosing one for the synthetic APEX generator.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

2. State, Action, Context and the Markov Question

Instructor objective: Design a causal telemetry schema and discover when the visible state is insufficient for predicting the future.

Separate what the world is, what you do, and what you cannot control



Speed is state; throttle is action; rain is context.

2. State, Action, Context and the Markov Question

The problem that earns this chapter

Suppose two cars have the same speed and throttle. One is on fresh soft tyres in dry weather; the other is on worn hards in rain. A model that sees only speed and throttle receives identical input for futures that should differ. The failure is not “the neural network needs more layers.” The state description is incomplete.

Predict before reading

Classify each variable as state, action, context, target, or identifier: speed, throttle, rain intensity, driver number, tyre age, next speed, track name, brake. Explain any variable whose category could change depending on the system boundary.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

State is the information the transition needs to carry. Action is the intervention being chosen. Context changes the transition but is not controlled by the agent inside the model. Identifiers describe an entity or grouping and should not automatically become numeric features.

The Markov property says the next state is conditionally independent of the distant past once the present state and action are known. Real telemetry is rarely perfectly Markov. Tyre temperature, fuel mass, setup, and previous cornering load may be hidden. A recurrent model compensates by compressing history into memory. But recurrence is not permission to ignore schema design: hidden memory cannot recover information that was never observed.

A practical state contract should be sufficient, measurable, stable across data sources, and safe to use at inference time. A variable known only after the race cannot be an input to an online simulator.

Vocabulary that now has a job

Concept: Markov state - Meaning in plain language: A state that contains enough information to predict the next transition when combined with the action. - **Role inside APEX:** The ideal telemetry state; approximated using features plus history.

Concept: Context - Meaning in plain language: An external condition that modifies dynamics. - **Role inside APEX:** Rain, track geometry, session phase and compound.

Concept: Identifier - Meaning in plain language: A label used for grouping or lookup, not automatically a causal feature. - **Role inside APEX:** Session ID, driver code and event ID.

Concept: Partial observability - Meaning in plain language: Important state exists but is not directly measured. - **Role inside APEX:** Tyre temperature or setup may need history, proxies or latent state.

Worked example: calculate it by hand

Consider the transition

$$v_{t+1} = v_t + (4a_t - 7b_t - 0.02v_t - 3r_t)\Delta t$$

where a is throttle, b is brake, and r is rain intensity. At v=50 m/s, throttle 0.8, brake 0, rain 0.5, and dt=0.2:

Engine contribution: $4 \times 0.8 = 3.2 \text{ m/s}^2$.

Drag contribution: $0.02 \times 50 = 1.0 \text{ m/s}^2$.

Rain/grip penalty: $3 \times 0.5 = 1.5 \text{ m/s}^2$.

Net acceleration: $3.2 - 1.0 - 1.5 = 0.7 \text{ m/s}^2$.

Speed change: $0.7 \times 0.2 = 0.14 \text{ m/s}$.

Next speed: 50.14 m/s.

If rain were omitted, the same visible state/action would predict 50.44 m/s. That 0.30 m/s discrepancy is a schema problem before it is a modelling problem.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/01_units_and_shapes

What we are about to build

A two-feature telemetry tensor with explicit time and feature axes. The point is not the arithmetic; it is learning to make shape and semantics part of the data contract.

Runnable implementation

```
import numpy as np
```

```
speed_kmh = np.array([0.0, 72.0, 144.0])
speed_mps = speed_kmh / 3.6
telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
print("speed_mps:", speed_mps)
print("telemetry shape [time, features]:", telemetry.shape)
assert telemetry.shape == (3, 2)
```

Observed output from the packaged solution

```
speed_mps: [ 0. 20. 40.]
telemetry shape [time, features]: (3, 2)
```

Read the important lines like English

Code: `speed_mps = speed_kmh / 3.6` - **What the line is doing:** Convert a display unit into the canonical physical unit. - **What to inspect:** The numerical values change while the physical quantity does not.

Code: `np.stack(..., axis=1)` - **What the line is doing:** Create one row per time step and one column per feature. - **What to inspect:** `axis=1` establishes [time, features].

Code: `assert telemetry.shape == (3, 2)` - **What the line is doing:** Turn the axis contract into an executable check. - **What to inspect:** A shape check cannot verify feature order or units by itself.

State and tensor trace

```
row 0 = [0.0 m/s, 0.0 throttle]
row 1 = [20.0 m/s, 0.5 throttle]
row 2 = [40.0 m/s, 1.0 throttle]
shape = [3 time steps, 2 features]
```

If you transpose it, the shape becomes [2 features, 3 time steps]; the numbers still exist, but every downstream interpretation changes.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Transpose the array and remove the shape assertion. Write a function that assumes rows are time. Observe that it can compute plausible but meaningless statistics.

Diagnose from the earliest failed contract

The earliest failed contract is axis semantics, not the model. Print shape, feature names, units, and one known row before debugging anything downstream.

Repair and lock the repair with a test

Create a dataclass or Pydantic model that stores `feature_names`, `units`, `sample_hz`, and the array. Add tests for feature order, legal ranges, and axis length.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Flat numeric vector - **Choose it when:** The state is small, fixed and well documented. - **Do not choose it when:** Variables have complex structure or missingness semantics.

Implementation: Typed record / dataframe - **Choose it when:** You are ingesting and validating human-readable telemetry. - **Do not choose it when:** You need high-throughput batched neural computation.

Implementation: History encoder - **Choose it when:** The observed frame is partially Markov and recent history contains useful proxies. - **Do not choose it when:** Missing variables are unrelated to anything observed.

Implementation: Latent state model - **Choose it when:** Hidden dynamics and uncertainty must be inferred. - **Do not choose it when:** A transparent state vector already predicts well enough.

APEX uses named tabular records at ingestion, then ordered tensors for learning. Conversion happens at an explicit boundary so readability and computational efficiency are both preserved.

Transfer the lesson into Project APEX

The canonical contract distinguishes state features, control features, context features and identifiers. The window builder constructs history and future-control tensors without exposing future target state.

Repository path to inspect

```
apex_engine/src/apexsim/contracts.py
apex_engine/src/apexsim/data/features.py
apex_engine/src/apexsim/data/windows.py
```

Connection to research

RSSMs exist partly because observations are incomplete. Their deterministic hidden state and stochastic latent state summarize what the agent believes about the world. That belief is only as good as the evidence and intervention contract supplied to it.

Check your understanding before continuing

Can tyre age be state and context at the same time?

Why is driver ID dangerous as a direct numeric feature?

What evidence would show that a single telemetry frame is not Markov enough?

Solutions and reasoning

1. The label depends on the chosen boundary. If tyre age evolves inside the simulator, it is state; if supplied as a fixed scenario condition, it behaves like context. 2. Numeric encoding invents an ordering and can let the model memorize identities instead of dynamics. Use a justified embedding or group-specific analysis. 3. A history-aware model consistently outperforming an equal-capacity frame model, especially after controlling leakage, suggests the past carries missing state information.

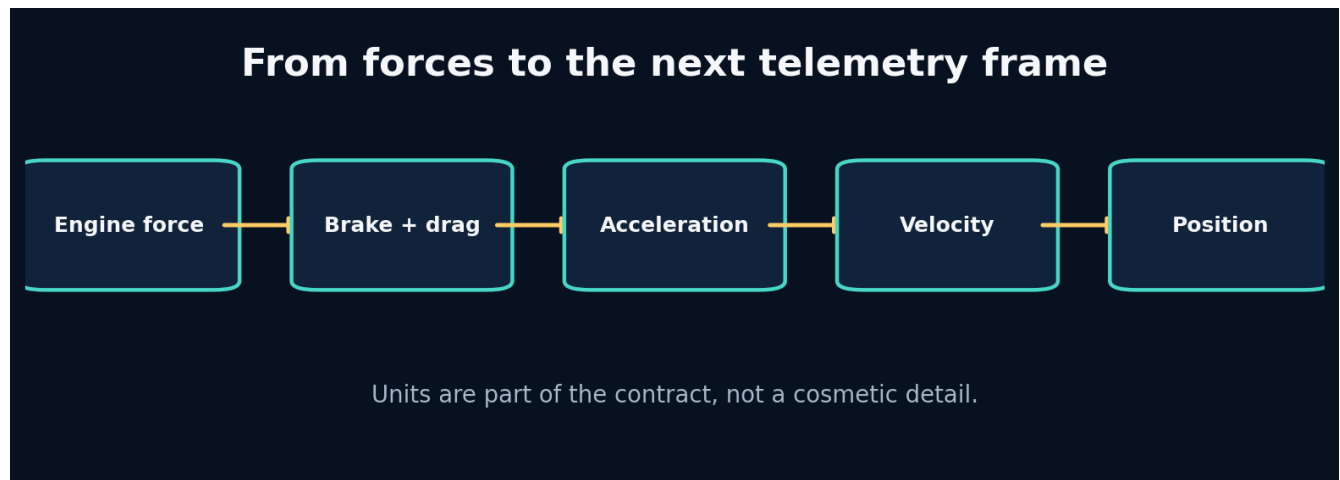
Independent build challenge

Write the complete APEX V1 schema on paper. For every field, state its category, unit, legal range, availability at inference, and expected causal relationship.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

3. Units, Forces and Numerical Integration

Instructor objective: Build a physically interpretable transition, use dimensional analysis to catch errors, and understand the limits of toy physics.



3. Units, Forces and Numerical Integration

The problem that earns this chapter

Telemetry sources use kilometres per hour, metres per second, percentages, normalized controls, degrees, radians and timestamps in different formats. A model can fit unit mistakes without raising an exception. The result may look statistically smooth while violating the world.

Predict before reading

A formula subtracts $0.02 * \text{speed}^2$ from acceleration, but speed is supplied in km/h although the coefficient was fitted in m/s. Predict whether the drag is too large or too small and by what factor.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Dimensional analysis asks whether the units on both sides of an equation agree. It is one of the cheapest and strongest debugging tools in simulation engineering. If force is in newtons and mass is in kilograms, acceleration is metres per second squared. If speed enters a squared term, a unit conversion error is squared too.

A toy longitudinal model combines engine force, braking force and drag. It will not reproduce downforce, tyre load sensitivity, energy recovery, differential behaviour or setup effects. That is fine when the model's purpose is to generate causal examples and test software contracts. It is dangerous if the UI presents the result as lap-time truth.

The hand-written model also teaches residual learning: instead of asking a neural network to rediscover every obvious relationship, we can predict the correction to a known physics approximation. Whether that helps is an empirical decision.

Vocabulary that now has a job

Concept: Dimensional analysis - Meaning in plain language: Check that quantities combined by an equation have compatible units. - **Role inside APEX:** Catches km/h–m/s and timestamp errors before training.

Concept: Force balance - Meaning in plain language: Net force is the sum of propulsive and resisting forces. - **Role inside APEX:** Provides an interpretable synthetic transition.

Concept: Residual model - Meaning in plain language: A learned correction added to a known baseline model. - **Role inside APEX:** Possible hybrid physics/ML architecture.

Concept: Numerical stability - Meaning in plain language: Whether repeated computation remains bounded and sensible. - **Role inside APEX:** Critical for long autoregressive rollouts and SSM state.

Worked example: calculate it by hand

Suppose the drag acceleration term is $c v^2$. Converting from m/s to km/h multiplies speed by 3.6. Squaring multiplies the drag term by $3.6^2 = 12.96$. The model therefore applies nearly thirteen times too much drag.

For a 50 m/s car with mass 800 kg:

Engine force: 8,000 N

Brake force: 0 N

Drag: 2,500 N

Net force: 5,500 N

Acceleration: $5500 / 800 = 6.875 \text{ m/s}^2$

At $dt=0.1$, velocity increases by 0.6875 m/s

If speed were incorrectly treated as 180 in a coefficient calibrated for 50, the quadratic term would dominate and could create impossible negative velocity.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/03_force_model

What we are about to build

A longitudinal force model with engine, brake and aerodynamic drag contributions. You will inspect every term separately before summing them.

Runnable implementation

from education.core **import** CarState, step_car

state = CarState(0.0, 0.0, 40.0)

for i **in** range(5):

state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)

print(i, state)

Observed output from the packaged solution

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/03_force_model/solution.py", line 1, in <module>

from education.core import CarState, step_car

ModuleNotFoundError: No module named 'education'

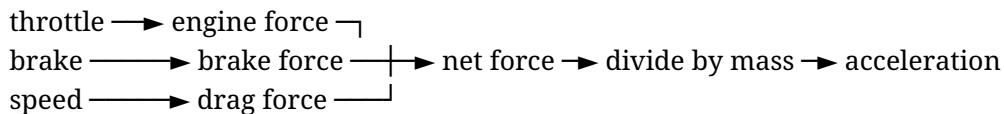
Read the important lines like English

Code: `engine = max_engine_force * throttle` - **What the line is doing:** Scale available forward force by a normalized control. - **What to inspect:** Throttle range must be validated as 0–1.

Code: `drag = drag_coefficient * speed2` - **What the line is doing:** Apply a quadratic resistance that grows rapidly with speed. - **What to inspect:** The coefficient depends on the speed unit.

Code: `acceleration = net_force / mass` - **What the line is doing:** Use Newton's second law to convert force into acceleration. - **What to inspect:** Mass must be positive and use kilograms.

State and tensor trace



At every arrow, annotate the unit. That annotation is an executable design review even before a test exists.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Feed speed in km/h without conversion. Then set mass to zero or throttle above one. Observe which failures become exceptions and which silently produce nonsense.

Diagnose from the earliest failed contract

Start with invariants: $mass > 0$, $0 \leq controls \leq 1$, speed unit known, $drag \geq 0$, and braking should not increase velocity. Then inspect individual force terms.

Repair and lock the repair with a test

Introduce typed conversion functions, range validation, and property tests: increasing brake at fixed state must not increase next speed; increasing throttle must not decrease it in a regime without traction limits.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Pure hand-written physics - Choose it when: You need interpretability, counterfactual control and known equations. - **Do not choose it when:** Unknown tyre/track interactions dominate error.

Implementation: Pure learned dynamics - Choose it when: You have broad representative data and care about predictive fit. - **Do not choose it when:** Safety or extrapolation requires hard physical guarantees.

Implementation: Residual learning - Choose it when: A baseline captures broad physics and data can learn systematic corrections. - **Do not choose it when:** The baseline is badly misspecified and constrains learning in the wrong direction.

Implementation: Physics-informed loss - Choose it when: Violations can be expressed as differentiable penalties. - **Do not choose it when:** The penalty is only a weak proxy and overwhelms observed evidence.

APEX V1 uses the toy force model to create controllable offline data, while the predictive models learn directly from canonical telemetry. Physical validity checks remain part of evaluation rather than being assumed from the architecture.

Transfer the lesson into Project APEX

Inspect how the synthetic generator computes curvature demand, grip, drag, controls and state updates. Then compare those equations with the features available to the learned models.

Repository path to inspect

```
projects/03_lap_physics/main.py
apex_engine/src/apexsim/data/synthetic.py
apex_engine/src/apexsim/evaluation.py
```

Connection to research

World models often optimize data likelihood or prediction error, not the laws of mechanics. Adding inductive bias can improve sample efficiency, but every constraint is a claim that must be tested under the target domain.

Check your understanding before continuing

Why can two unit errors cancel during training but still fail in deployment?

What is the advantage of predicting a residual?

Name one invariant that should hold for all APEX rollouts.

Solutions and reasoning

1. A model can learn coefficients adapted to the mistaken training scale; a different source or UI conversion changes the scale and exposes the mismatch. 2. The baseline supplies known structure, so the learner spends capacity on unmodelled effects and can be easier to interpret. 3. Examples include finite values, nonnegative speed, track progress wrapped into its legal interval, and controls within their defined range.

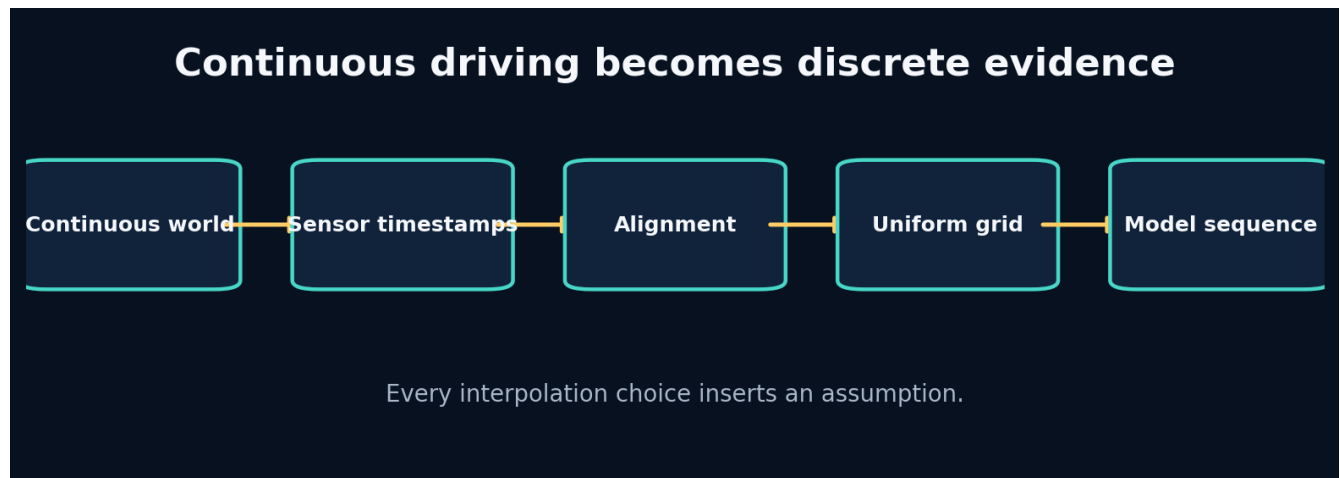
Independent build challenge

Add a tyre-grip limit to the force model. Show a case where more throttle no longer produces more acceleration, and explain why a monotonicity test must be conditional rather than universal.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

4. Sampling, Aliasing and Time Alignment

Instructor objective: Turn asynchronous telemetry into a defensible time grid without creating fake evidence or future leakage.



4. Sampling, Aliasing and Time Alignment

The problem that earns this chapter

Speed may arrive at one frequency, weather at another, and event data only when something changes. A neural network expects aligned rows. Joining by nearest timestamp can quietly attach a future weather measurement to an earlier car frame. Interpolation can make missing data look observed.

Predict before reading

An 8 Hz oscillation is sampled at 5 Hz. Sketch the samples. Will the observed sequence reveal an 8 Hz signal, a slower false signal, or no signal? Then decide what a model might learn from it.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Sampling converts a continuous process into discrete observations. If the sample rate is too low for the dynamics, high-frequency behaviour aliases into a different pattern. No architecture can reconstruct information that the sensor never captured without additional assumptions.

Alignment chooses which measurements describe the same model time. A backward as-of join says: at each car timestamp, use the latest context that was already available. A nearest join may use the future. A tolerance limits how stale a matched measurement may be. These are causal policies, not dataframe trivia.

Resampling onto a uniform grid simplifies windows and batch training, but interpolation must respect feature type. Continuous speed may be linearly interpolated across short gaps. Gear and flags are categorical and generally require forward fill or explicit unknown values. Long gaps should be marked invalid, not painted over.

Vocabulary that now has a job

Concept: Sampling rate - **Meaning in plain language:** How many observations are recorded per second. - **Role inside APEX:** Defines temporal resolution and sequence length.

Concept: Aliasing - **Meaning in plain language:** A high-frequency process appears as a different lower-frequency pattern. - **Role inside APEX:** Can hide braking spikes or steering oscillations.

Concept: As-of join - Meaning in plain language: Match each timestamp with a nearby record under a direction and tolerance policy. - **Role inside APEX:** Aligns weather, position and telemetry streams.

Concept: Staleness - Meaning in plain language: How old a context value is when reused. - **Role inside APEX:** A monitoring feature and validation rule.

Worked example: calculate it by hand

The Nyquist guideline says a sinusoid of frequency f requires sampling above $2f$ to avoid ambiguity. An 8 Hz oscillation needs more than 16 Hz. At 5 Hz, sample times are 0, 0.2, 0.4... seconds. The phase advances $8 \times 0.2 = 1.6$ cycles per sample, equivalent to an apparent 0.6-cycle advance after wrapping. The observed pattern therefore resembles a 3 Hz oscillation (0.6×5).

For alignment, suppose car timestamps are 0.0, 0.2, 0.4, 0.6 and rain observations are 0.05 and 0.55. A backward join at 0.4 uses rain from 0.05; a nearest join uses 0.55, leaking a future value by 0.15 seconds.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/19_time_alignment

What we are about to build

A backward merge_asof with an explicit tolerance. You will inspect which rain observation is attached to each car frame and identify unmatched stale rows.

Runnable implementation

import pandas **as** pd

```
car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
print(aligned)
```

Observed output from the packaged solution

```
t speed rain
0 0.0  10 NaN
1 0.2  12  0.0
2 0.4  14 NaN
3 0.6  16  0.3
```

Read the important lines like English

Code: sort_values('t') - **What the line is doing:** Satisfy the ordered-time precondition of an as-of join. - **What to inspect:** Unsorted data can fail or produce invalid matches.

Code: direction='backward' - **What the line is doing:** Use only context observations at or before the car timestamp. - **What to inspect:** This encodes online availability and blocks future leakage.

Code: tolerance=0.3 - **What the line is doing:** Reject a match older than the allowed staleness. - **What to inspect:** Choose tolerance from domain timing, not convenience.

State and tensor trace

car t=0.00 → no earlier rain measurement → missing
 car t=0.20 → rain at 0.05 (age 0.15 s) → valid
 car t=0.40 → rain at 0.05 (age 0.35 s) → rejected by 0.30 s tolerance
 car t=0.60 → rain at 0.55 (age 0.05 s) → valid

A table row should carry not only the matched value but ideally the age of that value.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change direction to nearest and remove the tolerance. Add a future rain spike at 0.41. Observe how an earlier car frame receives information from the future.

Diagnose from the earliest failed contract

Compare source timestamp, target timestamp and signed age. The earliest failed contract is availability time, not model performance.

Repair and lock the repair with a test

Persist measurement timestamps and create a test asserting `context_time <= frame_time` for online features. Add a maximum staleness threshold and an explicit missingness mask.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Backward as-of - Choose it when: Features must reflect information available online.

- **Do not choose it when:** The quantity is defined as a symmetric offline estimate.

Implementation: Nearest join - Choose it when: Both streams are synchronized measurements and future use is scientifically acceptable. - **Do not choose it when:** Forecasting, control or causal analysis.

Implementation: Linear interpolation - Choose it when: A continuous signal has short, well-sampled gaps. - **Do not choose it when:** Categorical states, discontinuities or long outages.

Implementation: Forward fill - Choose it when: A discrete state remains active until changed. - **Do not choose it when:** A rapidly varying continuous sensor.

APEX chooses a canonical frequency, aligns source streams under causal policies, records staleness, and rejects gaps that exceed configured limits. The policy is part of the dataset version.

Transfer the lesson into Project APEX

The FastF1 and OpenF1 adapters normalize timestamps and stream-specific fields before producing the canonical frame. Window construction assumes the alignment contract is already satisfied.

Repository path to inspect

labs/04_sampling_aliasing/solution.py
 labs/19_time_alignment/solution.py
 apex_engine/src/apexsim/data/fastf1_adapter.py
 apex_engine/src/apexsim/data/openf1_adapter.py

Connection to research

Sequence models are often blamed for missing rapid dynamics that were already destroyed by low-rate or misaligned observations. Data acquisition defines the ceiling of learnable temporal behaviour.

Check your understanding before continuing

Why does increasing model capacity not solve aliasing?

What does a join tolerance represent physically?

When can interpolation create false certainty?

Solutions and reasoning

1. The distinct high-frequency processes map to identical samples, so the information is absent from the data. 2. The maximum age or temporal mismatch at which a context measurement is still considered representative. 3. When it fills a long or discontinuous gap with smooth values and the training pipeline treats those values as directly observed.

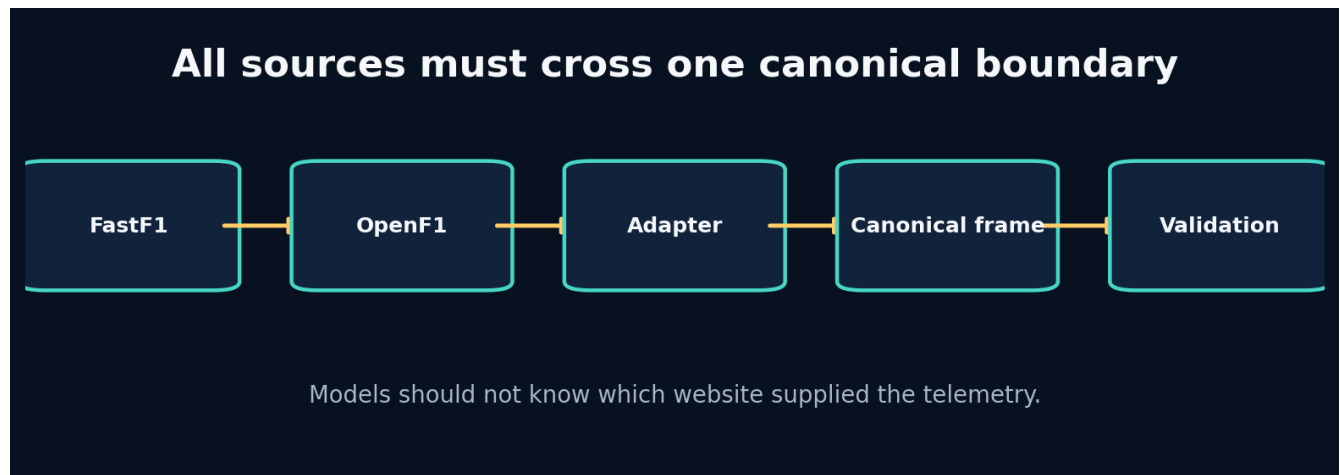
Independent build challenge

Create two asynchronous synthetic streams with a known causal delay. Compare nearest, backward and interpolated alignment by measuring how well each preserves the known delay.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

5. Canonical Contracts and Quality Gates

Instructor objective: Build one source-independent telemetry contract and make invalid evidence fail before it reaches a model.



5. Canonical Contracts and Quality Gates

The problem that earns this chapter

FastF1, OpenF1, synthetic data and future F1 25 UDP packets do not share identical names, units, timing or missingness. If every model contains source-specific branches, the system becomes impossible to test. We need a narrow boundary that all sources must cross.

Predict before reading

Imagine FastF1 supplies speed in km/h and a future UDP adapter supplies m/s. Both columns are called speed. Where should conversion happen: inside each model, inside the dataset, or inside the source adapter? Explain the consequences of each choice.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A canonical contract is not merely a dataframe schema. It specifies names, types, units, legal ranges, timing semantics, missingness, identifiers and invariants. Source adapters are translators: they may parse and convert, but they should not secretly perform model-specific feature engineering.

A quality gate separates admissible evidence from rejected or quarantined evidence. Some checks are hard errors—missing timestamp, impossible unit, duplicate key. Others are warnings—unusual prevalence or a short gap. The distinction should reflect whether downstream interpretation remains valid.

Contracts also enable replaceability. You can test the entire pipeline with synthetic data, then swap to FastF1 without changing the window builder or model. This is the central architectural move that lets APEX start before F1 25 is available.

Vocabulary that now has a job

Concept: Canonical contract - Meaning in plain language: One stable internal representation accepted by all downstream components. - **Role inside APEX:** The source-independent telemetry frame.

Concept: Adapter - Meaning in plain language: A boundary component translating one external source into the canonical representation. - **Role inside APEX:** FastF1, OpenF1 and future UDP adapters.

Concept: Quality gate - Meaning in plain language: Executable rules deciding whether evidence may proceed. - **Role inside APEX:** Range, timing, duplicate, finite-value and schema checks.

Concept: Quarantine - Meaning in plain language: Preserve questionable data separately instead of silently dropping or using it. - **Role inside APEX:** Allows investigation and reprocessing after a fix.

Worked example: calculate it by hand

Suppose the canonical speed unit is m/s and the legal range is 0–105 m/s. A source row says 320 km/h.

Adapter recognizes source unit km/h.

Convert: $320 / 3.6 = 88.89$ m/s.

Validate finite: yes.

Validate range: yes.

Store canonical value with source provenance.

If conversion were delayed until model code, validation would see 320 and reject a valid record—or the range would be widened and allow truly impossible m/s values. Therefore conversion belongs at the adapter boundary.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

A miniature pipeline that writes one immutable artifact per stage. Although small, it teaches the idea that every stage has named input/output and observable status.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `stages=['ingest','validate','window','train','evaluate','publish']` - **What the line is doing:**

Declare the lifecycle as distinct responsibilities. - **What to inspect:** A stage name should correspond to one recoverable contract.

Code: `run/f'{i:02d}_{name}.json'` - **What the line is doing:** Give each output a deterministic path within a run. - **What to inspect:** Production artifacts also need version and content metadata.

Code: `{'stage':name,'status':'succeeded'}` - **What the line is doing:** Persist machine-readable state instead of relying on console text. - **What to inspect:** Real status should be written atomically after success.

State and tensor trace

external row

↓ adapter parses names and units

canonical row

↓ validator checks invariants

admitted row + quality report

↓ window builder
model-ready example

Each arrow is a contract boundary with tests. A failure should name the earliest boundary it violates.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Rename one source column, duplicate a timestamp, insert infinite speed, and swap throttle/brake. Notice that only some defects are detectable from shape and range.

Diagnose from the earliest failed contract

Validate in layers: schema → types → units → temporal keys → ranges → cross-field relationships → distribution warnings. A semantic swap may require known examples or correlation checks.

Repair and lock the repair with a test

Create fixtures for one valid row and one row for every failure class. Make adapters produce the same canonical column order and metadata. Record source, adapter version and conversion policy.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Loose dataframe convention - Choose it when: Exploratory one-off analysis. - **Do not choose it when:** Multiple sources, teams or production reuse.

Implementation: Pydantic/dataclass contract - Choose it when: You need typed configuration and row/batch metadata validation. - **Do not choose it when:** Per-element validation becomes a throughput bottleneck; validate batches at boundaries.

Implementation: Schema registry - Choose it when: Many producers and consumers evolve independently. - **Do not choose it when:** A small local project where the operational overhead exceeds value.

Implementation: Silent coercion - Choose it when: Almost never; only for explicitly documented benign parsing. - **Do not choose it when:** Any conversion that can change scientific meaning.

APEX adapters own source parsing and canonical conversion. Core training code imports only the canonical contract. Validation produces a report and blocks hard failures before sequence construction.

Transfer the lesson into Project APEX

Read the contract and both adapters side by side. Confirm that training, evaluation and UI modules never call FastF1 or OpenF1 directly.

Repository path to inspect

```
apex_engine/src/apexsim/contracts.py
apex_engine/src/apexsim/data/validate.py
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
```

Connection to research

Research code often assumes a fixed benchmark tensor. Production world models need a stronger boundary because source evolution can alter semantics without changing array shape.

Check your understanding before continuing

Why should source adapters avoid model-specific feature engineering?

Give an example of a warning rather than a hard error.

What metadata is needed to reproduce a canonical dataset?

Solutions and reasoning

1. It couples data acquisition to one model and creates inconsistent versions across experiments; stable raw canonical features should precede reproducible transforms. 2. A session with unusually high rain or a slightly lower sample rate may be valid but deserves attention. 3. Source identifiers, retrieval time, adapter version, units, alignment policy, validation configuration, feature code version and dataset hash.

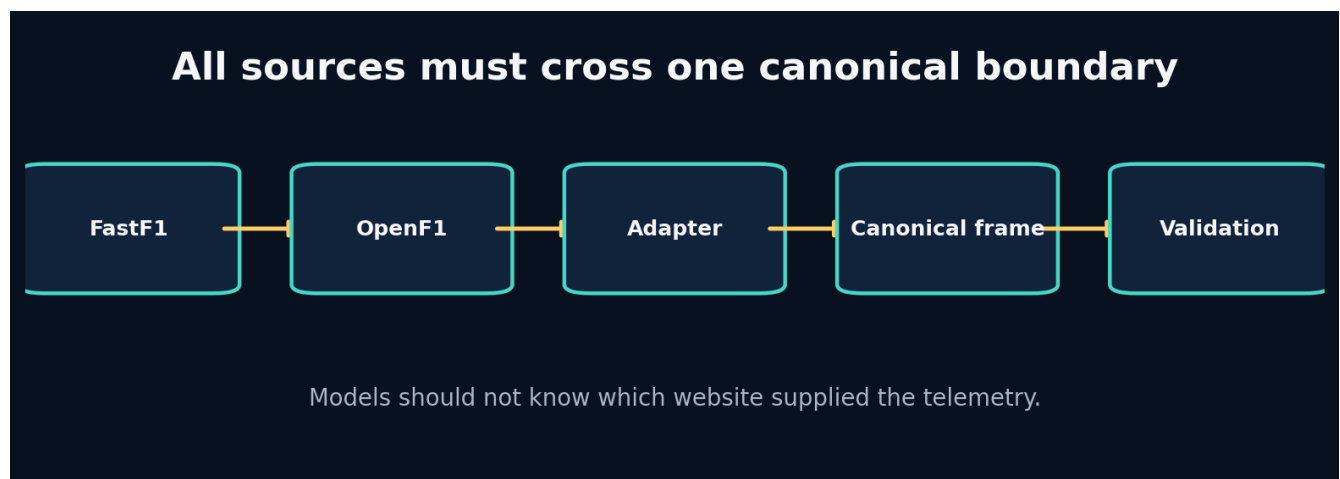
Independent build challenge

Define a canonical telemetry schema in JSON Schema or Pydantic. Implement two toy adapters with different names and units, and prove with tests that they produce identical canonical rows.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core

Instructor objective: Understand the complete data path from a public F1 source to canonical telemetry and learn to keep network concerns outside model code.



6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core

The problem that earns this chapter

APEX begins before you own F1 25, so historical evidence must come from public tooling. FastF1 is a Python library organized around events and sessions; OpenF1 exposes HTTP endpoints for telemetry, timing, position, weather and related records. Their access patterns differ. The engineering challenge is not “download a CSV”; it is to translate each source into evidence with the same meaning.

Predict before reading

A FastF1 session and an OpenF1 endpoint both provide speed, throttle and timestamps, but one uses relative session time and the other UTC timestamps. What information must an adapter preserve so the two sources can later be compared or combined?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Network retrieval, caching, parsing, source-specific naming and canonical conversion are separate steps. Keeping them separate lets you replay raw responses when an adapter changes, test conversion without the network, and avoid rate-limit failures inside training.

The safest ingestion process lands raw evidence first. A second deterministic step creates canonical data. This gives you two forensic layers: “what did the source return?” and “how did our code interpret it?” A model run should point to both.

FastF1 and OpenF1 can overlap without being interchangeable. Differences in sampling, provenance and field definitions should be measured rather than assumed away. APEX treats each adapter as an independently tested producer of the canonical contract.

Vocabulary that now has a job

Concept: Landing zone - Meaning in plain language: Immutable storage of source-native responses before interpretation. - **Role inside APEX:** Cached session/API data used for reproducible adapter tests.

Concept: Provenance - Meaning in plain language: Information describing where, when and how a record was obtained. - **Role inside APEX:** Source, event/session identifiers, driver, retrieval time and adapter version.

Concept: Idempotent ingestion - Meaning in plain language: Repeating the same request does not create conflicting outputs. - **Role inside APEX:** Deterministic paths and hashes for source snapshots.

Concept: Rate limit / cache - Meaning in plain language: External service constraints and local reuse strategy. - **Role inside APEX:** Prevents training from repeatedly hitting public services.

Worked example: calculate it by hand

Suppose OpenF1 returns UTC timestamps 12:00:00.100, 12:00:00.300, and FastF1 returns session-relative seconds 5.2, 5.4. To combine them, you need a session start reference or a shared event clock. If session start is 11:59:54.900, the UTC records map to 5.2 and 5.4 seconds.

The adapter should preserve:

Source timestamp exactly as received.

Parsed canonical timestamp.

Session and driver identifiers.

Source field names/units or adapter version.

Missingness and interpolation flags.

Without those fields, an apparent 0.2-second alignment error cannot be traced.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/19_time_alignment

What we are about to build

Use the alignment lab as an offline stand-in for real streams, then inspect the production FastF1 and OpenF1 adapters. The important skill is tracing a field from raw response to canonical column.

Runnable implementation

```
import pandas as pd
```

```
car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
```

```
weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
```

```
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
```

```
print(aligned)
```

Observed output from the packaged solution

```
t speed rain
0 0.0  10 NaN
1 0.2  12  0.0
2 0.4  14 NaN
3 0.6  16  0.3
```

Read the important lines like English

Code: `pd.merge_asof(...)` - **What the line is doing:** Join asynchronous records after each source has been parsed into a common time representation. - **What to inspect:** Alignment belongs after timestamp normalization.

Code: `direction="backward"` - **What the line is doing:** Preserve online causal availability. - **What to inspect:** Offline retrospective analysis may choose another policy, but must label it.

Code: `tolerance=...` - **What the line is doing:** Prevent arbitrarily stale context from being treated as current. - **What to inspect:** Record unmatched rows rather than silently filling all gaps.

State and tensor trace

network request

↓ raw JSON / library objects (cache this)

source parser

↓ typed source table

unit and timestamp conversion

↓ canonical columns

quality gate

↓ admitted canonical session + report

Training begins only after the last step. It should be possible to run everything after ingestion with no network.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Mock a source response where one field changes name, one timestamp is timezone-naive, and one speed column changes unit. Run the adapter tests and identify which change is detected.

Diagnose from the earliest failed contract

Compare the raw snapshot with the canonical output. If the raw source is correct but canonical data is wrong, the adapter is the earliest failed boundary. If the raw response itself changed, update the source fixture and conversion policy deliberately.

Repair and lock the repair with a test

Store representative source fixtures in tests. Assert exact canonical names, units, timestamp monotonicity and a few known values. Version adapter outputs whenever semantics change.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: FastF1 - Choose it when: Python-centric historical session analysis with convenient session abstractions and cached telemetry. - **Do not choose it when:** You require an HTTP-only integration or fields outside its supported representation.

Implementation: OpenF1 - Choose it when: Service-style access to historical/live endpoint data and independent stream retrieval. - **Do not choose it when:** You assume every endpoint is synchronized or identical to FastF1.

Implementation: Both as separate datasets - Choose it when: You want cross-source validation or broader coverage. - **Do not choose it when:** You have not measured semantic and temporal differences.

Implementation: Merge sources into one row - Choose it when: There is a justified key, timing policy and conflict-resolution rule. - **Do not choose it when:** Matching is based only on similar column names.

APEX V1 supports both adapters but does not require both for one run. Each produces the canonical contract; downstream code stays source-agnostic. Synthetic data remains the offline integration-test source.

Transfer the lesson into Project APEX

Run an adapter command when network access is available, save canonical output, validate it, then call the same run-canonical pipeline used for any source.

Repository path to inspect

```
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
apex_engine/src/apexsim/cli.py
apex_engine/README.md
```

Connection to research

Public datasets are observations, not the world itself. A world model trained on one source can inherit its sampling, filtering and missingness as if those were physical laws. Cross-source tests reveal some of these artifacts.

Check your understanding before continuing

Why should raw source snapshots be immutable?

What should happen when an external field disappears?

Why is network retrieval inside a PyTorch Dataset a bad default?

Solutions and reasoning

1. They allow exact replay, adapter debugging and proof of what the source returned at run time. 2. The adapter should fail clearly or mark the field unavailable according to a versioned policy; silently substituting another field can change meaning. 3. It makes samples nondeterministic, slow, rate-limit dependent and difficult to retry or reproduce.

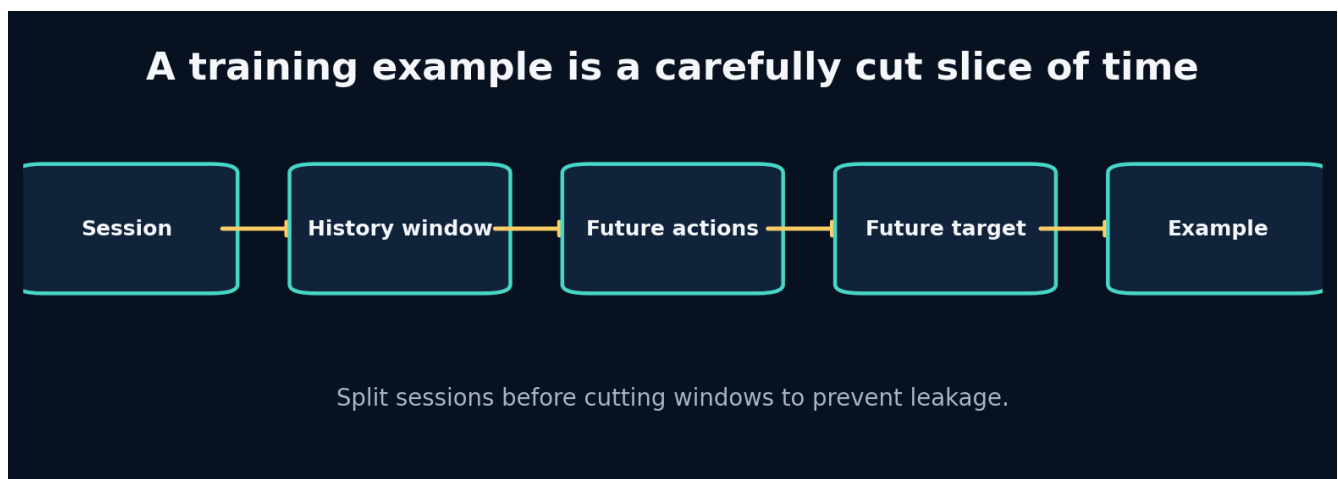
Independent build challenge

Create a recorded fixture for one mocked FastF1-like table and one OpenF1-like JSON response. Write adapters that produce byte-for-byte identical canonical CSV rows.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

7. Windows, Session Splits and Normalization Without Leakage

Instructor objective: Turn canonical sessions into causal training examples while keeping validation and test evidence genuinely independent.



7. Windows, Session Splits and Normalization Without Leakage

The problem that earns this chapter

A telemetry session yields thousands of overlapping windows. If you randomly split those windows, nearly identical frames from the same lap appear in both training and test. The test score becomes a measure of memorization under overlap, not generalization to a new session.

Predict before reading

A 100-frame session uses 20 history frames and 5 future frames with stride 1. How many windows can be created? If windows 1 and 2 are put in different splits, how many frames do they share?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A sequence example has three distinct pieces: history observations, future interventions/context, and future target state. The model must not receive the future target inside its inputs. This sounds obvious, but dataframe slicing and feature lists can quietly duplicate it.

Split units should match the deployment claim. If the engine will simulate a new race session, split by session or event before making windows. A time-based holdout within one session answers a narrower question. Random windows almost always overstate performance when overlap is high.

Normalization must be fitted only on training data. Validation and test are transformed using stored training statistics. Otherwise the mean and variance leak information about future or held-out distributions, and deployment cannot reproduce the transform.

Vocabulary that now has a job

Concept: History length - Meaning in plain language: How many past frames are visible before a forecast begins. - **Role inside APEX:** Controls memory evidence and input cost.

Concept: Forecast horizon - Meaning in plain language: How many future transitions are predicted. - **Role inside APEX:** Defines simulation duration and error accumulation.

Concept: Split unit - Meaning in plain language: The independent entity assigned wholly to train, validation or test. - **Role inside APEX:** Usually session/event, not overlapping windows.

Concept: Train-only normalizer - Meaning in plain language: Scaling statistics fitted exclusively on training observations. - **Role inside APEX:** Persisted with the model artifact.

Worked example: calculate it by hand

For length $N=100$, history $H=20$, future $F=5$, stride 1, the first window consumes frames 0–24 and the last begins at 75, so the count is

$$N - H - F + 1 = 100 - 20 - 5 + 1 = 76.$$

Window 1 uses frames 0–24; window 2 uses 1–25. They share 24 of 25 frames. Assigning them to different splits creates almost complete leakage.

For normalization, training speeds [40, 50, 60] have mean 50 and standard deviation about 8.165. A test speed 70 becomes $(70-50)/8.165 \approx 2.45$. Recomputing the mean with test data would shrink that value and leak the test distribution.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/08_dataset_windows

What we are about to build

Create history/target windows and inspect shapes and boundaries. Then map the same logic to the production window builder, which also carries future controls and session identifiers.

Runnable implementation

```
import numpy as np
from education.core import sliding_windows

values=np.arange(30,dtype=float).reshape(10,3)
x,y=sliding_windows(values,history=4,horizon=2)
print("x",x.shape,"y",y.shape)
print("first history\n",x[0]); print("first future\n",y[0])
```

Observed output from the packaged solution

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/08_dataset_windows/solution.py", line 2, in <module>

```
from education.core import sliding_windows
```

ModuleNotFoundError: No module named 'education'

Read the important lines like English

Code: for start in range(...) - **What the line is doing:** Enumerate valid causal cut points. - **What to**

inspect: The upper bound must leave enough future target frames.

Code: history = data[start:start+H] - **What the line is doing:** Select only evidence available before forecast time. - **What to inspect:** Verify the last history timestamp precedes the first target timestamp.

Code: target = data[start+H:start+H+F] - **What the line is doing:** Select the future trajectory to learn. - **What to inspect:** Do not accidentally include target state in future inputs.

State and tensor trace

session frames: 0 1 2 3 4 5 6 7 8 9

history H=4: [0 1 2 3]

future controls: [4 5 6]

future targets: [4 5 6]

next window: [1 2 3 4] → [5 6 7]

Controls and target states share timestamps but not semantics. Future throttle may be supplied; future speed is what must be predicted.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Create all windows first and randomly split them. Train a nearest-neighbour or linear baseline. Compare its test score with a session-level split.

Diagnose from the earliest failed contract

Inspect source session IDs and frame-index overlap across splits. If any raw session contributes to multiple splits, your claimed generalization unit is false.

Repair and lock the repair with a test

Assign sessions/events deterministically to splits before windowing. Fit normalization on training windows only. Add tests for disjoint identifiers and for target columns absent from model inputs.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Random window split - Choose it when: Only for debugging shape/training code, never for a generalization claim with overlap. - **Do not choose it when:** Any realistic telemetry evaluation.

Implementation: Session split - Choose it when: Deployment targets unseen sessions under similar tracks/drivers. - **Do not choose it when:** You specifically study later portions of the same session.

Implementation: Event/track split - Choose it when: You need evidence of transfer to new circuits or events. - **Do not choose it when:** The dataset is too small to support that claim.

Implementation: Rolling temporal split - Choose it when: Production learns from past sessions and forecasts future dates. - **Do not choose it when:** Timestamp order does not correspond to deployment.

APEX V1 splits by session before creating windows and stores the split assignment. Normalization statistics are versioned artifacts loaded by training, evaluation, simulation and UI.

Transfer the lesson into Project APEX

Trace one WindowDataset sample. Print history, future inputs, targets, session ID, start timestamp and feature names. Then inspect the split tests.

Repository path to inspect

```
apex_engine/src/apexsim/data/windows.py
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/tests/test_windows.py
debugging_cases/04_random_window_split
debugging_cases/06_normalizer_leak
```

Connection to research

World-model papers often report benchmark splits that already define episodes. In custom telemetry projects, defining the episode and independence unit is part of the research contribution, not preprocessing trivia.

Check your understanding before continuing

Why can overlapping windows inflate scores even without exact duplicate rows?

Should validation statistics be used to choose normalization?

What deployment claim does a track holdout support?

Solutions and reasoning

1. They share most temporal context and local dynamics, so the model sees nearly the same trajectory during training. 2. No. Normalization is learned from training only; validation is used for model and hyperparameter selection under that fixed transform. 3. Evidence that the learned dynamics and representation transfer to an unseen circuit distribution, subject to the held-out tracks tested.

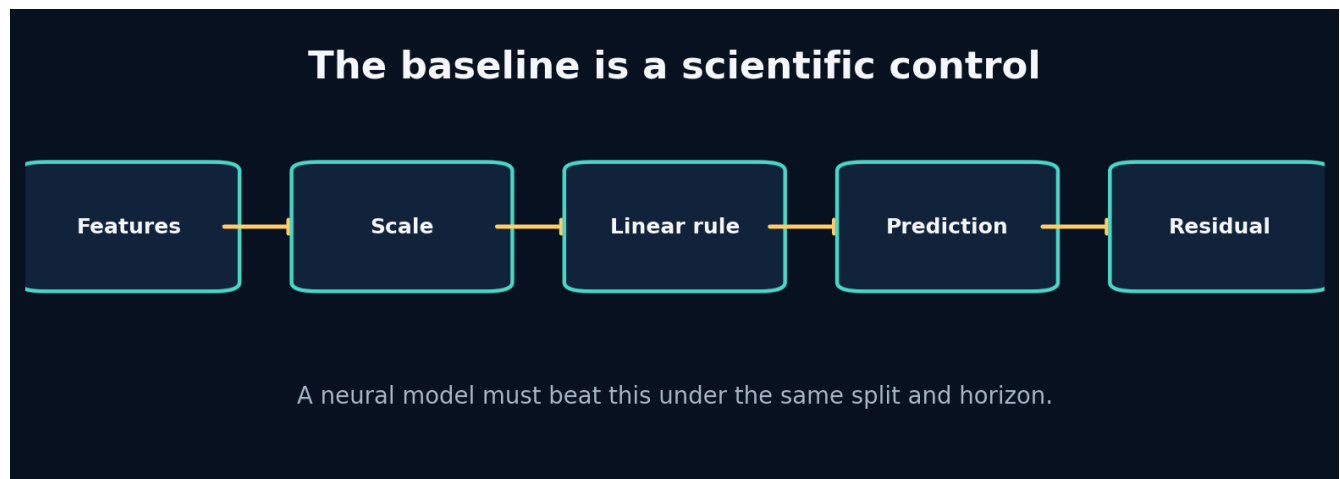
Independent build challenge

Implement a split audit that reports shared session IDs, shared raw frame identifiers, time overlap and distribution differences for every pair of splits.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

8. Statistics, Baselines and Uncertainty as Engineering Controls

Instructor objective: Use simple models and residual analysis to determine what the data contains before escalating to deep sequence architectures.



8. Statistics, Baselines and Uncertainty as Engineering Controls

The problem that earns this chapter

A GRU can produce a low loss for the wrong reason: leaked targets, easy persistence, narrow test conditions or an imbalanced metric. A baseline is the control group that tells you whether complexity has earned its place.

Predict before reading

At 5 Hz, speed changes slowly between adjacent frames. Which will be harder to beat for one-step speed prediction: a constant mean, persistence (next speed = current speed), or a linear action-conditioned model? Explain how the answer may change at a longer horizon.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Start with the distribution: means, spreads, missingness, correlations and conditional relationships. Then create baselines that correspond to real hypotheses. Persistence says the world changes slowly. Linear regression says the change is approximately additive in observed features. A physics baseline says known equations explain most of the transition.

Residuals—target minus prediction—show what the baseline failed to explain. Plot residuals against speed, curvature, rain, tyre age and horizon. Structured residuals are a blueprint for the next model. Random-looking residuals near measurement noise may mean a larger model has little room to help.

Uncertainty has several meanings. Aleatoric uncertainty reflects inherently variable outcomes or noisy measurements. Epistemic uncertainty reflects lack of knowledge, often rising outside the training distribution. A point predictor does not automatically communicate either.

Vocabulary that now has a job

Concept: Persistence baseline - Meaning in plain language: Predict that the next state equals the current state. - **Role inside APEX:** Strong short-horizon speed control.

Concept: Residual - Meaning in plain language: Observed target minus model prediction. - **Role inside APEX:** Reveals missing nonlinearities, context or delay.

Concept: Calibration - Meaning in plain language: Predicted uncertainty/probability matches empirical frequency or error. - **Role inside APEX:** Needed before presenting confidence in imagined futures.

Concept: Distribution shift - Meaning in plain language: Deployment evidence differs from training evidence. - **Role inside APEX:** New tracks, weather, drivers or game telemetry.

Worked example: calculate it by hand

Suppose true next speeds are [50.2, 60.1, 69.7] and persistence predicts [50.0, 60.0, 70.0].

Absolute errors are [0.2, 0.1, 0.3]; MAE is $(0.2+0.1+0.3)/3 = 0.2$ m/s.

A neural model with MAE 0.18 improves by 0.02 m/s, or 10% relative. That gain might matter—or it might vanish on a new session, cost far more compute, and introduce physical violations. Compare by horizon, subgroup, latency and stability before declaring victory.

If residuals under braking average +0.8 m/s (targets are higher than predictions), the baseline is overestimating deceleration. That points to brake scaling, delay or grip context.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/05_regression_baseline

What we are about to build

Fit a Ridge transition model to synthetic speed, throttle and brake data, inspect coefficients and calculate test MAE.

Runnable implementation

```
import numpy as np
from sklearn.linear_model import Ridge

rng=np.random.default_rng(4)
speed=rng.uniform(20,80,1000)
throttle=rng.uniform(0,1,1000)
brake=rng.uniform(0,1,1000)
y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
X=np.column_stack([speed,throttle,brake])
model=Ridge(alpha=1.0).fit(X[:800],y[:800])
print("coefficients:", model.coef_)
print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))
```

Observed output from the packaged solution

```
coefficients: [ 0.91911817  0.68984381 -1.03384695]
test MAE: 0.18453407932904015
```

Read the important lines like English

Code: `X=np.column_stack([speed,throttle,brake])` - **What the line is doing:** Construct one row per transition with named causal inputs. - **What to inspect:** Column order must match stored feature metadata.

Code: `Ridge(alpha=1.0).fit(X[:800],y[:800])` - **What the line is doing:** Estimate a regularized linear transition using training rows. - **What to inspect:** The split here is educational; real telemetry should split by session.

Code: `model.coef_` - **What the line is doing:** Expose the learned direction and relative scale of each feature. - **What to inspect:** Interpret coefficients only after accounting for units and scaling.

State and tensor trace

```
[speed, throttle, brake]
  ↓ linear weighted sum
predicted next speed
  ↓ subtract observed next speed
residual
  ↓ group/plot by condition
model-improvement hypothesis
```

The residual is not merely an error score; it is evidence about missing structure.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Add the target next speed as an input feature and watch MAE collapse. Then randomize brake while preserving speed. Compare coefficient signs and residuals.

Diagnose from the earliest failed contract

A suspiciously perfect baseline should trigger a feature audit and time-index trace before celebration. Check that every input is available at forecast time and that split entities are disjoint.

Repair and lock the repair with a test

Maintain an allow-list of input features, unit-test target exclusion, and report persistence beside every model. Add residual plots by horizon and condition to evaluation artifacts.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Persistence - Choose it when: Very short horizons and slowly changing states. - **Do not choose it when:** Actions or geometry cause rapid transitions.

Implementation: Linear / Ridge - Choose it when: You need interpretability, speed and a strong control. - **Do not choose it when:** Residuals show substantial nonlinear or temporal structure.

Implementation: Gradient-boosted trees - Choose it when: Tabular nonlinearities dominate and fixed windows can summarize history. - **Do not choose it when:** You need smooth autoregressive latent dynamics or differentiable planning.

Implementation: Probabilistic baseline - Choose it when: Forecast intervals or multi-modal outcomes matter. - **Do not choose it when:** You have not first validated deterministic error and calibration.

APEX keeps persistence and a linear transition baseline in the same evaluation harness as learned world models. A complex model is promoted only when it improves the relevant horizons without unacceptable violations or operational cost.

Transfer the lesson into Project APEX

Open the baseline implementation and evaluation report. Add a residual plot against curvature and a table comparing relative improvement over persistence.

Repository path to inspect

```
apex_engine/src/apexsim/models/baselines.py
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/pipeline/stages.py
debugging_cases/03_target_leakage
```

Connection to research

Strong baselines protect research from architecture-driven conclusions. If a latent world model cannot outperform persistence on the actual decision horizon, its representation may still be useful—but that claim needs a different evaluation.

Check your understanding before continuing

Why is relative improvement over persistence more informative than a standalone MAE?

What does a curved residual-vs-speed plot suggest?

Can uncertainty be low while the prediction is wrong?

Solutions and reasoning

1. It shows how much predictable change the model captures beyond the natural smoothness of telemetry. 2. The transition contains nonlinear speed dependence or the feature scaling/model form is misspecified. 3. Yes. An uncalibrated model can be confidently wrong, especially under distribution shift.

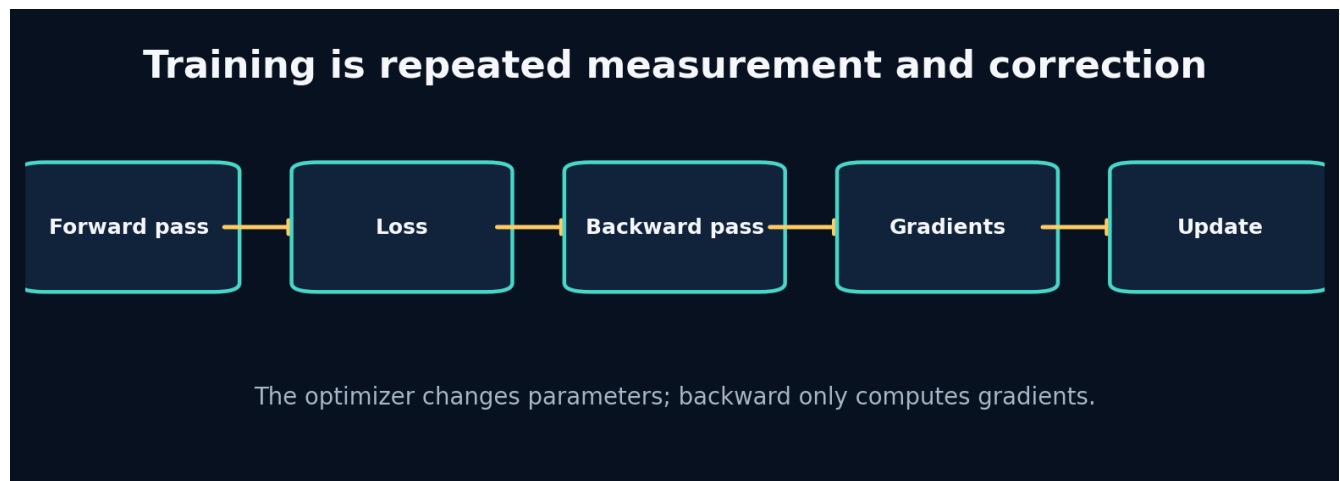
Independent build challenge

Create persistence, linear and polynomial baselines under the same session split. Produce a one-page model card explaining which baseline wins at 1, 5 and 20 steps and why.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

9. Tensors, Autograd and the Training Loop You Can Explain

Instructor objective: Build a PyTorch training loop while understanding every tensor, gradient and parameter update rather than memorizing boilerplate.



9. Tensors, Autograd and the Training Loop You Can Explain

The problem that earns this chapter

Many learners can write `loss.backward()` but cannot say what changed afterward. This becomes dangerous when gradients accumulate, validation tracks graphs, or tensor axes are swapped. We will turn the training loop into an observable state machine.

Predict before reading

Immediately after `loss.backward()` and before `optimizer.step()`, have the model weights changed? Where is the information needed to change them stored? What happens if `zero_grad()` is omitted for two iterations?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A tensor is an array plus shape, dtype, device and—when requested—a connection to a computation graph. The forward pass applies parameterized operations. The loss reduces prediction error to a scalar. Backpropagation computes derivatives of that scalar with respect to leaf parameters and stores them in `.grad`. The optimizer then uses those gradients to update parameters.

Gradients accumulate by default. That enables deliberate gradient accumulation but causes a bug when forgotten. Evaluation must switch behaviour-sensitive layers with `model.eval()` and disable graph construction with `torch.no_grad()`.

The most effective way to learn this is to print parameter values, gradients and loss before and after each line on a tiny problem whose correct mapping is known.

Vocabulary that now has a job

Concept: Computation graph - Meaning in plain language: The recorded chain of operations connecting inputs, parameters and loss. - **Role inside APEX:** Enables gradient-based training of all world models.

Concept: Gradient - Meaning in plain language: Local sensitivity of loss to a parameter. - **Role inside APEX:** Stored in each trainable parameter after backward.

Concept: Optimizer state - Meaning in plain language: Extra memory used to turn gradients into updates. - **Role inside APEX:** Adam moments are checkpointed with the model during resumable training.

Concept: Train/eval mode - Meaning in plain language: Switch for layers such as dropout and batch normalization. - **Role inside APEX:** Ensures deterministic, correctly normalized validation and rollout.

Worked example: calculate it by hand

For a one-parameter model $\hat{y} = w \times x$ with $x=2$, $y=6$, $w=1$, and squared loss $((y-\hat{y})^2$:

Prediction: $1 \times 2 = 2$.

Error: $2 - 6 = -4$.

Loss: 16.

Gradient: $dL/dw = 2(wx - y)x = 2(-4)(2) = -16$.

With learning rate 0.1, SGD update: $w \leftarrow 1 - 0.1(-16) = 2.6$.

New prediction: 5.2; new loss: 0.64.

Backward computes -16 ; the optimizer performs the update to 2.6.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/10_training_loop

What we are about to build

Train a linear PyTorch module to sum three input features. Print training loss, gradient norm and validation loss each epoch.

Runnable implementation

```
import torch
from torch import nn
```

```
torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
for epoch in range(8):
    model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
    grad=float(model.weight.grad.norm()); opt.step()
    model.eval();
    with torch.no_grad(): val=((model(x)-y)**2).mean()
    print(epoch,float(loss),grad,float(val))
```

Observed output from the packaged solution

```
0 3.2264039516448975 3.7724902629852295 2.9054489135742188
1 2.9054489135742188 3.5811667442321777 2.6067750453948975
2 2.6067750453948975 3.390789270401001 2.3303141593933105
3 2.3303141593933105 3.2017178535461426 2.0752670764923096
4 2.0752670764923096 3.0143868923187256 1.8400570154190063
5 1.8400570154190063 2.829359292984009 1.6230223178863525
6 1.6230223178863525 2.6473701000213623 1.4232583045959473
7 1.4232583045959473 2.469295024871826 1.2406574487686157
```

/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/10_training_loop/solution.py:11:

UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.

Consider using tensor.detach() first. (Triggered internally at

/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)

```
print(epoch,float(loss),grad,float(val))
```

Read the important lines like English

Code: opt.zero_grad() - **What the line is doing:** Clear gradients left by previous backward passes. -

What to inspect: It does not reset model weights.

Code: pred=model(x) - **What the line is doing:** Run the forward computation using current parameters. - **What to inspect:** Inspect prediction shape against target shape.

Code: loss.backward() - **What the line is doing:** Populate .grad for parameters connected to the loss. -

What to inspect: Weights have not yet changed.

Code: opt.step() - **What the line is doing:** Use gradients and optimizer state to update parameters. -

What to inspect: Compare parameter values before and after.

Code: with torch.no_grad() - **What the line is doing:** Compute validation outputs without constructing a backward graph. - **What to inspect:** Also call model.eval() for mode-dependent layers.

State and tensor trace

parameters θ_0

↓ forward

predictions $\hat{y}$

↓ loss

scalar L

↓ backward

$\theta.grad$ populated; θ still equals θ_0

↓ optimizer.step

parameters θ_1

Print `id(parameter)`, `parameter.detach()`, and `parameter.grad` around these steps once. That trace removes the mystery permanently.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove `zero_grad`, increase learning rate by 100×, and accidentally shape targets as `[B]` while predictions are `[B,1]`. Observe accumulation, divergence and broadcasting.

Diagnose from the earliest failed contract

Check shapes first, then finite values, loss scale, gradient norms, parameter-update magnitude and mode. Debug the smallest batch before scaling to full telemetry.

Repair and lock the repair with a test

Add assertions for exact prediction/target shape, finite loss and finite gradients. Add a one-batch overfit test and a deterministic seed. Make gradient accumulation an explicit configured feature if used.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: SGD - Choose it when: You want transparent updates and can tune schedules carefully. - **Do not choose it when:** Sparse/noisy gradients need faster adaptive progress.

Implementation: Adam/AdamW - Choose it when: A strong default for deep sequence models and fast iteration. - **Do not choose it when:** You assume it removes the need for learning-rate and regularization experiments.

Implementation: Gradient clipping - Choose it when: Recurrent or long-horizon training shows occasional exploding norms. - **Do not choose it when:** It hides systematically bad scaling or unstable architecture.

Implementation: Mixed precision - Choose it when: GPU throughput/memory matters and numerical checks pass. - **Do not choose it when:** You have not established a stable full-precision reference.

APEX training records seed, configuration, gradient-related settings, checkpoint metric and normalizer. Validation is performed under eval/no-grad and the best checkpoint is restored before test evaluation.

Transfer the lesson into Project APEX

Trace one batch through the GRU training function. Print every shape and compare parameter checksum before backward, after backward and after step.

Repository path to inspect

`apex_engine/src/apexsim/training.py`

`apex_engine/src/apexsim/models/gru_world_model.py`

debugging_cases/08_missing_eval_mode
 debugging_cases/09_gradient_accumulation

Connection to research

Research papers summarize optimization in a paragraph, but reproducing them depends on these details: gradient scale, target construction, optimizer state, evaluation mode, clipping, schedules and checkpoint selection.

Check your understanding before continuing

What exactly does `backward()` mutate?

Why can broadcasting produce a low-looking loss with the wrong semantics?

What does successful one-batch overfitting prove and not prove?

Solutions and reasoning

1. It accumulates gradients into `.grad` fields of leaf tensors requiring gradients; it does not update parameter values. 2. PyTorch expands compatible dimensions silently, comparing unintended pairs while still returning a scalar. 3. It proves the data path, model and optimizer can memorize a tiny sample; it does not prove generalization, correct splits or realistic rollout behaviour.

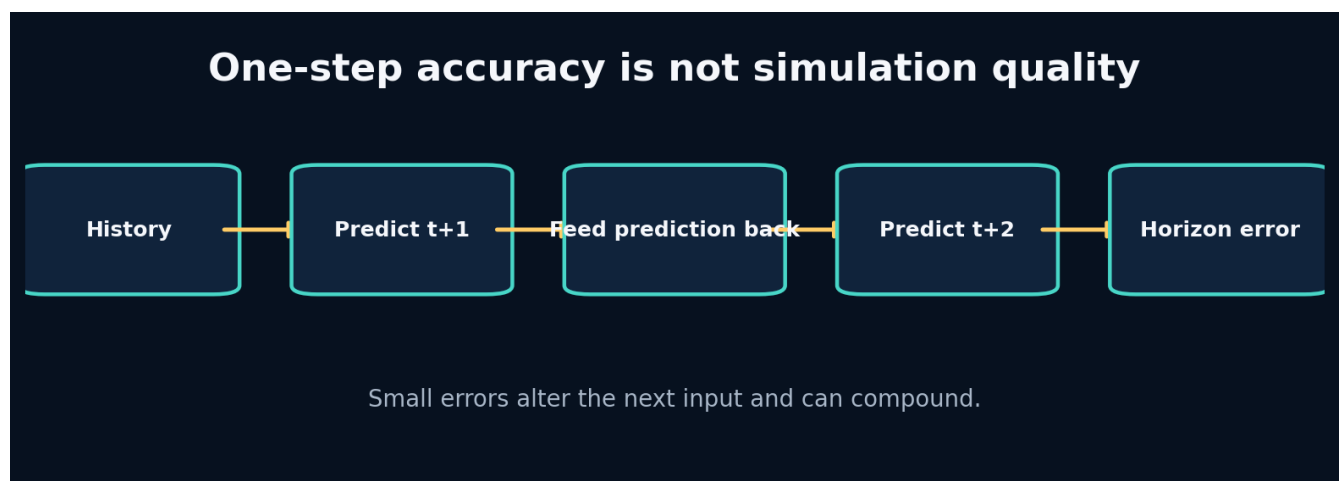
Independent build challenge

Instrument a training step to write a JSON trace containing input shapes, loss, gradient norms and parameter-update norms. Use it to diagnose three intentionally broken configurations.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

10. Transition Models and Why Rollouts Fail

Instructor objective: Build a one-step neural transition, convert it into an autoregressive simulator, and measure error compounding honestly.



10. Transition Models and Why Rollouts Fail

The problem that earns this chapter

A model can be excellent at predicting the next frame when given the true current frame, yet collapse when asked to simulate ten frames. During rollout it consumes its own imperfect predictions, a distribution it did not see during teacher-forced training.

Predict before reading

A model has a constant +0.1 m/s bias per predicted step and no corrective feedback. What speed bias do you expect after 20 autoregressive steps? What if the transition also multiplies the current error by 1.05 each step?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

One-step supervised training samples inputs from real data. Autoregressive inference samples later inputs from the model. This train–inference distribution shift is exposure bias. Even unbiased local noise can alter future controls' effects because the state enters nonlinear dynamics.

A rollout function must specify which future variables are known and which are predicted. In APEX, future controls/context can be provided by a scenario, while future state is generated. After each step, predicted state replaces the state portion of the next input; controls remain externally supplied.

Evaluation should report error by horizon rather than averaging all future steps. The shape of the curve distinguishes immediate bias, unstable compounding and long-memory failure.

Vocabulary that now has a job

Concept: Teacher forcing - **Meaning in plain language:** Train each transition using the true previous state. - **Role inside APEX:** Stable supervised training for GRU/RSSM components.

Concept: Autoregressive rollout - **Meaning in plain language:** Feed predicted state back to generate later states. - **Role inside APEX:** The actual simulation mode.

Concept: Exposure bias - **Meaning in plain language:** The model trains on true states but runs on its own imperfect states. - **Role inside APEX:** A central source of horizon degradation.

Concept: Horizon curve - **Meaning in plain language:** Error reported separately at each future step. - **Role inside APEX:** Primary evidence for useful simulation duration.

Worked example: calculate it by hand

With additive bias $e_{t+1}=e_t+0.1$, starting at zero, error after 20 steps is 2.0 m/s.

With amplification $e_{t+1}=1.05e_t+0.1$,

$$e_{20}=0.1 \frac{1.05^{20}-1}{1.05-1} \approx 3.31 \text{ m/s}.$$

The one-step bias is still only 0.1, but the system dynamics amplify it. This is why one-step MAE cannot certify a simulator.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/07_pytorch_module

What we are about to build

Use a small PyTorch transition module as the building block, then write a loop that feeds predicted state back while taking future controls from an external sequence.

Runnable implementation

```
import torch
from torch import nn

class Transition(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
    def forward(self,x): return self.net(x)
```

```
model=Transition(); x=torch.randn(4,5); y=model(x)
print(model); print("input",x.shape,"output",y.shape)
```

Observed output from the packaged solution

```
Transition(
  (net): Sequential(
    (0): Linear(in_features=5, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=2, bias=True)
  )
)
input torch.Size([4, 5]) output torch.Size([4, 2])
```

Read the important lines like English

Code: prediction = model(current_input) - **What the line is doing:** Predict the next state from current state/action/context. - **What to inspect:** Confirm output contains only target state variables.

Code: current_state = prediction - **What the line is doing:** Move the imagined world forward using its own result. - **What to inspect:** Detach only when appropriate; training through rollout may need gradients.

Code: next_input = concat(current_state, future_control[t]) - **What the line is doing:** Combine generated state with externally specified intervention. - **What to inspect:** Never overwrite the future control with a predicted target column.

State and tensor trace

real history → encoded state

step 1: predicted state₁ + supplied action₁ → input₂

step 2: predicted state₂ + supplied action₂ → input₃

step 3: predicted state₃ + supplied action₃ → input₄

At each step, log both normalized and physical-unit state. Violations may be hidden in normalized space.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

During rollout, accidentally keep feeding the last real state instead of each prediction. The multi-step score may look surprisingly strong because you are not actually simulating.

Diagnose from the earliest failed contract

Trace the source of every input tensor at every horizon. Mark each element as observed, supplied intervention, static context or model-generated. A rollout audit catches “teacher forcing at test time.”

Repair and lock the repair with a test

Add a test with a deterministic transition where the exact multi-step trajectory is known. Assert that changing a predicted state changes later predictions. Report teacher-forced and free-running metrics separately.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: One-step training - Choose it when: A stable starting point with abundant transition examples. - **Do not choose it when:** You treat its metric as rollout certification.

Implementation: Multi-step loss - Choose it when: Long-horizon accuracy matters and compute allows unrolled training. - **Do not choose it when:** Early training is unstable or the horizon curriculum is not controlled.

Implementation: Scheduled sampling - Choose it when: You want gradual exposure to model-generated states. - **Do not choose it when:** You assume it guarantees consistent probabilistic learning.

Implementation: Direct multi-horizon model - Choose it when: Fixed-horizon forecasts matter more than a reusable transition. - **Do not choose it when:** You need arbitrary-length interactive simulation.

APEX trains a stable sequence world model and evaluates it under free-running rollouts. Any reported simulation horizon is tied to its error and violation curves, not a marketing duration.

Transfer the lesson into Project APEX

Inspect `simulation.py` and identify the exact line where predicted state is inserted into the next model input. Add a debug mode that records provenance for every future feature.

Repository path to inspect

```
projects/04_mlp_transition/main.py
apex_engine/src/apexsim/simulation.py
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/models/gru_world_model.py
```

Connection to research

World-model planning magnifies rollout weaknesses because a planner searches for trajectories that exploit them. Reliable imagination requires evaluating model-generated state distributions, not only posterior-conditioned reconstruction.

Check your understanding before continuing

Why can zero-mean one-step noise still create biased long rollouts?

What future variables may legitimately be supplied to a simulator?

What does a flat horizon-error curve suggest?

Solutions and reasoning

1. Nonlinear dynamics and constraints can transform symmetric local noise into asymmetric state evolution. 2. Known scenario interventions and exogenous forecasts, provided their availability is explicit; future target state cannot be supplied. 3. The model may be stable over that range, or errors may be dominated by a constant initial bias rather than compounding; inspect conditions and baselines.

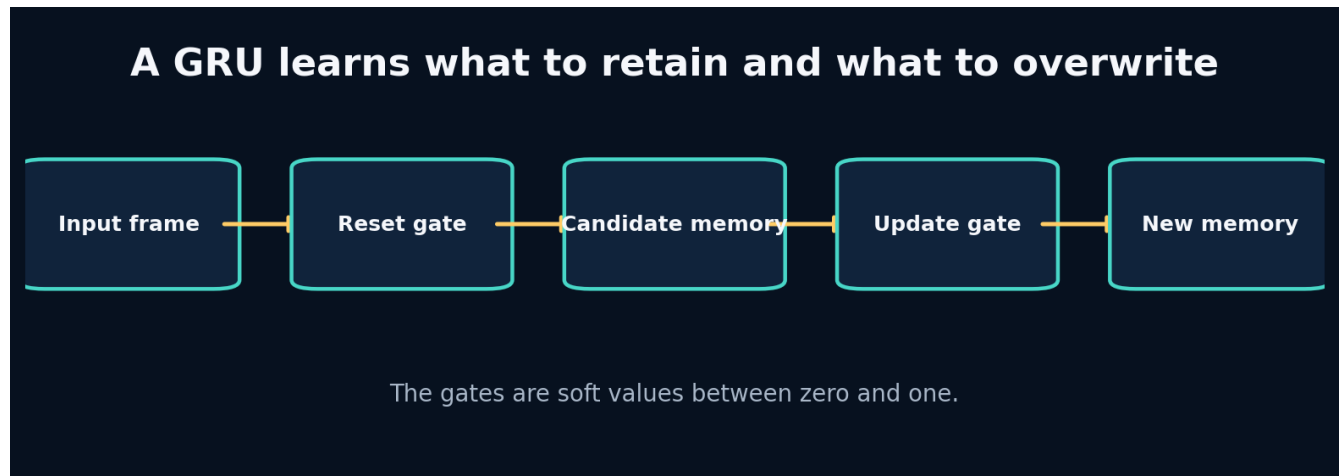
Independent build challenge

Train the same transition with one-step and five-step losses. Compare horizon curves, training stability and compute. Record which objective you choose for APEX V1 and why.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

11. Recurrent Memory and the GRU From the Inside

Instructor objective: Earn recurrent memory by exposing the limits of fixed-frame transitions, then inspect how GRU gates update hidden state.



11. Recurrent Memory and the GRU From the Inside

The problem that earns this chapter

Current speed, throttle and brake may not reveal whether the car has been accelerating for two seconds, exiting a corner, heating tyres or recovering from a braking event. A fixed-frame model sees only the present. We need a mechanism that carries a learned summary of the past.

Predict before reading

Two sequences end with the same final frame [speed=50, throttle=0.5, brake=0]. Sequence A accelerated steadily; sequence B braked hard and recovered. Should a recurrent model be able to produce different next-state predictions? What must differ internally?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A recurrent neural network updates hidden memory as each frame arrives. The final hidden state is not a literal copy of history; it is a learned summary optimized for the training objective. A vanilla recurrent cell can struggle when useful information must survive many updates because gradients repeatedly pass through the same transition.

A GRU adds gates. The update gate decides how much old memory to keep versus replace. The reset gate controls how much previous memory influences the candidate update. These are soft, feature-wise decisions, not binary switches. The same hidden dimension can retain long-term tyre information while rapidly updating braking state.

Using nn.GRU does not guarantee meaningful memory. You still need enough history, correct ordering, no hidden-state leakage across independent sequences, and evaluation that tests whether memory improves the target horizon.

Vocabulary that now has a job

Concept: Hidden state - Meaning in plain language: A learned vector carried across sequence steps. -

Role inside APEX: Compressed recent driving and unobserved dynamical context.

Concept: Update gate - Meaning in plain language: Controls interpolation between old memory and candidate memory. - **Role inside APEX:** Lets some information persist while other information changes.

Concept: Reset gate - Meaning in plain language: Controls how previous memory contributes to the candidate update. - **Role inside APEX:** Allows rapid forgetting when current evidence changes regime.

Concept: Sequence boundary - Meaning in plain language: The point where hidden memory must be reset or intentionally transferred. - **Role inside APEX:** Driver/session windows cannot share hidden state accidentally.

Worked example: calculate it by hand

A simplified update is

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

If old memory is $h=0.8$, candidate memory is -0.2 :

With update gate $z=0.1$: $h_t=0.9(0.8)+0.1(-0.2)=0.70$. Most old memory survives.

With $z=0.9$: $h_t=0.1(0.8)+0.9(-0.2)=-0.10$. The new evidence largely overwrites memory.

This interpolation makes a gate interpretable at the mechanism level, although individual learned dimensions may not map cleanly to human concepts.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/12_gru_gates

What we are about to build

Feed three simple one-hot-like inputs through a GRUCell and print hidden state after each step. The goal is to see memory as a changing vector rather than a black box.

Runnable implementation

import torch

```
torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
for x in [torch.tensor([[1,0]]),torch.tensor([[0,1]]),torch.tensor([[0,1]])]:
    h=cell(x,h); print(h.detach().numpy().round(3))
```

Observed output from the packaged solution

```
[[ 0.354 -0.151 -0.384  0.149]]
[[ 0.278  0.282 -0.371  0.408]]
[[ 0.322  0.39  -0.39  0.542]]
```

Read the important lines like English

Code: cell=torch.nn.GRUCell(2,4) - **What the line is doing:** Create a transition from a 2-feature input and 4-value memory to new memory. - **What to inspect:** The hidden size is model capacity, not a known physical variable count.

Code: h=torch.zeros(1,4) - **What the line is doing:** Initialize memory for one sequence. - **What to inspect:** Initial-state policy affects early predictions.

Code: h=cell(x,h) - **What the line is doing:** Combine current observation and previous memory into next memory. - **What to inspect:** Sequence order changes the result.

State and tensor trace

$$\begin{array}{l} x_1 + h_0=0 \rightarrow h_1 \\ x_2 + h_1 \rightarrow h_2 \\ x_3 + h_2 \rightarrow h_3 \\ \downarrow \\ \text{decoder predicts future} \end{array}$$

For batched sequences, PyTorch commonly uses [batch, time, features] when batch_first=True. The returned hidden state is typically [layers, batch, hidden]. Name these axes every time.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Reuse the hidden state from one session as the initial state for an unrelated session. Also shuffle time within each sequence while preserving rows.

Diagnose from the earliest failed contract

If validation varies with dataloader order or batch grouping, inspect hidden-state lifetime. Hidden memory should reset at independent sequence boundaries unless stateful streaming is an explicit design.

Repair and lock the repair with a test

Add a test where two sequences processed separately match the same sequences processed in a batch with zero initial states. Add session-boundary reset logic and make stateful mode explicit.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Vanilla RNN - Choose it when: Tiny educational tasks and very short dependencies. - **Do not choose it when:** Long/noisy telemetry histories.

Implementation: GRU - Choose it when: Strong compact recurrent baseline with fewer parameters than LSTM. - **Do not choose it when:** You have evidence that longer structured memory or parallel training is required.

Implementation: LSTM - Choose it when: You value an explicit cell state and gates and have sufficient compute/data. - **Do not choose it when:** Extra complexity provides no measured gain.

Implementation: Temporal convolution - Choose it when: Dependencies fit a fixed receptive field and parallel computation matters. - **Do not choose it when:** You need adaptive unbounded recurrence or stateful streaming.

The GRU is APEX V1's primary deterministic world model because it is understandable, stable, compact and strong enough to establish the pipeline. SSM and RSSM models are challengers, not automatic replacements.

Transfer the lesson into Project APEX

Trace the GRU world model from history encoder to autoregressive decoder. Print hidden shape, decoder input shape and output state at each horizon for a batch of one.

Repository path to inspect

```
labs/11_rnn_from_scratch/solution.py
labs/12_gru_gates/solution.py
projects/06_gru_forecaster/main.py
apex_engine/src/apexsim/models/gru_world_model.py
debugging_cases/07_hidden_state_reuse
```

Connection to research

Recurrent state is a deterministic belief summary. Dreamer's RSSM retains a recurrent deterministic state while adding a stochastic latent variable to represent uncertainty and ambiguity.

Check your understanding before continuing

Why can two sequences with the same final observed frame produce different hidden states?

Does a larger hidden size always improve memory?

When should hidden state persist across API requests?

Solutions and reasoning

1. The recurrence applies ordered updates, so earlier frames alter the memory entering later frames. 2. No. It increases capacity but may overfit, slow training or store irrelevant detail; history, objective and optimization also limit memory. 3. Only in an explicitly stateful stream with stable entity/session identity, ordering guarantees and reset rules. Stateless scenario requests should rebuild from supplied history.

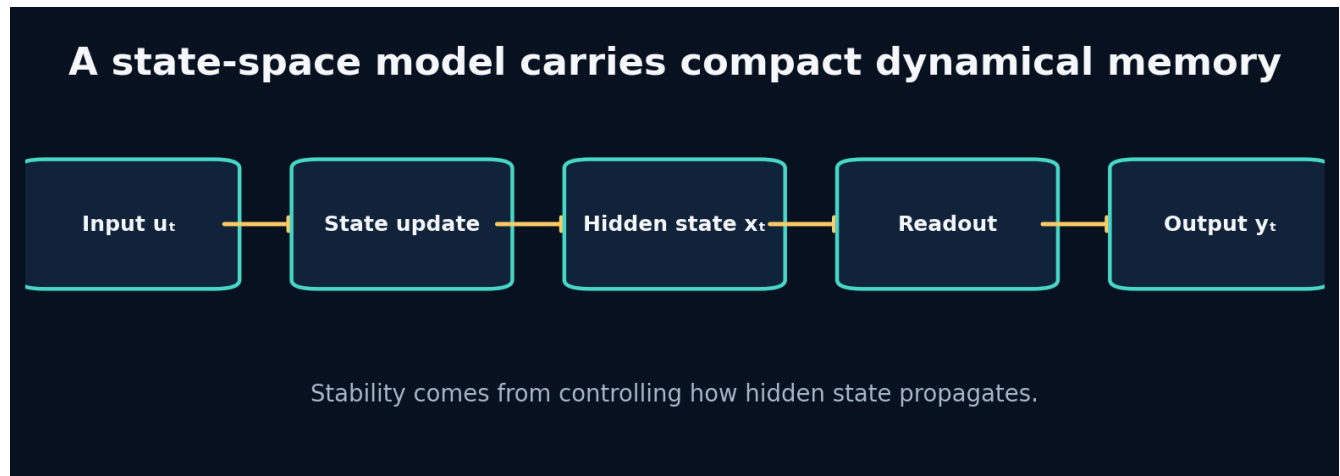
Independent build challenge

Implement a GRU forecaster and a frame-only MLP with matched parameter counts. Create a synthetic target dependent on an event ten steps ago and compare performance as history length changes.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

12. Classical State-Space Models: Dynamics as Memory

Instructor objective: Understand state-space recurrence mathematically, inspect stability, and connect classical dynamical systems to modern sequence models.



12. Classical State-Space Models: Dynamics as Memory

The problem that earns this chapter

GRUs learn memory through generic gates. State-space models begin with an explicit dynamical view: a hidden state evolves under a transition and receives input. This structure can represent long memory efficiently, but an unstable transition can explode over repeated steps.

Predict before reading

For the scalar recurrence $x_t = 1.05 x_{t-1} + u_t$, what happens after inputs become zero? What changes if the transition coefficient is 0.95?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A discrete linear state-space model is

$$x_t = A x_{t-1} + B u_t, y_t = C x_t + D u_t.$$

The hidden state x is memory; A controls how it persists and mixes; B writes input; C reads output. Repeated powers of A determine whether past information decays, persists, oscillates or explodes.

Many modern SSMs begin from continuous-time dynamics and discretize them for sampled sequences. This connects model behaviour to the time step. A change from 5 Hz to 20 Hz should not be treated as an arbitrary tensor resize when the transition is supposed to represent time.

Linear recurrence alone cannot model all nonlinear racing dynamics, but it exposes memory timescales and stability more directly than a generic black box.

Vocabulary that now has a job

Concept: State matrix A - Meaning in plain language: Controls hidden-state propagation between steps. - **Role inside APEX:** Determines memory decay, coupling and stability.

Concept: Input matrix B - Meaning in plain language: Controls how current telemetry writes into memory. - **Role inside APEX:** Maps actions/context into dynamical state.

Concept: Readout C - Meaning in plain language: Maps hidden memory to predicted output. - **Role inside APEX:** Produces next-state features from SSM memory.

Concept: Spectral radius - Meaning in plain language: Largest absolute eigenvalue of A ; a key discrete stability indicator. - **Role inside APEX:** Values above one can amplify state without bound.

Worked example: calculate it by hand

For $x_t = a x_{t-1}$ after input stops:

If $a=1.05$, then after 20 steps x is multiplied by $1.05^{20} \approx 2.65$.

If $a=0.95$, it is multiplied by $0.95^{20} \approx 0.358$.

If $a=1$, memory persists exactly.

If $a=-0.95$, memory alternates sign while decaying.

For a matrix, eigenvalues play the role of these scalar factors along different modes. Stability does not guarantee usefulness: a transition that decays too quickly forgets everything.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/13_linear_ssm

What we are about to build

Run a two-dimensional linear SSM under an input pulse and observe one state component react immediately while another integrates it more slowly.

Runnable implementation

```
import numpy as np
A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
```

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

```
for u in [1,1,0,0,0]:
    x=A@x+B[:,0]*u; print(x.round(3))
```

Observed output from the packaged solution

```
[0.2 0.05]
[0.39 0.112]
[0.37 0.135]
[0.352 0.151]
[0.334 0.164]
```

Read the important lines like English

Code: `A=np.array([[0.95,0.0],[0.1,0.85]])` - **What the line is doing:** Define decay and coupling of two hidden modes. - **What to inspect:** Diagonal values set persistence; off-diagonal 0.1 transfers state.

Code: `B=np.array([[0.2],[0.05]])` - **What the line is doing:** Define how the scalar input writes into both modes. - **What to inspect:** Different modes receive different input strength.

****Code:** `x=A@x+B[:,0]*u` - **What the line is doing:** **Apply one state-space transition.** - **What to inspect:**** Repeated multiplication reveals stability and memory.

State and tensor trace

$u_0=1 \rightarrow$ immediate mode rises strongly; slow mode rises slightly

$u_1=1 \rightarrow$ both accumulate

$u_2=0 \rightarrow$ immediate mode decays; slow mode still receives coupled memory

$u_3=0 \rightarrow$ both decay on different timescales

Plot each hidden dimension. A hidden state is useful because its dynamics differ, not because it has many numbers.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change a diagonal value in A to 1.2 and roll for 100 zero-input steps. Then set both diagonal values to 0.1.

Diagnose from the earliest failed contract

Measure hidden-state norm and eigenvalues. Explosion points to unstable propagation; rapid collapse points to memory that decays faster than the required dependency.

Repair and lock the repair with a test

Parameterize stable decay, constrain eigenvalues or add normalization/clipping as appropriate. Add tests that hidden state stays finite under bounded inputs for the intended horizon.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Fixed linear SSM - **Choose it when:** System identification, interpretability and approximately linear dynamics. - **Do not choose it when:** Strong context-dependent nonlinear transitions dominate.

Implementation: Nonlinear readout - Choose it when: Memory dynamics are simple but state-to-output mapping is nonlinear. - **Do not choose it when:** Input must alter the memory law itself.

Implementation: Input-dependent SSM - Choose it when: Different telemetry regimes require selective writing/forgetting. - **Do not choose it when:** A fixed transition already works and simplicity matters.

Implementation: GRU - Choose it when: You need a proven generic nonlinear recurrent baseline. - **Do not choose it when:** Long-sequence efficiency or structured timescales are the measured bottleneck.

APEX includes a small SSM-style challenger with bounded decay and input-dependent updates. It is taught as a selective recurrent model, not represented as a complete optimized Mamba implementation.

Transfer the lesson into Project APEX

Inspect the SSM cell parameters and plot hidden norms across a full session. Compare horizon curves and latency with the GRU under the same data and budget.

Repository path to inspect

```
labs/13_linear_ssm/solution.py
projects/07_selective_ssm/main.py
apex_engine/src/apexsim/models/ssm_world_model.py
debugging_cases/10_exploding_ssm
```

Connection to research

S4 and related structured SSMs use mathematical parameterizations and efficient convolution/recurrent forms to handle long sequences. The classical equation here is the conceptual root that makes those designs readable.

Check your understanding before continuing

Why can a stable SSM still be a bad memory model?

What does an off-diagonal element of A do?

How does sample rate affect a transition intended to represent physical time?

Solutions and reasoning

1. It may decay too quickly, ignore relevant inputs or lack nonlinear/selective behaviour. 2. It transfers information from one hidden component into another during propagation. 3. A smaller time step represents less elapsed time per transition; continuous-time parameterization/discretization can adjust accordingly, while a fixed discrete A changes physical meaning.

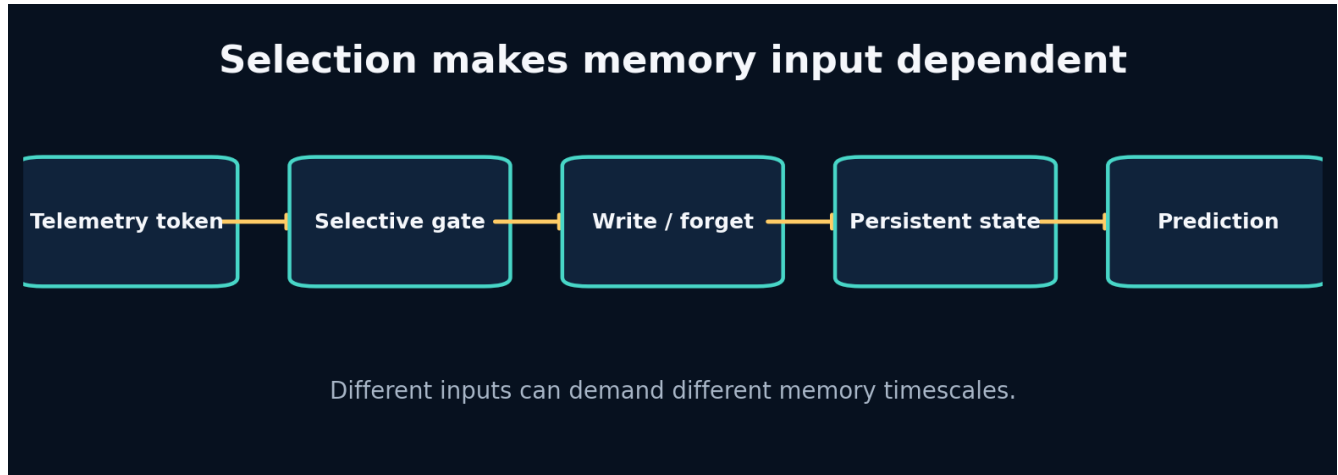
Independent build challenge

Construct a three-mode SSM with fast, medium and slow decay. Feed braking pulses of different duration and show which hidden mode retains each timescale.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

13. Selective State-Space Models and What Mamba Changes

Instructor objective: Build an input-dependent memory cell, understand selection, and separate the core Mamba idea from implementation hype.



13. Selective State-Space Models and What Mamba Changes

The problem that earns this chapter

A fixed SSM remembers every input according to the same transition. Racing telemetry is regime dependent: a yellow flag, braking spike or rain change may deserve a strong memory write, while repetitive straight-line frames may be compressed or forgotten.

Predict before reading

Imagine a memory of “recent heavy braking.” Should the same decay apply during ten ordinary straight-line frames and during a sudden lock-up indicator? How could the current input alter write and forget behaviour?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Selection means model parameters or gates depend on the current input. Instead of one fixed A and B, the sequence determines how strongly memory persists and what content is written. This gives content-aware recurrence while preserving a state-space perspective.

Mamba combines selective state-space dynamics with hardware-aware computation so long sequences can be processed efficiently. The important conceptual move for this curriculum is not “replace everything with Mamba.” It is: **make memory propagation conditional on the token when fixed dynamics are insufficient.**

A small selective cell can be written as $h_t = a(x_t) \odot h_{t-1} + (1 - a(x_t)) \odot b(x_t)$. It resembles a gated recurrent update. The difference between educational cell and research architecture must remain explicit.

Vocabulary that now has a job

Concept: Selection - Meaning in plain language: Current input changes how memory is written, retained or read. - **Role inside APEX:** Braking, weather or geometry can trigger different memory behaviour.

Concept: Input-dependent decay - Meaning in plain language: The retention factor is computed from the current token. - **Role inside APEX:** Adaptive timescales across race regimes.

Concept: Scan - Meaning in plain language: Efficiently apply a recurrence across sequence positions. - **Role inside APEX:** Important for long histories and GPU throughput.

Concept: Mamba block - Meaning in plain language: A complete architecture combining projections, selective SSM computation and implementation details. - **Role inside APEX:** A future challenger; not identical to the toy selective cell.

Worked example: calculate it by hand

Let old memory be $h=0.9$.

Ordinary input produces retention $a=0.95$ and candidate $b=0.2$: $h'=0.95(0.9)+0.05(0.2)=0.865$.

Critical braking input produces retention $a=0.2$ and candidate $b=-0.8$: $h'=0.2(0.9)+0.8(-0.8)=-0.46$.

The same memory dimension changes slowly during routine input and rapidly under a meaningful event. Selection is useful only if training learns gates that correspond to predictive needs rather than arbitrary noise.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/14_selective_ssm

What we are about to build

Implement a small PyTorch selective cell where decay and candidate write are functions of each input token. Feed basis vectors and inspect hidden updates.

Runnable implementation

```
import torch
from torch import nn

class SelectiveCell(nn.Module):
    def __init__(self,d):
        super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
    def forward(self,x,h):
        a=torch.sigmoid(self.decay(x))*0.99
        b=torch.tanh(self.write(x))
        return a*h+(1-a)*b
cell=SelectiveCell(3); h=torch.zeros(1,3)
for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))
```


Observed output from the packaged solution

```
[[[-0.114 -0.033 -0.16 ]]  
[[ 0.203  0.146 -0.244]]  
[[ 0.335 -0.015 -0.351]]
```

Read the important lines like English

****Code:** `a=torch.sigmoid(self.decay(x))*0.99` - What the line is doing: **Compute bounded input-dependent retention.** - What to inspect: ****** Multiplying by 0.99 prevents exact unit retention in this toy cell.

Code: `b=torch.tanh(self.write(x))` - **What the line is doing:** Compute candidate content to write. - **What to inspect:** Tanh bounds the candidate but can saturate.

Code: `return ah+(1-a)b` - **What the line is doing:** Interpolate between persistent memory and new content per dimension. - **What to inspect:** Inspect gate distributions across regimes.

State and tensor trace

```
input x_t  
├─> decay network → retention a_t ┐  
└─> write network → candidate b_t ┤  
                                └─> h_t  
previous h_{t-1} ─────────────────┘
```

Log a_t by feature regime. A gate that is always 0.5 is technically selective but may not be learning meaningful specialization.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove the sigmoid/bound and allow retention greater than one. Then initialize decay so gates saturate near zero or one.

Diagnose from the earliest failed contract

Track hidden norm, gate histograms, gradient norms and horizon errors. Explosion, frozen memory and always-overwritten memory have different signatures.

Repair and lock the repair with a test

Use stable parameterization, sensible initialization and regular diagnostic plots. Add tests for finite bounded-state behaviour under bounded inputs, but do not force gates to a preferred distribution without evidence.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Toy selective cell - **Choose it when:** Learning the mechanism and testing whether input-dependent memory helps. - **Do not choose it when:** Claiming Mamba-equivalent speed or quality.

Implementation: Official Mamba implementation - **Choose it when:** Long sequences and benchmarks justify the dependency and hardware path. - **Do not choose it when:** Your data are small, horizons short, or environment support is fragile.

Implementation: GRU - Choose it when: A stable gated recurrent baseline is sufficient. - **Do not choose it when:** Measured long-context/throughput limitations motivate another architecture.

Implementation: Transformer - Choose it when: Global content interactions and parallel token processing dominate. - **Do not choose it when:** Quadratic attention cost is unjustified and recurrent deployment state is useful.

APEX V1 retains a GRU primary and an SSM-style challenger. Fine-tuning or replacing with a full Mamba model should begin only after sequence length, data scale and latency measurements establish the need.

Transfer the lesson into Project APEX

Compare GRU and SSM models with identical windows, splits, normalization, hidden size budget and evaluation. Add gate plots to the SSM run artifacts.

Repository path to inspect

```
labs/14_selective_ssm/solution.py
projects/07_selective_ssm/main.py
apex_engine/src/apexsim/models/ssm_world_model.py
apex_engine/configs/ssm_fast.yaml
```

Connection to research

The Mamba paper motivates selection because content-dependent reasoning is a weakness of fixed linear time-invariant SSMs. Its efficiency claims depend on the complete algorithm and hardware-aware implementation, not merely using a gate.

Check your understanding before continuing

What makes a state update selective?

Why is bounded retention helpful but not sufficient?

What evidence would justify a full Mamba dependency in APEX?

Solutions and reasoning

1. The current input changes parameters controlling memory propagation, writing or reading. 2. It reduces explosion risk, but the model can still forget too quickly, ignore inputs or learn useless gates. 3. Long histories where GRU/Transformer baselines fail or are too slow, sufficient data, reproducible gains by horizon, and deployment hardware that benefits from the implementation.

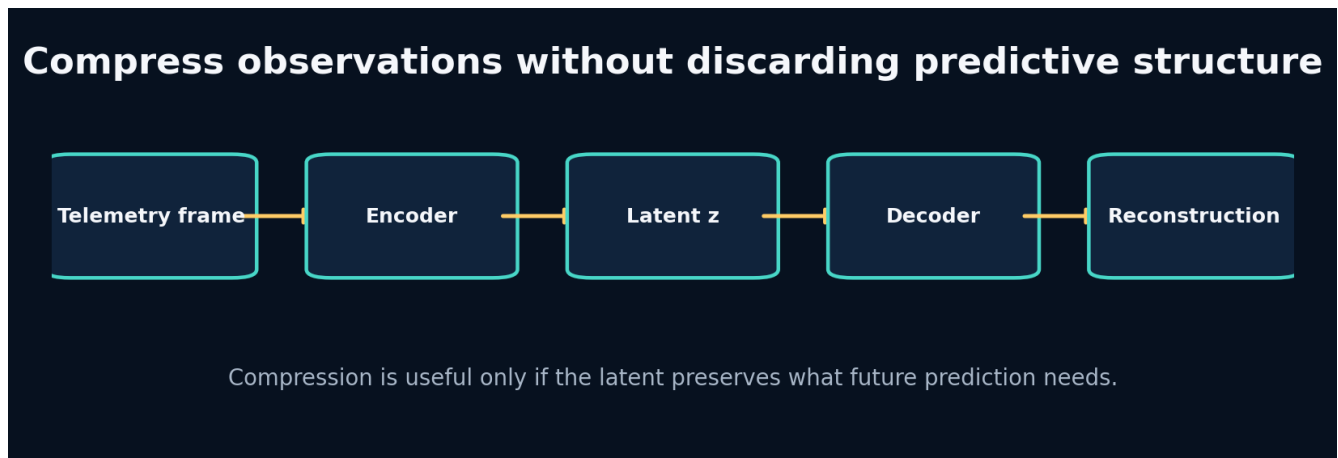
Independent build challenge

Create a dataset with rare event tokens that determine a target many steps later. Compare fixed linear SSM, selective cell and GRU while plotting learned retention around events.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

14. Autoencoders and Variational Latent State

Instructor objective: Learn representation compression by building it, then add probabilistic latent variables without confusing reconstruction with useful world modelling.



14. Autoencoders and Variational Latent State

The problem that earns this chapter

Raw observations can contain redundant or noisy dimensions. A world model may benefit from a compact latent state, but compression can discard precisely the small variable needed for future prediction. A good reconstruction is not automatically a good predictive representation.

Predict before reading

A telemetry vector has 12 features, but only speed, curvature and tyre age determine the next state. If an autoencoder compresses to three dimensions, will those three necessarily correspond to the three causal variables? Why or why not?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

An autoencoder learns an encoder $z=f(x)$ and decoder $\hat{x}=g(z)$ by minimizing reconstruction error. The bottleneck pressures the latent to preserve common information. Nothing requires each dimension to align with a human variable, and high-variance nuisance features may dominate.

A variational autoencoder predicts a distribution—usually mean and log variance—rather than one code. Sampling uses $z = \mu + \sigma \odot \epsilon$, which keeps the path differentiable with respect to μ and σ . A KL term regularizes the posterior toward a prior, enabling sampling but creating a tradeoff: too weak and the latent is irregular; too strong and it can ignore observations.

For world models, the latent should preserve information useful for dynamics and reward, not only instantaneous reconstruction. Temporal prediction losses or task probes test this.

Vocabulary that now has a job

Concept: Encoder - Meaning in plain language: Maps a high-dimensional observation into a compact representation. **- Role inside APEX:** Compresses telemetry/history into latent state.

Concept: Decoder - Meaning in plain language: Reconstructs or predicts observable state from latent representation. - **Role inside APEX:** Produces interpretable future telemetry.

Concept: Reparameterization - Meaning in plain language: Express stochastic sampling as differentiable transformation of parameter-free noise. - **Role inside APEX:** Allows RSSM/VAE posterior training.

Concept: KL divergence - Meaning in plain language: Penalty measuring difference between posterior and prior distributions. - **Role inside APEX:** Regularizes stochastic latent state and aligns imagination with inference.

Worked example: calculate it by hand

For one latent dimension with $\mu=0.2$, log variance=-1:

Variance: $\exp(-1)=0.3679$.

Standard deviation: $\exp(-0.5)=0.6065$.

If sampled noise $\epsilon=1.0$, then $z=0.2+0.6065=0.8065$.

KL to standard normal is $-0.5(1 + \log\text{var} - \mu^2 - \exp(\log\text{var}))$.

Substitute: $-0.5(1-1-0.04-0.3679)=0.20395$.

The KL is zero only when posterior mean is zero and variance is one. Driving every posterior to that point would erase observation information.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/16_vae_reparameterization

What we are about to build

Compute posterior standard deviation, sample with reparameterization, and calculate KL for a tiny batch. Pair it with the autoencoder lab to compare deterministic and stochastic codes.

Runnable implementation

```
import torch
```

```
mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[1.0,0.5]])
std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
print("mu",mu,"std",std,"sample",z,"KL",kl)
```

Observed output from the packaged solution

```
mu tensor([[ 0.2000, -0.4000]]) std tensor([[0.6065, 1.2840]]) sample tensor([[ -0.1015,  0.6276]]) KL
tensor([0.3583])
```

Read the important lines like English

****Code:** `std=torch.exp(0.5*logvar)` - What the line is doing: **Convert log variance into standard deviation.** - What to inspect:****** The factor 0.5 appears because std is square root of variance.

Code: `eps=torch.randn_like(std)` - **What the line is doing:** Sample parameter-free standard normal noise. - **What to inspect:** Random seed affects the sampled latent, not posterior parameters.

****Code:** `z=mu+std*eps` - **What the line is doing:** **Create a stochastic sample with gradients flowing through μ and std .** - **What to inspect:** ****** Inspect multiple samples to see uncertainty.

Code: `kl=...` - **What the line is doing:** Measure posterior departure from the unit Gaussian prior. - **What to inspect:** Monitor KL by dimension for collapse or overuse.

State and tensor trace

```

observation x
  ↓ encoder
 $\mu(x), \log\sigma^2(x)$ 
  ↓ sample  $\epsilon \sim N(0, I)$ 
 $z = \mu + \sigma \epsilon$ 
  ↓ decoder
reconstruction  $\hat{x}$ 

```

`loss = reconstruction error +  $\beta \times \text{KL}$`

Print latent means, standard deviations and KL—not only total loss.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Set KL weight extremely high and inspect whether reconstructions become generic. Then set it to zero and sample from the prior at inference.

Diagnose from the earliest failed contract

High KL with poor reconstruction suggests excessive regularization. Near-zero KL in every dimension plus decoder independence from z suggests posterior collapse. Good reconstruction with nonsensical prior samples suggests an unstructured latent.

Repair and lock the repair with a test

Use KL warm-up/free-nats or capacity control only after measuring the failure. Add latent probes for future speed, curvature and tyre state, and compare reconstruction versus predictive usefulness.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Deterministic autoencoder - **Choose it when:** Compression and reconstruction matter; uncertainty is not required. - **Do not choose it when:** You need coherent sampling or probabilistic belief.

Implementation: VAE - **Choose it when:** A smooth sampleable latent distribution is useful. - **Do not choose it when:** KL tradeoffs add complexity without serving the downstream task.

Implementation: Predictive encoder - **Choose it when:** Future-relevant structure matters more than reconstructing every input detail. - **Do not choose it when:** You still require high-fidelity observation generation.

Implementation: No bottleneck - Choose it when: State is already small and semantically meaningful.
- Do not choose it when: High-dimensional noisy observations overwhelm dynamics learning.

APEX telemetry is already compact, so V1 can predict state directly. The RSSM uses stochastic latent state to study uncertainty and imagination, not because compression is automatically necessary.

Transfer the lesson into Project APEX

Run autoencoder and VAE labs, then inspect RSSM posterior statistics. Create a probe predicting future speed from latent state and compare it with reconstruction loss.

Repository path to inspect

```
labs/15_autoencoder/solution.py
labs/16_vae_reparameterization/solution.py
apex_engine/src/apexsim/models/rssm.py
debugging_cases/11_kl_collapse
```

Connection to research

JEPA-style approaches challenge the need to reconstruct raw observations, arguing that useful representations can be learned by predicting abstract target embeddings. This is especially relevant when raw detail is unpredictable or irrelevant.

Check your understanding before continuing

Why can low reconstruction error coexist with poor future prediction?

What is the purpose of the KL term?

Why does reparameterization enable gradients?

Solutions and reasoning

1. The encoder may preserve visually/statistically dominant current detail while discarding small causal variables needed later. 2. It regularizes the posterior toward the prior so latent space can support coherent prior sampling/imagination. 3. The random variable is expressed as a deterministic differentiable function of parameters and independent noise, so gradients pass through μ and σ .

Independent build challenge

Train an autoencoder on synthetic telemetry with nuisance noise. Compare latent probes for future speed under reconstruction-only and reconstruction-plus-future-prediction objectives.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

15. RSSMs and Dreamer: Learning to Imagine Without Observations

Instructor objective: Implement the logic of a recurrent state-space model, distinguish posterior training from prior imagination, and understand Dreamer as a system rather than a buzzword.

RSSM separates deterministic memory from stochastic uncertainty



Training uses posterior evidence; imagination must survive on the prior.

15. RSSMs and Dreamer: Learning to Imagine Without Observations

The problem that earns this chapter

A deterministic recurrent model predicts one future. Real driving contains unobserved factors, noisy evidence and multiple plausible outcomes. An RSSM maintains deterministic memory plus a stochastic latent belief. The hardest requirement is that imagination must continue when future observations are absent.

Predict before reading

During training you know the next observation; during planning you do not. Which distribution may use the observation: the prior, posterior, both or neither? What would happen if imagination secretly used posterior information?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

An RSSM commonly has deterministic state h_t and stochastic state z_t . The transition updates h_t from previous latent/action. The prior predicts a distribution over z_t from h_t . During training, an encoder provides observation evidence and a posterior refines that distribution. The decoder predicts observation/state from h_t, z_t .

The KL term teaches the prior to resemble the posterior enough that prior-only imagination remains useful. Reconstruction or prediction terms teach the latent to represent evidence. These objectives can conflict. Posterior collapse, overconfident priors and compounding uncertainty are practical failures, not footnotes.

Dreamer adds reward/continuation models and learns actor/critic behaviour from trajectories imagined by the world model. Before trusting control, APEX must establish that interventions produce realistic, calibrated rollouts.

Vocabulary that now has a job

Concept: Deterministic state h - Meaning in plain language: Recurrent memory summarizing past latent/action sequence. - **Role inside APEX:** Carries stable temporal context.

Concept: Prior - Meaning in plain language: Latent distribution predicted without current observation. - **Role inside APEX:** Used during imagination and planning.

Concept: Posterior - Meaning in plain language: Latent distribution conditioned on current observation evidence. - **Role inside APEX:** Used during training/state inference.

Concept: Imagination - Meaning in plain language: Rollout using learned prior dynamics without future observations. - **Role inside APEX:** Generates counterfactual telemetry trajectories.

Worked example: calculate it by hand

Suppose posterior is $N(\mu_q=1, \sigma_q=0.5)$ and prior is $N(\mu_p=0, \sigma_p=1)$. The one-dimensional Gaussian KL $KL(q || p)$ is

$$\log(\sigma_p/\sigma_q) + \frac{\sigma_q^2 + (\mu_q - \mu_p)^2}{2\sigma_p^2} - \frac{1}{2}.$$

Substitute:

$\log(1/0.5)=0.693$

numerator $0.25 + 1 = 1.25$

divide by 2 gives 0.625

subtract 0.5

total 0.818

The gap signals that the prior cannot yet reproduce the posterior belief. Reducing it by making both distributions uninformative would also be bad; reconstruction/prediction must remain strong.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/17_rssm_step

What we are about to build

Construct one RSSM step: update deterministic memory, compute prior statistics, then compute posterior statistics using observation embedding. Inspect all shapes.

Runnable implementation

```
import torch
from torch import nn

cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
h=cell(torch.cat([prev_z,action],-1),h)
prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

Observed output from the packaged solution

h torch.Size([2, 8]) prior torch.Size([2, 4]) posterior torch.Size([2, 4])

Read the important lines like English

Code: `h=cell(torch.cat([prev_z,action],-1),h)` - **What the line is doing:** Advance deterministic memory using previous latent and current action. - **What to inspect:** Current observation is intentionally absent from this transition.

Code: `prior_stats=prior(h)` - **What the line is doing:** Predict latent distribution using only imagined memory. - **What to inspect:** This path must work at inference with no future observation.

Code: `post_stats=posterior(torch.cat([h,obs_embed],-1))` - **What the line is doing:** Refine latent belief using current evidence during training. - **What to inspect:** Posterior information must not leak into prior-only evaluation.

State and tensor trace

training step:

```
prev z + action → h_t → prior p(z_t|h_t)
                + observation embedding → posterior q(z_t|h_t,o_t)
posterior sample + h_t → decode target
```

imagination step:

prev z + action $\rightarrow$ h_t $\rightarrow$ prior sample $\rightarrow$ decode prediction
(no observation branch)

Evaluate both posterior reconstruction and prior rollout. They answer different questions.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Use posterior samples during test rollout. Then set KL weight to zero and compare prior imagination with posterior reconstruction.

Diagnose from the earliest failed contract

If posterior metrics are good but prior rollout is poor, inspect prior-posterior KL by horizon and latent dimension. If KL is near zero but outputs ignore observations, inspect collapse.

Repair and lock the repair with a test

Separate evaluation functions for posterior reconstruction and prior imagination. Add a test that imagination accepts no future observation tensor. Tune KL with explicit diagnostics, not only total loss.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Deterministic GRU world model - **Choose it when:** One likely future and stable V1 forecasting are sufficient. - **Do not choose it when:** Uncertainty/multi-modal futures are central.

Implementation: RSSM - Choose it when: You need a learned belief state and prior imagination. - **Do not choose it when:** Data/budget cannot support stable latent training or benefits are unmeasured.

Implementation: Ensemble deterministic models - Choose it when: You want practical epistemic uncertainty with simple training. - **Do not choose it when:** Compute or storage prevents multiple models.

Implementation: Full Dreamer actor-critic - Choose it when: Dynamics, reward and continuation models are validated and control is the target. - **Do not choose it when:** The world model can still be exploited or interventions are poorly represented.

APEX includes a compact RSSM challenger and preserves its weaker short-budget result as a debugging lesson. The deterministic GRU remains V1 primary until stochastic imagination demonstrates decision value and calibration.

Transfer the lesson into Project APEX

Train GRU and RSSM under the same session split. Compare posterior reconstruction, prior horizon error, KL, physical violations and scenario sensitivity—not just total training loss.

Repository path to inspect

```
projects/08_rssm_imagination/main.py
apex_engine/src/apexsim/models/rssm.py
apex_engine/configs/rssm_fast.yaml
debugging_cases/11_kl_collapse
```

Connection to research

DreamerV3 is an integrated agent: world model, imagined trajectories, reward/continuation prediction and actor-critic learning. Copying an RSSM cell is not equivalent to reproducing Dreamer’s training system or robustness.

Check your understanding before continuing

Why must the prior exclude current observation evidence?

What does good posterior reconstruction but poor prior rollout mean?

Why might a deterministic model outperform an RSSM on APEX V1?

Solutions and reasoning

1. The future observation is unavailable during imagination; including it leaks the answer and invalidates the simulator. 2. The latent encoder can explain observed frames, but learned dynamics cannot predict the corresponding latent beliefs without evidence. 3. Small data, short training, near-deterministic telemetry, KL optimization difficulty and evaluation focused on mean trajectory can all favour the simpler model.

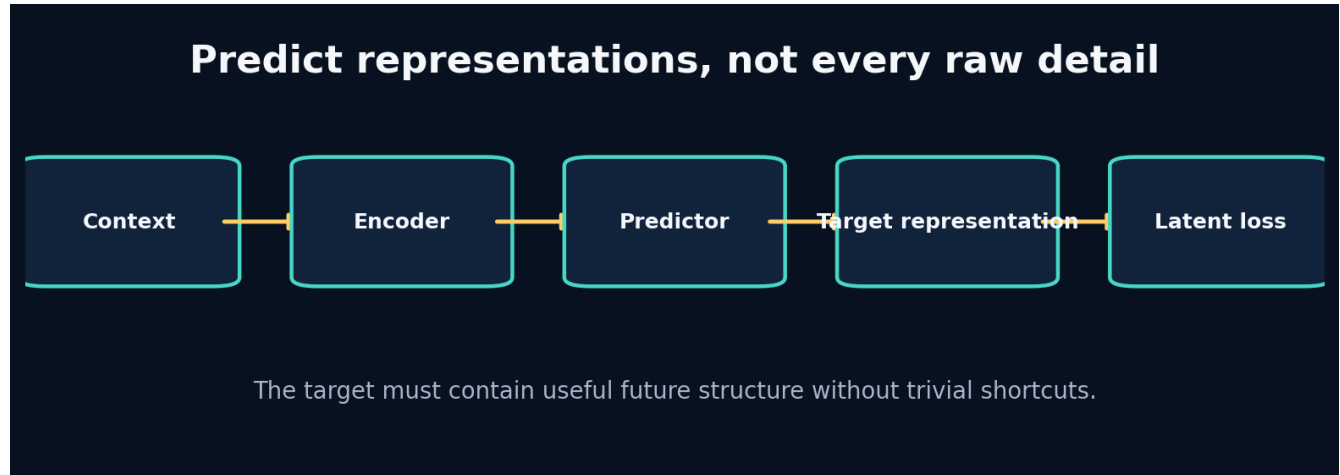
Independent build challenge

Implement a two-dimensional RSSM on a stochastic toy car. Plot posterior and prior distributions over time, then show a case where uncertainty widens under an unseen control sequence.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

16. JEPA, LeWorldModel and Fine-Tuning Existing Representations

Instructor objective: Understand predictive representation learning, decide when not to reconstruct raw observations, and build a disciplined fine-tuning strategy.



16. JEPA, LeWorldModel and Fine-Tuning Existing Representations

The problem that earns this chapter

Pixel or telemetry reconstruction forces a model to spend capacity on every observable detail, including noise that may not matter for future decisions. JEPA-style methods predict target representations instead. LeWorldModel-style work explores efficient latent world modelling and planning. The engineering question is not which acronym is newest; it is what target contains the information APEX needs.

Predict before reading

Suppose two future telemetry frames differ only in an unpredictable sensor-noise feature. A raw reconstruction loss treats them as different. Should a predictive representation loss treat them as different? What determines the answer?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Joint-embedding predictive architectures encode context and target views, then train a predictor to infer the target representation from context. The target encoder defines what counts as meaningful similarity. Because the loss is in representation space, the model can ignore some raw variation—if the representation has learned to ignore it.

This creates a new failure possibility: representation collapse or shortcuts. Stop-gradient/target-encoder design, masking strategy and representation diagnostics matter. In telemetry, a future-latent target could emphasize dynamics while de-emphasizing sensor noise, but it might also discard rare safety-critical events.

Fine-tuning an existing model makes sense when its input semantics, sequence structure and learned invariances transfer. Adaptation choices range from a frozen encoder plus new head, through

parameter-efficient adapters, to full fine-tuning. Start with the least invasive option that can represent the domain shift, and compare against training a small model from scratch.

Vocabulary that now has a job

Concept: Representation target - Meaning in plain language: The encoded feature vector the predictor must infer. - **Role inside APEX:** Potential future-dynamics target instead of raw telemetry.

Concept: Stop gradient / target encoder - Meaning in plain language: Prevents both sides from trivially co-adapting in unstable ways. - **Role inside APEX:** Part of a non-collapsing predictive objective.

Concept: Linear probe - Meaning in plain language: A simple supervised head used to test what information a representation contains. - **Role inside APEX:** Measures speed, curvature, tyre or event information in latent state.

Concept: Fine-tuning - Meaning in plain language: Adapt pretrained parameters to a new domain/task. - **Role inside APEX:** Possible when a relevant sequence/world model checkpoint exists.

Worked example: calculate it by hand

Compare two losses for target vector $y=[10.0, 0.02]$ where the second feature is pure noise. Prediction A is $[9.8, 0.50]$; prediction B is $[9.8, -0.40]$. Raw MSE differs because of noise.

If a target encoder maps both raw targets to a dynamics representation $[\text{speed_bin}=10, \text{regime}=\text{straight}]$, their representation loss can be identical. This is desirable only if the discarded noise truly has no predictive or decision value.

For fine-tuning, suppose a pretrained encoder has one million parameters. A frozen-head experiment trains 5,000 parameters; an adapter trains 50,000; full tuning changes all million. These are nested hypotheses about how much the representation must change.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/15_autoencoder

What we are about to build

Use the autoencoder lab as a concrete representation baseline, then modify its objective conceptually: predict a future latent code rather than reconstruct the current input. Add a linear probe for a known future variable.

Runnable implementation

```
import torch
from torch import nn

torch.manual_seed(3); x=torch.randn(256,12)
enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
for _ in range(80):
    opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward();
    opt.step()
print("latent",z.shape,"loss",float(loss))
```

Observed output from the packaged solution

```
latent torch.Size([256, 3]) loss 0.7505914568901062
/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/15_autoencoder/solution.py:8: UserWarning:
Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
print("latent",z.shape,"loss",float(loss))
```

Read the important lines like English

Code: `z=torch.tanh(enc(x))` - What the line is doing: Produce a learned representation through a bottleneck. - **What to inspect:** Inspect variance and pairwise similarity to detect collapse.

Code: `recon=dec(z)` - What the line is doing: The original lab asks the latent to reconstruct raw input. - **What to inspect:** Replace/augment this with future-target representation prediction.

Code: `loss=((recon-x)2).mean()` - What the line is doing:** Raw reconstruction values every feature according to scale and frequency. - **What to inspect:** Normalize or weight features, or choose a different target.

State and tensor trace

```
context history → context encoder → predictor → predicted target embedding
future target → target encoder → target embedding
                        ↓
                    latent-space loss
```

After training, freeze the encoder and probe whether latent state predicts physical variables and future events.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Train both context and target encoders with no asymmetry or variance checks until they output nearly constant vectors. Observe low latent loss and useless probes.

Diagnose from the earliest failed contract

Track per-dimension variance, embedding covariance, nearest-neighbour diversity and probe accuracy. Low loss with collapsed variance is not successful representation learning.

Repair and lock the repair with a test

Use an appropriate target/stop-gradient design, masking that prevents shortcuts, normalization, and diagnostic probes. Add rare-event retrieval tests so abstraction does not erase critical evidence.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Raw reconstruction - Choose it when: Exact observation generation or interpretable state recovery matters. - **Do not choose it when:** Unpredictable detail dominates and decisions depend on abstract dynamics.

Implementation: Predictive latent loss - Choose it when: Future-relevant representation is the main goal. - **Do not choose it when:** You cannot validate what information the latent discarded.

Implementation: Frozen pretrained encoder - Choose it when: Source and target semantics align and data are limited. - **Do not choose it when:** The representation lacks essential F1 variables.

Implementation: Full fine-tuning - Choose it when: You have sufficient data/compute and major domain adaptation is needed. - **Do not choose it when:** Small data risks catastrophic overfitting or a small scratch model is already competitive.

APEX V1 trains small models from scratch because the canonical telemetry state is compact and no assumed checkpoint perfectly matches its action/context semantics. The curriculum nevertheless prepares an adapter/probe-first fine-tuning experiment for future larger datasets.

Transfer the lesson into Project APEX

Add a latent probe suite to GRU, SSM and RSSM hidden states. If using a pretrained sequence encoder, freeze it first, train the same prediction head, and compare data efficiency and horizon curves.

Repository path to inspect

```
labs/15_autoencoder/solution.py
apex_engine/src/apexsim/models/
apex_engine/src/apexsim/evaluation.py
research_reading/
```

Connection to research

I-JEPA predicts representations of target image regions from context rather than reconstructing pixels. LeWorldModel explores latent world modelling and planning efficiency. Transfer their design principles only after mapping observation, action, target and evaluation semantics to the F1 domain.

Check your understanding before continuing

Why can latent loss be low under representation collapse?

What should a linear probe tell you?

What is the safest first fine-tuning experiment?

Solutions and reasoning

1. If both target and prediction are nearly constant, they match without containing information. 2. Whether a simple readout can recover a specified physical or predictive variable, revealing what information is accessible in the representation. 3. Freeze the pretrained backbone, train a small task head under the same split, and compare with simple baselines before unfreezing parameters.

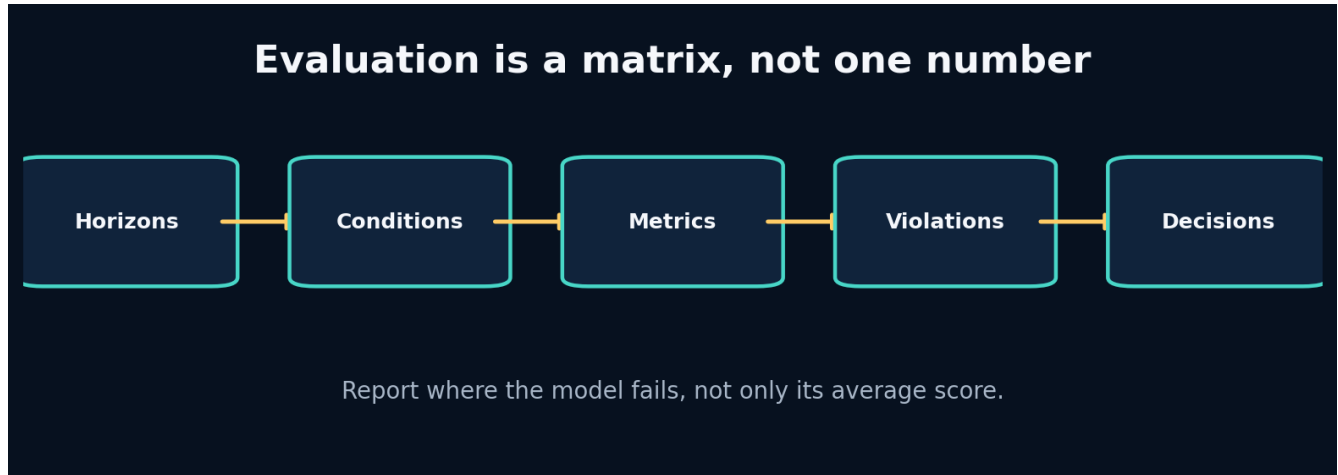
Independent build challenge

Build a future-latent predictor on synthetic telemetry. Evaluate reconstruction, future-state probes, rare-event retrieval and rollout value. Write a decision record comparing it with an RSSM objective.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

17. Evaluation, Ablation and Out-of-Distribution Testing

Instructor objective: Build an evaluation matrix that reveals where a world model works, why it works, and when it should refuse to be trusted.



17. Evaluation, Ablation and Out-of-Distribution Testing

The problem that earns this chapter

A single average MAE can hide catastrophic braking failures, worsening long-horizon error, impossible speeds, or performance limited to one track. A simulation engine needs evidence across horizons, features, conditions and interventions.

Predict before reading

Model A has lower average speed MAE than Model B, but A predicts negative speed in 2% of wet-braking rollouts while B never does. Which model is better? State the missing operational information required to decide.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Evaluation begins with the decision. If a race engineer uses the simulator to compare a six-second throttle scenario, horizon-six causal sensitivity and physical validity matter more than one-step average fit. Metrics are measurements of requirements, not a scoreboard detached from use.

Ablation removes or changes one information source while holding other factors fixed. It answers causal questions about the implementation: Does weather context help? Does VH-like second channel—here, perhaps curvature or tyre age—matter? Randomly changing many settings answers nothing cleanly.

Out-of-distribution tests define shifts deliberately: unseen track, heavier rain, longer tyre age, unusual controls, different sample rate. Failure under OOD is expected; the goal is to detect and quantify it, then decide whether to abstain, retrain or constrain use.

Vocabulary that now has a job

Concept: Horizon metric - Meaning in plain language: Error measured separately at each future step.

- **Role inside APEX:** Defines useful simulation duration.

Concept: Guardrail metric - Meaning in plain language: A metric that must remain within a safety/validity limit even if primary error improves. - **Role inside APEX:** Negative speed, progress bounds and control sensitivity.

Concept: Ablation - Meaning in plain language: A controlled removal or replacement testing one hypothesis. - **Role inside APEX:** State-only, no weather, no geometry, model-family comparisons.

Concept: OOD test - Meaning in plain language: Evaluation under a deliberately shifted distribution. -

Role inside APEX: Unseen circuits, weather or control regimes.

Worked example: calculate it by hand

Suppose Model A and B have:

Metric

A

B

1-step MAE

0.4

0.5

20-step MAE

8.0

5.0

Negative-speed rate

2%

0%

Scenario latency

30 ms

12 ms

If the UI serves 20-step scenarios, B is stronger despite worse one-step fit. If only one-step filtering is needed, A may be acceptable if violations are constrained. The “best model” is a vector of requirement tradeoffs plus thresholds, not one scalar.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/09_loss_comparison

What we are about to build

Compare loss functions on controlled predictions, then connect optimization loss to evaluation metrics. The loss trains parameters; the metric judges the product and need not be identical.

Runnable implementation

```
import torch
from torch.nn import functional as F

pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
print("MSE",F.mse_loss(pred,target).item())
print("MAE",F.l1_loss(pred,target).item())
print("Huber",F.huber_loss(pred,target).item())
```

Observed output from the packaged solution

MSE 72.25
MAE 4.25
Huber 4.125

Read the important lines like English

Code: mse = ... - **What the line is doing:** Penalize squared residuals, emphasizing large errors. - **What to inspect:** Sensitive to scale and outliers.

Code: mae = ... - **What the line is doing:** Measure average absolute physical error. - **What to inspect:** More robust but less smooth at zero.

Code: compare losses - **What the line is doing:** Reveal how optimization priorities differ on the same predictions. - **What to inspect:** Do not select by training loss alone.

State and tensor trace

```
run artifact
├─ overall metrics
├─ horizon curves
├─ per-feature errors
├─ subgroup tables
├─ physical violations
├─ intervention sensitivity
└─ baseline-relative improvement
```

Every plot should include sample count and split identity.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Average all horizons and all conditions into one MAE. Then remove weather from inputs but accidentally change random seed and training epochs too.

Diagnose from the earliest failed contract

Ask whether compared runs share dataset hash, split, normalizer, seed set, budget and evaluation code. If multiple variables changed, the result is not an ablation.

Repair and lock the repair with a test

Store experiment manifests and compare matched runs. Add confidence intervals across seeds/sessions where feasible. Make promotion gates explicit: primary metric, guardrails, latency and artifact completeness.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: MAE/RMSE - Choose it when: Continuous state error in interpretable units. - **Do not choose it when:** They are the only evidence for a simulator.

Implementation: Horizon curves - Choose it when: Autoregressive use or planning matters. - **Do not choose it when:** Never; at least inspect them for rollouts.

Implementation: Physical penalties - Choose it when: Known invalid regions can be expressed and monitored. - **Do not choose it when:** They replace empirical outcome evaluation.

Implementation: Human scenario review - Choose it when: Qualitative causal behaviour and UI trust matter. - **Do not choose it when:** It substitutes for reproducible quantitative tests.

APEX reports baseline-relative error by horizon, per-feature metrics, physical violations, scenario sensitivity and run metadata. Model promotion is a documented decision, not automatic best-MAE selection.

Transfer the lesson into Project APEX

Run the GRU, SSM and RSSM references. Build one comparison report with matched data and budgets, then explain why the simplest winning model remains primary.

Repository path to inspect

```
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/artifacts/runs/
field_workbook/
```

Connection to research

World-model papers choose benchmarks and metrics that reflect their claims. When adapting a paper to F1, reproduce the mechanism but redefine evaluation around telemetry dynamics, interventions, uncertainty and deployment conditions.

Check your understanding before continuing

What makes an experiment a true ablation?

Why should error be normalized as well as reported in physical units?

What is an abstention policy?

Solutions and reasoning

1. One targeted component changes while data, split, budget, evaluation and other variables remain fixed. 2. Physical units are interpretable; normalized error helps compare features with different scales. Both are useful. 3. A rule that withholds or flags predictions when evidence is outside trusted conditions or uncertainty/violations exceed thresholds.

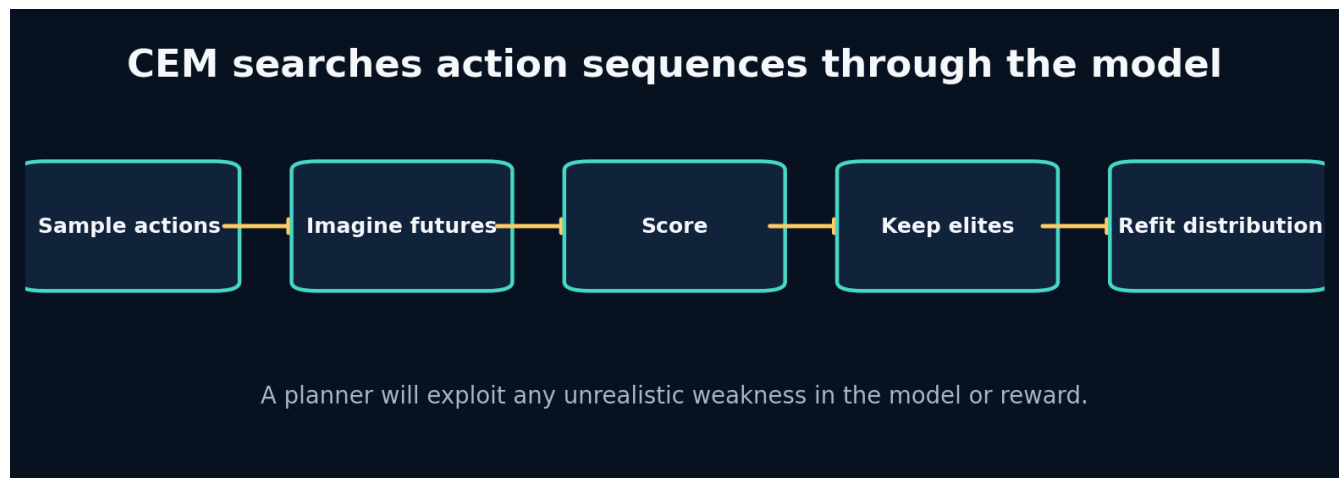
Independent build challenge

Design a complete APEX evaluation matrix across three horizons, three tracks, wet/dry conditions and five metrics. Define promotion and abstention criteria before seeing results.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

18. Planning With MPC and the Cross-Entropy Method

Instructor objective: Use a world model to compare action sequences, understand CEM search, and learn why planners exploit model and reward defects.



18. Planning With MPC and the Cross-Entropy Method

The problem that earns this chapter

Prediction answers “what happens if we do this?” Planning asks “what should we do?” A planner proposes action sequences, imagines outcomes through the world model, scores them, and improves its proposal. This creates a powerful adversary: it actively searches for unrealistic trajectories that maximize the score.

Predict before reading

A reward gives +1 for speed and no penalty for leaving the track or braking instability. What action sequence will a planner prefer? Why is this not primarily a planner bug?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Model predictive control repeatedly plans over a finite horizon, executes only the first action, observes the new state, and replans. This feedback limits drift compared with committing to a long open-loop sequence.

The cross-entropy method maintains a distribution over action sequences. It samples candidates, rolls them through the model, keeps an elite fraction, and refits mean/variance toward those elites. Repetition concentrates search around high-scoring actions.

Planner quality is bounded by world-model validity and reward completeness. Optimization pressure finds loopholes humans do not notice in average evaluation. Therefore planning tests should include adversarial actions, uncertainty penalties, constraints and real-environment verification before control is trusted.

Vocabulary that now has a job

Concept: Model predictive control - Meaning in plain language: Plan over a horizon, execute a short prefix, observe, and replan. - **Role inside APEX:** Future scenario/strategy layer after dynamics validation.

Concept: CEM - Meaning in plain language: Sampling optimizer that iteratively refits an action distribution to elite candidates. - **Role inside APEX:** Simple derivative-free planner through imagined telemetry.

Concept: Reward model - Meaning in plain language: Function scoring predicted trajectories for the decision objective. - **Role inside APEX:** Could combine progress, stability, tyre use and constraint penalties.

Concept: Model exploitation - Meaning in plain language: Planner finds actions that score well because the learned model is wrong. - **Role inside APEX:** A major risk under unusual controls.

Worked example: calculate it by hand

Suppose CEM samples 500 six-step throttle sequences. It scores final-speed error to a target plus smoothness. It keeps 50 elites (top 10%). New mean and standard deviation are the elite sample statistics.

If initial mean is 0.5 and standard deviation 0.3, one iteration may shift the early-step means toward 0.8 where acceleration helps. After several iterations variance shrinks. Too-rapid variance collapse can trap search; a minimum standard deviation preserves exploration.

MPC then applies only the first action, receives new evidence and replans. The remaining five actions are proposals, not a fixed command commitment.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/18_cem_planning

What we are about to build

Run CEM against a tiny differentiable-free speed simulator. Watch mean action sequence and best score improve over five iterations.

Runnable implementation

import numpy **as** np

```
rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
for it in range(5):
    actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
    speed=40+np.cumsum(2*actions-0.4,axis=1)
    score=-(speed[:,1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
    elite=actions[np.argsort(score)[-50:]]
    mean,std=elite.mean(0),elite.std(0)+1e-3
    print(it,mean.round(2),score.max().round(3))
```

Observed output from the packaged solution

```
0 [0.71 0.68 0.7 0.62 0.67 0.71] -1.123
1 [0.84 0.8 0.82 0.82 0.83 0.82] -0.019
2 [0.86 0.84 0.86 0.88 0.87 0.9 ] -0.005
3 [0.87 0.85 0.85 0.87 0.89 0.87] -0.001
4 [0.86 0.84 0.85 0.88 0.89 0.87] -0.001
```

Read the important lines like English

Code: `actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)` - **What the line is doing:** Sample bounded candidate action sequences from the current proposal. - **What to inspect:** Clipping changes the distribution near limits.

Code: `score=...` - **What the line is doing:** Evaluate imagined outcome and smoothness under the toy world model. - **What to inspect:** Every omitted constraint becomes a possible exploit.

Code: `elite=actions[np.argsort(score)[-50:]]` - **What the line is doing:** Select the highest-scoring candidate sequences. - **What to inspect:** Elite fraction controls search pressure.

Code: `mean,std=elite.mean(0),elite.std(0)+1e-3` - **What the line is doing:** Refit proposal distribution and preserve minimum exploration. - **What to inspect:** Monitor premature variance collapse.

State and tensor trace

proposal distribution over action sequences

↓ sample N candidates

world model rollout for each candidate

↓ reward + constraints

select top K elites

↓ fit new mean/std

repeat → execute first action → observe → replan

Log the best candidate, elite diversity, uncertainty and physical violations each iteration.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove the smoothness term and allow throttle beyond one. Add a model region where extreme throttle incorrectly increases speed without penalty. Watch CEM discover it.

Diagnose from the earliest failed contract

Replay planned trajectories under a trusted simulator or held-out real transitions. Compare planner actions with the training action distribution. Large OOD actions and low predicted uncertainty are red flags.

Repair and lock the repair with a test

Enforce hard action bounds, add physically justified constraints, penalize uncertainty/OOD distance, and use receding-horizon verification. Create adversarial planner tests as part of model evaluation.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Grid search - Choose it when: Action space and horizon are tiny. - **Do not choose it when:** Combinatorial sequences.

Implementation: CEM - Choose it when: Continuous bounded actions, no gradients required and batch rollout is cheap. - **Do not choose it when:** Model evaluations are extremely expensive or multi-modal search collapses.

Implementation: Gradient planning - Choose it when: World model/reward are differentiable and smooth. - **Do not choose it when:** Discrete actions, poor local optima or unstable gradients.

Implementation: Actor network - Choose it when: Fast repeated decisions justify amortizing planning through policy learning. - **Do not choose it when:** World model/reward is not validated or policy can exploit it unseen.

APEX V1 exposes manual counterfactual controls and includes a latent MPC project, but does not claim autonomous race control. Planning becomes a later stage after dynamics and reward validation.

Transfer the lesson into Project APEX

Connect CEM to the scenario rollout interface using bounded throttle/brake actions. Evaluate planned sequences against the synthetic ground-truth environment before any live integration.

Repository path to inspect

```
labs/18_cem_planning/solution.py
projects/09_latent_mpc/main.py
apex_engine/src/apexsim/simulation.py
debugging_cases/12_planner_exploitation
```

Connection to research

Dreamer replaces repeated online search with an actor and critic trained from imagined trajectories, but the exploitation problem remains. Better optimization can make model defects more dangerous, not less.

Check your understanding before continuing

Why does MPC execute only the first planned action?

What does elite variance tell you?

How can uncertainty be used in planning?

Solutions and reasoning

1. New observations correct model drift and changing conditions before the next action. 2. How concentrated high-scoring candidates are; rapid collapse can indicate convergence or premature loss of exploration. 3. Penalize uncertain trajectories, constrain planning to trusted regions, or trigger abstention/human review.

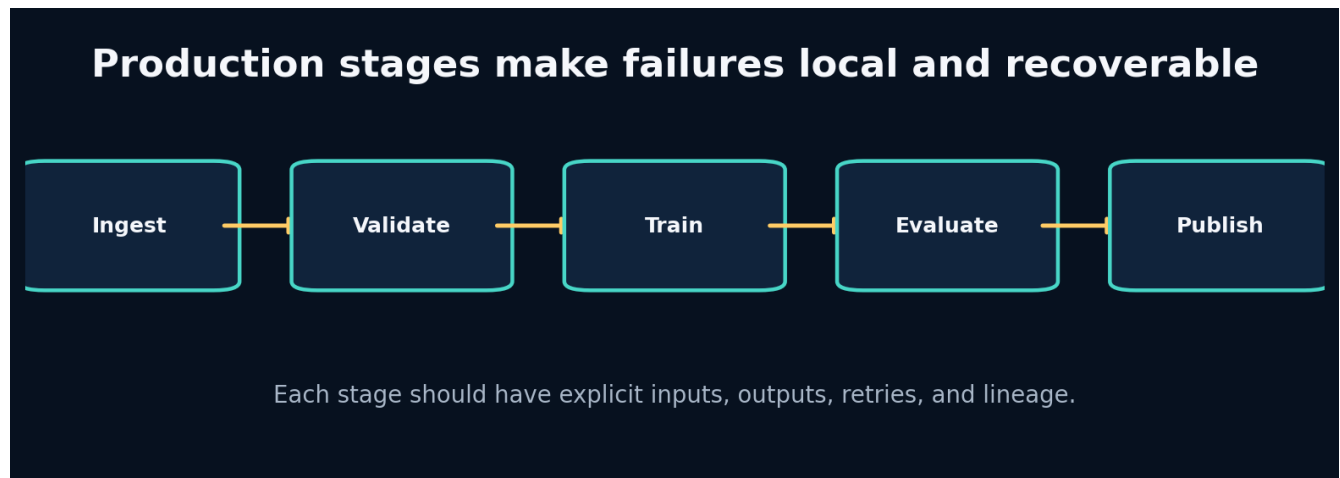
Independent build challenge

Add braking and curvature constraints to the CEM lab. Create a hidden model defect, show the planner exploit it, then design an evaluation that catches the exploit before deployment.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

19. Production Pipelines, Airflow, Registry and Monitoring

Instructor objective: Refactor experimental code into idempotent stages with observable lineage, then understand when orchestration becomes necessary.



19. Production Pipelines, Airflow, Registry and Monitoring

The problem that earns this chapter

A notebook can ingest, train and plot once. A production system must retry failed downloads, validate versions, reproduce models, backfill sessions, avoid overwriting artifacts, and tell operators which stage failed. Orchestration should coordinate trustworthy functions, not rescue tangled code.

Predict before reading

A pipeline fails after training but before publication. On retry, should it retrain from scratch, reuse the checkpoint, or overwrite the run directory? What information is needed to choose safely?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A stage has explicit inputs, outputs, configuration, failure modes and idempotency policy. Ingestion writes canonical data; validation writes a report; training writes a checkpoint; evaluation writes metrics; publication writes UI/API products. Each artifact is immutable within a run and linked by identifiers/hashes.

Airflow schedules and tracks task dependencies, retries and backfills. Business logic should remain in ordinary tested Python functions so the same stage runs locally, in tests and under orchestration. Passing large arrays through orchestration metadata is a smell; pass durable artifact references.

A registry records runs, configurations, dataset versions, metrics and status. Monitoring then covers pipeline health, data drift, model behaviour and product delivery. Logging without stable run IDs and structured fields is not lineage.

Vocabulary that now has a job

Concept: Idempotency - Meaning in plain language: Repeating a stage with the same inputs yields the same result or safely reuses it. - **Role inside APEX:** Enables retries and backfills.

Concept: Artifact lineage - Meaning in plain language: Trace which data, code and configuration produced an output. - **Role inside APEX:** Every UI prediction links to model run and dataset.

Concept: Orchestrator - Meaning in plain language: Coordinates when stages run and how failures/retries are managed. - **Role inside APEX:** Airflow DAG wraps tested stage functions.

Concept: Registry - Meaning in plain language: Queryable record of runs, statuses, metrics and artifact locations. - **Role inside APEX:** SQLite V1, expandable to a service/database.

Worked example: calculate it by hand

Assume training checkpoint hash abc123 exists and evaluation failed because disk was temporarily full.

A safe retry checks:

Same run ID and immutable configuration hash.

Same dataset/normalizer hashes.

Checkpoint exists, is complete and matches metadata.

Training stage status is succeeded.

Evaluation output is absent or marked failed.

Then retry evaluation only. If configuration changed, create a new run rather than silently reusing the checkpoint. If checkpoint integrity is uncertain, retrain in a new or repaired stage path.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

Use the miniature stage artifact writer to learn explicit stage boundaries, then trace the production APEX runner and Airflow DAG that call the same business functions.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `run.mkdir(parents=True,exist_ok=True)` - **What the line is doing:** Create a stable run namespace for artifacts. - **What to inspect:** A production run directory should not mix different configuration hashes.

Code: `path.write_text(json.dumps(...))` - **What the line is doing:** Persist machine-readable stage evidence. - **What to inspect:** Use atomic temporary-file replacement for critical artifacts.

Code: `stages=[...]` - **What the line is doing:** Make dependencies visible and independently testable. - **What to inspect:** Do not create one giant “do everything” task.

State and tensor trace

CLI / Airflow / test

↓

ordinary Python stage function

↓

durable artifact + metadata

↓

registry status transition

↓

next stage receives artifact path

A retry should start from the last valid durable boundary.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Write every run to latest/, pass a large tensor through Airflow XCom, and combine ingestion/training/publication in one task. Simulate a publication failure.

Diagnose from the earliest failed contract

Look for overwritten evidence, ambiguous status and work that must repeat unnecessarily. The earliest failed design is usually the artifact/stage contract, not the scheduler.

Repair and lock the repair with a test

Use immutable run IDs, atomic writes, explicit stage manifests and small task messages containing paths/IDs. Add pipeline tests that run twice and assert safe reuse or deterministic replacement.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Local Python runner - Choose it when: Development, CI and small reproducible workflows. - **Do not choose it when:** Complex schedules, backfills and multi-worker operations.

Implementation: Airflow - Choose it when: Batch dependencies, retries, schedules and historical backfills matter. - **Do not choose it when:** You need low-latency event streaming or have only one trivial script.

Implementation: Experiment tracker - Choose it when: Comparing many model runs and artifacts. - **Do not choose it when:** You assume it replaces data/version contracts and orchestration.

Implementation: Model registry - Choose it when: Promotion, rollback and serving require a governed model identity. - **Do not choose it when:** A notebook checkpoint with no deployment lifecycle.

APEX implements a local runner as the source of truth, a thin Airflow DAG, a run registry and immutable run artifacts. The full Airflow service is an optional deployment layer, not required for learning core models.

Transfer the lesson into Project APEX

Run the same configuration twice under different IDs. Compare manifests, then deliberately fail publication and resume from the last successful stage. Inspect API/UI links back to the run directory.

Repository path to inspect

```
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/src/apexsim/pipeline/runner.py
apex_engine/src/apexsim/registry.py
apex_engine/dags/apexsim_dag.py
debugging_cases/13_artifact_overwrite
debugging_cases/14_airflow_payload
```

Connection to research

Reproducible world-model research depends on production habits: immutable datasets, exact split policies, seeds, configuration, checkpoints and evaluation code. Without them, architecture comparisons are anecdotes.

Check your understanding before continuing

Why should Airflow tasks pass paths instead of arrays?

What makes a stage idempotent?

What four layers should monitoring cover?

Solutions and reasoning

1. Durable storage survives retries/workers, avoids metadata-size limits and preserves lineage. 2. Same logical inputs/config produce the same content or a safely detected reusable artifact without duplicating side effects. 3. Pipeline execution, input data/contract, model behaviour/drift and delivered product/API/UI health.

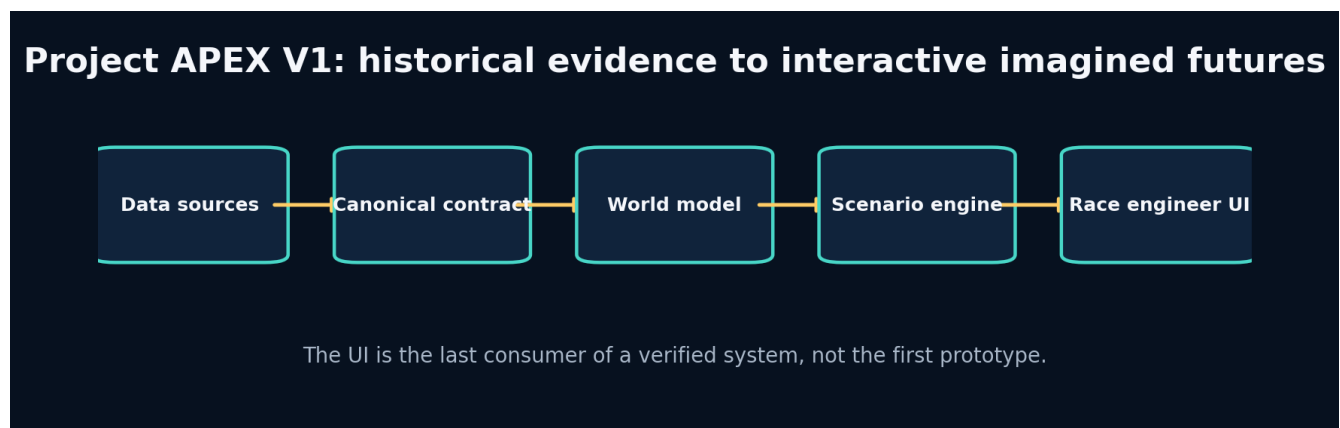
Independent build challenge

Create a two-run backfill that ingests two sessions, trains one model per session and publishes reports. Kill the process mid-run, restart it, and prove no successful work is corrupted or needlessly repeated.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

20. Build Project APEX V1 End to End—and Then Outgrow It

Instructor objective: Assemble the complete system, run a verified world-model simulation, use the UI responsibly, and define the research path toward F1 25 and an original architecture.



20. Build Project APEX V1 End to End—and Then Outgrow It

The problem that earns this chapter

The final challenge is integration. Correct individual pieces can still form a wrong system: adapters may disagree with windows, normalizers may differ between training and UI, scenario controls may overwrite state, or a beautiful interface may present false precision. We now build one traceable path from evidence to an imagined future.

Predict before reading

Before running anything, list the artifacts that must exist for a UI scenario to be reproducible: data, split, normalizer, model, evaluation, scenario inputs and software identity. Which of these should the UI be allowed to modify?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

APEX V1 is a historical telemetry world-simulation engine. It does not claim complete car physics or autonomous strategy. It answers a narrower, testable question: given recent canonical telemetry and an explicit sequence of future controls/context, can a learned transition model generate plausible short-horizon future state?

The system begins with synthetic data so every causal rule is inspectable and offline tests always run. It accepts FastF1/OpenF1 canonical data when available. A deterministic GRU is primary; an SSM-style model and RSSM are challengers. Evaluation determines the trusted horizon. The scenario engine modifies only designated future controls/context. The UI displays recorded and imagined trajectories with model/run identity and limitations.

The next stages are research, not feature accumulation: improve state observability, calibrate uncertainty, add track/generalization splits, ingest F1 25 UDP, learn richer latent dynamics, and validate planning. An original architecture should be proposed only after a measured failure cannot be fixed by data, objective or simpler model choices.

Vocabulary that now has a job

Concept: Scenario contract - Meaning in plain language: Exactly which future controls/context may be changed and how. - **Role inside APEX:** Throttle, brake, rain/grip and tyre assumptions under bounded transformations.

Concept: Trusted horizon - Meaning in plain language: Maximum forecast duration meeting error and guardrail criteria. - **Role inside APEX:** Displayed with every scenario rather than claiming unlimited simulation.

Concept: Product provenance - Meaning in plain language: User-visible link from prediction to run/model/data/config. - **Role inside APEX:** Prevents anonymous, irreproducible UI output.

Concept: Research hypothesis - Meaning in plain language: A falsifiable explanation for a measured failure and proposed change. - **Role inside APEX:** The basis for new world-model architecture work.

Worked example: calculate it by hand

A UI scenario uses history length 32 at 5 Hz (6.4 seconds) and predicts 8 frames (1.6 seconds). Suppose speed MAE by horizon is [1.0,1.3,1.8,2.5,3.4,4.6,6.2,8.5] km/h, with a guardrail of MAE ≤ 5 km/h and zero

physical violations. The trusted horizon is step 6, or 1.2 seconds, even though the model emits 1.6 seconds.

This is not failure. The system should shade or label steps 7–8 as exploratory, refuse high-stakes conclusions, and direct research toward the compounding error observed after step 6.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

Run the production APEX fast configuration, inspect every artifact, launch the UI, modify one scenario control, and trace the resulting prediction back through normalizer, model and run metadata.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `apexsim run -config configs/fast.yaml -run-id ...` - **What the line is doing:** Execute the complete source-independent pipeline. - **What to inspect:** Use a new immutable run ID and inspect stage logs.

Code: `apexsim ui -run-dir ...` - **What the line is doing:** Launch the race-engineering interface from verified artifacts. - **What to inspect:** The UI must load the same normalizer/model/config used in evaluation.

Code: `apexsim run-canonical -input-path ...` - **What the line is doing:** Feed adapter-produced canonical data through the identical pipeline. - **What to inspect:** Validate source and split semantics before comparing runs.

State and tensor trace

```
FastF1 / OpenF1 / synthetic / future UDP
    ↓ canonical contract + quality report
session split → windows → train-only normalizer
    ↓
```

GRU / SSM / RSSM checkpoint

↓ matched evaluation + trusted horizon

scenario controls → autoregressive rollout

↓

artifact bundle → registry → API/UI

For one UI point, record: source frame indices, normalized input, scenario transform, model output, inverse transform, plotted value and run ID.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Load a checkpoint with the wrong normalizer, let the UI modify future target speed directly, hide the model identity, or display more decimal precision than validation supports.

Diagnose from the earliest failed contract

Reproduce the scenario outside the UI using the same artifact paths. Compare canonical input, normalized tensor and raw model output. If CLI and UI differ, the serving boundary is broken; if both fail, move earlier.

Repair and lock the repair with a test

Bundle model, normalizer, feature order and config; validate hashes on load. Make scenario fields allow-listed. Display uncertainty/trusted horizon and provenance. Add end-to-end tests comparing UI callback output with direct simulation.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Gradio UI - Choose it when: Rapid educational interactive scenarios with Python integration. - **Do not choose it when:** You require a highly customized production frontend at scale.

Implementation: FastAPI + web frontend - Choose it when: Stable service contracts, multiple clients and custom UX. - **Do not choose it when:** The API/model contract is still changing daily.

Implementation: Synthetic-first - Choose it when: Offline causality, testing and controlled failure injection. - **Do not choose it when:** You confuse synthetic accuracy with real F1 fidelity.

Implementation: F1 25 UDP migration - Choose it when: Game packets are available and canonical mapping/latency tests are ready. - **Do not choose it when:** You let packet-specific fields leak through the entire core architecture.

The delivered V1 is a complete learning and research platform, not the final Dreamer-for-F1 system. Its value is that every future architecture competes inside a tested data, rollout, evaluation and product framework.

Transfer the lesson into Project APEX

Follow START_HERE.md, run tests, execute a fast reference, launch the UI, complete Projects 1–9, then rebuild one core module without reading the solution. Use the source companion only after your own attempt.

Repository path to inspect

apex_engine/START_HERE.md
 apex_engine/src/apexsim/
 apex_engine/notebooks/
 apex_engine/tests/
 projects/10_project_apex/
 source_code_companion/

Connection to research

The path toward a Dreamer-like F1 system is staged: validated dynamics → probabilistic belief/calibration → reward/continuation model → planning under constraints → actor/critic imagination → F1 25 online data → cross-domain fine-tuning. Each arrow needs evidence.

Check your understanding before continuing

Why must the UI use the training normalizer?

What should determine the displayed simulation horizon?

When is a new architecture justified?

Solutions and reasoning

1. The model learned parameters in that transformed feature space; a different transform changes the meaning of every input/output. 2. Matched held-out horizon error, physical guardrails, uncertainty and operational tolerance—not the configured output length. 3. When a specific measured failure survives data/contract fixes, objective improvements and simpler baselines, and the proposed mechanism directly targets that failure.

Independent build challenge

Rebuild APEX from an empty repository using only the canonical contract, acceptance tests and architecture diagram. Then propose one original model change with a falsifiable hypothesis, matched ablation and rollback condition.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

Ten-project ladder

The small labs teach one idea in isolation. The projects force multiple ideas to coexist.

One-dimensional car integrator

Telemetry quality laboratory

Lap physics sandbox

Nonlinear transition model

Track segment encoder

GRU telemetry forecaster

Selective SSM memory laboratory

RSSM imagination laboratory

Latent MPC planner

Project APEX production F1 world-simulation engine

For every project, write an Architecture Decision Record before opening the supplied solution. Record the problem, options, evidence, decision, and revisit condition.

Primary references

Hafner et al., *Mastering Diverse Domains through World Models (DreamerV3)*, arXiv:2301.04104.

Gu and Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, arXiv:2312.00752.

Assran et al., *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture*, arXiv:2301.08243.

LeWorldModel authors, project paper and official implementation.

FastF1 official documentation.

OpenF1 official documentation.

Electronic Arts, *F1 25 Data Output Specification*.

PyTorch official documentation for Dataset, DataLoader, GRU, autograd and optimization.

Project APEX Executable Lab Manual

Twenty guided experiments from units to latent planning

OpenAI

August 2026

Table of Contents

1 How to run a lab

For every lab, use the same discipline:

Read the goal and predict the output.

Copy the solution into a temporary file by hand or complete your own starter.

Run it once unchanged.

Explain every printed number and tensor axis.

Introduce the specified defect deliberately.

Find the earliest contract that fails.

Repair it and add a regression assertion.

Complete the extension without copying.

Record how the idea changes Project APEX.

The lab is complete only when you can alter it and still explain the result.

2 Lab 1: Units and tensor axes

2.1 Why this lab exists

A number without a unit and an array without axis names are both incomplete evidence. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

2.2 Predict before running

Predict the converted speeds and the exact [time, feature] shape before running. Write the expected output, shapes, units and likely failure modes.

2.3 Execute

```
cd labs/01_units_and_shapes
python solution.py
```

2.4 Complete code with line numbers

```

1: import numpy as np
2:
3: speed_kmh = np.array([0.0, 72.0, 144.0])
4: speed_mps = speed_kmh / 3.6
5: telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
6: print("speed_mps:", speed_mps)
7: print("telemetry shape [time, features]:", telemetry.shape)
8: assert telemetry.shape == (3, 2)

```

2.5 Packaged reference output

```

speed_mps: [ 0. 20. 40.]
telemetry shape [time, features]: (3, 2)

```

2.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

2.7 Break it deliberately

Transpose the array or skip km/h conversion. Add assertions for units, shape and feature order.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

2.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

2.9 Extension

Add a typed telemetry batch carrying values, names, units and sample rate.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

2.10 Where it enters APEX

Canonical adapter and contract validation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

2.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

3 Lab 2: Differentiate and integrate a speed trace

3.1 Why this lab exists

Numerical derivatives magnify noise; numerical integration accumulates bias. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

3.2 Predict before running

Predict whether reconstructing speed from a numerical gradient will be exact. Write the expected output, shapes, units and likely failure modes.

3.3 Execute

```
cd labs/02_vectorized_kinematics
python solution.py
```

3.4 Complete code with line numbers

```
1: import numpy as np
2:
3: dt = 0.2
4: time = np.arange(0, 5, dt)
5: speed = 20 + 4*np.sin(time)
6: accel = np.gradient(speed, dt)
7: reconstructed = speed[0] + np.cumsum(accel)*dt
8: print("first accelerations:", accel[:5].round(3))
9: print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

3.5 Packaged reference output

first accelerations: [3.973 3.894 3.66 3.279 2.768]

max reconstruction error: 0.7946773231802453

3.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

3.7 Break it deliberately

Change dt without changing time samples; compare errors and locate the contract mismatch.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

3.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

3.9 Extension

Compare forward, central and smoothed derivatives.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

3.10 Where it enters APEX

Synthetic causal generator and physical sanity checks. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

3.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

4 Lab 3: Longitudinal forces

4.1 Why this lab exists

Separating drive, brake and drag makes causal errors visible. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

4.2 Predict before running

Calculate net force and acceleration for one chosen row by hand. Write the expected output, shapes, units and likely failure modes.

4.3 Execute

```
cd labs/03_force_model
python solution.py
```

4.4 Complete code with line numbers

```
1: from education.core import CarState, step_car
2:
3: state = CarState(0.0, 0.0, 40.0)
4: for i in range(5):
5:     state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)
6:     print(i, state)
```

4.5 Packaged reference output

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/03_force_model/solution.py", line 1, in
<module>

```
    from education.core import CarState, step_car
```

ModuleNotFoundError: No module named 'education'

4.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

4.7 Break it deliberately

Feed km/h into a coefficient calibrated for m/s and add a property test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

4.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

4.9 Extension

Add grip-limited traction and show where more throttle stops helping.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

4.10 Where it enters APEX

Synthetic telemetry dynamics and residual modelling. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

4.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

5 Lab 4: Sampling and aliasing

5.1 Why this lab exists

The model cannot learn temporal information the sensor never captured. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

5.2 Predict before running

Predict the apparent signal at 5, 12 and 40 Hz. Write the expected output, shapes, units and likely failure modes.

5.3 Execute

```
cd labs/04_sampling_aliasing
python solution.py
```

5.4 Complete code with line numbers

```
1: import numpy as np
2:
3: for hz in [5, 12, 40]:
4:     t=np.arange(0,2,1/hz)
5:     signal=np.sin(2*np.pi*8*t)
6:     print(hz, "Hz samples:", np.round(signal[:8],3))
```

5.5 Packaged reference output

```
5 Hz samples: [ 0. -0.588 0.951 -0.951 0.588 -0. -0.588 0.951]
12 Hz samples: [ 0. -0.866 0.866 -0. -0.866 0.866 -0. -0.866]
40 Hz samples: [ 0.  0.951 0.588 -0.588 -0.951 -0.  0.951 0.588]
```

5.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

5.7 Break it deliberately

Choose a signal above Nyquist and show two different continuous signals sharing samples.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

5.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

5.9 Extension

Build an anti-alias filter experiment.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

5.10 Where it enters APEX

Canonical frequency and F1 25 telemetry-rate decisions. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

5.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

6 Lab 5: Ridge transition baseline

6.1 Why this lab exists

A simple control tells you whether complexity earns its cost. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

6.2 Predict before running

Predict coefficient signs for speed, throttle and brake. Write the expected output, shapes, units and likely failure modes.

6.3 Execute

```
cd labs/05_regression_baseline
python solution.py
```

6.4 Complete code with line numbers

```
1: import numpy as np
2: from sklearn.linear_model import Ridge
3:
4: rng=np.random.default_rng(4)
5: speed=rng.uniform(20,80,1000)
6: throttle=rng.uniform(0,1,1000)
```



```

7: brake=rng.uniform(0,1,1000)
8: y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
9: X=np.column_stack([speed,throttle,brake])
10: model=Ridge(alpha=1.0).fit(X[:800],y[:800])
11: print("coefficients:", model.coef_)
12: print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))

```

6.5 Packaged reference output

```

coefficients: [ 0.91911817  0.68984381 -1.03384695]
test MAE: 0.18453407932904015

```

6.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

6.7 Break it deliberately

Leak the target into features, observe near-perfect error, then prevent it with an allow-list.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

6.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

6.9 Extension

Add polynomial speed features and inspect residuals.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

6.10 Where it enters APEX

Baseline-relative model evaluation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

6.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

7 Lab 6: Learning a physical coefficient

7.1 Why this lab exists

Autograd is easiest to trust when the expected parameter is known. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

7.2 Predict before running

Predict the sign of the drag-coefficient gradient from an under-estimate. Write the expected output, shapes, units and likely failure modes.

7.3 Execute

```
cd labs/06_autograd_drag
python solution.py
```

7.4 Complete code with line numbers

```
1: import torch
2:
3: speed=torch.linspace(5,70,200)
4: true_drag=0.002
5: target=-true_drag*speed**2
6: drag=torch.tensor(0.01,requires_grad=True)
7: opt=torch.optim.SGD([drag],lr=1e-7)
8: for step in range(200):
9:     opt.zero_grad()
10:    pred=-drag*speed**2
11:    loss=((pred-target)**2).mean()
12:    loss.backward(); opt.step()
13: print("learned drag:", drag.item(), "loss:", loss.item())
```

7.5 Packaged reference output

learned drag: 0.0020000000949949026 loss: 0.0

7.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

7.7 Break it deliberately

Omit `zero_grad` and increase learning rate until divergence.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

7.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

7.9 Extension

Estimate two correlated force parameters and study identifiability.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

7.10 Where it enters APEX

Training-loop diagnostics and hybrid physics learning. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

7.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

8 Lab 7: A transition MLP

8.1 Why this lab exists

A module is a parameterized function with a precise shape contract. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

8.2 Predict before running

Write input/output shapes for one batch before executing. Write the expected output, shapes, units and likely failure modes.

8.3 Execute

```
cd labs/07_pytorch_module
python solution.py
```

8.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: class Transition(nn.Module):
5:     def __init__(self):
6:         super().__init__()
7:         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
8:     def forward(self,x): return self.net(x)
9:
10: model=Transition(); x=torch.randn(4,5); y=model(x)
11: print(model); print("input",x.shape,"output",y.shape)
```

8.5 Packaged reference output

```
Transition(
  (net): Sequential(
    (0): Linear(in_features=5, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=2, bias=True)
  )
)
input torch.Size([4, 5]) output torch.Size([4, 2])
```

8.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

8.7 Break it deliberately

Swap batch and feature axes; block the bug with exact assertions.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

8.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

8.9 Extension

Add a residual connection predicting state delta instead of absolute state.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

8.10 Where it enters APEX

Frame transition baseline and shape tracing. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

8.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

9 Lab 8: Causal history and target windows

9.1 Why this lab exists

Sequence slicing defines what the model is allowed to know. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

9.2 Predict before running

List raw indices for the first and final example. Write the expected output, shapes, units and likely failure modes.

9.3 Execute

```
cd labs/08_dataset_windows
python solution.py
```

9.4 Complete code with line numbers

```
1: import numpy as np
2: from education.core import sliding_windows
3:
4: values=np.arange(30,dtype=float).reshape(10,3)
5: x,y=sliding_windows(values,history=4,horizon=2)
6: print("x",x.shape,"y",y.shape)
7: print("first history\n",x[0]); print("first future\n",y[0])
```

9.5 Packaged reference output

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/08_dataset_windows/solution.py", line 2, in <module>

```
    from education.core import sliding_windows
```

ModuleNotFoundError: No module named 'education'

9.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

9.7 Break it deliberately

Put target state into future inputs and add a leakage test.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

9.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

9.9 Extension

Add future controls as a separate tensor and session identifiers.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

9.10 Where it enters APEX

APEX WindowDataset and rollout contract. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

9.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

10 Lab 9: Loss functions under outliers

10.1 Why this lab exists

Loss determines which errors receive optimization pressure. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

10.2 Predict before running

Rank MSE, MAE and Huber sensitivity to one extreme residual. Write the expected output, shapes, units and likely failure modes.

10.3 Execute

```
cd labs/09_loss_comparison
python solution.py
```

10.4 Complete code with line numbers

```
1: import torch
2: from torch.nn import functional as F
3:
4: pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
5: print("MSE",F.mse_loss(pred,target).item())
6: print("MAE",F.l1_loss(pred,target).item())
7: print("Huber",F.huber_loss(pred,target).item())
```

10.5 Packaged reference output

```
MSE 72.25
MAE 4.25
Huber 4.125
```

10.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

10.7 Break it deliberately

Scale one feature by 100 without normalization and observe domination.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

10.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

10.9 Extension

Create a weighted multi-feature telemetry loss.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

10.10 Where it enters APEX

Model training versus operational metrics. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

10.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

11 Lab 10: Gradients and updates

11.1 Why this lab exists

Backward computes gradients; the optimizer changes parameters. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

11.2 Predict before running

State what changes after each training-loop line. Write the expected output, shapes, units and likely failure modes.

11.3 Execute

```
cd labs/10_training_loop
python solution.py
```

11.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
5: model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
```

```

6: for epoch in range(8):
7:   model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
8:   grad=float(model.weight.grad.norm()); opt.step()
9:   model.eval();
10:  with torch.no_grad(): val=((model(x)-y)**2).mean()
11:  print(epoch,float(loss),grad,float(val))

```

11.5 Packaged reference output

```

0 3.2264039516448975 3.7724902629852295 2.9054489135742188
1 2.9054489135742188 3.5811667442321777 2.6067750453948975
2 2.6067750453948975 3.390789270401001 2.3303141593933105
3 2.3303141593933105 3.2017178535461426 2.0752670764923096
4 2.0752670764923096 3.0143868923187256 1.8400570154190063
5 1.8400570154190063 2.829359292984009 1.6230223178863525
6 1.6230223178863525 2.6473701000213623 1.4232583045959473
7 1.4232583045959473 2.469295024871826 1.2406574487686157

```

/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/10_training_loop/solution.py:11:

UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.

Consider using tensor.detach() first. (Triggered internally at

/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)

```
print(epoch,float(loss),grad,float(val))
```

11.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

11.7 Break it deliberately

Remove zeroing, omit eval mode and create broadcasting targets.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

11.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

11.9 Extension

Write a JSON trace of gradients and update norms.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

11.10 Where it enters APEX

APEX training and checkpoint validation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

11.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

12 Lab 11: Recurrent state updates

12.1 Why this lab exists

A recurrent model is a transition over hidden memory. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

12.2 Predict before running

Compute the first scalar hidden update by hand. Write the expected output, shapes, units and likely failure modes.

12.3 Execute

```
cd labs/11_rnn_from_scratch
python solution.py
```

12.4 Complete code with line numbers

```
1: import torch
2:
3: torch.manual_seed(0); W=torch.randn(3,2); U=torch.randn(3,3); b=torch.zeros(3); h=torch.zeros(3)
4: sequence=torch.tensor([[1.,0.],[0.,1.],[1.,1.]])
```

```
5: for t,x in enumerate(sequence):
6:   h=torch.tanh(W@x+U@h+b); print(t,h.numpy().round(3))
```

12.5 Packaged reference output

```
0 [ 0.912 -0.975 -0.795]
1 [-0.169  0.563 -0.984]
2 [ 0.982 -0.968 -0.582]
```

12.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

12.7 Break it deliberately

Reverse sequence order and reuse state across sequences.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

12.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

12.9 Extension

Create a long-delay memory task.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

12.10 Where it enters APEX

Why APEX needs history encoding. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

12.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

13 Lab 12: GRU memory gates

13.1 Why this lab exists

Gates learn how much old memory to retain or overwrite. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

13.2 Predict before running

Predict whether repeating the same input yields identical hidden states. Write the expected output, shapes, units and likely failure modes.

13.3 Execute

```
cd labs/12_gru_gates
python solution.py
```

13.4 Complete code with line numbers

```
1: import torch
2:
3: torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
4: for x in [torch.tensor([[1.,0.]]),torch.tensor([[0.,1.]]),torch.tensor([[0.,1.]]):
5:     h=cell(x,h); print(h.detach().numpy().round(3))
```

13.5 Packaged reference output

```
[[ 0.354 -0.151 -0.384  0.149]]
[[ 0.278  0.282 -0.371  0.408]]
[[ 0.322  0.39  -0.39  0.542]]
```

13.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

13.7 Break it deliberately

Force update gates to saturation and inspect memory.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

13.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

13.9 Extension

Compare GRU and matched RNN on delayed events.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

13.10 Where it enters APEX

Primary deterministic world model. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

13.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

14 Lab 13: Linear state-space memory

14.1 Why this lab exists

Eigenvalues determine decay, persistence, oscillation and explosion. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

14.2 Predict before running

Predict the two hidden components after the first pulse. Write the expected output, shapes, units and likely failure modes.

14.3 Execute

```
cd labs/13_linear_ssm
python solution.py
```

14.4 Complete code with line numbers

```
1: import numpy as np
2: A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
3: for u in [1,1,0,0,0]:
4:   x=A@x+B[:,0]*u; print(x.round(3))
```

14.5 Packaged reference output

```
[0.2 0.05]
[0.39 0.112]
[0.37 0.135]
[0.352 0.151]
[0.334 0.164]
```

14.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

14.7 Break it deliberately

Set spectral radius above one and create a stability test.

Before fixing the defect, write:

expected symptom;
 earliest failed contract;
 one misleading downstream symptom;
 one assertion that should catch it;
 why a model or optimizer change would not be the correct first response.

14.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

14.9 Extension

Design fast, medium and slow memory modes.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

14.10 Where it enters APEX

SSM challenger and hidden-norm monitoring. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

14.11 Exit questions

What assumption did this lab make deliberately simple?
 What output would immediately make you distrust the result?
 Which test protects the idea rather than this exact code?
 When would you choose a different implementation?

15 Lab 14: Input-dependent state-space memory

15.1 Why this lab exists

Selection lets current content alter retention and writing. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

15.2 Predict before running

Predict how basis-vector order changes final memory. Write the expected output, shapes, units and likely failure modes.

15.3 Execute

```
cd labs/14_selective_ssm
python solution.py
```


15.4 Complete code with line numbers

```

1: import torch
2: from torch import nn
3:
4: class SelectiveCell(nn.Module):
5:     def __init__(self,d):
6:         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
7:     def forward(self,x,h):
8:         a=torch.sigmoid(self.decay(x))*0.99
9:         b=torch.tanh(self.write(x))
10:        return a*h+(1-a)*b
11: cell=SelectiveCell(3); h=torch.zeros(1,3)
12: for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))

```

15.5 Packaged reference output

```

[[-0.169 -0.022  0.151]]
[[ 0.072  0.103 -0.15  ]]
[[ 0.048  0.153 -0.225]]

```

15.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

15.7 Break it deliberately

Remove gate bounds and inspect explosion/frozen memory.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

15.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

15.9 Extension

Log retention by braking/straight regimes.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

15.10 Where it enters APEX

Selective SSM challenger and Mamba preparation. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

15.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

16 Lab 15: Latent compression

16.1 Why this lab exists

Reconstruction pressure does not guarantee predictive representations. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

16.2 Predict before running

Predict latent and reconstruction shapes. Write the expected output, shapes, units and likely failure modes.

16.3 Execute

```
cd labs/15_autoencoder
python solution.py
```

16.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: torch.manual_seed(3); x=torch.randn(256,12)
5: enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
```

```

6: for _ in range(80):
7:     opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward();
   opt.step()
8: print("latent",z.shape,"loss",float(loss))

```

16.5 Packaged reference output

```

latent torch.Size([256, 3]) loss 0.7505914568901062
/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/15_autoencoder/solution.py:8: UserWarning:
Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
   print("latent",z.shape,"loss",float(loss))

```

16.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

16.7 Break it deliberately

Add high-variance nuisance features and observe what latent preserves.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

16.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

16.9 Extension

Add a future-state probe and predictive auxiliary loss.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

16.10 Where it enters APEX

Representation learning before RSSM/JEPA. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

16.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

17 Lab 16: Stochastic latent variables

17.1 Why this lab exists

Reparameterization separates noise from differentiable distribution parameters. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

17.2 Predict before running

Compute standard deviations and KL by hand. Write the expected output, shapes, units and likely failure modes.

17.3 Execute

```
cd labs/16_vae_reparameterization
python solution.py
```

17.4 Complete code with line numbers

```
1: import torch
2:
3: mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[ -1.0,0.5]])
4: std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
5: kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
6: print("mu",mu,"std",std,"sample",z,"KL",kl)
```

17.5 Packaged reference output

```
mu tensor([[ 0.2000, -0.4000]]) std tensor([[0.6065, 1.2840]]) sample tensor([[0.8058, 0.3872]]) KL
tensor([0.3583])
```

17.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

17.7 Break it deliberately

Set extreme log variance and inspect numerical behaviour.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

17.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

17.9 Extension

Sample repeatedly and compare empirical moments.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

17.10 Where it enters APEX

RSSM posterior/prior distributions. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

17.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

18 Lab 17: Prior and posterior belief

18.1 Why this lab exists

Training may use observation evidence; imagination may not. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

18.2 Predict before running

Name the inputs to `h`, prior and posterior before running. Write the expected output, shapes, units and likely failure modes.

18.3 Execute

```
cd labs/17_rssm_step
python solution.py
```

18.4 Complete code with line numbers

```
1: import torch
2: from torch import nn
3:
4: cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
5: h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
6: h=cell(torch.cat([prev_z,action],-1),h)
7: prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
8: print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

18.5 Packaged reference output

```
h torch.Size([2, 8]) prior torch.Size([2, 4]) posterior torch.Size([2, 4])
```

18.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

18.7 Break it deliberately

Leak observation into the prior and write a no-future-observation test.

Before fixing the defect, write:

expected symptom;
 earliest failed contract;
 one misleading downstream symptom;
 one assertion that should catch it;
 why a model or optimizer change would not be the correct first response.

18.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

18.9 Extension

Add sampling, KL and decoder loss for multiple steps.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

18.10 Where it enters APEX

Dreamer-style latent dynamics. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

18.11 Exit questions

What assumption did this lab make deliberately simple?
 What output would immediately make you distrust the result?
 Which test protects the idea rather than this exact code?
 When would you choose a different implementation?

19 Lab 18: Cross-entropy planning

19.1 Why this lab exists

Planning repeatedly samples, imagines, keeps elites and refits. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

19.2 Predict before running

Predict the direction of the first action mean. Write the expected output, shapes, units and likely failure modes.

19.3 Execute

```
cd labs/18_cem_planning
python solution.py
```

19.4 Complete code with line numbers

```

1: import numpy as np
2:
3: rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
4: for it in range(5):
5:     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
6:     speed=40+np.cumsum(2*actions-0.4,axis=1)
7:     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
8:     elite=actions[np.argsort(score)[-50:]]
9:     mean,std=elite.mean(0),elite.std(0)+1e-3
10:    print(it,mean.round(2),score.max().round(3))

```

19.5 Packaged reference output

```

0 [0.71 0.68 0.7  0.62 0.67 0.71] -1.123
1 [0.84 0.8  0.82 0.82 0.83 0.82] -0.019
2 [0.86 0.84 0.86 0.88 0.87 0.9 ] -0.005
3 [0.87 0.85 0.85 0.87 0.89 0.87] -0.001
4 [0.86 0.84 0.85 0.88 0.89 0.87] -0.001

```

19.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

19.7 Break it deliberately

Remove constraints and create a model loophole for the planner to exploit.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

19.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

19.9 Extension

Add brake actions, uncertainty penalty and receding horizon.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

19.10 Where it enters APEX

Scenario planning and later control. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

19.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

20 Lab 19: Causal asynchronous joins

20.1 Why this lab exists

Alignment policy determines whether future information leaks backward. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

20.2 Predict before running

Manually match every car timestamp under backward tolerance. Write the expected output, shapes, units and likely failure modes.

20.3 Execute

```
cd labs/19_time_alignment
python solution.py
```

20.4 Complete code with line numbers

```
1: import pandas as pd
2:
3: car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
4: weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
5:
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=
```

0.3)

6: print(aligned)

20.5 Packaged reference output

```
t speed rain
0 0.0 10 NaN
1 0.2 12 0.0
2 0.4 14 NaN
3 0.6 16 0.3
```

20.6 Instructor trace

Read the program in this order:

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

20.7 Break it deliberately

Switch to nearest and add a future spike.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

20.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

20.9 Extension

Persist source age and missingness masks.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

20.10 Where it enters APEX

FastF1/OpenF1 stream alignment. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

20.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

21 Lab 20: Staged artifact workflow

21.1 Why this lab exists

Durable boundaries make retries, lineage and debugging possible. The program is intentionally small enough that you can track every value. Do not add abstraction until the mechanism is obvious.

21.2 Predict before running

Predict the exact output tree. Write the expected output, shapes, units and likely failure modes.

21.3 Execute

```
cd labs/20_pipeline_contract
python solution.py
```

21.4 Complete code with line numbers

```
1: from pathlib import Path
2: import json
3:
4: run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
5: stages=['ingest','validate','window','train','evaluate','publish']
6: for i,name in enumerate(stages):
7:     path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
8:     print(path)
```

21.5 Packaged reference output

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

21.6 Instructor trace

Read the program in this order:

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

Identify persistent state and trainable parameters, if any.

Identify the input evidence and its axes/units.

Identify the transformation performed by each statement.

Identify what is printed and what is hidden.

Recompute at least one printed value manually.

State which line would first reveal an invalid assumption.

Now add temporary prints after every meaningful assignment. For tensors, print shape, dtype, minimum, maximum and one row. For iterative systems, print state before and after the transition. For learned models, print loss, gradient and parameter values at least once.

21.7 Break it deliberately

Overwrite one shared latest path and simulate failure halfway.

Before fixing the defect, write:

expected symptom;

earliest failed contract;

one misleading downstream symptom;

one assertion that should catch it;

why a model or optimizer change would not be the correct first response.

21.8 Repair test

Turn your diagnosis into a small automated check. The repair should fail on the broken implementation and pass on the corrected one. Prefer a known numerical answer, invariant, exact shape/ordering check or deterministic causal relationship.

21.9 Extension

Add content hashes, atomic writes and resumable status.

Do not copy from another lab. Write a short decision note comparing the original and extended implementation: what new capability was added, what complexity it introduced, and what evidence justifies keeping it.

21.10 Where it enters APEX

Local runner, Airflow DAG and registry. Find the corresponding production file using the Code Atlas and compare the small mechanism with the production abstraction. Explain what production concerns were added: configuration, batching, validation, artifacts, logging, tests or serving.

21.11 Exit questions

What assumption did this lab make deliberately simple?

What output would immediately make you distrust the result?

Which test protects the idea rather than this exact code?

When would you choose a different implementation?

Project APEX Tensor, Memory and Data-Flow Visual Atlas

Twenty diagrams for tracing the complete simulation system

OpenAI

August 2026

Table of Contents

How to use a systems diagram

Do not look at a diagram and say “I get it.” Cover the right half. For each arrow, predict the data type, shape, unit, source and whether gradients can cross it. Then reveal the next block. A diagram becomes understanding when you can reconstruct the implementation and name the failure created by breaking one arrow.

For every atlas plate:

Read the blocks left to right.

Write the contract at each arrow.

Trace one numerical example.

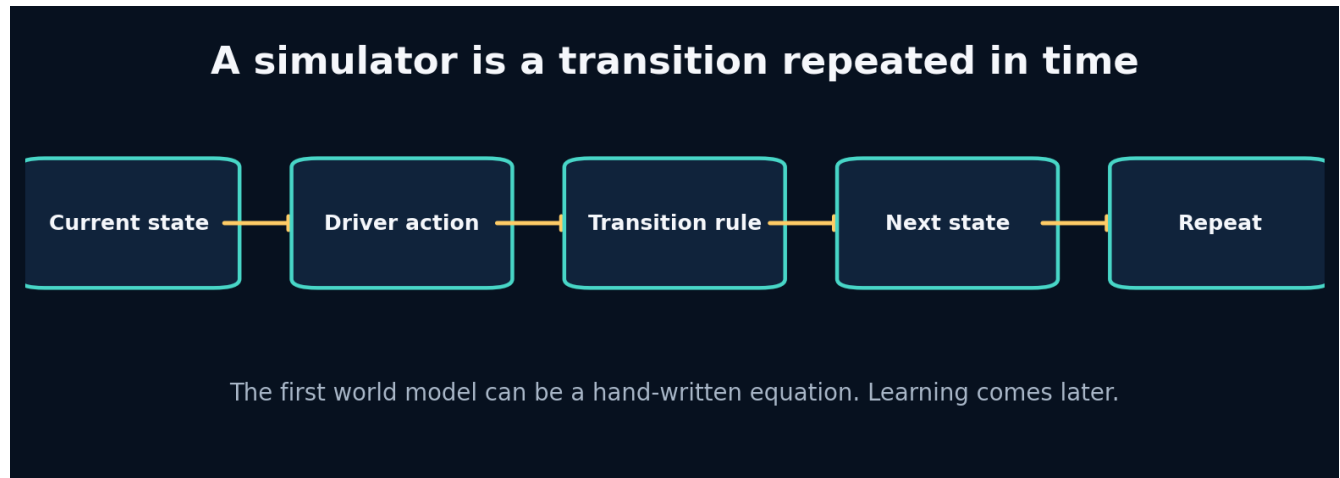
Mark observed, supplied, learned and generated values.

Mark training-only and deployment-available information.

Identify one invariant and one failure probe.

Locate the corresponding APEX source path.

Plate 1: 1. Build a Simulator Before You Build a Neural Network



1. Build a Simulator Before You Build a Neural Network

What the picture is claiming

Understand simulation as repeated state transition, then build and inspect the smallest causal driving world.

A simulator stores a description of the world, receives an intervention, applies a transition, and returns a new description. In our first world, the state is only velocity and position. The action is a throttle value. The transition is an equation. Repeating the equation produces a rollout.

This hand-written simulator is already a world model in the broad engineering sense: it predicts a future state from a present state and an action. It is not a *learned* world model. That difference matters. A learned model replaces part of the transition rule with parameters estimated from evidence. The interface should remain stable: state in, action in, next state out.

Starting here gives us an oracle for later debugging. If a neural network predicts that stronger braking increases velocity, we can compare it with the toy causal rule and ask whether the data contract, target alignment, or learned relationship is broken.

Label every block

State: The minimum information carried from one step to the next. In APEX: Speed, progress, tyre state and other dynamic variables.

Action: A controllable input applied during a transition. In APEX: Throttle, brake, steering and gear choices.

Transition: The rule that maps current state and action to next state. In APEX: Hand-written physics, a GRU, an SSM or an RSSM.

Rollout: Repeatedly applying the transition to create a future trajectory. In APEX: Imagining several seconds of future telemetry.

Walk one value through it

Euler integration uses

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

$$v_{t+1} = v_t + a_t \Delta t$$

and

$$x_{t+1} = x_t + v_t \Delta t.$$

With $v_t = 20$, $a_t = 3$, and $\Delta t = 0.2$:

Velocity change: $3 \times 0.2 = 0.6$ m/s.

Next velocity: $20 + 0.6 = 20.6$ m/s.

Distance using the current velocity: $20 \times 0.2 = 4.0$ m.

Next position, if starting at zero: 4.0 m.

A semi-implicit variant updates velocity first and then position, producing $20.6 \times 0.2 = 4.12$ m. Neither is universally “the answer”; the numerical method is an implementation choice whose error shrinks with smaller time steps.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Change Δt from 0.1 to 1.0 without changing the acceleration profile. Compare the trajectory. Then update position with the newly computed velocity and compare again.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Explicit Euler: choose when you need the simplest transparent baseline and small time steps. Avoid when long unstable rollouts or stiff dynamics.

Semi-implicit Euler: choose when velocity drives position and you want slightly better stability at similar cost. Avoid when you need high-order accuracy.

Runge-Kutta: choose when the hand-written dynamics are smooth and numerical accuracy matters. Avoid when the transition is learned from discrete telemetry where solver precision is not the main bottleneck.

APEX implementation map

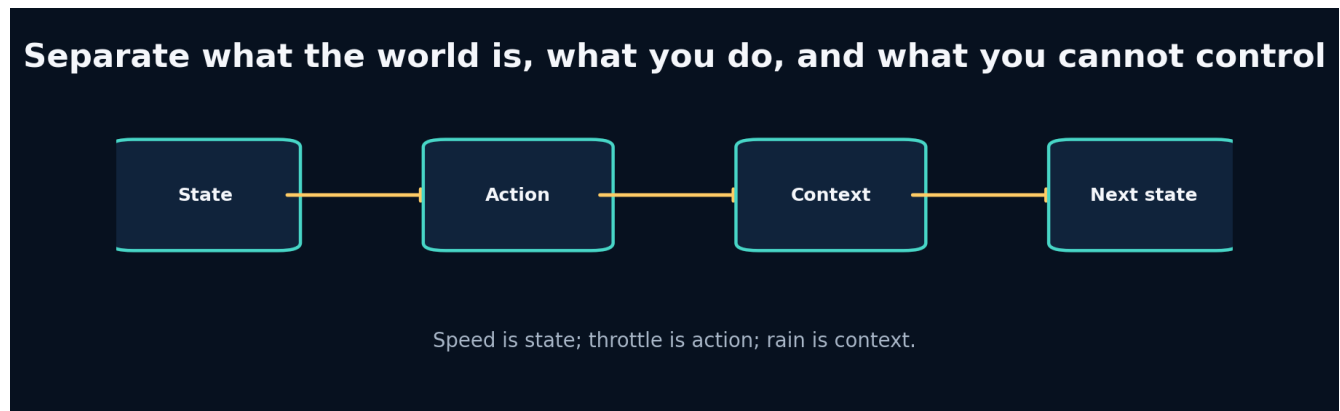
projects/01_car_integrator/main.py
 apex_engine/src/apexsim/data/synthetic.py
 apex_engine/src/apexsim/simulation.py

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 2: 2. State, Action, Context and the Markov Question



2. State, Action, Context and the Markov Question

What the picture is claiming

Design a causal telemetry schema and discover when the visible state is insufficient for predicting the future.

State is the information the transition needs to carry. Action is the intervention being chosen. Context changes the transition but is not controlled by the agent inside the model. Identifiers describe an entity or grouping and should not automatically become numeric features.

The Markov property says the next state is conditionally independent of the distant past once the present state and action are known. Real telemetry is rarely perfectly Markov. Tyre temperature, fuel mass, setup, and previous cornering load may be hidden. A recurrent model compensates by compressing history into memory. But recurrence is not permission to ignore schema design: hidden memory cannot recover information that was never observed.

A practical state contract should be sufficient, measurable, stable across data sources, and safe to use at inference time. A variable known only after the race cannot be an input to an online simulator.

Label every block

Markov state: A state that contains enough information to predict the next transition when combined with the action. In APEX: The ideal telemetry state; approximated using features plus history.

Context: An external condition that modifies dynamics. In APEX: Rain, track geometry, session phase and compound.

Identifier: A label used for grouping or lookup, not automatically a causal feature. In APEX: Session ID, driver code and event ID.

Partial observability: Important state exists but is not directly measured. In APEX: Tyre temperature or setup may need history, proxies or latent state.

Walk one value through it

Consider the transition

$$v_{t+1} = v_t + (4a_t - 7b_t - 0.02v_t - 3r_t)\Delta t$$

where a is throttle, b is brake, and r is rain intensity. At $v=50$ m/s, throttle 0.8, brake 0, rain 0.5, and $dt=0.2$:

Engine contribution: $4 \times 0.8 = 3.2$ m/s².

Drag contribution: $0.02 \times 50 = 1.0$ m/s².

Rain/grip penalty: $3 \times 0.5 = 1.5$ m/s².

Net acceleration: $3.2 - 1.0 - 1.5 = 0.7$ m/s².

Speed change: $0.7 \times 0.2 = 0.14$ m/s.

Next speed: 50.14 m/s.

If rain were omitted, the same visible state/action would predict 50.44 m/s. That 0.30 m/s discrepancy is a schema problem before it is a modelling problem.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Transpose the array and remove the shape assertion. Write a function that assumes rows are time. Observe that it can compute plausible but meaningless statistics.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Flat numeric vector: choose when the state is small, fixed and well documented. Avoid when variables have complex structure or missingness semantics.

Typed record / dataframe: choose when you are ingesting and validating human-readable telemetry. Avoid when you need high-throughput batched neural computation.

History encoder: choose when the observed frame is partially markov and recent history contains useful proxies. Avoid when missing variables are unrelated to anything observed.

APEX implementation map

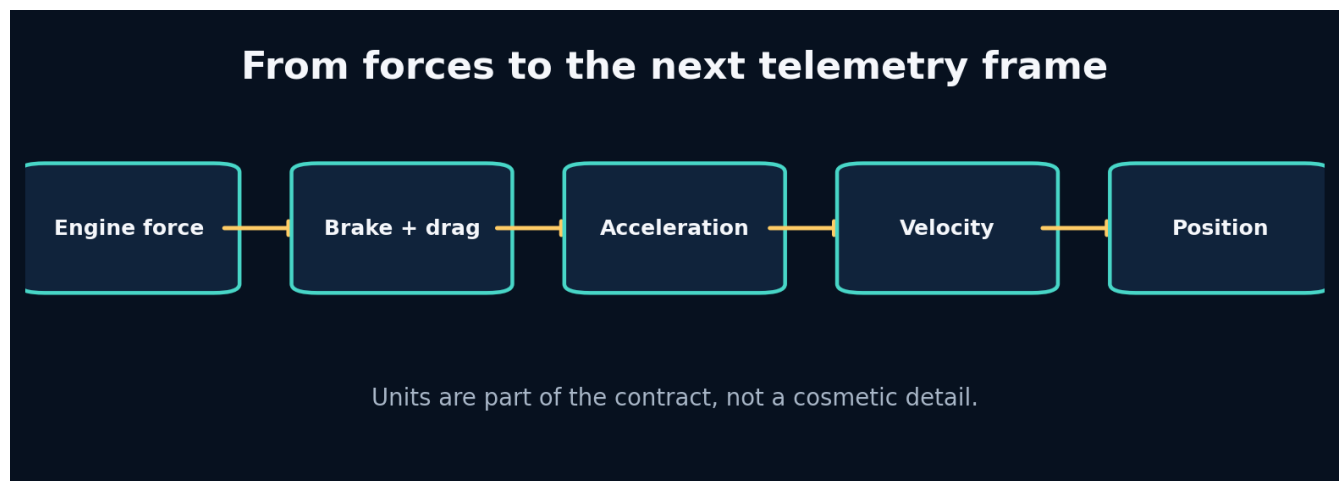
```
apex_engine/src/apexsim/contracts.py
apex_engine/src/apexsim/data/features.py
apex_engine/src/apexsim/data/windows.py
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 3: 3. Units, Forces and Numerical Integration



3. Units, Forces and Numerical Integration

What the picture is claiming

Build a physically interpretable transition, use dimensional analysis to catch errors, and understand the limits of toy physics.

Dimensional analysis asks whether the units on both sides of an equation agree. It is one of the cheapest and strongest debugging tools in simulation engineering. If force is in newtons and mass is in kilograms,

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

acceleration is metres per second squared. If speed enters a squared term, a unit conversion error is squared too.

A toy longitudinal model combines engine force, braking force and drag. It will not reproduce downforce, tyre load sensitivity, energy recovery, differential behaviour or setup effects. That is fine when the model's purpose is to generate causal examples and test software contracts. It is dangerous if the UI presents the result as lap-time truth.

The hand-written model also teaches residual learning: instead of asking a neural network to rediscover every obvious relationship, we can predict the correction to a known physics approximation. Whether that helps is an empirical decision.

Label every block

Dimensional analysis: Check that quantities combined by an equation have compatible units. In APEX: Catches km/h–m/s and timestamp errors before training.

Force balance: Net force is the sum of propulsive and resisting forces. In APEX: Provides an interpretable synthetic transition.

Residual model: A learned correction added to a known baseline model. In APEX: Possible hybrid physics/ML architecture.

Numerical stability: Whether repeated computation remains bounded and sensible. In APEX: Critical for long autoregressive rollouts and SSM state.

Walk one value through it

Suppose the drag acceleration term is $c v^2$. Converting from m/s to km/h multiplies speed by 3.6. Squaring multiplies the drag term by $3.6^2 = 12.96$. The model therefore applies nearly thirteen times too much drag.

For a 50 m/s car with mass 800 kg:

Engine force: 8,000 N

Brake force: 0 N

Drag: 2,500 N

Net force: 5,500 N

Acceleration: $5500 / 800 = 6.875 \text{ m/s}^2$

At $dt=0.1$, velocity increases by 0.6875 m/s

If speed were incorrectly treated as 180 in a coefficient calibrated for 50, the quadratic term would dominate and could create impossible negative velocity.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Feed speed in km/h without conversion. Then set mass to zero or throttle above one. Observe which failures become exceptions and which silently produce nonsense.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Pure hand-written physics: choose when you need interpretability, counterfactual control and known equations. Avoid when unknown tyre/track interactions dominate error.

Pure learned dynamics: choose when you have broad representative data and care about predictive fit. Avoid when safety or extrapolation requires hard physical guarantees.

Residual learning: choose when a baseline captures broad physics and data can learn systematic corrections. Avoid when the baseline is badly misspecified and constrains learning in the wrong direction.

APEX implementation map

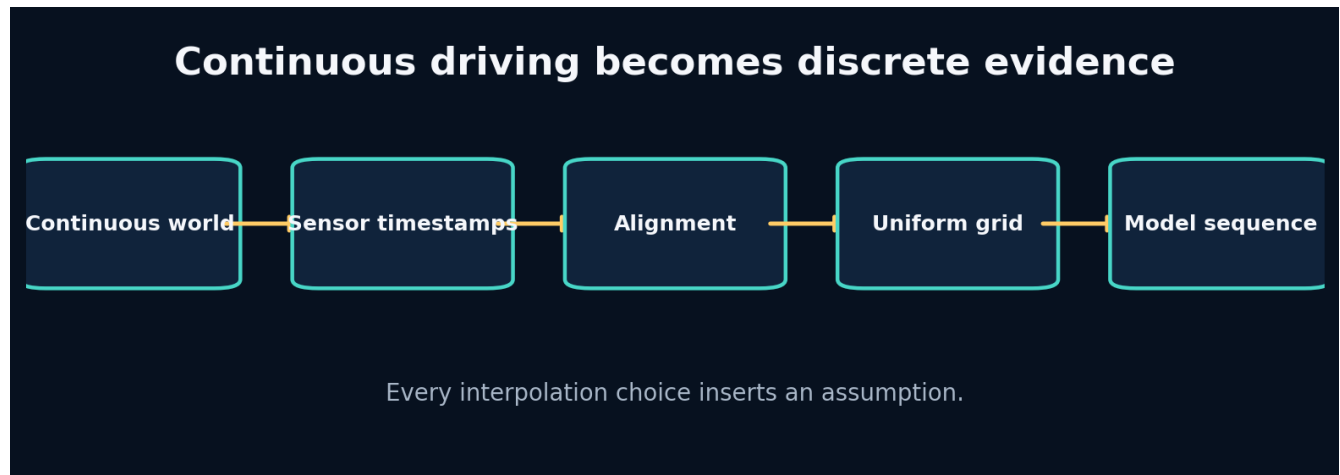
```
projects/03_lap_physics/main.py
apex_engine/src/apexsim/data/synthetic.py
apex_engine/src/apexsim/evaluation.py
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 4: 4. Sampling, Aliasing and Time Alignment



4. Sampling, Aliasing and Time Alignment

What the picture is claiming

Turn asynchronous telemetry into a defensible time grid without creating fake evidence or future leakage.

Sampling converts a continuous process into discrete observations. If the sample rate is too low for the dynamics, high-frequency behaviour aliases into a different pattern. No architecture can reconstruct information that the sensor never captured without additional assumptions.

Alignment chooses which measurements describe the same model time. A backward as-of join says: at each car timestamp, use the latest context that was already available. A nearest join may use the future. A tolerance limits how stale a matched measurement may be. These are causal policies, not dataframe trivia.

Resampling onto a uniform grid simplifies windows and batch training, but interpolation must respect feature type. Continuous speed may be linearly interpolated across short gaps. Gear and flags are categorical and generally require forward fill or explicit unknown values. Long gaps should be marked invalid, not painted over.

Label every block

Sampling rate: How many observations are recorded per second. In APEX: Defines temporal resolution and sequence length.

Aliasing: A high-frequency process appears as a different lower-frequency pattern. In APEX: Can hide braking spikes or steering oscillations.

As-of join: Match each timestamp with a nearby record under a direction and tolerance policy. In APEX: Aligns weather, position and telemetry streams.

Staleness: How old a context value is when reused. In APEX: A monitoring feature and validation rule.

Walk one value through it

The Nyquist guideline says a sinusoid of frequency f requires sampling above $2f$ to avoid ambiguity. An 8 Hz oscillation needs more than 16 Hz. At 5 Hz, sample times are 0, 0.2, 0.4... seconds. The phase

advances $8 \times 0.2 = 1.6$ cycles per sample, equivalent to an apparent 0.6-cycle advance after wrapping. The observed pattern therefore resembles a 3 Hz oscillation (0.6×5).

For alignment, suppose car timestamps are 0.0, 0.2, 0.4, 0.6 and rain observations are 0.05 and 0.55. A backward join at 0.4 uses rain from 0.05; a nearest join uses 0.55, leaking a future value by 0.15 seconds.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Change direction to nearest and remove the tolerance. Add a future rain spike at 0.41. Observe how an earlier car frame receives information from the future.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Backward as-of: choose when features must reflect information available online. Avoid when the quantity is defined as a symmetric offline estimate.

Nearest join: choose when both streams are synchronized measurements and future use is scientifically acceptable. Avoid when forecasting, control or causal analysis.

Linear interpolation: choose when a continuous signal has short, well-sampled gaps. Avoid when categorical states, discontinuities or long outages.

APEX implementation map

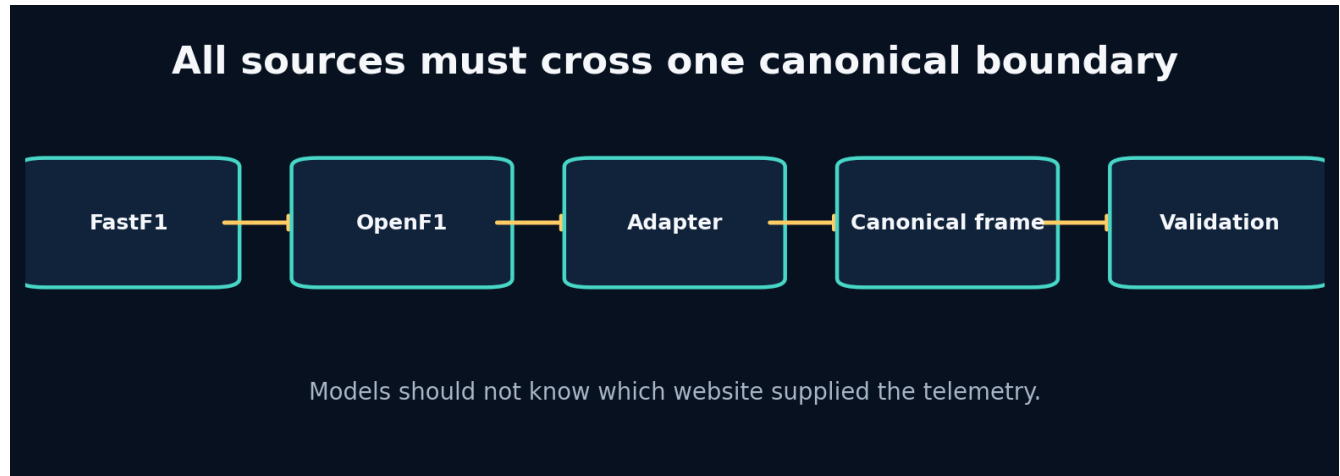
```
labs/04_sampling_aliasing/solution.py
labs/19_time_alignment/solution.py
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 5: 5. Canonical Contracts and Quality Gates



5. Canonical Contracts and Quality Gates

What the picture is claiming

Build one source-independent telemetry contract and make invalid evidence fail before it reaches a model.

A canonical contract is not merely a dataframe schema. It specifies names, types, units, legal ranges, timing semantics, missingness, identifiers and invariants. Source adapters are translators: they may parse and convert, but they should not secretly perform model-specific feature engineering.

A quality gate separates admissible evidence from rejected or quarantined evidence. Some checks are hard errors—missing timestamp, impossible unit, duplicate key. Others are warnings—unusual prevalence or a short gap. The distinction should reflect whether downstream interpretation remains valid.

Contracts also enable replaceability. You can test the entire pipeline with synthetic data, then swap to FastF1 without changing the window builder or model. This is the central architectural move that lets APEX start before F1 25 is available.

Label every block

Canonical contract: One stable internal representation accepted by all downstream components. In APEX: The source-independent telemetry frame.

Adapter: A boundary component translating one external source into the canonical representation. In APEX: FastF1, OpenF1 and future UDP adapters.

Quality gate: Executable rules deciding whether evidence may proceed. In APEX: Range, timing, duplicate, finite-value and schema checks.

Quarantine: Preserve questionable data separately instead of silently dropping or using it. In APEX: Allows investigation and reprocessing after a fix.

Walk one value through it

Suppose the canonical speed unit is m/s and the legal range is 0–105 m/s. A source row says 320 km/h.

Adapter recognizes source unit km/h.

Convert: $320 / 3.6 = 88.89$ m/s.

Validate finite: yes.

Validate range: yes.

Store canonical value with source provenance.

If conversion were delayed until model code, validation would see 320 and reject a valid record—or the range would be widened and allow truly impossible m/s values. Therefore conversion belongs at the adapter boundary.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Rename one source column, duplicate a timestamp, insert infinite speed, and swap throttle/brake. Notice that only some defects are detectable from shape and range.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Loose dataframe convention: choose when exploratory one-off analysis. Avoid when multiple sources, teams or production reuse.

Pydantic/dataclass contract: choose when you need typed configuration and row/batch metadata validation. Avoid when per-element validation becomes a throughput bottleneck; validate batches at boundaries.

Schema registry: choose when many producers and consumers evolve independently. Avoid when a small local project where the operational overhead exceeds value.

APEX implementation map

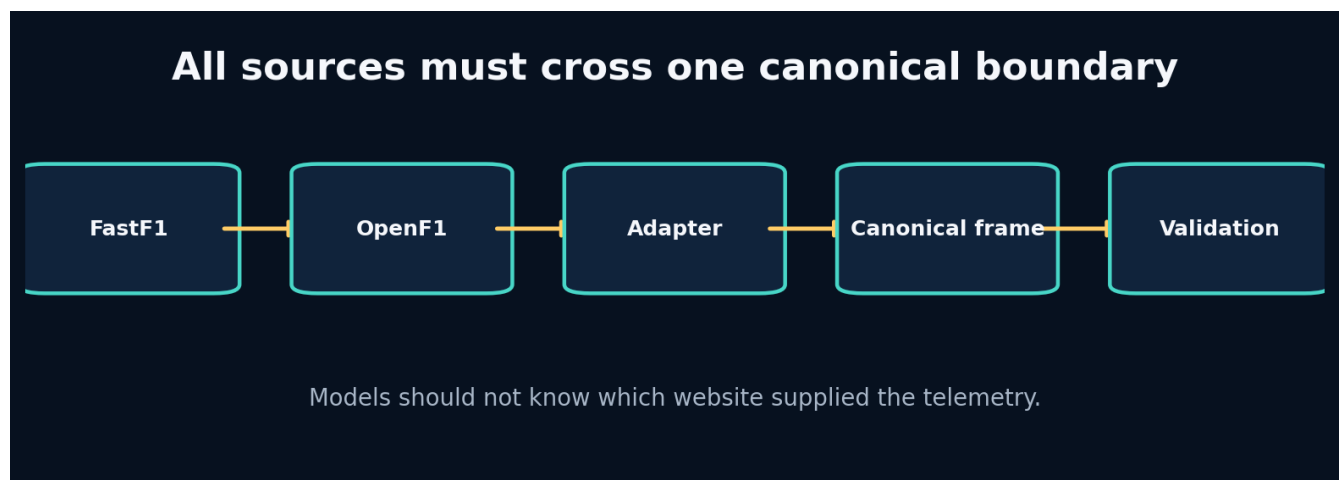
apex_engine/src/apexsim/contracts.py
 apex_engine/src/apexsim/data/validate.py
 apex_engine/src/apexsim/data/fastf1_adapter.py
 apex_engine/src/apexsim/data/openf1_adapter.py

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 6: 6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core



6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core

What the picture is claiming

Understand the complete data path from a public F1 source to canonical telemetry and learn to keep network concerns outside model code.

Network retrieval, caching, parsing, source-specific naming and canonical conversion are separate steps. Keeping them separate lets you replay raw responses when an adapter changes, test conversion without the network, and avoid rate-limit failures inside training.

The safest ingestion process lands raw evidence first. A second deterministic step creates canonical data. This gives you two forensic layers: “what did the source return?” and “how did our code interpret it?” A model run should point to both.

FastF1 and OpenF1 can overlap without being interchangeable. Differences in sampling, provenance and field definitions should be measured rather than assumed away. APEX treats each adapter as an independently tested producer of the canonical contract.

Label every block

Landing zone: Immutable storage of source-native responses before interpretation. In APEX: Cached session/API data used for reproducible adapter tests.

Provenance: Information describing where, when and how a record was obtained. In APEX: Source, event/session identifiers, driver, retrieval time and adapter version.

Idempotent ingestion: Repeating the same request does not create conflicting outputs. In APEX: Deterministic paths and hashes for source snapshots.

Rate limit / cache: External service constraints and local reuse strategy. In APEX: Prevents training from repeatedly hitting public services.

Walk one value through it

Suppose OpenF1 returns UTC timestamps 12:00:00.100, 12:00:00.300, and FastF1 returns session-relative seconds 5.2, 5.4. To combine them, you need a session start reference or a shared event clock. If session start is 11:59:54.900, the UTC records map to 5.2 and 5.4 seconds.

The adapter should preserve:

Source timestamp exactly as received.

Parsed canonical timestamp.

Session and driver identifiers.

Source field names/units or adapter version.

Missingness and interpolation flags.

Without those fields, an apparent 0.2-second alignment error cannot be traced.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Mock a source response where one field changes name, one timestamp is timezone-naive, and one speed column changes unit. Run the adapter tests and identify which change is detected.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

FastF1: choose when python-centric historical session analysis with convenient session abstractions and cached telemetry. Avoid when you require an http-only integration or fields outside its supported representation.

OpenF1: choose when service-style access to historical/live endpoint data and independent stream retrieval. Avoid when you assume every endpoint is synchronized or identical to fastf1.

Both as separate datasets: choose when you want cross-source validation or broader coverage. Avoid when you have not measured semantic and temporal differences.

APEX implementation map

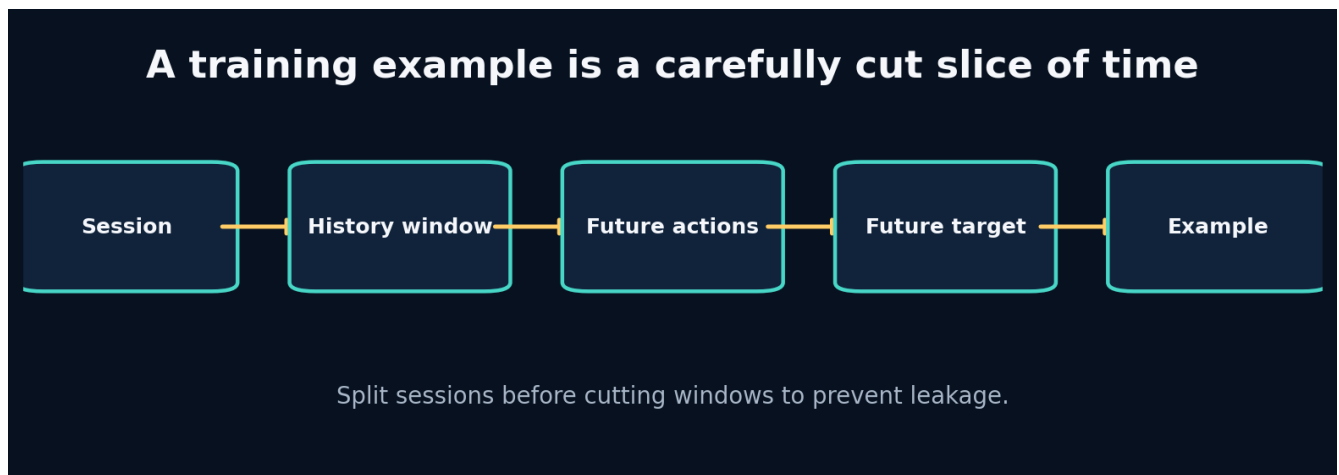
```
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
apex_engine/src/apexsim/cli.py
apex_engine/README.md
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 7: 7. Windows, Session Splits and Normalization Without Leakage



7. Windows, Session Splits and Normalization Without Leakage

What the picture is claiming

Turn canonical sessions into causal training examples while keeping validation and test evidence genuinely independent.

A sequence example has three distinct pieces: history observations, future interventions/context, and future target state. The model must not receive the future target inside its inputs. This sounds obvious, but dataframe slicing and feature lists can quietly duplicate it.

Split units should match the deployment claim. If the engine will simulate a new race session, split by session or event before making windows. A time-based holdout within one session answers a narrower question. Random windows almost always overstate performance when overlap is high.

Normalization must be fitted only on training data. Validation and test are transformed using stored training statistics. Otherwise the mean and variance leak information about future or held-out distributions, and deployment cannot reproduce the transform.

Label every block

History length: How many past frames are visible before a forecast begins. In APEX: Controls memory evidence and input cost.

Forecast horizon: How many future transitions are predicted. In APEX: Defines simulation duration and error accumulation.

Split unit: The independent entity assigned wholly to train, validation or test. In APEX: Usually session/event, not overlapping windows.

Train-only normalizer: Scaling statistics fitted exclusively on training observations. In APEX: Persisted with the model artifact.

Walk one value through it

For length $N=100$, history $H=20$, future $F=5$, stride 1, the first window consumes frames 0–24 and the last begins at 75, so the count is

$$N - H - F + 1 = 100 - 20 - 5 + 1 = 76.$$

Window 1 uses frames 0–24; window 2 uses 1–25. They share 24 of 25 frames. Assigning them to different splits creates almost complete leakage.

For normalization, training speeds [40, 50, 60] have mean 50 and standard deviation about 8.165. A test speed 70 becomes $(70-50)/8.165 \approx 2.45$. Recomputing the mean with test data would shrink that value and leak the test distribution.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Create all windows first and randomly split them. Train a nearest-neighbour or linear baseline. Compare its test score with a session-level split.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Random window split: choose when only for debugging shape/training code, never for a generalization claim with overlap. Avoid when any realistic telemetry evaluation.

Session split: choose when deployment targets unseen sessions under similar tracks/drivers. Avoid when you specifically study later portions of the same session.

Event/track split: choose when you need evidence of transfer to new circuits or events. Avoid when the dataset is too small to support that claim.

APEX implementation map

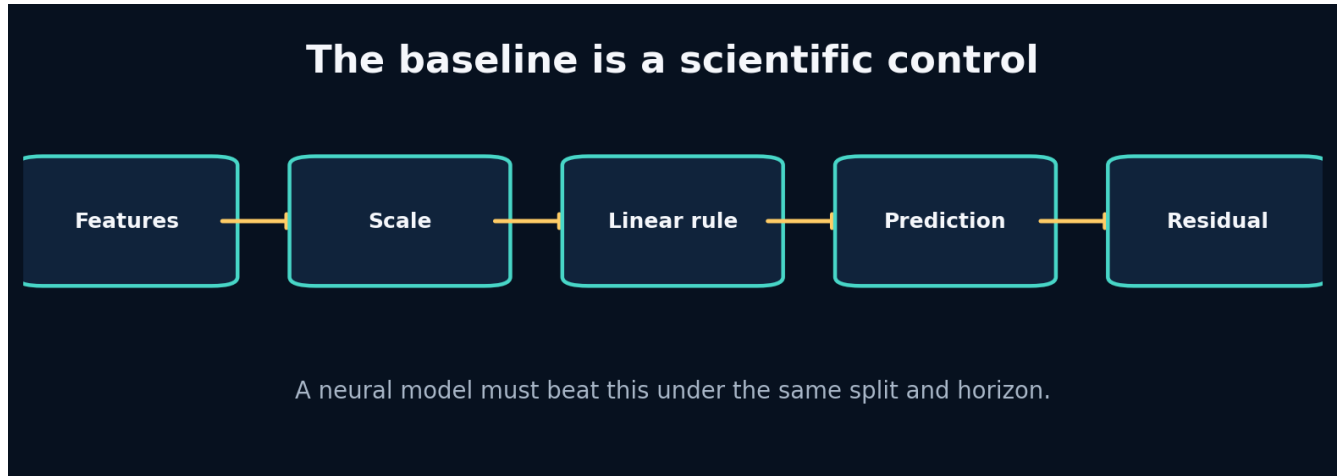
```
apex_engine/src/apexsim/data/windows.py
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/tests/test_windows.py
debugging_cases/04_random_window_split
debugging_cases/06_normalizer_leak
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 8: 8. Statistics, Baselines and Uncertainty as Engineering Controls



8. Statistics, Baselines and Uncertainty as Engineering Controls

What the picture is claiming

Use simple models and residual analysis to determine what the data contains before escalating to deep sequence architectures.

Start with the distribution: means, spreads, missingness, correlations and conditional relationships. Then create baselines that correspond to real hypotheses. Persistence says the world changes slowly. Linear regression says the change is approximately additive in observed features. A physics baseline says known equations explain most of the transition.

Residuals—target minus prediction—show what the baseline failed to explain. Plot residuals against speed, curvature, rain, tyre age and horizon. Structured residuals are a blueprint for the next model. Random-looking residuals near measurement noise may mean a larger model has little room to help.

Uncertainty has several meanings. Aleatoric uncertainty reflects inherently variable outcomes or noisy measurements. Epistemic uncertainty reflects lack of knowledge, often rising outside the training distribution. A point predictor does not automatically communicate either.

Label every block

Persistence baseline: Predict that the next state equals the current state. In APEX: Strong short-horizon speed control.

Residual: Observed target minus model prediction. In APEX: Reveals missing nonlinearities, context or delay.

Calibration: Predicted uncertainty/probability matches empirical frequency or error. In APEX: Needed before presenting confidence in imagined futures.

Distribution shift: Deployment evidence differs from training evidence. In APEX: New tracks, weather, drivers or game telemetry.

Walk one value through it

Suppose true next speeds are [50.2, 60.1, 69.7] and persistence predicts [50.0, 60.0, 70.0].

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

Absolute errors are [0.2, 0.1, 0.3]; MAE is $(0.2+0.1+0.3)/3 = 0.2$ m/s.

A neural model with MAE 0.18 improves by 0.02 m/s, or 10% relative. That gain might matter—or it might vanish on a new session, cost far more compute, and introduce physical violations. Compare by horizon, subgroup, latency and stability before declaring victory.

If residuals under braking average +0.8 m/s (targets are higher than predictions), the baseline is overestimating deceleration. That points to brake scaling, delay or grip context.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;
 observation, action, context, target or identifier;
 raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Add the target next speed as an input feature and watch MAE collapse. Then randomize brake while preserving speed. Compare coefficient signs and residuals.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Persistence: choose when very short horizons and slowly changing states. Avoid when actions or geometry cause rapid transitions.

Linear / Ridge: choose when you need interpretability, speed and a strong control. Avoid when residuals show substantial nonlinear or temporal structure.

Gradient-boosted trees: choose when tabular nonlinearities dominate and fixed windows can summarize history. Avoid when you need smooth autoregressive latent dynamics or differentiable planning.

APEX implementation map

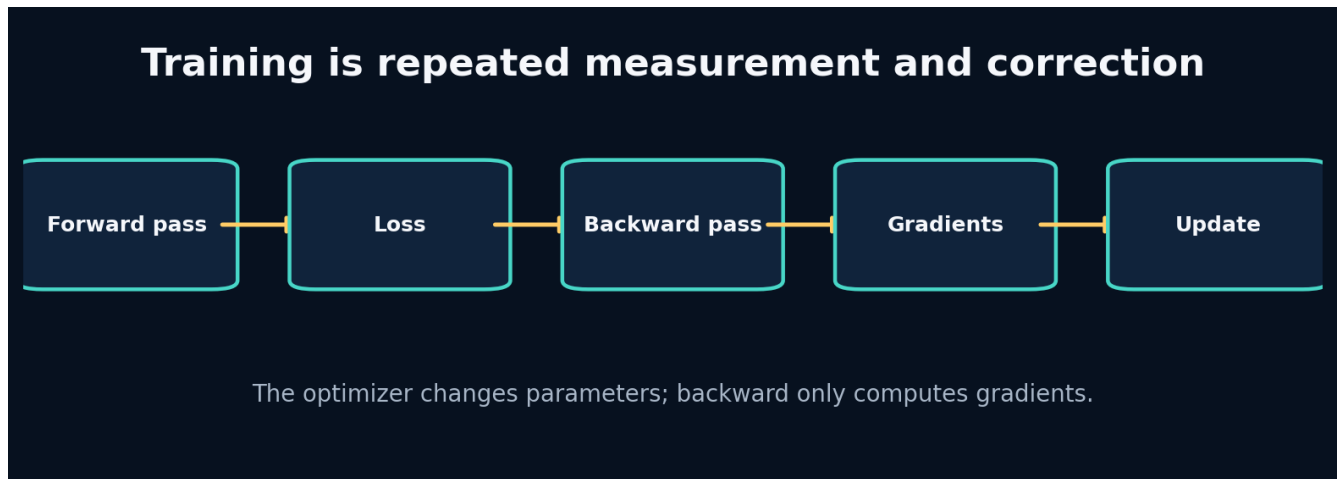
apex_engine/src/apexsim/models/baselines.py
 apex_engine/src/apexsim/evaluation.py
 apex_engine/src/apexsim/pipeline/stages.py
 debugging_cases/03_target_leakage

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 9: 9. Tensors, Autograd and the Training Loop You Can Explain



9. Tensors, Autograd and the Training Loop You Can Explain

What the picture is claiming

Build a PyTorch training loop while understanding every tensor, gradient and parameter update rather than memorizing boilerplate.

A tensor is an array plus shape, dtype, device and—when requested—a connection to a computation graph. The forward pass applies parameterized operations. The loss reduces prediction error to a scalar. Backpropagation computes derivatives of that scalar with respect to leaf parameters and stores them in `.grad`. The optimizer then uses those gradients to update parameters.

Gradients accumulate by default. That enables deliberate gradient accumulation but causes a bug when forgotten. Evaluation must switch behaviour-sensitive layers with `model.eval()` and disable graph construction with `torch.no_grad()`.

The most effective way to learn this is to print parameter values, gradients and loss before and after each line on a tiny problem whose correct mapping is known.

Label every block

Computation graph: The recorded chain of operations connecting inputs, parameters and loss. In APEX: Enables gradient-based training of all world models.

Gradient: Local sensitivity of loss to a parameter. In APEX: Stored in each trainable parameter after backward.

Optimizer state: Extra memory used to turn gradients into updates. In APEX: Adam moments are checkpointed with the model during resumable training.

Train/eval mode: Switch for layers such as dropout and batch normalization. In APEX: Ensures deterministic, correctly normalized validation and rollout.

Walk one value through it

For a one-parameter model $\hat{y} = w \times x$ with $x=2$, $y=6$, $w=1$, and squared loss $((y-\hat{y})^2)$:

Prediction: $1 \times 2 = 2$.

Error: $2 - 6 = -4$.

Loss: 16.

Gradient: $dL/dw = 2(wx-y)x = 2(-4)(2) = -16$.

With learning rate 0.1, SGD update: $w \leftarrow 1 - 0.1(-16) = 2.6$.

New prediction: 5.2; new loss: 0.64.

Backward computes -16 ; the optimizer performs the update to 2.6.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Remove `zero_grad`, increase learning rate by 100×, and accidentally shape targets as [B] while predictions are [B,1]. Observe accumulation, divergence and broadcasting.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

SGD: choose when you want transparent updates and can tune schedules carefully. Avoid when sparse/noisy gradients need faster adaptive progress.

Adam/AdamW: choose when a strong default for deep sequence models and fast iteration. Avoid when you assume it removes the need for learning-rate and regularization experiments.

Gradient clipping: choose when recurrent or long-horizon training shows occasional exploding norms. Avoid when it hides systematically bad scaling or unstable architecture.

APEX implementation map

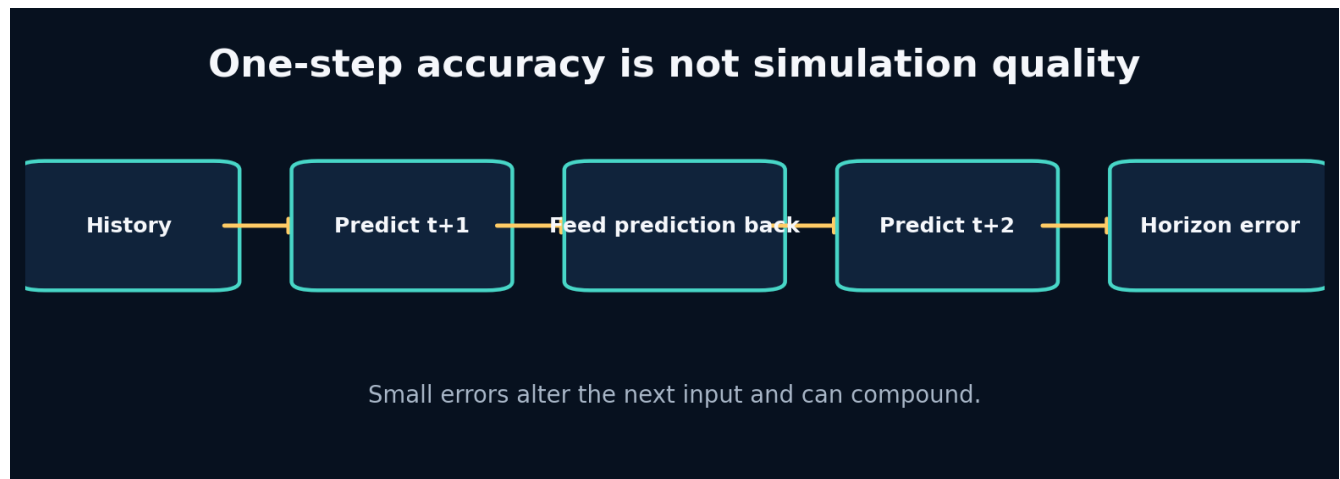
apex_engine/src/apexsim/training.py
 apex_engine/src/apexsim/models/gru_world_model.py
 debugging_cases/08_missing_eval_mode
 debugging_cases/09_gradient_accumulation

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 10: 10. Transition Models and Why Rollouts Fail



10. Transition Models and Why Rollouts Fail

What the picture is claiming

Build a one-step neural transition, convert it into an autoregressive simulator, and measure error compounding honestly.

One-step supervised training samples inputs from real data. Autoregressive inference samples later inputs from the model. This train–inference distribution shift is exposure bias. Even unbiased local noise can alter future controls’ effects because the state enters nonlinear dynamics.

A rollout function must specify which future variables are known and which are predicted. In APEX, future controls/context can be provided by a scenario, while future state is generated. After each step, predicted state replaces the state portion of the next input; controls remain externally supplied.

Evaluation should report error by horizon rather than averaging all future steps. The shape of the curve distinguishes immediate bias, unstable compounding and long-memory failure.

Label every block

Teacher forcing: Train each transition using the true previous state. In APEX: Stable supervised training for GRU/RSSM components.

Autoregressive rollout: Feed predicted state back to generate later states. In APEX: The actual simulation mode.

Exposure bias: The model trains on true states but runs on its own imperfect states. In APEX: A central source of horizon degradation.

Horizon curve: Error reported separately at each future step. In APEX: Primary evidence for useful simulation duration.

Walk one value through it

With additive bias $e_{t+1}=e_t+0.1$, starting at zero, error after 20 steps is 2.0 m/s.

With amplification $e_{t+1}=1.05e_t+0.1$,

$$e_{20}=0.1 \frac{1.05^{20}-1}{1.05-1} \approx 3.31 \text{ m/s}.$$

The one-step bias is still only 0.1, but the system dynamics amplify it. This is why one-step MAE cannot certify a simulator.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

During rollout, accidentally keep feeding the last real state instead of each prediction. The multi-step score may look surprisingly strong because you are not actually simulating.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

One-step training: choose when a stable starting point with abundant transition examples. Avoid when you treat its metric as rollout certification.

Multi-step loss: choose when long-horizon accuracy matters and compute allows unrolled training. Avoid when early training is unstable or the horizon curriculum is not controlled.

Scheduled sampling: choose when you want gradual exposure to model-generated states. Avoid when you assume it guarantees consistent probabilistic learning.

APEX implementation map

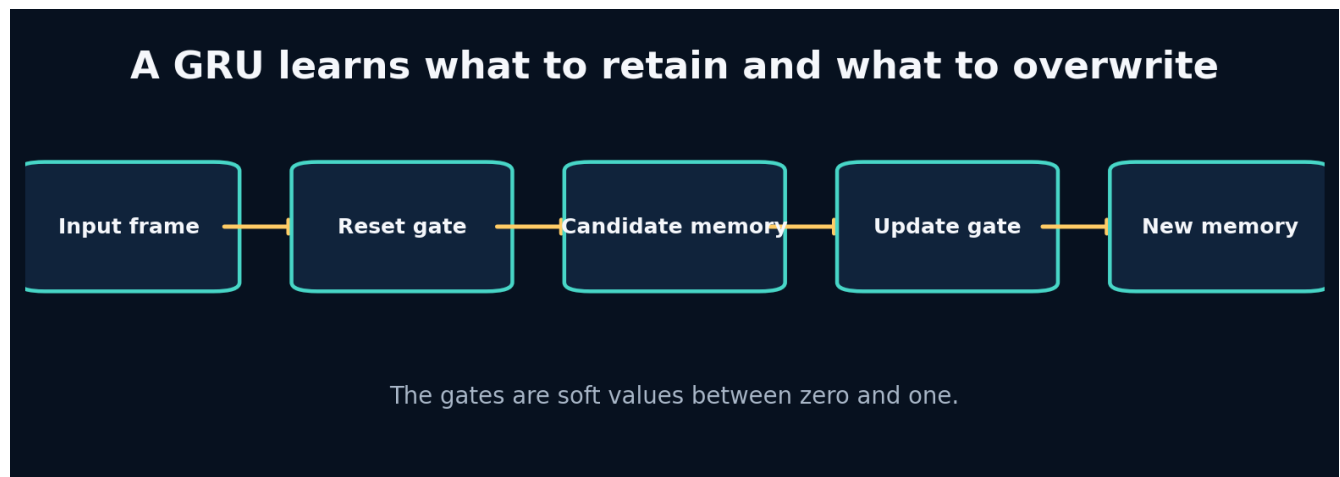
```
projects/04_mlp_transition/main.py
apex_engine/src/apexsim/simulation.py
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/models/gru_world_model.py
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 11: 11. Recurrent Memory and the GRU From the Inside



11. Recurrent Memory and the GRU From the Inside

What the picture is claiming

Earn recurrent memory by exposing the limits of fixed-frame transitions, then inspect how GRU gates update hidden state.

A recurrent neural network updates hidden memory as each frame arrives. The final hidden state is not a literal copy of history; it is a learned summary optimized for the training objective. A vanilla recurrent cell can struggle when useful information must survive many updates because gradients repeatedly pass through the same transition.

A GRU adds gates. The update gate decides how much old memory to keep versus replace. The reset gate controls how much previous memory influences the candidate update. These are soft, feature-wise decisions, not binary switches. The same hidden dimension can retain long-term tyre information while rapidly updating braking state.

Using nn.GRU does not guarantee meaningful memory. You still need enough history, correct ordering, no hidden-state leakage across independent sequences, and evaluation that tests whether memory improves the target horizon.

Label every block

Hidden state: A learned vector carried across sequence steps. In APEX: Compressed recent driving and unobserved dynamical context.

Update gate: Controls interpolation between old memory and candidate memory. In APEX: Lets some information persist while other information changes.

Reset gate: Controls how previous memory contributes to the candidate update. In APEX: Allows rapid forgetting when current evidence changes regime.

Sequence boundary: The point where hidden memory must be reset or intentionally transferred. In APEX: Driver/session windows cannot share hidden state accidentally.

Walk one value through it

A simplified update is

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

If old memory is $h=0.8$, candidate memory is -0.2 :

With update gate $z=0.1$: $h_t=0.9(0.8)+0.1(-0.2)=0.70$. Most old memory survives.

With $z=0.9$: $h_t=0.1(0.8)+0.9(-0.2)=-0.10$. The new evidence largely overwrites memory.

This interpolation makes a gate interpretable at the mechanism level, although individual learned dimensions may not map cleanly to human concepts.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Reuse the hidden state from one session as the initial state for an unrelated session. Also shuffle time within each sequence while preserving rows.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Vanilla RNN: choose when tiny educational tasks and very short dependencies. Avoid when long/noisy telemetry histories.

GRU: choose when strong compact recurrent baseline with fewer parameters than lstm. Avoid when you have evidence that longer structured memory or parallel training is required.

LSTM: choose when you value an explicit cell state and gates and have sufficient compute/data. Avoid when extra complexity provides no measured gain.

APEX implementation map

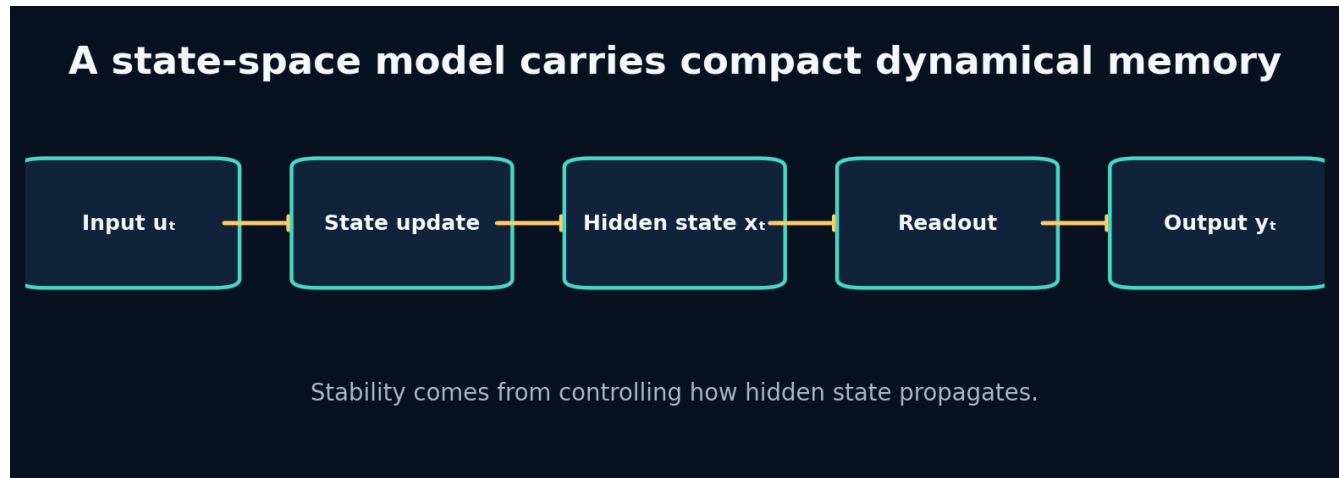
```
labs/11_rnn_from_scratch/solution.py
labs/12_gru_gates/solution.py
projects/06_gru_forecaster/main.py
apex_engine/src/apexsim/models/gru_world_model.py
debugging_cases/07_hidden_state_reuse
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 12: 12. Classical State-Space Models: Dynamics as Memory



12. Classical State-Space Models: Dynamics as Memory

What the picture is claiming

Understand state-space recurrence mathematically, inspect stability, and connect classical dynamical systems to modern sequence models.

A discrete linear state-space model is

$$x_t = A x_{t-1} + B u_t, y_t = C x_t + D u_t.$$

The hidden state x is memory; A controls how it persists and mixes; B writes input; C reads output. Repeated powers of A determine whether past information decays, persists, oscillates or explodes.

Many modern SSMs begin from continuous-time dynamics and discretize them for sampled sequences. This connects model behaviour to the time step. A change from 5 Hz to 20 Hz should not be treated as an arbitrary tensor resize when the transition is supposed to represent time.

Linear recurrence alone cannot model all nonlinear racing dynamics, but it exposes memory timescales and stability more directly than a generic black box.

Label every block

State matrix A : Controls hidden-state propagation between steps. In APEX: Determines memory decay, coupling and stability.

Input matrix B : Controls how current telemetry writes into memory. In APEX: Maps actions/context into dynamical state.

Readout C : Maps hidden memory to predicted output. In APEX: Produces next-state features from SSM memory.

Spectral radius: Largest absolute eigenvalue of A ; a key discrete stability indicator. In APEX: Values above one can amplify state without bound.

Walk one value through it

For $x_t = A x_{t-1} + B u_t$ after input stops:

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

If $a=1.05$, then after 20 steps x is multiplied by $1.05^{20} \approx 2.65$.

If $a=0.95$, it is multiplied by $0.95^{20} \approx 0.358$.

If $a=1$, memory persists exactly.

If $a=-0.95$, memory alternates sign while decaying.

For a matrix, eigenvalues play the role of these scalar factors along different modes. Stability does not guarantee usefulness: a transition that decays too quickly forgets everything.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;
 observation, action, context, target or identifier;
 raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Change a diagonal value in A to 1.2 and roll for 100 zero-input steps. Then set both diagonal values to 0.1.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Fixed linear SSM: choose when system identification, interpretability and approximately linear dynamics. Avoid when strong context-dependent nonlinear transitions dominate.

Nonlinear readout: choose when memory dynamics are simple but state-to-output mapping is nonlinear. Avoid when input must alter the memory law itself.

Input-dependent SSM: choose when different telemetry regimes require selective writing/forgetting. Avoid when a fixed transition already works and simplicity matters.

APEX implementation map

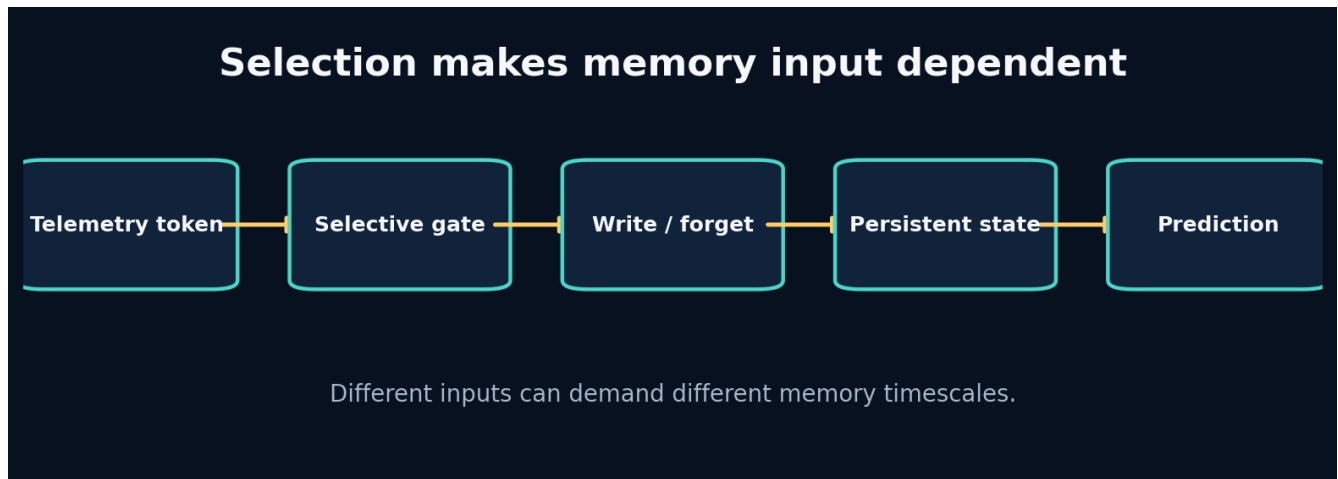
labs/13_linear_ssm/solution.py
 projects/07_selective_ssm/main.py
 apex_engine/src/apexsim/models/ssm_world_model.py
 debugging_cases/10_exploding_ssm

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 13: 13. Selective State-Space Models and What Mamba Changes



13. Selective State-Space Models and What Mamba Changes

What the picture is claiming

Build an input-dependent memory cell, understand selection, and separate the core Mamba idea from implementation hype.

Selection means model parameters or gates depend on the current input. Instead of one fixed A and B, the sequence determines how strongly memory persists and what content is written. This gives content-aware recurrence while preserving a state-space perspective.

Mamba combines selective state-space dynamics with hardware-aware computation so long sequences can be processed efficiently. The important conceptual move for this curriculum is not “replace everything with Mamba.” It is: **make memory propagation conditional on the token when fixed dynamics are insufficient.**

A small selective cell can be written as $h_t = a(x_t) \odot h_{t-1} + (1 - a(x_t)) \odot b(x_t)$. It resembles a gated recurrent update. The difference between educational cell and research architecture must remain explicit.

Label every block

Selection: Current input changes how memory is written, retained or read. In APEX: Braking, weather or geometry can trigger different memory behaviour.

Input-dependent decay: The retention factor is computed from the current token. In APEX: Adaptive timescales across race regimes.

Scan: Efficiently apply a recurrence across sequence positions. In APEX: Important for long histories and GPU throughput.

Mamba block: A complete architecture combining projections, selective SSM computation and implementation details. In APEX: A future challenger; not identical to the toy selective cell.

Walk one value through it

Let old memory be $h=0.9$.

Ordinary input produces retention $a=0.95$ and candidate $b=0.2$: $h'=0.95(0.9)+0.05(0.2)=0.865$.

Critical braking input produces retention $a=0.2$ and candidate $b=-0.8$: $h'=0.2(0.9)+0.8(-0.8)=-0.46$.

The same memory dimension changes slowly during routine input and rapidly under a meaningful event. Selection is useful only if training learns gates that correspond to predictive needs rather than arbitrary noise.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;
 observation, action, context, target or identifier;
 raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Remove the sigmoid/bound and allow retention greater than one. Then initialize decay so gates saturate near zero or one.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Toy selective cell: choose when learning the mechanism and testing whether input-dependent memory helps. Avoid when claiming mamba-equivalent speed or quality.

Official Mamba implementation: choose when long sequences and benchmarks justify the dependency and hardware path. Avoid when your data are small, horizons short, or environment support is fragile.

GRU: choose when a stable gated recurrent baseline is sufficient. Avoid when measured long-context/throughput limitations motivate another architecture.

APEX implementation map

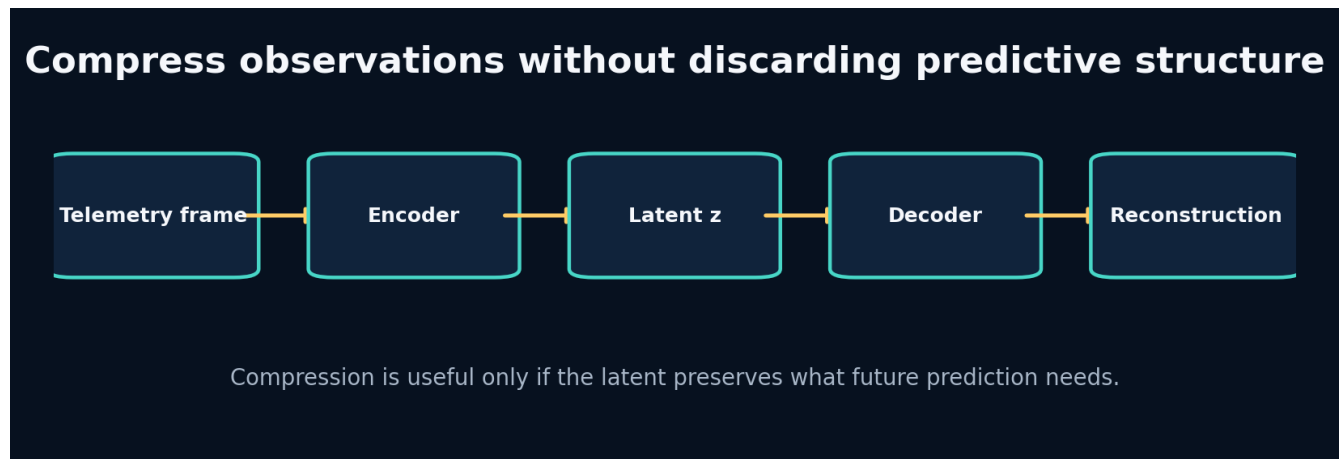
labs/14_selective_ssm/solution.py
 projects/07_selective_ssm/main.py
 apex_engine/src/apexsim/models/ssm_world_model.py
 apex_engine/configs/ssm_fast.yaml

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 14: 14. Autoencoders and Variational Latent State



14. Autoencoders and Variational Latent State

What the picture is claiming

Learn representation compression by building it, then add probabilistic latent variables without confusing reconstruction with useful world modelling.

An autoencoder learns an encoder $z=f(x)$ and decoder $\hat{x}=g(z)$ by minimizing reconstruction error. The bottleneck pressures the latent to preserve common information. Nothing requires each dimension to align with a human variable, and high-variance nuisance features may dominate.

A variational autoencoder predicts a distribution—usually mean and log variance—rather than one code. Sampling uses $z = \mu + \sigma \odot \epsilon$, which keeps the path differentiable with respect to μ and σ . A KL term regularizes the posterior toward a prior, enabling sampling but creating a tradeoff: too weak and the latent is irregular; too strong and it can ignore observations.

For world models, the latent should preserve information useful for dynamics and reward, not only instantaneous reconstruction. Temporal prediction losses or task probes test this.

Label every block

Encoder: Maps a high-dimensional observation into a compact representation. In APEX: Compresses telemetry/history into latent state.

Decoder: Reconstructs or predicts observable state from latent representation. In APEX: Produces interpretable future telemetry.

Reparameterization: Express stochastic sampling as differentiable transformation of parameter-free noise. In APEX: Allows RSSM/VAE posterior training.

KL divergence: Penalty measuring difference between posterior and prior distributions. In APEX: Regularizes stochastic latent state and aligns imagination with inference.

Walk one value through it

For one latent dimension with $\mu=0.2$, $\log \text{variance}=-1$:

Variance: $\exp(-1)=0.3679$.

Standard deviation: $\exp(-0.5)=0.6065$.

If sampled noise $\epsilon=1.0$, then $z=0.2+0.6065=0.8065$.

KL to standard normal is $-0.5(1 + \log \text{var} - \mu^2 - \exp(\log \text{var}))$.

Substitute: $-0.5(1 - 1 - 0.04 - 0.3679)=0.20395$.

The KL is zero only when posterior mean is zero and variance is one. Driving every posterior to that point would erase observation information.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Set KL weight extremely high and inspect whether reconstructions become generic. Then set it to zero and sample from the prior at inference.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Deterministic autoencoder: choose when compression and reconstruction matter; uncertainty is not required. Avoid when you need coherent sampling or probabilistic belief.

VAE: choose when a smooth sampleable latent distribution is useful. Avoid when kl tradeoffs add complexity without serving the downstream task.

Predictive encoder: choose when future-relevant structure matters more than reconstructing every input detail. Avoid when you still require high-fidelity observation generation.

APEX implementation map

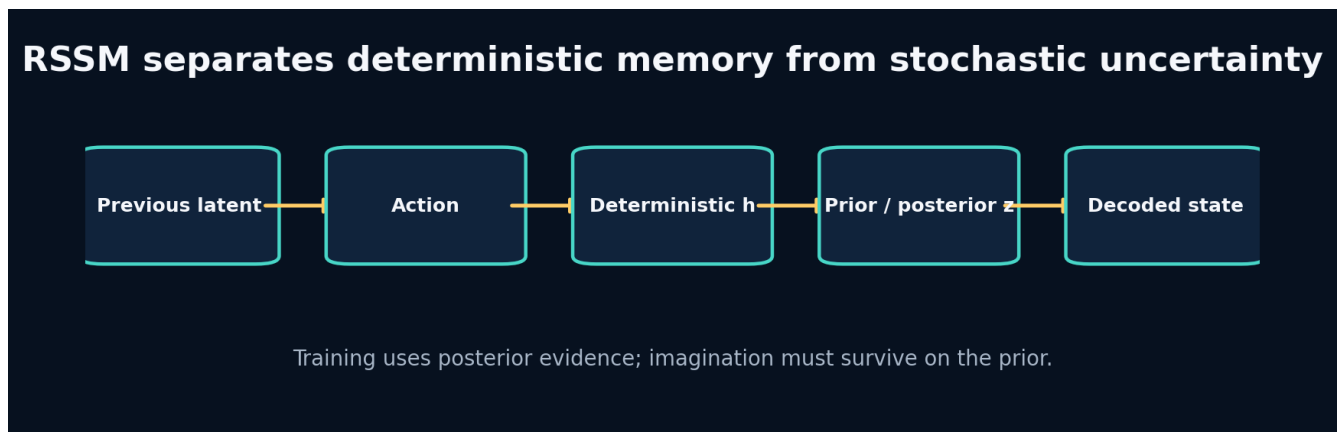
labs/15_autoencoder/solution.py
 labs/16_vae_reparameterization/solution.py
 apex_engine/src/apexsim/models/rssm.py
 debugging_cases/11_kl_collapse

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 15: 15. RSSMs and Dreamer: Learning to Imagine Without Observations



15. RSSMs and Dreamer: Learning to Imagine Without Observations

What the picture is claiming

Implement the logic of a recurrent state-space model, distinguish posterior training from prior imagination, and understand Dreamer as a system rather than a buzzword.

An RSSM commonly has deterministic state h_t and stochastic state z_t . The transition updates h_t from previous latent/action. The prior predicts a distribution over z_t from h_t . During training, an encoder provides observation evidence and a posterior refines that distribution. The decoder predicts observation/state from h_t, z_t .

The KL term teaches the prior to resemble the posterior enough that prior-only imagination remains useful. Reconstruction or prediction terms teach the latent to represent evidence. These objectives can conflict. Posterior collapse, overconfident priors and compounding uncertainty are practical failures, not footnotes.

Dreamer adds reward/continuation models and learns actor/critic behaviour from trajectories imagined by the world model. Before trusting control, APEX must establish that interventions produce realistic, calibrated rollouts.

Label every block

Deterministic state h: Recurrent memory summarizing past latent/action sequence. In APEX: Carries stable temporal context.

Prior: Latent distribution predicted without current observation. In APEX: Used during imagination and planning.

Posterior: Latent distribution conditioned on current observation evidence. In APEX: Used during training/state inference.

Imagination: Rollout using learned prior dynamics without future observations. In APEX: Generates counterfactual telemetry trajectories.

Walk one value through it

Suppose posterior is $N(\mu_q=1, \sigma_q=0.5)$ and prior is $N(\mu_p=0, \sigma_p=1)$. The one-dimensional Gaussian KL $KL(q || p)$ is

$$\log(\sigma_p/\sigma_q) + \frac{\sigma_q^2 + (\mu_q - \mu_p)^2}{2\sigma_p^2} - \frac{1}{2}.$$

Substitute:

$\log(1/0.5)=0.693$

numerator $0.25 + 1 = 1.25$

divide by 2 gives 0.625

subtract 0.5

total 0.818

The gap signals that the prior cannot yet reproduce the posterior belief. Reducing it by making both distributions uninformative would also be bad; reconstruction/prediction must remain strong.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Use posterior samples during test rollout. Then set KL weight to zero and compare prior imagination with posterior reconstruction.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Deterministic GRU world model: choose when one likely future and stable v1 forecasting are sufficient. Avoid when uncertainty/multi-modal futures are central.

RSSM: choose when you need a learned belief state and prior imagination. Avoid when data/budget cannot support stable latent training or benefits are unmeasured.

Ensemble deterministic models: choose when you want practical epistemic uncertainty with simple training. Avoid when compute or storage prevents multiple models.

APEX implementation map

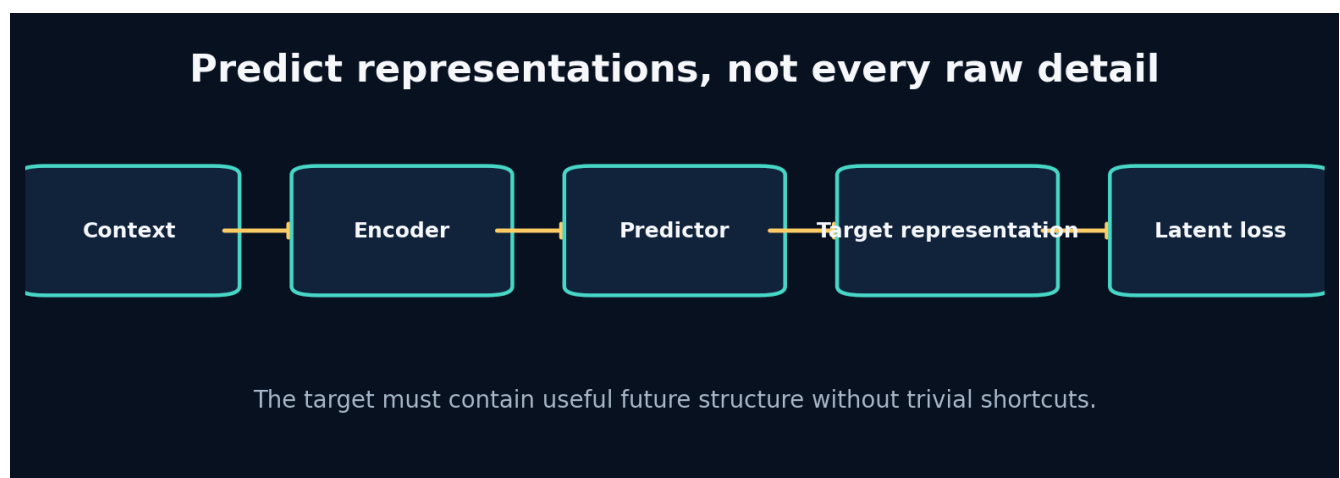
```
projects/08_rssm_imagination/main.py
apex_engine/src/apexsim/models/rssm.py
apex_engine/configs/rssm_fast.yaml
debugging_cases/11_kl_collapse
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 16: 16. JEPA, LeWorldModel and Fine-Tuning Existing Representations



16. JEPA, LeWorldModel and Fine-Tuning Existing Representations

What the picture is claiming

Understand predictive representation learning, decide when not to reconstruct raw observations, and build a disciplined fine-tuning strategy.

Joint-embedding predictive architectures encode context and target views, then train a predictor to infer the target representation from context. The target encoder defines what counts as meaningful similarity. Because the loss is in representation space, the model can ignore some raw variation—if the representation has learned to ignore it.

This creates a new failure possibility: representation collapse or shortcuts. Stop-gradient/target-encoder design, masking strategy and representation diagnostics matter. In telemetry, a future-latent target could emphasize dynamics while de-emphasizing sensor noise, but it might also discard rare safety-critical events.

Fine-tuning an existing model makes sense when its input semantics, sequence structure and learned invariances transfer. Adaptation choices range from a frozen encoder plus new head, through parameter-efficient adapters, to full fine-tuning. Start with the least invasive option that can represent the domain shift, and compare against training a small model from scratch.

Label every block

Representation target: The encoded feature vector the predictor must infer. In APEX: Potential future-dynamics target instead of raw telemetry.

Stop gradient / target encoder: Prevents both sides from trivially co-adapting in unstable ways. In APEX: Part of a non-collapsing predictive objective.

Linear probe: A simple supervised head used to test what information a representation contains. In APEX: Measures speed, curvature, tyre or event information in latent state.

Fine-tuning: Adapt pretrained parameters to a new domain/task. In APEX: Possible when a relevant sequence/world model checkpoint exists.

Walk one value through it

Compare two losses for target vector $y=[10.0, 0.02]$ where the second feature is pure noise. Prediction A is $[9.8, 0.50]$; prediction B is $[9.8, -0.40]$. Raw MSE differs because of noise.

If a target encoder maps both raw targets to a dynamics representation $[\text{speed_bin}=10, \text{regime}=\text{straight}]$, their representation loss can be identical. This is desirable only if the discarded noise truly has no predictive or decision value.

For fine-tuning, suppose a pretrained encoder has one million parameters. A frozen-head experiment trains 5,000 parameters; an adapter trains 50,000; full tuning changes all million. These are nested hypotheses about how much the representation must change.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;
 observation, action, context, target or identifier;
 raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Train both context and target encoders with no asymmetry or variance checks until they output nearly constant vectors. Observe low latent loss and useless probes.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Raw reconstruction: choose when exact observation generation or interpretable state recovery matters. Avoid when unpredictable detail dominates and decisions depend on abstract dynamics.

Predictive latent loss: choose when future-relevant representation is the main goal. Avoid when you cannot validate what information the latent discarded.

Frozen pretrained encoder: choose when source and target semantics align and data are limited. Avoid when the representation lacks essential f1 variables.

APEX implementation map

```

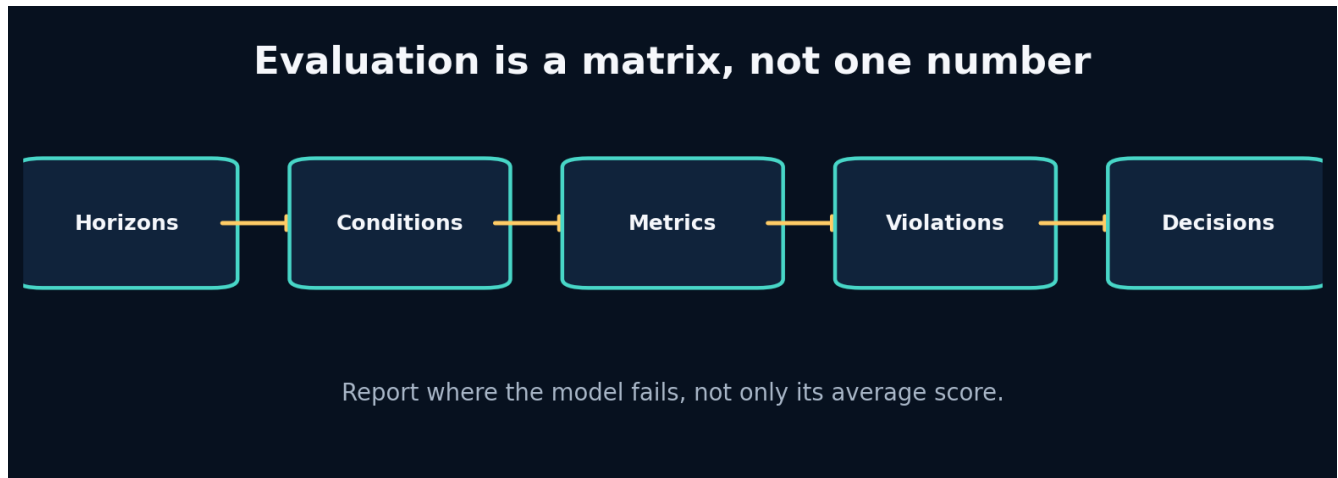
labs/15_autoencoder/solution.py
apex_engine/src/apexsim/models/
apex_engine/src/apexsim/evaluation.py
research_reading/
  
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 17: 17. Evaluation, Ablation and Out-of-Distribution Testing



17. Evaluation, Ablation and Out-of-Distribution Testing

What the picture is claiming

Build an evaluation matrix that reveals where a world model works, why it works, and when it should refuse to be trusted.

Evaluation begins with the decision. If a race engineer uses the simulator to compare a six-second throttle scenario, horizon-six causal sensitivity and physical validity matter more than one-step average fit. Metrics are measurements of requirements, not a scoreboard detached from use.

Ablation removes or changes one information source while holding other factors fixed. It answers causal questions about the implementation: Does weather context help? Does VH-like second channel—here, perhaps curvature or tyre age—matter? Randomly changing many settings answers nothing cleanly.

Out-of-distribution tests define shifts deliberately: unseen track, heavier rain, longer tyre age, unusual controls, different sample rate. Failure under OOD is expected; the goal is to detect and quantify it, then decide whether to abstain, retrain or constrain use.

Label every block

Horizon metric: Error measured separately at each future step. In APEX: Defines useful simulation duration.

Guardrail metric: A metric that must remain within a safety/validity limit even if primary error improves. In APEX: Negative speed, progress bounds and control sensitivity.

Ablation: A controlled removal or replacement testing one hypothesis. In APEX: State-only, no weather, no geometry, model-family comparisons.

OOD test: Evaluation under a deliberately shifted distribution. In APEX: Unseen circuits, weather or control regimes.

Walk one value through it

Suppose Model A and B have:

Metric

A

B

1-step MAE

0.4

0.5

20-step MAE

8.0

5.0

Negative-speed rate

2%

0%

Scenario latency

30 ms

12 ms

If the UI serves 20-step scenarios, B is stronger despite worse one-step fit. If only one-step filtering is needed, A may be acceptable if violations are constrained. The “best model” is a vector of requirement tradeoffs plus thresholds, not one scalar.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Average all horizons and all conditions into one MAE. Then remove weather from inputs but accidentally change random seed and training epochs too.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

MAE/RMSE: choose when continuous state error in interpretable units. Avoid when they are the only evidence for a simulator.

Horizon curves: choose when autoregressive use or planning matters. Avoid when never; at least inspect them for rollouts.

Physical penalties: choose when known invalid regions can be expressed and monitored. Avoid when they replace empirical outcome evaluation.

APEX implementation map

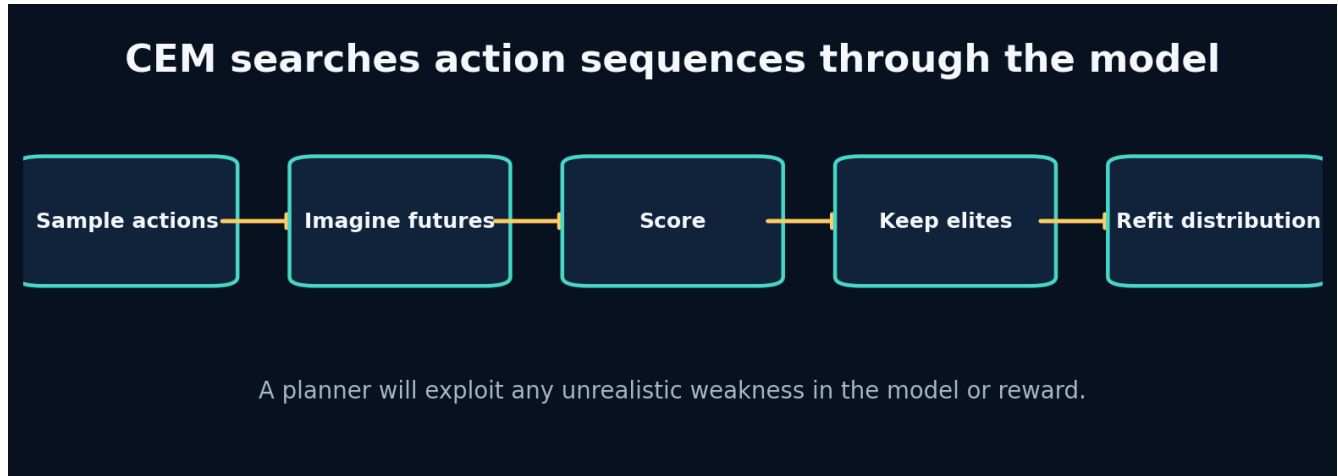
apex_engine/src/apexsim/evaluation.py
 apex_engine/src/apexsim/pipeline/stages.py
 apex_engine/artifacts/runs/
 field_workbook/

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 18: 18. Planning With MPC and the Cross-Entropy Method



18. Planning With MPC and the Cross-Entropy Method

What the picture is claiming

Use a world model to compare action sequences, understand CEM search, and learn why planners exploit model and reward defects.

Model predictive control repeatedly plans over a finite horizon, executes only the first action, observes the new state, and replans. This feedback limits drift compared with committing to a long open-loop sequence.

The cross-entropy method maintains a distribution over action sequences. It samples candidates, rolls them through the model, keeps an elite fraction, and refits mean/variance toward those elites. Repetition concentrates search around high-scoring actions.

Planner quality is bounded by world-model validity and reward completeness. Optimization pressure finds loopholes humans do not notice in average evaluation. Therefore planning tests should include adversarial actions, uncertainty penalties, constraints and real-environment verification before control is trusted.

Label every block

Model predictive control: Plan over a horizon, execute a short prefix, observe, and replan. In APEX: Future scenario/strategy layer after dynamics validation.

CEM: Sampling optimizer that iteratively refits an action distribution to elite candidates. In APEX: Simple derivative-free planner through imagined telemetry.

Reward model: Function scoring predicted trajectories for the decision objective. In APEX: Could combine progress, stability, tyre use and constraint penalties.

Model exploitation: Planner finds actions that score well because the learned model is wrong. In APEX: A major risk under unusual controls.

Walk one value through it

Suppose CEM samples 500 six-step throttle sequences. It scores final-speed error to a target plus smoothness. It keeps 50 elites (top 10%). New mean and standard deviation are the elite sample statistics.

If initial mean is 0.5 and standard deviation 0.3, one iteration may shift the early-step means toward 0.8 where acceleration helps. After several iterations variance shrinks. Too-rapid variance collapse can trap search; a minimum standard deviation preserves exploration.

MPC then applies only the first action, receives new evidence and replans. The remaining five actions are proposals, not a fixed command commitment.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;
 observation, action, context, target or identifier;
 raw or normalized units;
 deterministic value or distribution parameters/sample;
 training-time evidence or deployment-time evidence;
 in-memory tensor or durable artifact reference.

Break one arrow

Remove the smoothness term and allow throttle beyond one. Add a model region where extreme throttle incorrectly increases speed without penalty. Watch CEM discover it.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Grid search: choose when action space and horizon are tiny. Avoid when combinatorial sequences.

CEM: choose when continuous bounded actions, no gradients required and batch rollout is cheap. Avoid when model evaluations are extremely expensive or multi-modal search collapses.

Gradient planning: choose when world model/reward are differentiable and smooth. Avoid when discrete actions, poor local optima or unstable gradients.

APEX implementation map

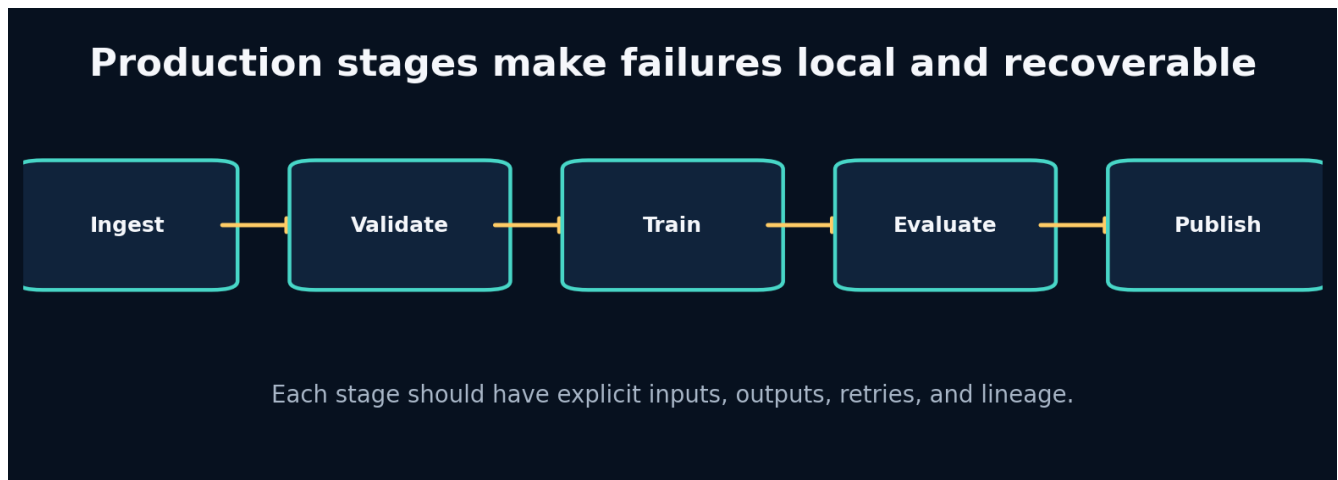
labs/18_cem_planning/solution.py
 projects/09_latent_mpc/main.py
 apex_engine/src/apexsim/simulation.py
 debugging_cases/12_planner_exploitation

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 19: 19. Production Pipelines, Airflow, Registry and Monitoring



19. Production Pipelines, Airflow, Registry and Monitoring

What the picture is claiming

Refactor experimental code into idempotent stages with observable lineage, then understand when orchestration becomes necessary.

A stage has explicit inputs, outputs, configuration, failure modes and idempotency policy. Ingestion writes canonical data; validation writes a report; training writes a checkpoint; evaluation writes metrics; publication writes UI/API products. Each artifact is immutable within a run and linked by identifiers/hashes.

Airflow schedules and tracks task dependencies, retries and backfills. Business logic should remain in ordinary tested Python functions so the same stage runs locally, in tests and under orchestration. Passing large arrays through orchestration metadata is a smell; pass durable artifact references.

A registry records runs, configurations, dataset versions, metrics and status. Monitoring then covers pipeline health, data drift, model behaviour and product delivery. Logging without stable run IDs and structured fields is not lineage.

Label every block

Idempotency: Repeating a stage with the same inputs yields the same result or safely reuses it. In APEX: Enables retries and backfills.

Artifact lineage: Trace which data, code and configuration produced an output. In APEX: Every UI prediction links to model run and dataset.

Orchestrator: Coordinates when stages run and how failures/retries are managed. In APEX: Airflow DAG wraps tested stage functions.

Registry: Queryable record of runs, statuses, metrics and artifact locations. In APEX: SQLite V1, expandable to a service/database.

Walk one value through it

Assume training checkpoint hash abc123 exists and evaluation failed because disk was temporarily full.

A safe retry checks:

Same run ID and immutable configuration hash.

Same dataset/normalizer hashes.

Checkpoint exists, is complete and matches metadata.

Training stage status is succeeded.

Evaluation output is absent or marked failed.

Then retry evaluation only. If configuration changed, create a new run rather than silently reusing the checkpoint. If checkpoint integrity is uncertain, retrain in a new or repaired stage path.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

persistent state or temporary activation;

observation, action, context, target or identifier;

raw or normalized units;

deterministic value or distribution parameters/sample;

training-time evidence or deployment-time evidence;

in-memory tensor or durable artifact reference.

Break one arrow

Write every run to latest/, pass a large tensor through Airflow XCom, and combine ingestion/training/publication in one task. Simulate a publication failure.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Local Python runner: choose when development, ci and small reproducible workflows. Avoid when complex schedules, backfills and multi-worker operations.

Airflow: choose when batch dependencies, retries, schedules and historical backfills matter. Avoid when you need low-latency event streaming or have only one trivial script.

Experiment tracker: choose when comparing many model runs and artifacts. Avoid when you assume it replaces data/version contracts and orchestration.

APEX implementation map

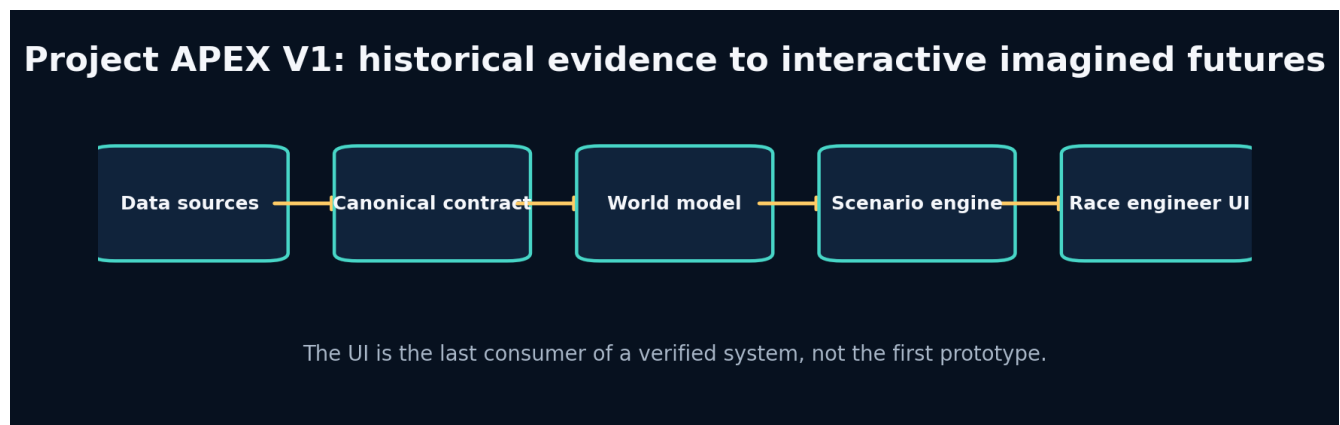
```
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/src/apexsim/pipeline/runner.py
apex_engine/src/apexsim/registry.py
apex_engine/dags/apexsim_dag.py
debugging_cases/13_artifact_overwrite
debugging_cases/14_airflow_payload
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Plate 20: 20. Build Project APEX V1 End to End—and Then Outgrow It



20. Build Project APEX V1 End to End—and Then Outgrow It

What the picture is claiming

Assemble the complete system, run a verified world-model simulation, use the UI responsibly, and define the research path toward F1 25 and an original architecture.

APEX V1 is a historical telemetry world-simulation engine. It does not claim complete car physics or autonomous strategy. It answers a narrower, testable question: given recent canonical telemetry and an explicit sequence of future controls/context, can a learned transition model generate plausible short-horizon future state?

The system begins with synthetic data so every causal rule is inspectable and offline tests always run. It accepts FastF1/OpenF1 canonical data when available. A deterministic GRU is primary; an SSM-style model and RSSM are challengers. Evaluation determines the trusted horizon. The scenario engine modifies only designated future controls/context. The UI displays recorded and imagined trajectories with model/run identity and limitations.

The next stages are research, not feature accumulation: improve state observability, calibrate uncertainty, add track/generalization splits, ingest F1 25 UDP, learn richer latent dynamics, and validate planning. An original architecture should be proposed only after a measured failure cannot be fixed by data, objective or simpler model choices.

Label every block

Scenario contract: Exactly which future controls/context may be changed and how. In APEX: Throttle, brake, rain/grip and tyre assumptions under bounded transformations.

Trusted horizon: Maximum forecast duration meeting error and guardrail criteria. In APEX: Displayed with every scenario rather than claiming unlimited simulation.

Product provenance: User-visible link from prediction to run/model/data/config. In APEX: Prevents anonymous, irreproducible UI output.

Research hypothesis: A falsifiable explanation for a measured failure and proposed change. In APEX: The basis for new world-model architecture work.

Walk one value through it

A UI scenario uses history length 32 at 5 Hz (6.4 seconds) and predicts 8 frames (1.6 seconds). Suppose speed MAE by horizon is [1.0,1.3,1.8,2.5,3.4,4.6,6.2,8.5] km/h, with a guardrail of $\text{MAE} \leq 5$ km/h and zero physical violations. The trusted horizon is step 6, or 1.2 seconds, even though the model emits 1.6 seconds.

This is not failure. The system should shade or label steps 7–8 as exploratory, refuse high-stakes conclusions, and direct research toward the compounding error observed after step 6.

Now repeat the trace using one value from the packaged lab. Write its shape and unit before and after every transformation. If a block contains a learned model, distinguish its parameters from its temporary activation/state.

What crosses each arrow?

Use this checklist:

- persistent state or temporary activation;
- observation, action, context, target or identifier;
- raw or normalized units;
- deterministic value or distribution parameters/sample;
- training-time evidence or deployment-time evidence;
- in-memory tensor or durable artifact reference.

Break one arrow

Load a checkpoint with the wrong normalizer, let the UI modify future target speed directly, hide the model identity, or display more decimal precision than validation supports.

The first visible symptom may occur several blocks later. The debugging objective is to identify the earliest arrow whose actual contract differs from the diagram.

Choosing a different route

Gradio UI: choose when rapid educational interactive scenarios with python integration. Avoid when you require a highly customized production frontend at scale.

FastAPI + web frontend: choose when stable service contracts, multiple clients and custom ux. Avoid when the api/model contract is still changing daily.

Synthetic-first: choose when offline causality, testing and controlled failure injection. Avoid when you confuse synthetic accuracy with real f1 fidelity.

APEX implementation map

```
apex_engine/START_HERE.md
apex_engine/src/apexsim/
apex_engine/notebooks/
apex_engine/tests/
projects/10_project_apex/
source_code_companion/
```

Open those files and redraw the diagram with exact function names and tensor shapes. Then add one missing production block—validation, logging, registry, monitoring or UI provenance—that the conceptual diagram intentionally omits.

Explain it without vocabulary

Explain the plate to a five-year-old using objects moving through boxes. Then explain it to an engineer using contracts, shapes, units and failure modes. If both explanations are accurate, the idea is becoming durable.

Project APEX Ten-Project Instructor Ladder

Progressive builds that culminate in the production simulation engine

OpenAI

August 2026

Table of Contents

1 How the project ladder works

Each project is larger than a lab and smaller than the final system. Start from an empty file. Use the supplied implementation only after your acceptance tests run against your own attempt. A project is complete when you can explain its contracts, reproduce its results, inject its listed failure and compare at least one alternative.

Use one branch or directory per project. At the end, write an Architecture Decision Record with: context, options, evidence, decision, consequences and revisit condition.

2 Project 1: One-Dimensional Car Integrator

2.1 Mission

Create an explicit state/action transition and a deterministic rollout.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

2.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

2.3 Acceptance criteria

Position and velocity arrays match an analytical constant-acceleration case.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

2.4 Deliberate failure

Wrong time-step use and update order.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

2.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

2.6 Transfer into APEX

The simulation interface used by every later model.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

2.7 Repository

projects/01_car_integrator/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

3 Project 2: Telemetry Quality Laboratory

3.1 Mission

Define a typed telemetry contract and reject corrupt units, ranges, timestamps and axes.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

3.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

3.3 Acceptance criteria

Every injected defect fails at the intended boundary with a useful message.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

3.4 Deliberate failure

Silent coercion and semantic swaps.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

3.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

3.6 Transfer into APEX

Canonical contract and ingestion quality gate.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

3.7 Repository

projects/02_telemetry_quality/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

4 Project 3: Lap Physics Sandbox

4.1 Mission

Add track progress, curvature demand, grip, tyre degradation and weather to a causal toy environment.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

4.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

4.3 Acceptance criteria

Changing each scenario variable produces a directionally justified trajectory change.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

4.4 Deliberate failure

Impossible controls and discontinuous progress.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

4.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

4.6 Transfer into APEX

Offline data source and trusted evaluator for planner tests.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

4.7 Repository

projects/03_lap_physics/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

5 Project 4: Nonlinear Transition Model

5.1 Mission

Train an MLP to predict next state and compare with persistence and linear controls.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

5.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

5.3 Acceptance criteria

Model improves matched test transitions and residual plots explain the gain.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

5.4 Deliberate failure

Target leakage and one-step-only conclusions.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

5.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

5.6 Transfer into APEX

First learned dynamics component.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

5.7 Repository

projects/04_mlp_transition/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

6 Project 5: Track Segment Encoder

6.1 Mission

Represent cyclic progress and local geometry without discontinuity at start/finish.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

6.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

6.3 Acceptance criteria

Nearby physical positions have nearby representations across the wrap boundary.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

6.4 Deliberate failure

Treating progress as an ordinary unbounded scalar.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

6.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

6.6 Transfer into APEX

Geometry/context representation for unseen-track transfer.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

6.7 Repository

projects/05_track_encoder/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

7 Project 6: GRU Telemetry Forecaster

7.1 Mission

Encode history and generate an autoregressive future under supplied controls.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

7.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

7.3 Acceptance criteria

Free-running horizon curves beat controls within a defined trusted range.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

7.4 Deliberate failure

Hidden-state reuse and teacher forcing at test.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

7.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

7.6 Transfer into APEX

Primary deterministic APEX V1 model.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

7.7 Repository

projects/06_gru_forecaster/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

8 Project 7: Selective SSM Memory Laboratory

8.1 Mission

Build stable input-dependent retention and compare with GRU on long/rare dependencies.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

8.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

8.3 Acceptance criteria

Selection provides measurable memory or latency value under matched budget.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

8.4 Deliberate failure

Exploding state and architecture-name overclaiming.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

8.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

8.6 Transfer into APEX

SSM challenger and Mamba preparation.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

8.7 Repository

projects/07_selective_ssm/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

9 Project 8: RSSM Imagination Laboratory

9.1 Mission

Separate posterior evidence from prior imagination and train stochastic latent dynamics.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

9.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

9.3 Acceptance criteria

Prior rollouts are useful and uncertainty diagnostics are calibrated on a stochastic task.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

9.4 Deliberate failure

Posterior leakage and KL collapse.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

9.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

9.6 Transfer into APEX

Dreamer-style belief model challenger.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

9.7 Repository

projects/08_rssm_imagination/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

10 Project 9: Latent MPC Planner

10.1 Mission

Use CEM to optimize bounded action sequences through a learned model.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

10.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

10.3 Acceptance criteria

Selected actions improve trusted-environment outcome without OOD exploitation.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

10.4 Deliberate failure

Reward loopholes and variance collapse.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

10.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

10.6 Transfer into APEX

Planning layer after dynamics certification.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

10.7 Repository

projects/09_latent_mpc/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

11 Project 10: Project APEX Production Engine

11.1 Mission

Integrate ingestion, contracts, windows, models, evaluation, pipeline, registry, API and UI.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

11.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

11.3 Acceptance criteria

A fresh run is reproducible, tested, traceable and usable through the interface.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

11.4 Deliberate failure

Artifact mismatch, UI false precision and source coupling.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

11.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

11.6 Transfer into APEX

The capstone and base for F1 25/research expansion.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

11.7 Repository

projects/10_project_apex/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

Project APEX Production Debugging Casebook

Fifteen failures investigated from symptom to regression test

OpenAI

August 2026

Table of Contents

1 Debugging protocol

Do not begin by changing the model. Reproduce the failure, state the violated invariant, and find the earliest boundary where observed evidence diverges from expected evidence.

For every case, complete this investigation before reading the diagnosis:

Exact symptom and smallest reproducer.

Expected invariant.

First probe and why it is high-information.

Earliest failed contract.

Root cause.

Minimal repair.

Regression test.

Misleading repair that must be rejected.

Monitoring signal that would catch recurrence.

The casebook intentionally spans data, tensors, optimization, latent dynamics, planning, pipelines and UI. A production world model is only as strong as its weakest boundary.

2 Case 1: Unit mismatch

2.1 Incident report

Primary symptom: Speed is converted twice and rollout acceleration is wrong.

Secondary evidence: A speed distribution has a plausible shape but is centred near 7 m/s instead of 90 m/s; drag and progress become inconsistent.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

2.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

2.3 High-information first probe

Print one raw value, source unit, canonical value and expected hand conversion at the adapter boundary.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

2.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

2.5 Root cause

Conversion is performed both by the source adapter and a downstream feature transform.

2.6 Correct repair

Choose one canonical unit, convert once in the adapter, store unit metadata, and delete downstream implicit conversion.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

2.7 Regression test

A fixture containing 360 km/h must become exactly 100 m/s and remain 100 m/s through window construction.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

2.8 The tempting wrong repair

Changing model coefficients or clipping speed masks the defect and corrupts every downstream interpretation.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

2.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

2.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

2.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

3 Case 2: Axis swap

3.1 Incident report

Primary symptom: A [batch,time,feature] tensor is passed as [time,batch,feature].

Secondary evidence: Training runs because dimensions are numerically compatible, but hidden state mixes examples and sequence length changes with batch size.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

3.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

3.3 High-information first probe

Use unequal dimensions such as batch=3, time=7, features=5; print named shape at model entry.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

3.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

3.5 Root cause

The dataset/collate output and model configuration disagree about `batch_first`.

3.6 Correct repair

Standardize one axis convention, assert it at the boundary, and transpose only in one named adapter when required.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

3.7 Regression test

A batch processed together must match each example processed separately with zero initial state.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

3.8 The tempting wrong repair

Increasing hidden size or training longer cannot repair wrong semantic axes.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

3.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

3.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

3.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

4 Case 3: Target leakage

4.1 Incident report

Primary symptom: Future speed accidentally appears in model inputs.

Secondary evidence: Validation error collapses unrealistically and remains strong even when controls are shuffled.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

4.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

4.3 High-information first probe

Print feature names and source frame indices for one window; train a linear model and inspect suspicious coefficient on future speed.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

4.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

4.5 Root cause

Window feature allow-list includes a target column at the same or later timestamp.

4.6 Correct repair

Separate state, future-control and target schemas; construct tensors from explicit allow-lists.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

4.7 Regression test

Assert no target feature/timestamp appears in history or future-input tensors and run a shuffled-target sanity test.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

4.8 The tempting wrong repair

Celebrating the score or adding regularization preserves the invalid experiment.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

4.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

4.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

4.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

5 Case 4: Random window split

5.1 Incident report

Primary symptom: Overlapping windows from one session enter every split.

Secondary evidence: Test performance is extremely stable and much better than unseen-session performance.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

5.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

5.3 High-information first probe

Report unique session IDs and raw frame IDs in each split; calculate overlap and nearest temporal distance.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

5.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

5.5 Root cause

Windows are generated first and then randomly assigned.

5.6 Correct repair

Assign sessions/events before windowing; persist deterministic split mapping.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

5.7 Regression test

No session ID or raw frame identifier may appear in more than one split.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

5.8 The tempting wrong repair

Reducing window overlap does not solve the generalization-claim mismatch.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

5.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

5.10 Operational prevention

Define:

- a precondition checked before the stage;

- a postcondition checked after the stage;

- a metric or alert for recurrence;

- provenance needed for forensic replay;

a rollback or quarantine action.

5.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

6 Case 5: Stale as-of join

6.1 Incident report

Primary symptom: Weather is forward-filled across a long outage.

Secondary evidence: Rain appears constant for minutes despite missing source records, and the model becomes overconfident.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

6.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

6.3 High-information first probe

Store matched source timestamp and compute age for every joined row; plot age distribution.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

6.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

6.5 Root cause

Forward fill or backward join has no tolerance and missingness is erased.

6.6 Correct repair

Use causal join with domain tolerance, carry age/missing masks, quarantine long gaps.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

6.7 Regression test

Every matched context timestamp must be $\leq$ frame time and age $\leq$ configured tolerance.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

6.8 The tempting wrong repair

Blind interpolation creates fabricated certainty rather than recovering evidence.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

6.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

6.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

6.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

7 Case 6: Normalizer leak

7.1 Incident report

Primary symptom: Standardization is fitted on all data.

Secondary evidence: Held-out features have suspiciously centred distributions and deployment transformation cannot be recreated from training artifacts.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

7.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

7.3 High-information first probe

Recompute normalizer hash from training rows only and compare; inspect fitted row identifiers.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

7.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

7.5 Root cause

Preprocessing is applied before split or fit is called on concatenated partitions.

7.6 Correct repair

Split first, fit on train, serialize statistics, transform validation/test without refit.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

7.7 Regression test

Changing validation/test values must not change stored normalizer statistics.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair.

Then decide whether the same invariant belongs in runtime validation or monitoring.

7.8 The tempting wrong repair

The score inflation may be small, but the protocol and reproducibility are invalid.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

7.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

7.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;
 provenance needed for forensic replay;
 a rollback or quarantine action.

7.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

8 Case 7: Hidden-state reuse

8.1 Incident report

Primary symptom: Hidden state leaks between unrelated sessions.

Secondary evidence: Predictions depend on dataloader order and the first frames of a session contain information from a previous driver.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

8.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

8.3 High-information first probe

Run the same sessions in reversed batch order and compare outputs; print hidden norm at boundaries.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

8.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

8.5 Root cause

Stateful recurrence is used without entity/session reset policy.

8.6 Correct repair

Zero/reset hidden state at independent boundaries; make streaming state keyed and explicit.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

8.7 Regression test

Separate processing and batched processing with zero state must agree.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

8.8 The tempting wrong repair

Randomizing order can hide the symptom without defining correct state lifetime.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

8.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

8.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

8.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

9 Case 8: Missing eval mode

9.1 Incident report

Primary symptom: Dropout remains active during evaluation.

Secondary evidence: Repeated inference for identical input produces different predictions and reported metrics vary.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

9.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

9.3 High-information first probe

Call the model twice with fixed input/seed; inspect `model.training`.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

9.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

9.5 Root cause

Evaluation uses `no_grad` but never calls `model.eval()`.

9.6 Correct repair

Switch to eval before validation/inference and restore train mode when resuming optimization.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

9.7 Regression test

Two deterministic eval calls must match for deterministic models; validation helper must assert mode.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

9.8 The tempting wrong repair

Averaging many stochastic passes is not a repair unless MC dropout is an intentional uncertainty method.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

9.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

9.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

9.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

10 Case 9: Gradient accumulation

10.1 Incident report

Primary symptom: Gradients are never cleared.

Secondary evidence: Update magnitude grows across batches, loss oscillates and gradient norms depend on batch position.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

10.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

10.3 High-information first probe

Print one parameter gradient before backward each iteration.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

10.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

10.5 Root cause

`zero_grad` is omitted while optimizer steps every batch.

10.6 Correct repair

Clear gradients at the intended accumulation boundary and document accumulation factor.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

10.7 Regression test

Two identical independent batches should produce identical gradients when starting from same parameters.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

10.8 The tempting wrong repair

Lowering learning rate may reduce symptoms but leaves unintended history in every update.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

10.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

10.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

10.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

11 Case 10: Exploding SSM

11.1 Incident report

Primary symptom: Unconstrained recurrent decay exceeds one.

Secondary evidence: Hidden norm grows exponentially during zero-input rollout, then outputs become `inf/NaN`.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

11.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

11.3 High-information first probe

Plot hidden norm over 200 zero-input steps and inspect transition eigenvalues/gates.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

11.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

11.5 Root cause

State propagation permits amplification greater than one without stabilizing design.

11.6 Correct repair

Use stable parameterization/bounds, initialization and horizon stress tests; clip only as secondary protection.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

11.7 Regression test

Bounded input must produce finite bounded hidden state over the certified horizon.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

11.8 The tempting wrong repair

Output clipping conceals unstable internal memory and gradients.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

11.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

11.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

11.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

12 Case 11: KL collapse

12.1 Incident report

Primary symptom: RSSM posterior ignores stochastic state.

Secondary evidence: KL is nearly zero, posterior and prior are identical, and decoder predictions barely change when latent is resampled.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

12.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

12.3 High-information first probe

Report KL per latent dimension, latent variance and decoder sensitivity to z .

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

12.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

12.5 Root cause

KL pressure/decoder capacity lets deterministic path solve loss while stochastic latent carries no information.

12.6 Correct repair

Tune balance with warm-up/free-nats/capacity changes after diagnostics; force evaluation of prior and latent probes.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

12.7 Regression test

Changing z while holding h fixed should alter decoded predictions on a task designed to require uncertainty.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

12.8 The tempting wrong repair

Simply increasing stochastic dimension adds unused variables and can worsen optimization.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

12.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

12.10 Operational prevention

Define:

a precondition checked before the stage;

a postcondition checked after the stage;

a metric or alert for recurrence;

provenance needed for forensic replay;

a rollback or quarantine action.

12.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

13 Case 12: Planner exploitation

13.1 Incident report

Primary symptom: CEM finds actions outside training support.

Secondary evidence: Imagined reward is excellent but actions saturate at boundaries and fail in trusted simulator/replay.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

13.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

13.3 High-information first probe

Measure action-distribution distance, uncertainty and violations for elite candidates.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

13.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

13.5 Root cause

Reward/model lacks constraints and planner searches OOD regions where dynamics are inaccurate.

13.6 Correct repair

Enforce bounds, OOD/uncertainty penalties, trusted-model validation and receding-horizon replanning.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

13.7 Regression test

A hidden loophole fixture must be rejected or penalized despite high predicted reward.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

13.8 The tempting wrong repair

Reducing CEM iterations may hide exploitation but does not repair model/reward validity.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

13.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

13.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

13.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

14 Case 13: Artifact overwrite

14.1 Incident report

Primary symptom: Two runs write the same model path.

Secondary evidence: Metrics refer to one config while checkpoint silently belongs to another; rollback is impossible.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

14.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

14.3 High-information first probe

Compare checkpoint hash, config hash and timestamps in registry/run directory.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

14.4 Investigation ladder

Reproduce with the smallest data/model configuration.
 Freeze randomness and preserve the failing artifact.
 Inspect identifiers, timestamps, units, shapes and feature names.
 Compare one value by hand with expected transformation.
 Move one boundary earlier until evidence becomes correct.
 The defect lies between the last correct and first incorrect boundary.
 Repair the cause and rerun the minimal reproducer.
 Run the full relevant test set and compare unaffected behaviour.

14.5 Root cause

Mutable latest/model.pt is used as both identity and storage.

14.6 Correct repair

Use immutable run IDs/content hashes; make latest a pointer, not the sole artifact.
 A repair should reduce ambiguity. It should not merely make the current metric look normal.

14.7 Regression test

Concurrent runs must produce distinct immutable paths and provenance must resolve exactly.
 Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair.
 Then decide whether the same invariant belongs in runtime validation or monitoring.

14.8 The tempting wrong repair

Adding timestamps to filenames without registry metadata still weakens lineage.
 This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

14.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

14.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

14.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

15 Case 14: Airflow payload

15.1 Incident report

Primary symptom: Large arrays are passed through XCom.

Secondary evidence: Scheduler metadata grows, serialization fails and workers spend time moving data through the control plane.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

15.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

15.3 High-information first probe

Inspect XCom sizes and task logs; trace where durable files already exist.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

15.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

15.5 Root cause

Task interfaces return in-memory arrays instead of artifact references.

15.6 Correct repair

Persist arrays in object/file storage and pass small path/ID/hash messages.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

15.7 Regression test

Task output metadata must remain below a chosen size and downstream task must reload/validate artifact.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

15.8 The tempting wrong repair

Raising database limits turns an architectural problem into a larger operational problem.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

15.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

15.10 Operational prevention

Define:

- a precondition checked before the stage;
- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

15.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

16 Case 15: UI false precision

16.1 Incident report

Primary symptom: The UI reports more precision than the model supports.

Secondary evidence: A scenario displays speed to 0.001 km/h beyond the measured error and shows untrusted horizons as equally certain.

Assume the symptom was first noticed in a model or UI output. Your task is to resist debugging at the last visible layer.

16.2 Pause and predict

Write the invariant that should hold. List three plausible causes from different layers. Rank them by how early they occur in the data-to-product flow. Choose one probe that can eliminate the largest number of hypotheses.

16.3 High-information first probe

Compare displayed digits/horizon with calibration and held-out metrics; reproduce one point outside UI.

This probe is valuable because it inspects the contract closest to the suspected transformation. A good probe is small, deterministic and interpretable; a full retraining run is a poor first probe.

16.4 Investigation ladder

Reproduce with the smallest data/model configuration.

Freeze randomness and preserve the failing artifact.

Inspect identifiers, timestamps, units, shapes and feature names.

Compare one value by hand with expected transformation.

Move one boundary earlier until evidence becomes correct.

The defect lies between the last correct and first incorrect boundary.

Repair the cause and rerun the minimal reproducer.

Run the full relevant test set and compare unaffected behaviour.

16.5 Root cause

Presentation is designed independently of model uncertainty, provenance and trusted range.

16.6 Correct repair

Round to meaningful resolution, label trusted horizon, show run/model identity and exploratory regions.

A repair should reduce ambiguity. It should not merely make the current metric look normal.

16.7 Regression test

Snapshot/UI test verifies provenance, units, rounding and horizon labels for a known run.

Write the test before deleting the broken version. Prove that it fails on the defect and passes after repair. Then decide whether the same invariant belongs in runtime validation or monitoring.

16.8 The tempting wrong repair

A beautiful chart can increase harm when it makes uncertain outputs look authoritative.

This matters because production incidents are often prolonged by symptom-level fixes that make evidence less visible.

16.9 APEX tracing exercise

Locate the corresponding path in the production repository. Draw the call path from CLI/UI input to the failed value. Mark every place that can transform it. Add one structured log or artifact field at the earliest useful boundary.

16.10 Operational prevention

Define:

a precondition checked before the stage;

- a postcondition checked after the stage;
- a metric or alert for recurrence;
- provenance needed for forensic replay;
- a rollback or quarantine action.

16.11 Interview-level explanation

Explain the incident in four sentences: impact, root cause, repair, and prevention. Then explain why the most obvious model-level change would have been wrong.

Project APEX Research Reading and Design Guide

From paper diagrams to falsifiable F1 world-model experiments

OpenAI

August 2026

Table of Contents

1 How to read a world-model paper as an engineer

A research paper is not a tutorial, a repository is not a proof, and a benchmark result is not automatically relevant to F1 telemetry. The job of this guide is to turn each paper into six concrete objects:

Problem claim — what limitation the paper says exists.

Mechanism — the smallest architectural or objective change that addresses it.

Training contract — what data, targets, losses and optimization are required.

Evaluation claim — what evidence the authors provide and what they do not provide.

Reproduction slice — the smallest runnable implementation that isolates the mechanism.

APEX experiment — a matched ablation whose result could change an engineering decision.

Before reading equations, write down the interfaces:

observation o_t

state / belief s_t

intervention a_t

context c_t

reward or task target r_t

next observation / representation

Then label what is available during training, posterior inference, open-loop imagination and deployment. Most accidental misunderstandings come from letting a training-only signal leak into imagination.

1.1 The three-pass method

1.1.1 Pass 1: Claim map

Read title, abstract, introduction, figures and conclusion. Do not stop to decode every symbol. Write one sentence for the problem, one for the mechanism and one for the result. If you cannot, the paper has not yet become a mental model.

1.1.2 Pass 2: Execution map

Trace one batch through the method. List tensor shapes, gradient paths and stop-gradient paths. Mark which network is updated by which loss. Mark any target network, replay buffer, latent sample or normalization rule.

1.1.3 Pass 3: Evidence map

For every table or curve, ask:

What changed between control and treatment?

What remained fixed?

How many seeds or tasks support the claim?

Does the metric test the claimed mechanism?

What failure is hidden by the average?

Which assumption changes in APEX?

Finish by writing a reproduction test that could fail. “Implement Dreamer” is not a test. “Under the same session split and parameter budget, does a stochastic RSSM improve calibrated 20-step telemetry coverage over a deterministic GRU?” is testable.

2 DreamerV3: from latent dynamics to imagined behaviour

2.1 The problem it addresses

Model-free reinforcement learning learns behaviour directly from real interaction and can be data hungry. Dreamer learns a world model from experience, then improves behaviour using trajectories generated inside that model. DreamerV3 focuses on one configuration that performs across varied domains without per-domain tuning.

The system is not merely an RSSM. It is a collection of contracts:

experience replay

↓

encoder + recurrent state-space model

↓

observation / reward / continuation predictions

↓

latent imagined trajectories

↓

actor and critic learning

If you implement only a stochastic recurrent transition, you have built one component, not the full algorithm.

2.2 Mechanism map

The recurrent state combines deterministic memory with stochastic latent state. During representation learning, current observations inform a posterior latent. A prior predicts the latent without seeing the

observation. Reconstruction/prediction losses preserve useful evidence; KL-related terms align posterior and prior enough for imagination.

During imagination, the model receives latent state and actions but no future observation. Reward and continuation heads score imagined outcomes. Actor and critic updates occur on those imagined trajectories.

2.3 Paper-to-code questions

Before implementing, answer:

Which tensors are detached during actor/critic updates?

Does the actor receive stochastic samples, distribution statistics or both?

How are returns normalized or transformed?

How are reward scales handled across domains?

How is the KL objective balanced to prevent collapse or an unusable prior?

What is the imagination horizon and how does it interact with model error?

2.4 Minimal reproduction slice

Do not begin with an F1 agent. Build a stochastic one-dimensional car where hidden grip changes between episodes. Train:

Deterministic GRU transition.

RSSM with posterior/prior and state decoder.

Reward head for progress minus instability.

CEM planner using the RSSM.

Only then, a small actor trained on imagined trajectories.

Your first milestone is not reward. It is showing that prior rollouts widen uncertainty when grip is ambiguous and that the posterior narrows after evidence.

2.5 APEX experiment

Question: Does stochastic latent belief improve decisions under hidden grip compared with deterministic recurrence?

Control: GRU with matched hidden/parameter budget.

Treatment: RSSM with deterministic memory, stochastic latent and calibrated prediction intervals.

Held fixed: canonical data, session split, history, horizon, optimizer budget, scenario controls and evaluation.

Primary metric: negative log likelihood or interval coverage by horizon under hidden-grip sessions.

Guardrails: mean speed MAE, physical violations, latency, prior-posterior gap and calibration.

Decision rule: keep RSSM only if uncertainty improves decision safety or planning under ambiguity without unacceptable degradation in mean trajectory.

3 Mamba and selective state-space models

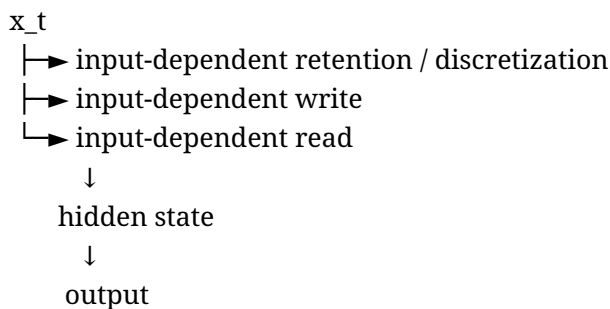
3.1 The problem it addresses

Transformers use content-dependent attention but attention cost grows quadratically with sequence length. Structured state-space models process long sequences efficiently but traditional linear time-invariant dynamics have limited content-dependent selection. Mamba introduces input-dependent selective state-space parameters and an implementation designed for modern hardware.

The transferable insight is not “Mamba is faster.” It is that a sequence token can determine what enters memory, what persists and what is read, while retaining a recurrent/state-space computation.

3.2 Mechanism map

A conceptual selective recurrence is:



The complete Mamba block contains more than a gate. Its empirical and throughput claims rely on parameterization, projections, local mixing and a selective-scan implementation. A custom $a(x)h + (1-a(x))b(x)$ cell is a teaching model, not an equivalent reproduction.

3.3 Paper-to-code questions

Which parameters depend on input and which remain global?

How is stability enforced or encouraged?

What is computed recurrently during inference?

What is parallelized during training?

What sequence lengths and hardware are required for throughput advantage?

Are gains caused by selection, parameter count, training recipe or implementation?

3.4 Minimal reproduction slice

Create a delayed-event task. Most frames are routine; one rare token determines a target 200 steps later. Compare:

Fixed linear SSM.

GRU.

Toy selective SSM.

Official Mamba implementation, only after the first three are understood.

Plot retention/write behaviour around the rare token. Match parameter counts and training steps.

Measure memory accuracy and wall-clock throughput separately.

3.5 APEX experiment

Question: Does input-dependent memory improve long-horizon telemetry representation when rare regime changes matter?

Use histories long enough to create a real bottleneck. Test sessions with safety-car changes, rain onset or sustained tyre degradation. Compare speed/latency against GRU and temporal convolution. A short 32-frame window is unlikely to justify architectural migration by itself.

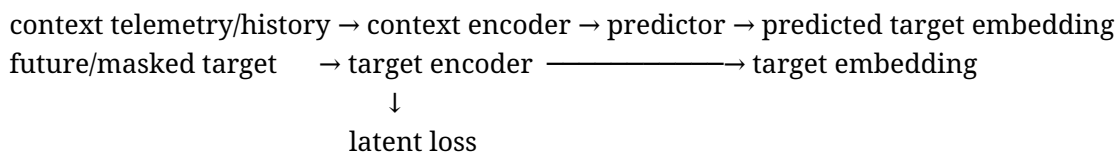
4 I-JEPA and predictive representations

4.1 The problem it addresses

Reconstructive self-supervision asks a model to reproduce every raw detail. A joint-embedding predictive architecture predicts representations of missing/target regions from context. The target representation can discard unpredictable local detail while preserving semantic structure.

For telemetry, the analogue is predicting a future or masked-state representation instead of every sensor value. This can be beneficial if sensor noise and redundant channels dominate reconstruction. It can be harmful if the target encoder discards rare transition cues.

4.2 Mechanism map



The asymmetry between context and target branches, stop-gradient/target updates, masking and representation regularity prevent trivial collapse.

4.3 Diagnostics before celebration

A low latent loss is meaningless without:

Per-dimension variance.

Embedding covariance/effective rank.

Nearest-neighbour diversity.

Linear probes for physical variables.

Future-event retrieval.

Downstream rollout or decision value.

Checks that masked information cannot be copied through shortcuts.

4.4 APEX experiment

Pretrain an encoder on unlabeled canonical sessions by predicting future target embeddings. Freeze it and train a small dynamics head. Compare with:

Random frozen encoder.

Autoencoder-pretrained encoder.

End-to-end supervised GRU.

JEPA-style pretrained encoder plus head.

Report data efficiency curves—not only final performance. The strongest transfer claim is improved performance with fewer labelled/high-quality sessions under the same held-out events.

5 LeWorldModel and efficient planning-oriented latent models

5.1 Reading discipline

When a paper claims faster planning, separate:

Latent transition speed.

Number of model evaluations required by the planner.

Representation/encoder cost.

Training cost.

Hardware and batch size.

Quality of imagined trajectories.

Task success under equal wall-clock budget.

A model can be faster per step but require more samples, or produce lower-quality rollouts that mislead planning. Planning speed and decision quality must be reported together.

5.2 APEX reproduction slice

Use the same CEM planner with two dynamics backends:

Deterministic GRU latent/state transition.

Alternative efficient latent transition inspired by the paper.

Hold candidate count, horizon, reward and constraints fixed. Measure:

Rollouts per second.

Best score found under fixed wall-clock time.

Real/synthetic-environment outcome of the selected action.

Model exploitation rate.

Memory and startup cost.

The project should adopt the method only if planning throughput improves while selected actions remain valid under a trusted evaluator.

6 Fine-tuning an existing world or sequence model

Fine-tuning begins with interface compatibility, not checkpoint popularity.

6.1 Compatibility audit

Question

Why it matters

What was the observation type?

Pixels, tokens and telemetry have different invariances.

What actions were used?

A latent trained without interventions may not support counterfactual control.

What time step and horizon?

Dynamics meaning changes with sample rate.

What normalization and units?

Parameter scales encode the source representation.

What objective shaped the latent?

Reconstruction, contrastive, predictive and control objectives preserve different information.

What domain diversity?

Narrow pretraining may transfer less than a small domain model.

Is the implementation/license operationally usable?

Research value and deployability differ.

6.2 Adapter ladder

Frozen features + linear probe. Cheapest test of accessible information.

Frozen backbone + nonlinear head. Tests whether representation is useful but readout is nonlinear.

Parameter-efficient adapters or selected-layer tuning. Adapts domain while preserving most weights.

Gradual unfreezing. Expands capacity only when validation evidence supports it.

Full fine-tuning. Highest flexibility and highest overfitting/forgetting risk.

Train from scratch baseline. Always retain this control for compact telemetry models.

Track representational drift, source-task regression when relevant, and performance by data volume. Fine-tuning is justified by better data efficiency, final performance or operational capability—not by using a larger name.

7 Designing your own APEX world model

An original architecture begins with a measured failure.

7.1 Failure-to-mechanism worksheet

Measured failure

Candidate mechanism

Simpler alternative to rule out

Long history forgotten

Structured/selective memory

Longer window, GRU tuning, temporal convolution

Multi-modal future

Stochastic latent or mixture head

Ensemble, quantile regression

Poor unseen-track transfer

Track representation / equivariance

Better split, geometry features, augmentation

Planner exploitation

Uncertainty/conservative objective

Hard constraints, OOD action limits

Slow planning

Efficient latent transition

Batching, smaller model, fewer candidates

Rare events erased

Event-aware objective/memory

Reweighting, targeted windows

7.2 Architecture proposal template

Write the following before code:

Observed failure: metric, horizon, subgroup and examples.

Hypothesis: why the current mechanism fails.

New mechanism: exact information path being added or changed.

Control: strongest simpler model.

Matched budget: parameters, data, steps and compute.

Primary evidence: metric that directly tests the mechanism.

Guardrails: validity, latency, calibration and baseline regression.

Ablations: remove each new component independently.

Failure/rollback condition: result that causes rejection.

Reproduction package: code, config, data hash, seeds and report.

7.3 Example original direction: geometry-conditioned selective RSSM

A possible research model could combine:

Deterministic recurrent belief.

Stochastic latent for unobserved grip.

Track-geometry encoder.

Input-dependent selective memory rates conditioned on curvature/regime.

Prior imagination under future controls.

Calibrated state distribution and physical validity penalties.

This sounds impressive, but it contains several hypotheses. Test them separately:

Does geometry conditioning improve unseen-track transfer?

Does stochastic latent improve uncertainty under hidden grip?

Does selection improve retention of rare regime changes?

Does combining them produce gains beyond each component?

Only the ablation matrix turns the architecture into research.

8 Paper reading exercises

For each paper:

Draw the training-time graph and imagination-time graph separately.

Mark every gradient and stop-gradient path.

List every distribution and what conditions it sees.

Reproduce the smallest mechanism on a toy sequence.

Design a failure case where the mechanism should help.

Design a case where it should not help.

State the APEX adaptation and the assumption it changes.

Write a matched ablation and rejection criterion.

9 Primary source list

DreamerV3: [arXiv:2301.04104](#)

Mamba: [arXiv:2312.00752](#)

I-JEPA: [arXiv:2301.08243](#) and the official implementation

LeWorldModel: primary paper and official implementation

FastF1 official documentation

OpenF1 official documentation

PyTorch official documentation

EA F1 25 Data Output Specification

Project APEX Field Workbook

*20 labs • 15 debugging investigations • paper-to-code workshops •
independent design records*

This workbook is intentionally write-heavy. Complete it beside the seven-volume curriculum. A page is not finished until it contains a prediction, runtime evidence, an explanation, a deliberately introduced failure, a repair and an engineering decision.

Evidence Standard

- Prediction written before execution
- Command or code version recorded
- Observed shapes, values or plots captured
- Failure localized to the earliest contract
- Repair protected by a regression test
- Implementation choice justified against an alternative

Lab Worksheet 1: 01 Units And Shapes

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 2: 02 Vectorized Kinematics

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 3: 03 Force Model

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 4: 04 Sampling Aliasing

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 5: 05 Regression Baseline

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 6: 06 Autograd Drag

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 7: 07 Pytorch Module

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 8: 08 Dataset Windows

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 9: 09 Loss Comparison

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 10: 10 Training Loop

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 11: 11 Rnn From Scratch

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 12: 12 Gru Gates

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 13: 13 Linear Ssm

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 14: 14 Selective Ssm

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 15: 15 Autoencoder

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Write predictions before outputs. Evidence before architecture.

Decision record

Question	Evidence-based response

Lab Worksheet 16: 16 Vae Reparameterization

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 17: 17 Rssm Step

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 18: 18 Cem Planning

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 19: 19 Time Alignment

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Lab Worksheet 20: 20 Pipeline Contract

Before you run

Prediction	Your answer

Runtime trace

Line / operation	Input	Output	Meaning

Break and repair

Injected defect:

Earliest failed invariant:

Regression test:

Decision record

Question	Evidence-based response

Debugging Investigation 1: Unit Mismatch

Speed is converted twice and rollout acceleration is wrong.

Do not repair yet

- 1. What should be invariant?
- 2. What is the earliest component that can violate it?
- 3. What is the smallest input that reproduces the problem?
- 4. Which probe distinguishes your top two hypotheses?
- 5. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 2: Axis Swap

A [batch,time,feature] tensor is passed as [time,batch,feature].

Do not repair yet

- 6. What should be invariant?
- 7. What is the earliest component that can violate it?
- 8. What is the smallest input that reproduces the problem?
- 9. Which probe distinguishes your top two hypotheses?
- 10. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 3: Target Leakage

Future speed accidentally appears in model inputs.

Do not repair yet

- 11. What should be invariant?
- 12. What is the earliest component that can violate it?
- 13. What is the smallest input that reproduces the problem?
- 14. Which probe distinguishes your top two hypotheses?
- 15. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 4: Random Window Split

Overlapping windows from one session enter every split.

Do not repair yet

- 16. What should be invariant?
- 17. What is the earliest component that can violate it?
- 18. What is the smallest input that reproduces the problem?
- 19. Which probe distinguishes your top two hypotheses?
- 20. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 5: Stale Asof Join

Weather is forward-filled across a long outage.

Do not repair yet

- 21. What should be invariant?
- 22. What is the earliest component that can violate it?
- 23. What is the smallest input that reproduces the problem?
- 24. Which probe distinguishes your top two hypotheses?
- 25. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 6: Normalizer Leak

Standardization is fitted on all data.

Do not repair yet

- 26. What should be invariant?
- 27. What is the earliest component that can violate it?
- 28. What is the smallest input that reproduces the problem?
- 29. Which probe distinguishes your top two hypotheses?
- 30. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 7: Hidden State Reuse

Hidden state leaks between unrelated sessions.

Do not repair yet

- 31. What should be invariant?
- 32. What is the earliest component that can violate it?
- 33. What is the smallest input that reproduces the problem?
- 34. Which probe distinguishes your top two hypotheses?
- 35. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 8: Missing Eval Mode

Dropout remains active during evaluation.

Do not repair yet

- 36. What should be invariant?
- 37. What is the earliest component that can violate it?
- 38. What is the smallest input that reproduces the problem?
- 39. Which probe distinguishes your top two hypotheses?
- 40. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 9: Gradient Accumulation

Gradients are never cleared.

Do not repair yet

- 41. What should be invariant?
- 42. What is the earliest component that can violate it?
- 43. What is the smallest input that reproduces the problem?
- 44. Which probe distinguishes your top two hypotheses?
- 45. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 10: Exploding Ssm

Unconstrained recurrent decay exceeds one.

Do not repair yet

- 46. What should be invariant?
- 47. What is the earliest component that can violate it?
- 48. What is the smallest input that reproduces the problem?
- 49. Which probe distinguishes your top two hypotheses?
- 50. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 11: KI Collapse

RSSM posterior ignores stochastic state.

Do not repair yet

- 51. What should be invariant?
- 52. What is the earliest component that can violate it?
- 53. What is the smallest input that reproduces the problem?
- 54. Which probe distinguishes your top two hypotheses?
- 55. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 12: Planner Exploitation

CEM finds actions outside training support.

Do not repair yet

- 56. What should be invariant?
- 57. What is the earliest component that can violate it?
- 58. What is the smallest input that reproduces the problem?
- 59. Which probe distinguishes your top two hypotheses?
- 60. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 13: Artifact Overwrite

Two runs write the same model path.

Do not repair yet

- 61. What should be invariant?
- 62. What is the earliest component that can violate it?
- 63. What is the smallest input that reproduces the problem?
- 64. Which probe distinguishes your top two hypotheses?
- 65. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 14: Airflow Payload

Large arrays are passed through XCom.

Do not repair yet

- 66. What should be invariant?
- 67. What is the earliest component that can violate it?
- 68. What is the smallest input that reproduces the problem?
- 69. Which probe distinguishes your top two hypotheses?
- 70. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Debugging Investigation 15: Ui False Precision

The UI reports more precision than the model supports.

Do not repair yet

- 71. What should be invariant?
- 72. What is the earliest component that can violate it?
- 73. What is the smallest input that reproduces the problem?
- 74. Which probe distinguishes your top two hypotheses?
- 75. What downstream symptom could distract you?

Repair record

Item	Your evidence

Regression test specification

Given:

When:

Then:

Paper-to-Code Workshop 1: DreamerV3

Reference: arXiv:2301.04104

Focus: World model, reward/continuation heads, imagined actor-critic learning and robustness transformations.

One-page paper decomposition

Question	Your implementation interpretation
What problem is being solved?	
What is the input and target?	
What information is hidden or latent?	
What loss terms exist?	
What prevents collapse or exploitation?	
What is the inference-time path?	
Which baseline is load-bearing?	
Which claim can APEX test cheaply?	

Code mapping

Write the modules, tensors and tests required for the smallest faithful reproduction.

Paper component	APEX module / tensor / test

Write predictions before outputs. Evidence before architecture.

Paper-to-Code Workshop 2: Mamba

Reference: arXiv:2312.00752

Focus: Input-dependent selective state-space dynamics and hardware-aware sequence computation.

One-page paper decomposition

Question	Your implementation interpretation
What problem is being solved?	
What is the input and target?	
What information is hidden or latent?	
What loss terms exist?	
What prevents collapse or exploitation?	
What is the inference-time path?	
Which baseline is load-bearing?	
Which claim can APEX test cheaply?	

Code mapping

Write the modules, tensors and tests required for the smallest faithful reproduction.

Paper component	APEX module / tensor / test

Write predictions before outputs. Evidence before architecture.

Paper-to-Code Workshop 3: I-JEPA

Reference: arXiv:2301.08243

Focus: Context-to-target representation prediction without pixel reconstruction.

One-page paper decomposition

Question	Your implementation interpretation
What problem is being solved?	
What is the input and target?	
What information is hidden or latent?	
What loss terms exist?	
What prevents collapse or exploitation?	
What is the inference-time path?	
Which baseline is load-bearing?	
Which claim can APEX test cheaply?	

Code mapping

Write the modules, tensors and tests required for the smallest faithful reproduction.

Paper component	APEX module / tensor / test

Write predictions before outputs. Evidence before architecture.

Paper-to-Code Workshop 4: LeWorldModel

Reference: arXiv:2603.19312

Focus: End-to-end reconstruction-free world-model learning with embedding prediction and latent regularization.

One-page paper decomposition

Question	Your implementation interpretation
What problem is being solved?	
What is the input and target?	
What information is hidden or latent?	
What loss terms exist?	
What prevents collapse or exploitation?	
What is the inference-time path?	
Which baseline is load-bearing?	
Which claim can APEX test cheaply?	

Code mapping

Write the modules, tensors and tests required for the smallest faithful reproduction.

Paper component	APEX module / tensor / test

Write predictions before outputs. Evidence before architecture.

Paper-to-Code Workshop 5: F1 25 UDP Specification

Reference: EA Data Output from F1 25 Game v3

Focus: Packed little-endian packet contracts, frame synchronization, rates, privacy restrictions and replay requirements.

One-page paper decomposition

Question	Your implementation interpretation
What problem is being solved?	
What is the input and target?	
What information is hidden or latent?	
What loss terms exist?	
What prevents collapse or exploitation?	
What is the inference-time path?	
Which baseline is load-bearing?	
Which claim can APEX test cheaply?	

Code mapping

Write the modules, tensors and tests required for the smallest faithful reproduction.

Paper component	APEX module / tensor / test

Write predictions before outputs. Evidence before architecture.

Original World-Model Design Studio

Do not begin with an architecture name. Begin with a measured failure in APEX.

1. Failure definition

76. Which held-out situation fails?

77. What metric and horizon expose it?

78. What simpler explanation has been ruled out?

2. State design

79. Which variables belong in state, action, context and observation?

80. What must be latent?

81. What must remain physically interpretable?

3. Memory design

82. What duration matters?

83. Is access content-based, recurrent or hierarchical?

84. What is the compute budget?

4. Objective design

85. What should be predicted?

86. Which loss terms are necessary?

87. How will collapse or exploitation be detected?

5. Evaluation design

88. What baseline must be beaten?

89. Which ablation isolates the contribution?

90. What physical guardrails block promotion?

Final Independent Build Checklist

- ☐ Canonical contract created from memory
- ☐ Group split proven leakage-free
- ☐ Baseline reproduced
- ☐ GRU rollout rebuilt
- ☐ Selective SSM challenger implemented
- ☐ RSSM prior/posterior paths explained
- ☐ CEM planner constrained
- ☐ Ablations recorded
- ☐ Pipeline stages idempotent
- ☐ API contract versioned
- ☐ UI communicates uncertainty
- ☐ F1 25 replay adapter designed
- ☐ One original architecture proposed and falsifiable

Project APEX Annotated Source Code Companion

Every production module, line-numbered source, block explanations and line-by-line Markdown notes

Use this companion with the editor open. First read a function signature and predict its contract. Then follow the numbered code. The annotation states what the line does and what invariant to verify; you should add the concrete tensor shape, unit or artifact path from your own run.

Call Path

1. CLI / UI / API
2. Configuration and source adapter
3. Canonical contract and validation
4. Features, splits, normalization and windows
5. Model factory and world-model architecture
6. Training and checkpointing
7. Evaluation, scenario rollout and plotting
8. Pipeline stages, registry and published artifacts

How to Read a Module

- Role: why the file exists.
- Imports: architectural dependencies.
- Public definitions: boundaries used by other modules.
- State: values that persist between calls or time steps.
- Transformations: equations and tensor shape changes.
- Failure paths: invariants, exceptions and fallbacks.
- Artifacts: what is written, returned or registered.
- Tests: evidence required before changing the implementation.

src/apexsim/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-3

```
0001 """Project APEX: an educational F1 telemetry world-model simulation engine."""
0002
0003 __version__ = "0.1.0"
```

Line	Code signal	What it is doing / what to verify
1	"""Project APEX: an educational F1 telemetry world-model simulation engine."""	Begins or ends documentation describing the module, class or function contract.
3	__version__ = "0.1.0"	Creates or updates `__version__`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

src/apexsim/cli.py

Routes human commands to tested domain functions.

A CLI should be thin. It parses intent, validates inputs and calls the same adapters and stages used by tests and orchestration.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import typer
0007
0008 from apexsim.config import load_config
0009 from apexsim.data.fastf1_adapter import ingest_fastf1_session
0010 from apexsim.data.openf1_adapter import ingest_openf1_session
0011 from apexsim.data.synthetic import generate_synthetic_sessions
0012 from apexsim.data.validate import validate_canonical_frame
0013 from apexsim.pipeline.runner import run_pipeline
0014
0015 app = typer.Typer(no_args_is_help=True, help="Project APEX F1 world-simulation engine")
0016
0017
0018 @app.command()
0019 def generate(config: Path = Path("configs/fast.yaml"), output: Path = Path("data/raw/synthetic.csv")) -> None:
0020     cfg = load_config(config)
0021     frame = generate_synthetic_sessions(cfg, output)
0022     typer.echo(f"Generated {len(frame):,} canonical telemetry rows at {output}")

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import typer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.config import load_config	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.data.fastf1_adapter import ingest_fastf1_session	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.data.openf1_adapter import ingest_openf1_session	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.data.synthetic import generate_synthetic_sessions	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
12	from apexsim.data.validate import validate_canonical_frame	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.pipeline.runner import run_pipeline	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	app = typer.Typer(no_args_is_help=True, help="Project APEX F1 world-simulation e	Creates or updates `app`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
19	def generate(config: Path = Path("configs/fast.yaml"), output: Path = Path("data	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
20	cfg = load_config(config)	Creates or updates `cfg`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	frame = generate_synthetic_sessions(cfg, output)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	typer.echo(f"Generated {len(frame):,} canonical telemetry rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 23-44

```

0023
0024
0025 @app.command("ingest-fastf1")
0026 def ingest_fastf1(
0027     year: int,
0028     event: str,
0029     session: str,
0030     driver: str,
0031     output: Path = Path("data/raw/fastf1.csv"),
0032     sample_hz: int = 5,
0033 ) -> None:
0034     frame = ingest_fastf1_session(year, event, session, driver, output, sample_hz)
0035     typer.echo(f"Ingested {len(frame):,} FastF1 rows at {output}")
0036
0037
0038 @app.command("ingest-openf1")
0039 def ingest_openf1(
0040     session_key: int,
0041     driver_number: int,
0042     output: Path = Path("data/raw/openf1.csv"),
0043     sample_hz: int = 4,
0044 ) -> None:

```

Line	Code signal	What it is doing / what to verify
25	@app.command("ingest-fastf1")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
26	def ingest_fastf1(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
27	year: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	event: str,	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
29	session: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	driver: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	output: Path = Path("data/raw/fastf1.csv"),	Creates or updates `output: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	sample_hz: int = 5,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
34	frame = ingest_fastf1_session(year, event, session, driver, output, sample_hz)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	typer.echo(f"Ingested {len(frame):,} FastF1 rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	@app.command("ingest-openf1")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
39	def ingest_openf1(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	session_key: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	driver_number: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	output: Path = Path("data/raw/openf1.csv"),	Creates or updates `output: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	sample_hz: int = 4,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-66

```

0045     frame = ingest_openf1_session(session_key, driver_number, output, sample_hz)
0046     typer.echo(f"Ingested {len(frame):,} OpenF1 rows at {output}")
0047
0048
0049     @app.command()
0050     def validate(path: Path) -> None:
0051         import pandas as pd
0052
0053         report = validate_canonical_frame(pd.read_csv(path), strict=False)
0054         typer.echo(json.dumps(report.to_dict(), indent=2))
0055         if not report.passed:

```

Source → contract → runtime evidence → failure test

```

0056         raise typer.Exit(code=1)
0057
0058
0059 @app.command()
0060 def run(
0061     config: Path = Path("configs/fast.yaml"),
0062     run_id: str = "reference_gru",
0063 ) -> None:
0064     summary = run_pipeline(load_config(config), run_id)
0065     typer.echo(json.dumps(summary, indent=2))
0066

```

Line	Code signal	What it is doing / what to verify
45	frame = ingest_openf1_session(session_key, driver_number, output, sample_hz)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	typer.echo(f"Ingested {len(frame);} OpenF1 rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
49	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
50	def validate(path: Path) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
51	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
53	report = validate_canonical_frame(pd.read_csv(path), strict=False)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
54	typer.echo(json.dumps(report.to_dict(), indent=2))	Creates or updates `typer.echo(json.dumps(report.to_dict(), indent)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	if not report.passed:	Branches only when this condition is true; verify both the true and false paths in tests.
56	raise typer.Exit(code=1)	Stops execution because an invariant or supported-condition check failed.
59	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
60	def run(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
61	config: Path = Path("configs/fast.yaml"),	Creates or updates `config: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	run_id: str = "reference_gru",	Creates or updates `run_id: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	summary = run_pipeline(load_config(config), run_id)	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	typer.echo(json.dumps(summary, indent=2))	Creates or updates

		<code>`typer.echo(json.dumps(summary, indent`.</code> Check its unit, shape, lifetime and whether it is derived only from information available at this point.
--	--	---

Lines 67-88

```

0067
0068 @app.command("run-canonical")
0069 def run_canonical(
0070     input_path: Path,
0071     config: Path = Path("configs/fast.yaml"),
0072     run_id: str = "historical_world_model",
0073 ) -> None:
0074     """Run the full pipeline from a validated canonical FastF1/OpenF1 CSV."""
0075     cfg = load_config(config)
0076     if not input_path.exists():
0077         raise typer.BadParameter(f"Canonical input does not exist: {input_path}")
0078     cfg.data.canonical_input_path = input_path
0079     summary = run_pipeline(cfg, run_id)
0080     typer.echo(json.dumps(summary, indent=2))
0081
0082
0083 @app.command()
0084 def ui(
0085     run_dir: Path = Path("artifacts/runs/reference_gru"),
0086     config: Path = Path("configs/fast.yaml"),
0087     share: bool = False,
0088 ) -> None:

```

Line	Code signal	What it is doing / what to verify
68	<code>@app.command("run-canonical")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
69	<code>def run_canonical(</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
70	<code>input_path: Path,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	<code>config: Path = Path("configs/fast.yaml"),</code>	Creates or updates <code>`config: Path`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	<code>run_id: str = "historical_world_model",</code>	Creates or updates <code>`run_id: str`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
73	<code>) -> None:</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	<code>"""Run the full pipeline from a validated canonical FastF1/OpenF1 CSV."""</code>	Begins or ends documentation describing the module, class or function contract.
75	<code>cfg = load_config(config)</code>	Creates or updates <code>`cfg`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>if not input_path.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
77	<code>raise typer.BadParameter(f"Canonical input does not exist: {input_path}")</code>	Stops execution because an invariant or supported-condition check failed.
78	<code>cfg.data.canonical_input_path = input_path</code>	Creates or updates <code>`cfg.data.canonical_input_path`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at

		this point.
79	summary = run_pipeline(cfg, run_id)	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	typer.echo(json.dumps(summary, indent=2))	Creates or updates `typer.echo(json.dumps(summary, indent=2))`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
84	def ui(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
85	run_dir: Path = Path("artifacts/runs/reference_gru"),	Creates or updates `run_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	config: Path = Path("configs/fast.yaml"),	Creates or updates `config: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	share: bool = False,	Creates or updates `share: bool`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
88	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-107

```

0089     from apexsim.ui import launch
0090
0091     launch(run_dir, config, share=share)
0092
0093
0094 @app.command()
0095 def api(
0096     artifacts_dir: Path = Path("artifacts/runs"),
0097     host: str = "127.0.0.1",
0098     port: int = 8000,
0099 ) -> None:
0100     import uvicorn
0101     from apexsim.serving import create_api
0102
0103     uvicorn.run(create_api(artifacts_dir), host=host, port=port)
0104
0105
0106 if __name__ == "__main__":
0107     app()

```

Line	Code signal	What it is doing / what to verify
89	from apexsim.ui import launch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
91	launch(run_dir, config, share=share)	Creates or updates `launch(run_dir, config, share)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.

95	def api(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
96	artifacts_dir: Path = Path("artifacts/runs"),	Creates or updates `artifacts_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	host: str = "127.0.0.1",	Creates or updates `host: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
98	port: int = 8000,	Creates or updates `port: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
99	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	import uvicorn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
101	from apexsim.serving import create_api	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
103	uvicorn.run(create_api(artifacts_dir), host=host, port=port)	Creates or updates `uvicorn.run(create_api(artifacts_dir), host)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	if __name__ == "__main__":	Branches only when this condition is true; verify both the true and false paths in tests.
107	app()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

src/apexsim/config.py

Loads and validates experiment choices before expensive work begins.

Typed configuration turns hidden constants into reproducible evidence and stops incompatible history, horizon or feature choices at the boundary.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004 from typing import Literal
0005
0006 import yaml
0007 from pydantic import BaseModel, Field, model_validator
0008
0009
0010 class DataConfig(BaseModel):
0011     source: Literal["synthetic", "fastf1", "openf1"] = "synthetic"
0012     raw_dir: Path = Path("data/raw")
0013     processed_dir: Path = Path("data/processed")
0014     canonical_input_path: Path | None = None
0015     sample_hz: int = Field(5, ge=1, le=50)
0016     sessions: int = Field(8, ge=3)
0017     laps_per_session: int = Field(3, ge=1)
0018     track_length_m: float = Field(5200.0, gt=500)
0019     session_seconds: int = Field(210, ge=30)
0020     train_fraction: float = 0.625
0021     val_fraction: float = 0.125
0022     test_fraction: float = 0.25
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from typing import Literal	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import yaml	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from pydantic import BaseModel, Field, model_validator	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	class DataConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
11	source: Literal["synthetic", "fastf1", "openf1"] = "synthetic"	Creates or updates `source: Literal["synthetic", "fastf1", "openf1"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
12	raw_dir: Path = Path("data/raw")	Creates or updates `raw_dir: Path`. Check its

Source → contract → runtime evidence → failure test

		unit, shape, lifetime and whether it is derived only from information available at this point.
13	processed_dir: Path = Path("data/processed")	Creates or updates `processed_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
14	canonical_input_path: Path None = None	Creates or updates `canonical_input_path: Path   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	sample_hz: int = Field(5, ge=1, le=50)	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	sessions: int = Field(8, ge=3)	Creates or updates `sessions: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	laps_per_session: int = Field(3, ge=1)	Creates or updates `laps_per_session: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	track_length_m: float = Field(5200.0, gt=500)	Creates or updates `track_length_m: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	session_seconds: int = Field(210, ge=30)	Creates or updates `session_seconds: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	train_fraction: float = 0.625	Creates or updates `train_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	val_fraction: float = 0.125	Creates or updates `val_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	test_fraction: float = 0.25	Creates or updates `test_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     sequence_length: int = Field(32, ge=4)
0024     prediction_horizon: int = Field(8, ge=1)
0025
0026     @model_validator(mode="after")
0027     def fractions_sum_to_one(self) -> "DataConfig":
0028         total = self.train_fraction + self.val_fraction + self.test_fraction
0029         if abs(total - 1.0) > 1e-6:
0030             raise ValueError("train/val/test fractions must sum to 1.0")
0031         return self
0032
0033
0034     class ModelConfig(BaseModel):
0035         kind: Literal["gru", "rsm", "ssm", "linear"] = "gru"
0036         hidden_dim: int = Field(64, ge=8)
0037         latent_dim: int = Field(16, ge=2)
0038         layers: int = Field(1, ge=1, le=8)
0039         dropout: float = Field(0.0, ge=0.0, lt=1.0)
0040
0041

```

```

0042 class TrainingConfig(BaseModel):
0043     epochs: int = Field(4, ge=1)
0044     batch_size: int = Field(64, ge=1)

```

Line	Code signal	What it is doing / what to verify
23	sequence_length: int = Field(32, ge=4)	Creates or updates `sequence_length: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	prediction_horizon: int = Field(8, ge=1)	Creates or updates `prediction_horizon: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	@model_validator(mode="after")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
27	def fractions_sum_to_one(self) -> "DataConfig":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
28	total = self.train_fraction + self.val_fraction + self.test_fraction	Creates or updates `total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	if abs(total - 1.0) > 1e-6:	Branches only when this condition is true; verify both the true and false paths in tests.
30	raise ValueError("train/val/test fractions must sum to 1.0")	Stops execution because an invariant or supported-condition check failed.
31	return self	Returns the result to the caller; this is the function output boundary that tests should assert.
34	class ModelConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
35	kind: Literal["gru", "rsm", "ssm", "linear"] = "gru"	Creates or updates `kind: Literal["gru", "rsm", "ssm", "linear"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	hidden_dim: int = Field(64, ge=8)	Creates or updates `hidden_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	latent_dim: int = Field(16, ge=2)	Creates or updates `latent_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	layers: int = Field(1, ge=1, le=8)	Creates or updates `layers: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	dropout: float = Field(0.0, ge=0.0, lt=1.0)	Creates or updates `dropout: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	class TrainingConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
43	epochs: int = Field(4, ge=1)	Creates or updates `epochs: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	batch_size: int = Field(64, ge=1)	Creates or updates `batch_size: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

this point.

Lines 45-66

```

0045     learning_rate: float = Field(1e-3, gt=0)
0046     weight_decay: float = Field(1e-4, ge=0)
0047     grad_clip: float = Field(1.0, gt=0)
0048     patience: int = Field(3, ge=1)
0049     device: Literal["cpu", "cuda", "mps"] = "cpu"
0050     max_train_windows: int | None = Field(3500, ge=32)
0051
0052
0053     class SimulationConfig(BaseModel):
0054         rollout_steps: int = Field(80, ge=1)
0055         throttle_scale: float = Field(1.0, ge=0.0, le=2.0)
0056         brake_scale: float = Field(1.0, ge=0.0, le=2.0)
0057         grip_multiplier: float = Field(1.0, ge=0.3, le=1.5)
0058         rain_delta: float = Field(0.0, ge=-1.0, le=1.0)
0059         tyre_degradation_multiplier: float = Field(1.0, ge=0.1, le=3.0)
0060
0061
0062     class ProjectConfig(BaseModel):
0063         project_name: str = "Project APEX"
0064         seed: int = 42
0065         artifacts_dir: Path = Path("artifacts/runs")
0066         data: DataConfig = DataConfig()

```

Line	Code signal	What it is doing / what to verify
45	learning_rate: float = Field(1e-3, gt=0)	Creates or updates `learning_rate: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	weight_decay: float = Field(1e-4, ge=0)	Creates or updates `weight_decay: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	grad_clip: float = Field(1.0, gt=0)	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
48	patience: int = Field(3, ge=1)	Creates or updates `patience: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	device: Literal["cpu", "cuda", "mps"] = "cpu"	Creates or updates `device: Literal["cpu", "cuda", "mps"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	max_train_windows: int None = Field(3500, ge=32)	Creates or updates `max_train_windows: int   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	class SimulationConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
54	rollout_steps: int = Field(80, ge=1)	Creates or updates `rollout_steps: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	throttle_scale: float = Field(1.0, ge=0.0, le=2.0)	Creates or updates `throttle_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	brake_scale: float = Field(1.0, ge=0.0, le=2.0)	Creates or updates `brake_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	grip_multiplier: float = Field(1.0, ge=0.3,	Creates or updates `grip_multiplier: float`.

	le=1.5)	Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	rain_delta: float = Field(0.0, ge=-1.0, le=1.0)	Creates or updates `rain_delta: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	tyre_degradation_multiplier: float = Field(1.0, ge=0.1, le=3.0)	Creates or updates `tyre_degradation_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	class ProjectConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
63	project_name: str = "Project APEX"	Creates or updates `project_name: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	seed: int = 42	Creates or updates `seed: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	artifacts_dir: Path = Path("artifacts/runs")	Creates or updates `artifacts_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	data: DataConfig = DataConfig()	Creates or updates `data: DataConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-78

```

0067     model: ModelConfig = ModelConfig()
0068     training: TrainingConfig = TrainingConfig()
0069     simulation: SimulationConfig = SimulationConfig()
0070
0071
0072 def load_config(path: str | Path) -> ProjectConfig:
0073     config_path = Path(path)
0074     if not config_path.exists():
0075         raise FileNotFoundError(f"Configuration file does not exist: {config_path}")
0076     with config_path.open("r", encoding="utf-8") as handle:
0077         payload = yaml.safe_load(handle) or {}
0078     return ProjectConfig.model_validate(payload)

```

Line	Code signal	What it is doing / what to verify
67	model: ModelConfig = ModelConfig()	Creates or updates `model: ModelConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	training: TrainingConfig = TrainingConfig()	Creates or updates `training: TrainingConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	simulation: SimulationConfig = SimulationConfig()	Creates or updates `simulation: SimulationConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	def load_config(path: str Path) -> ProjectConfig:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
73	config_path = Path(path)	Creates or updates `config_path`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
74	if not config_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
75	raise FileNotFoundError(f"Configuration file does not exist: {config_path}")	Stops execution because an invariant or supported-condition check failed.
76	with config_path.open("r", encoding="utf-8") as handle:	Opens a managed resource or context and guarantees cleanup when the block exits.
77	payload = yaml.safe_load(handle) or {}	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	return ProjectConfig.model_validate(payload)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/contracts.py

Defines the canonical telemetry language used by every data source and model.

Treat field order, unit and availability time as part of the API. A model checkpoint is compatible only with the exact contract and preprocessing version it was trained against.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004 from typing import Final
0005
0006 import pandas as pd
0007
0008 # Canonical columns are deliberately independent of FastF1, OpenF1, and F1 25.
0009 # Every source adapter must translate into this stable contract.
0010 IDENTITY_COLUMNS: Final[list[str]] = [
0011     "session_id",
0012     "source",
0013     "track_id",
0014     "driver_id",
0015     "timestamp_s",
0016     "lap_number",
0017 ]
0018
0019 STATE_COLUMNS: Final[list[str]] = [
0020     "speed_mps",
0021     "acceleration_mps2",
0022     "track_progress_sin",
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from typing import Final	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	# Canonical columns are deliberately independent of FastF1, OpenF1, and F1 25.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
9	# Every source adapter must translate into this stable contract.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
10	IDENTITY_COLUMNS: Final[list[str]] = [	Creates or updates `IDENTITY_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
11	"session_id",	Executes this statement inside the current

Source → contract → runtime evidence → failure test

		control-flow block; connect it to the nearest function contract and testable invariant.
12	"source",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
13	"track_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	"driver_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	"timestamp_s",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	"lap_number",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	STATE_COLUMNS: Final[list[str]] = [	Creates or updates `STATE_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	"speed_mps",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	"acceleration_mps2",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	"track_progress_sin",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     "track_progress_cos",
0024     "tyre_age_laps",
0025 ]
0026
0027 ACTION_COLUMNS: Final[list[str]] = [
0028     "throttle",
0029     "brake",
0030     "gear_norm",
0031     "drs",
0032     "steering_proxy",
0033 ]
0034
0035 CONTEXT_COLUMNS: Final[list[str]] = [
0036     "curvature",
0037     "air_temp_c",
0038     "track_temp_c",
0039     "rainfall",
0040     "wind_speed_mps",
0041     "grip_level",
0042 ]
0043
0044 AUX_COLUMNS: Final[list[str]] = [

```

Line	Code signal	What it is doing / what to verify
23	"track_progress_cos",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	"tyre_age_laps",	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
25	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	ACTION_COLUMNS: Final[list[str]] = [	Creates or updates `ACTION_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	"throttle",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	"brake",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	"gear_norm",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	"drs",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	"steering_proxy",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	CONTEXT_COLUMNS: Final[list[str]] = [	Creates or updates `CONTEXT_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	"curvature",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	"air_temp_c",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	"track_temp_c",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	"rainfall",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	"wind_speed_mps",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	"grip_level",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	AUX_COLUMNS: Final[list[str]] = [	Creates or updates `AUX_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     "lap_distance_m",
0046     "x_m",
0047     "y_m",

```

```

0048     "rpm",
0049     "gear",
0050     "compound_id",
0051     "is_pit",
0052     "safety_car",
0053 ]
0054
0055 REQUIRED_COLUMNS: Final[list[str]] = (
0056     IDENTITY_COLUMNS + STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS + AUX_COLUMNS
0057 )
0058
0059 MODEL_INPUT_COLUMNS: Final[list[str]] = STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS
0060 TARGET_COLUMNS: Final[list[str]] = STATE_COLUMNS
0061
0062
0063 @dataclass(frozen=True)
0064 class ContractReport:
0065     rows: int
0066     sessions: int

```

Line	Code signal	What it is doing / what to verify
45	"lap_distance_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	"x_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	"y_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	"rpm",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	"gear",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	"compound_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	"is_pit",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	"safety_car",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
55	REQUIRED_COLUMNS: Final[list[str]] = (	Creates or updates `REQUIRED_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	IDENTITY_COLUMNS + STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS + AUX_COLUMN	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
59	MODEL_INPUT_COLUMNS: Final[list[str]] = STATE_COLUMNS + ACTION_COLUMNS + CONTEXT	Creates or updates `MODEL_INPUT_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	TARGET_COLUMNS: Final[list[str]] = STATE_COLUMNS	Creates or updates `TARGET_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime

		and whether it is derived only from information available at this point.
63	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
64	class ContractReport:	Defines a type that owns related state and behaviour behind a named interface.
65	rows: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
66	sessions: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 67-81

```

0067     drivers: int
0068     tracks: int
0069     null_cells: int
0070     duplicate_frames: int
0071     out_of_range_cells: int
0072     passed: bool
0073
0074     def to_dict(self) -> dict[str, int | bool]:
0075         return self.__dict__.copy()
0076
0077
0078 def assert_canonical_frame(frame: pd.DataFrame) -> None:
0079     missing = sorted(set(REQUIRED_COLUMNS) - set(frame.columns))
0080     if missing:
0081         raise ValueError(f"Canonical frame is missing columns: {missing}")

```

Line	Code signal	What it is doing / what to verify
67	drivers: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
68	tracks: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
69	null_cells: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
70	duplicate_frames: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	out_of_range_cells: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
72	passed: bool	Provides an intentionally empty block; verify that this is a deliberate extension point.
74	def to_dict(self) -> dict[str, int bool]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
75	return self.__dict__.copy()	Returns the result to the caller; this is the function output boundary that tests should assert.
78	def assert_canonical_frame(frame: pd.DataFrame) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
79	missing = sorted(set(REQUIRED_COLUMNS) - set(frame.columns))	Creates or updates `missing`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

80	if missing:	Branches only when this condition is true; verify both the true and false paths in tests.
81	raise ValueError(f'Canonical frame is missing columns: {missing}')	Stops execution because an invariant or supported-condition check failed.

src/apexsim/data/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """Data ingestion, validation, alignment, feature, and windowing utilities."""
```

Line	Code signal	What it is doing / what to verify
1	"""Data ingestion, validation, alignment, feature, and windowing utilities."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/data/fastf1_adapter.py

Converts FastF1 session objects into the canonical telemetry contract.

Source-specific names, rates and units end here; downstream model code must not depend on FastF1 internals.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import numpy as np
0006 import pandas as pd
0007
0008
0009 def ingest_fastf1_session(
0010     year: int,
0011     event: str,
0012     session_code: str,
0013     driver: str,
0014     output_path: str | Path,
0015     sample_hz: int = 5,
0016 ) -> pd.DataFrame:
0017     """Fetch a historical FastF1 session and translate it into APEX's canonical contract.
0018
0019     FastF1 is an optional dependency because the complete project must remain runnable offline.
0020     """
0021     try:
0022         import fastf1

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def ingest_fastf1_session(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	year: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	event: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	session_code: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

13	driver: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	output_path: str Path,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	sample_hz: int = 5,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	) -> pd.DataFrame:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	"""Fetch a historical FastF1 session and translate it into APEX's canonical cont	Begins or ends documentation describing the module, class or function contract.
19	FastF1 is an optional dependency because the complete project must remain runnab	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	"""	Begins or ends documentation describing the module, class or function contract.
21	try:	Begins an operation that may fail for an expected reason.
22	import fastf1	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Lines 23-44

```

0023     except ImportError as exc:
0024         raise RuntimeError(
0025             "FastF1 is not installed. Install the project with the 'real-data' extra."
0026         ) from exc
0027
0028     cache_dir = Path(".cache/fastf1")
0029     cache_dir.mkdir(parents=True, exist_ok=True)
0030     fastf1.Cache.enable_cache(str(cache_dir))
0031     session = fastf1.get_session(year, event, session_code)
0032     session.load(telemetry=True, laps=True, weather=True, messages=False)
0033
0034     driver_laps = session.laps.pick_drivers(driver)
0035     rows: list[pd.DataFrame] = []
0036     weather = session.weather_data.copy()
0037     if "Time" in weather:
0038         weather["timestamp_s"] = weather["Time"].dt.total_seconds()
0039
0040     for _, lap in driver_laps.iterlaps():
0041         telemetry = lap.get_telemetry().copy()
0042         if telemetry.empty:
0043             continue
0044         telemetry["timestamp_s"] = telemetry["SessionTime"].dt.total_seconds()

```

Line	Code signal	What it is doing / what to verify
23	except ImportError as exc:	Handles a specific failure mode. The handler should preserve useful diagnostic evidence.
24	raise RuntimeError(	Stops execution because an invariant or supported-condition check failed.
25	"FastF1 is not installed. Install the project with the 'real-data' extra."	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	) from exc	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	cache_dir = Path(".cache/fastf1")	Creates or updates `cache_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
29	<code>cache_dir.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>`cache_dir.mkdir(parents)`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	<code>fastf1.Cache.enable_cache(str(cache_dir))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	<code>session = fastf1.get_session(year, event, session_code)</code>	Creates or updates <code>`session`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>session.load(telemetry=True, laps=True, weather=True, messages=False)</code>	Creates or updates <code>`session.load(telemetry)`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>driver_laps = session.laps.pick_drivers(driver)</code>	Creates or updates <code>`driver_laps`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>rows: list[pd.DataFrame] = []</code>	Creates or updates <code>`rows: list[pd.DataFrame]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>weather = session.weather_data.copy()</code>	Creates or updates <code>`weather`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	<code>if "Time" in weather:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
38	<code>weather["timestamp_s"] = weather["Time"].dt.total_seconds()</code>	Creates or updates <code>`weather["timestamp_s"]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>for _, lap in driver_laps.iterlaps():</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
41	<code>telemetry = lap.get_telemetry().copy()</code>	Creates or updates <code>`telemetry`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>if telemetry.empty:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
43	<code>continue</code>	Skips the remainder of the current iteration and advances to the next item.
44	<code>telemetry["timestamp_s"] = telemetry["SessionTime"].dt.total_seconds()</code>	Creates or updates <code>`telemetry["timestamp_s"]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     start, end = telemetry.timestamp_s.min(), telemetry.timestamp_s.max()
0046     regular_time = np.arange(start, end, 1.0 / sample_hz)
0047     telemetry = telemetry.set_index("timestamp_s").reindex(regular_time).interpolate().ffill().bfill()
0048     telemetry.index.name = "timestamp_s"
0049     telemetry = telemetry.reset_index()
0050     telemetry["lap_number"] = int(lap.LapNumber)
0051     telemetry["tyre_age_laps"] = float(lap.TyreLife or 0) + (
0052         telemetry["Distance"] / max(float(telemetry["Distance"].max()), 1.0)
0053     )

```

```

0054     rows.append(telemetry)
0055
0056     if not rows:
0057         raise RuntimeError("FastF1 returned no telemetry for the requested driver/session.")
0058     tele = pd.concat(rows, ignore_index=True).sort_values("timestamp_s")
0059     distance = tele["Distance"].fillna(method="ffill").fillna(0.0)
0060     progress = (distance / max(float(distance.max()), 1.0)) % 1.0
0061     speed_mps = tele["Speed"].astype(float) / 3.6
0062     dt = tele.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)
0063     acceleration = speed_mps.diff().fillna(0.0) / dt
0064     x = tele.get("X", pd.Series(np.zeros(len(tele)))).astype(float)
0065     y = tele.get("Y", pd.Series(np.zeros(len(tele)))).astype(float)
0066     heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))

```

Line	Code signal	What it is doing / what to verify
45	start, end = telemetry.timestamp_s.min(), telemetry.timestamp_s.max()	Creates or updates `start, end`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	regular_time = np.arange(start, end, 1.0 / sample_hz)	Creates or updates `regular_time`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	telemetry = telemetry.set_index("timestamp_s").reindex(regular_time).interpolate	Creates or updates `telemetry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	telemetry.index.name = "timestamp_s"	Creates or updates `telemetry.index.name`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	telemetry = telemetry.reset_index()	Creates or updates `telemetry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	telemetry["lap_number"] = int(lap.LapNumber)	Creates or updates `telemetry["lap_number"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	telemetry["tyre_age_laps"] = float(lap.TyreLife or 0) + (	Creates or updates `telemetry["tyre_age_laps"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	telemetry["Distance"] / max(float(telemetry["Distance"].max()), 1.0)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
53	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
54	rows.append(telemetry)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	if not rows:	Branches only when this condition is true; verify both the true and false paths in tests.
57	raise RuntimeError("FastF1 returned no telemetry for the requested driver/sessio	Stops execution because an invariant or supported-condition check failed.
58	tele = pd.concat(rows, ignore_index=True).sort_values("timestamp_ s")	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
59	distance = tele["Distance"].fillna(method="ffill").fillna(0. 0)	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
60	<code>progress = (distance / max(float(distance.max()), 1.0)) % 1.0</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>speed_mps = tele["Speed"].astype(float) / 3.6</code>	Creates or updates `speed_mps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	<code>dt = tele.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)</code>	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	<code>acceleration = speed_mps.diff().fillna(0.0) / dt</code>	Creates or updates `acceleration`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	<code>x = tele.get("X", pd.Series(np.zeros(len(tele))))).astype(float)</code>	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>y = tele.get("Y", pd.Series(np.zeros(len(tele))))).astype(float)</code>	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	<code>heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067 steering = np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1)
0068 curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)
0069
0070 canonical = pd.DataFrame(
0071     {
0072         "session_id": f"FF1_{year}_{event}_{session_code}",
0073         "source": "fastf1",
0074         "track_id": str(event),
0075         "driver_id": str(driver),
0076         "timestamp_s": tele.timestamp_s,
0077         "lap_number": tele.lap_number,
0078         "speed_mps": speed_mps,
0079         "acceleration_mps2": acceleration,
0080         "track_progress_sin": np.sin(2 * np.pi * progress),
0081         "track_progress_cos": np.cos(2 * np.pi * progress),
0082         "tyre_age_laps": tele.tyre_age_laps,
0083         "throttle": tele["Throttle"].astype(float) / 100.0,
0084         "brake": tele["Brake"].astype(float),
0085         "gear_norm": tele["nGear"].astype(float).clip(0, 8) / 8.0,
0086         "drs": (tele["DRS"].astype(float) > 9).astype(float),
0087         "steering_proxy": steering,
0088         "curvature": curvature,

```

Line	Code signal	What it is doing / what to verify
67	<code>steering = np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
68	<code>curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
70	<code>canonical = pd.DataFrame(</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
72	"session_id": f"FF1_{year}_{event}" _session_code",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	"source": "fastf1",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	"track_id": str(event),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
75	"driver_id": str(driver),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	"timestamp_s": tele.timestamp_s,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	"lap_number": tele.lap_number,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	"speed_mps": speed_mps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	"acceleration_mps2": acceleration,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
80	"track_progress_sin": np.sin(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
81	"track_progress_cos": np.cos(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
82	"tyre_age_laps": tele.tyre_age_laps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
83	"throttle": tele["Throttle"].astype(float) / 100.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
84	"brake": tele["Brake"].astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
85	"gear_norm": tele["nGear"].astype(float).clip(0, 8) / 8.0,	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
86	"drs": (tele["DRS"].astype(float) > 9).astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
87	"steering_proxy": steering,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-107

```

0089         "air_temp_c": 25.0,
0090         "track_temp_c": 35.0,
0091         "rainfall": 0.0,
0092         "wind_speed_mps": 2.0,
0093         "grip_level": 1.0,
0094         "lap_distance_m": distance,
0095         "x_m": x,
```

Source → contract → runtime evidence → failure test

```

0096         "y_m": y,
0097         "rpm": tele["RPM"].astype(float),
0098         "gear": tele["nGear"].astype(int),
0099         "compound_id": 0,
0100         "is_pit": 0,
0101         "safety_car": 0,
0102     }
0103 )
0104 output = Path(output_path)
0105 output.parent.mkdir(parents=True, exist_ok=True)
0106 canonical.to_csv(output, index=False)
0107 return canonical

```

Line	Code signal	What it is doing / what to verify
89	"air_temp_c": 25.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
90	"track_temp_c": 35.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"rainfall": 0.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	"wind_speed_mps": 2.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
93	"grip_level": 1.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
94	"lap_distance_m": distance,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	"rpm": tele["RPM"].astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	"gear": tele["nGear"].astype(int),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	"compound_id": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
102	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
103	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
104	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

105	<code>output.parent.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>`output.parent.mkdir(parents`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	<code>canonical.to_csv(output, index=False)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
107	<code>return canonical</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/features.py

Creates causal derived signals and feature arrays.

Every feature must be available at prediction time and have one stable unit/order definition.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import pandas as pd
0007
0008 from apexsim.contracts import MODEL_INPUT_COLUMNS, TARGET_COLUMNS
0009
0010
0011 @dataclass
0012 class Standardizer:
0013     input_mean: np.ndarray
0014     input_std: np.ndarray
0015     target_mean: np.ndarray
0016     target_std: np.ndarray
0017
0018     @classmethod
0019     def fit(cls, frame: pd.DataFrame) -> "Standardizer":
0020         x = frame[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0021         y = frame[TARGET_COLUMNS].to_numpy(np.float32)
0022         x_std = x.std(axis=0)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.contracts import MODEL_INPUT_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	@dataclass	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
12	class Standardizer:	Defines a type that owns related state and behaviour behind a named interface.
13	input_mean: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	input_std: np.ndarray	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
15	target_mean: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	target_std: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	@classmethod	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
19	def fit(cls, frame: pd.DataFrame) -> "Standardizer":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
20	x = frame[MODEL_INPUT_COLUMNS].to_numpy(np.float32)	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	y = frame[TARGET_COLUMNS].to_numpy(np.float32)	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	x_std = x.std(axis=0)	Creates or updates `x_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     y_std = y.std(axis=0)
0024     return cls(
0025         input_mean=x.mean(axis=0),
0026         input_std=np.where(x_std < 1e-6, 1.0, x_std),
0027         target_mean=y.mean(axis=0),
0028         target_std=np.where(y_std < 1e-6, 1.0, y_std),
0029     )
0030
0031     def transform_inputs(self, values: np.ndarray) -> np.ndarray:
0032         return (values - self.input_mean) / self.input_std
0033
0034     def transform_targets(self, values: np.ndarray) -> np.ndarray:
0035         return (values - self.target_mean) / self.target_std
0036
0037     def inverse_targets(self, values: np.ndarray) -> np.ndarray:
0038         return values * self.target_std + self.target_mean
0039
0040     def to_dict(self) -> dict[str, list[float]]:
0041         return {
0042             "input_mean": self.input_mean.tolist(),
0043             "input_std": self.input_std.tolist(),
0044             "target_mean": self.target_mean.tolist(),

```

Line	Code signal	What it is doing / what to verify
23	y_std = y.std(axis=0)	Creates or updates `y_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	return cls(	Returns the result to the caller; this is the function output boundary that tests should assert.
25	input_mean=x.mean(axis=0),	Creates or updates `input_mean`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	input_std=np.where(x_std < 1e-6, 1.0, x_std),	Creates or updates `input_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
27	target_mean=y.mean(axis=0),	Creates or updates `target_mean`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	target_std=np.where(y_std < 1e-6, 1.0, y_std),	Creates or updates `target_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	def transform_inputs(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	return (values - self.input_mean) / self.input_std	Returns the result to the caller; this is the function output boundary that tests should assert.
34	def transform_targets(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
35	return (values - self.target_mean) / self.target_std	Returns the result to the caller; this is the function output boundary that tests should assert.
37	def inverse_targets(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
38	return values * self.target_std + self.target_mean	Returns the result to the caller; this is the function output boundary that tests should assert.
40	def to_dict(self) -> dict[str, list[float]]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
41	return {	Returns the result to the caller; this is the function output boundary that tests should assert.
42	"input_mean": self.input_mean.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	"input_std": self.input_std.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	"target_mean": self.target_mean.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-50

```

0045     "target_std": self.target_std.tolist(),
0046     }
0047
0048     @classmethod
0049     def from_dict(cls, payload: dict[str, list[float]]) -> "Standardizer":
0050         return cls(**{key: np.asarray(value, dtype=np.float32) for key, value in payload.items()})

```

Line	Code signal	What it is doing / what to verify
45	"target_std": self.target_std.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

48	@classmethod	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
49	def from_dict(cls, payload: dict[str, list[float]]) -> "Standardizer":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
50	return cls(**{key: np.asarray(value, dtype=np.float32) for key, value in payload	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/openf1_adapter.py

Retrieves and aligns OpenF1 endpoint streams.

Every as-of join needs a direction, tolerance and freshness interpretation; nearest data is not automatically simultaneous data.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import numpy as np
0006 import pandas as pd
0007 import requests
0008
0009
0010 BASE_URL = "https://api.openf1.org/v1"
0011
0012
0013 def _get(endpoint: str, params: dict[str, int | str], timeout: int = 60) -> list[dict]:
0014     response = requests.get(f"{BASE_URL}/{endpoint}", params=params, timeout=timeout)
0015     response.raise_for_status()
0016     payload = response.json()
0017     if not isinstance(payload, list):
0018         raise ValueError(f"Expected a list from OpenF1 {endpoint}, got {type(payload).__name__}")
0019     return payload
0020
0021
0022 def ingest_openf1_session(
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import requests	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	BASE_URL = "https://api.openf1.org/v1"	Creates or updates `BASE_URL`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	def _get(endpoint: str, params: dict[str, int str], timeout: int = 60) -> list	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
14	response =	Creates or updates `response`. Check its unit,

Source → contract → runtime evidence → failure test

	<code>requests.get(f'{BASE_URL}/{endpoint}', params=params, timeout=timeout</code>	shape, lifetime and whether it is derived only from information available at this point.
15	<code>response.raise_for_status()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>payload = response.json()</code>	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>if not isinstance(payload, list):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
18	<code>raise ValueError(f'Expected a list from OpenF1 {endpoint}, got {type(payload)}.')</code>	Stops execution because an invariant or supported-condition check failed.
19	<code>return payload</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
22	<code>def ingest_openf1_session(</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.

Lines 23-44

```

0023     session_key: int,
0024     driver_number: int,
0025     output_path: str | Path,
0026     sample_hz: int = 4,
0027 ) -> pd.DataFrame:
0028     """Translate OpenF1 car/location/weather streams into the canonical frame.
0029
0030     Historical OpenF1 data is rich but the endpoints are sampled independently. We therefore align
0031     streams on time instead of assuming row N in one endpoint matches row N in another.
0032     """
0033     car = pd.DataFrame(_get("car_data", {"session_key": session_key, "driver_number": driver_number}))
0034     location = pd.DataFrame(_get("location", {"session_key": session_key, "driver_number": driver_number}))
0035     laps = pd.DataFrame(_get("laps", {"session_key": session_key, "driver_number": driver_number}))
0036     weather = pd.DataFrame(_get("weather", {"session_key": session_key}))
0037     if car.empty or location.empty:
0038         raise RuntimeError("OpenF1 returned no car or location data for the request.")
0039
0040     for frame in (car, location, weather):
0041         frame["date"] = pd.to_datetime(frame["date"], utc=True)
0042     car = car.sort_values("date")
0043     location = location.sort_values("date")
0044     weather = weather.sort_values("date")

```

Line	Code signal	What it is doing / what to verify
23	<code>session_key: int,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>driver_number: int,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
25	<code>output_path: str Path,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	<code>sample_hz: int = 4,</code>	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	<code>) -> pd.DataFrame:</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	<code>"""Translate OpenF1 car/location/weather streams into the canonical frame.</code>	Begins or ends documentation describing the module, class or function contract.

30	Historical OpenF1 data is rich but the endpoints are sampled independently. We t	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	streams on time instead of assuming row N in one endpoint matches row N in anoth	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	""	Begins or ends documentation describing the module, class or function contract.
33	car = pd.DataFrame(_get("car_data", {"session_key": session_key, "driver_number"	Creates or updates `car`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	location = pd.DataFrame(_get("location", {"session_key": session_key, "driver_nu	Creates or updates `location`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	laps = pd.DataFrame(_get("laps", {"session_key": session_key, "driver_number": d	Creates or updates `laps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	weather = pd.DataFrame(_get("weather", {"session_key": session_key}))	Creates or updates `weather`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	if car.empty or location.empty:	Branches only when this condition is true; verify both the true and false paths in tests.
38	raise RuntimeError("OpenF1 returned no car or location data for the request.")	Stops execution because an invariant or supported-condition check failed.
40	for frame in (car, location, weather):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
41	frame["date"] = pd.to_datetime(frame["date"], utc=True)	Creates or updates `frame["date"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	car = car.sort_values("date")	Creates or updates `car`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	location = location.sort_values("date")	Creates or updates `location`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	weather = weather.sort_values("date")	Creates or updates `weather`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045 merged = pd.merge_asof(car, location, on="date", direction="nearest", tolerance=pd.Timedelta("1s"))
0046 merged = pd.merge_asof(merged, weather, on="date", direction="nearest", tolerance=pd.Timedelta("20s"),
suffixes=("", "_weather"))
0047 merged = merged.dropna(subset=["speed", "x", "y"])
0048 start = merged.date.min()
0049 merged["timestamp_s"] = (merged.date - start).dt.total_seconds()
0050 regular_time = np.arange(0, merged.timestamp_s.max(), 1.0 / sample_hz)
0051 numeric = merged.select_dtypes(include=[np.number]).copy()
0052 numeric.index = merged.timestamp_s
0053 numeric = numeric[~numeric.index.duplicated()].reindex(regular_time).interpolate().ffill().bfill()
0054 numeric.index.name = "timestamp_s"
0055 numeric = numeric.reset_index()
0056
0057 speed_mps = numeric["speed"] / 3.6

```

Source → contract → runtime evidence → failure test

```

0058 dt = numeric.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)
0059 accel = speed_mps.diff().fillna(0.0) / dt
0060 x, y = numeric["x"], numeric["y"]
0061 heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))
0062 curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)
0063 distance = np.cumsum(speed_mps.to_numpy() / sample_hz)
0064 track_length = max(float(np.quantile(distance, 0.99)), 1.0)
0065 progress = (distance / track_length) % 1.0
0066

```

Line	Code signal	What it is doing / what to verify
45	<code>merged = pd.merge_asof(car, location, on="date", direction="nearest", tolerance=</code>	Aligns asynchronous time streams by nearby timestamps; direction, tolerance and sort order are part of the scientific contract.
46	<code>merged = pd.merge_asof(merged, weather, on="date", direction="nearest", toleranc</code>	Aligns asynchronous time streams by nearby timestamps; direction, tolerance and sort order are part of the scientific contract.
47	<code>merged = merged.dropna(subset=["speed", "x", "y"])</code>	Creates or updates `merged`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	<code>start = merged.date.min()</code>	Creates or updates `start`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>merged["timestamp_s"] = (merged.date - start).dt.total_seconds()</code>	Creates or updates `merged["timestamp_s"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>regular_time = np.arange(0, merged.timestamp_s.max(), 1.0 / sample_hz)</code>	Creates or updates `regular_time`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	<code>numeric = merged.select_dtypes(include=[np.number]).copy()</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	<code>numeric.index = merged.timestamp_s</code>	Creates or updates `numeric.index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	<code>numeric = numeric[~numeric.index.duplicated()].reindex(regular_time).interpolate</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	<code>numeric.index.name = "timestamp_s"</code>	Creates or updates `numeric.index.name`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	<code>numeric = numeric.reset_index()</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	<code>speed_mps = numeric["speed"] / 3.6</code>	Creates or updates `speed_mps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	<code>dt = numeric.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)</code>	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	<code>accel = speed_mps.diff().fillna(0.0) / dt</code>	Creates or updates `accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
60	<code>x, y = numeric["x"], numeric["y"]</code>	Creates or updates `x, y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	<code>curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
63	<code>distance = np.cumsum(speed_mps.to_numpy0 / sample_hz)</code>	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	<code>track_length = max(float(np.quantile(distance, 0.99)), 1.0)</code>	Creates or updates `track_length`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>progress = (distance / track_length) % 1.0</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     canonical = pd.DataFrame(
0068     {
0069         "session_id": f"OF1_{session_key}",
0070         "source": "openf1",
0071         "track_id": f"session_{session_key}",
0072         "driver_id": str(driver_number),
0073         "timestamp_s": numeric.timestamp_s,
0074         "lap_number": (distance // track_length + 1).astype(int),
0075         "speed_mps": speed_mps,
0076         "acceleration_mps2": accel,
0077         "track_progress_sin": np.sin(2 * np.pi * progress),
0078         "track_progress_cos": np.cos(2 * np.pi * progress),
0079         "tyre_age_laps": distance / track_length,
0080         "throttle": numeric.get("throttle", pd.Series(np.zeros(len(numeric)))) / 100.0,
0081         "brake": numeric.get("brake", pd.Series(np.zeros(len(numeric))).clip(0, 1),
0082         "gear_norm": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) / 8.0,
0083         "drs": numeric.get("drs", pd.Series(np.zeros(len(numeric))).clip(0, 1),
0084         "steering_proxy": np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1),
0085         "curvature": curvature,
0086         "air_temp_c": numeric.get("air_temperature", pd.Series(np.full(len(numeric), 25.0))),
0087         "track_temp_c": numeric.get("track_temperature", pd.Series(np.full(len(numeric), 35.0))),
0088         "rainfall": numeric.get("rainfall", pd.Series(np.zeros(len(numeric))).clip(0, 1),

```

Line	Code signal	What it is doing / what to verify
67	<code>canonical = pd.DataFrame(</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
69	<code>"session_id": f"OF1_{session_key}",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
70	<code>"source": "openf1",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	<code>"track_id": f"session_{session_key}",</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
72	"driver_id": str(driver_number),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	"timestamp_s": numeric.timestamp_s,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	"lap_number": (distance // track_length + 1).astype(int),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
75	"speed_mps": speed_mps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	"acceleration_mps2": accel,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	"track_progress_sin": np.sin(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	"track_progress_cos": np.cos(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	"tyre_age_laps": distance / track_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
80	"throttle": numeric.get("throttle", pd.Series(np.zeros(len(numeric)))) / 100.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
81	"brake": numeric.get("brake", pd.Series(np.zeros(len(numeric)))) .clip(0, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
82	"gear_norm": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) / 8.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
83	"drs": numeric.get("drs", pd.Series(np.zeros(len(numeric)))) .clip(0, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
84	"steering_proxy": np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
85	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
86	"air_temp_c": numeric.get("air_temperature", pd.Series(np.full(len(numeric), 25.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
87	"track_temp_c": numeric.get("track_temperature", pd.Series(np.full(len(numeric),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	"rainfall": numeric.get("rainfall", pd.Series(np.zeros(len(numeric)))) .clip(0, 1	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.

Lines 89-104

```

0089     "wind_speed_mps": numeric.get("wind_speed", pd.Series(np.full(len(numeric), 2.0))),
0090     "grip_level": 1.0,
0091     "lap_distance_m": distance % track_length,
0092     "x_m": x,
0093     "y_m": y,
0094     "rpm": numeric.get("rpm", pd.Series(np.full(len(numeric), 9000.0))),
0095     "gear": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) .astype(int),

```

Source → contract → runtime evidence → failure test

```

0096         "compound_id": 0,
0097         "is_pit": 0,
0098         "safety_car": 0,
0099     }
0100 )
0101 output = Path(output_path)
0102 output.parent.mkdir(parents=True, exist_ok=True)
0103 canonical.to_csv(output, index=False)
0104 return canonical

```

Line	Code signal	What it is doing / what to verify
89	"wind_speed_mps": numeric.get("wind_speed", pd.Series(np.full(len(numeric), 2.0))	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
90	"grip_level": 1.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"lap_distance_m": distance % track_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
93	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
94	"rpm": numeric.get("rpm", pd.Series(np.full(len(numeric), 9000.0))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	"gear": numeric.get("n_gear", pd.Series(np.ones(len(numeric))))).astype(int) ,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	"compound_id": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
101	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
102	output.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `output.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
103	canonical.to_csv(output, index=False)	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
104	return canonical	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/synthetic.py

Creates a controlled causal telemetry world for offline learning and tests.

Synthetic data is used to verify contracts and interventions, not to justify real-world accuracy claims.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import math
0004 from pathlib import Path
0005
0006 import numpy as np
0007 import pandas as pd
0008
0009 from apexsim.config import ProjectConfig
0010
0011
0012 def _track_geometry(track_length_m: float, points: int, phase: float) -> pd.DataFrame:
0013     distance = np.linspace(0.0, track_length_m, points, endpoint=False)
0014     theta = 2.0 * np.pi * distance / track_length_m
0015     # A closed, non-circular educational track. It is not an F1 circuit replica.
0016     radius = 780.0 + 135.0 * np.sin(3 * theta + phase) + 70.0 * np.sin(7 * theta)
0017     x = radius * np.cos(theta) + 90.0 * np.sin(2 * theta)
0018     y = 0.78 * radius * np.sin(theta) + 65.0 * np.sin(5 * theta + phase)
0019     dx = np.gradient(x, distance)
0020     dy = np.gradient(y, distance)
0021     heading = np.unwrap(np.arctan2(dy, dx))
0022     curvature = np.abs(np.gradient(heading, distance))
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import math	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
12	def _track_geometry(track_length_m: float, points: int, phase: float) -> pd.DataFrame:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
13	distance = np.linspace(0.0, track_length_m, points, endpoint=False)	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
14	<code>theta = 2.0 * np.pi * distance / track_length_m</code>	Creates or updates `theta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	<code># A closed, non-circular educational track. It is not an F1 circuit replica.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
16	<code>radius = 780.0 + 135.0 * np.sin(3 * theta + phase) + 70.0 * np.sin(7 * theta)</code>	Creates or updates `radius`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>x = radius * np.cos(theta) + 90.0 * np.sin(2 * theta)</code>	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>y = 0.78 * radius * np.sin(theta) + 65.0 * np.sin(5 * theta + phase)</code>	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	<code>dx = np.gradient(x, distance)</code>	Creates or updates `dx`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>dy = np.gradient(y, distance)</code>	Creates or updates `dy`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	<code>heading = np.unwrap(np.arctan2(dy, dx))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	<code>curvature = np.abs(np.gradient(heading, distance))</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     curvature /= max(float(np.quantile(curvature, 0.99)), 1e-6)
0024     curvature = np.clip(curvature, 0.0, 1.5)
0025     return pd.DataFrame(
0026         {
0027             "lap_distance_m": distance,
0028             "x_m": x,
0029             "y_m": y,
0030             "curvature": curvature,
0031             "heading": heading,
0032         }
0033     )
0034
0035
0036 def _gear_from_speed(speed_mps: float) -> int:
0037     # Simple gear map for a teaching simulator.
0038     kmh = speed_mps * 3.6
0039     thresholds = [0, 70, 105, 140, 180, 220, 260, 300]
0040     return int(np.clip(np.searchsorted(thresholds, kmh, side="right"), 1, 8))
0041
0042
0043 def generate_synthetic_sessions(config: ProjectConfig, output_path: str | Path) -> pd.DataFrame:
0044     """Generate causal F1-like telemetry with actions, weather, tyre wear, and track geometry.

```

Line	Code signal	What it is doing / what to verify
23	<code>curvature /= max(float(np.quantile(curvature, 0.99)), 1e-6)</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	<code>curvature = np.clip(curvature, 0.0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it

		may hide distribution shift.
25	return pd.DataFrame(	Returns the result to the caller; this is the function output boundary that tests should assert.
26	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	"lap_distance_m": distance,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	"heading": heading,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	def gear_from_speed(speed_mps: float) -> int:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
37	# Simple gear map for a teaching simulator.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
38	kmh = speed_mps * 3.6	Creates or updates `kmh`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	thresholds = [0, 70, 105, 140, 180, 220, 260, 300]	Creates or updates `thresholds`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	return int(np.clip(np.searchsorted(thresholds, kmh, side="right"), 1, 8))	Returns the result to the caller; this is the function output boundary that tests should assert.
43	def generate_synthetic_sessions(config: ProjectConfig, output_path: str Path)	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
44	"""Generate causal F1-like telemetry with actions, weather, tyre wear, and track	Begins or ends documentation describing the module, class or function contract.

Lines 45-66

```

0045
0046     The generator gives us a controlled world where the hidden rules are known. This is invaluable
0047     for testing whether the pipeline and models recover cause-and-effect rather than dataset leakage.
0048     """
0049     rng = np.random.default_rng(config.seed)
0050     hz = config.data.sample_hz
0051     dt = 1.0 / hz
0052     frames_per_session = int(config.data.session_seconds * hz)
0053     output_frames: list[pd.DataFrame] = []

```

Source → contract → runtime evidence → failure test

```

0054
0055     for session_idx in range(config.data.sessions):
0056         track_id = f"synthetic_track_{session_idx % 3}"
0057         session_id = f"SYN_{session_idx:03d}"
0058         driver_id = f"DRV_{session_idx % 5:02d}"
0059         source = "synthetic"
0060         phase = (session_idx % 3) * 0.7
0061         geometry = _track_geometry(config.data.track_length_m, 2048, phase)
0062
0063         driver_aggression = 0.93 + 0.14 * rng.random()
0064         driver_smoothness = 0.85 + 0.14 * rng.random()
0065         wet_session = rng.random() < 0.28
0066         rainfall_base = rng.uniform(0.18, 0.65) if wet_session else rng.uniform(0.0, 0.035)

```

Line	Code signal	What it is doing / what to verify
46	The generator gives us a controlled world where the hidden rules are known. This	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	for testing whether the pipeline and models recover cause-and-effect rather than	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
48	"""	Begins or ends documentation describing the module, class or function contract.
49	rng = np.random.default_rng(config.seed)	Creates or updates `rng`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	hz = config.data.sample_hz	Creates or updates `hz`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	dt = 1.0 / hz	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	frames_per_session = int(config.data.session_seconds * hz)	Creates or updates `frames_per_session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	output_frames: list[pd.DataFrame] = []	Creates or updates `output_frames: list[pd.DataFrame]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	for session_idx in range(config.data.sessions):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
56	track_id = f"synthetic_track_{session_idx % 3}"	Creates or updates `track_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	session_id = f"SYN_{session_idx:03d}"	Creates or updates `session_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	driver_id = f"DRV_{session_idx % 5:02d}"	Creates or updates `driver_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	source = "synthetic"	Creates or updates `source`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	phase = (session_idx % 3) * 0.7	Creates or updates `phase`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

61	geometry = _track_geometry(config.data.track_length_m, 2048, phase)	Creates or updates `geometry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	driver_aggression = 0.93 + 0.14 * rng.random()	Creates or updates `driver_aggression`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	driver_smoothness = 0.85 + 0.14 * rng.random()	Creates or updates `driver_smoothness`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	wet_session = rng.random() < 0.28	Creates or updates `wet_session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	rainfall_base = rng.uniform(0.18, 0.65) if wet_session else rng.uniform(0.0, 0.0)	Creates or updates `rainfall_base`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     air_temp = rng.uniform(18.0, 31.0)
0068     track_temp = air_temp + rng.uniform(4.0, 18.0)
0069     wind = rng.uniform(0.5, 7.0)
0070     grip = rng.uniform(0.92, 1.06) * (1.0 - 0.35 * rainfall_base)
0071     compound_id = int(rng.integers(0, 3))
0072     tyre_deg_rate = [0.000035, 0.000022, 0.000015][compound_id]
0073
0074     speed = rng.uniform(45.0, 60.0)
0075     distance_total = 0.0
0076     previous_speed = speed
0077     previous_heading = 0.0
0078     records: list[dict[str, float | int | str]] = []
0079
0080     for frame_idx in range(frames_per_session):
0081         timestamp = frame_idx * dt
0082         progress = (distance_total % config.data.track_length_m) / config.data.track_length_m
0083         geo_idx = min(int(progress * len(geometry)), len(geometry) - 1)
0084         point = geometry.iloc[geo_idx]
0085         curvature = float(point.curvature)
0086         tyre_age_laps = distance_total / config.data.track_length_m
0087         lap_number = int(tyre_age_laps) + 1
0088

```

Line	Code signal	What it is doing / what to verify
67	air_temp = rng.uniform(18.0, 31.0)	Creates or updates `air_temp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	track_temp = air_temp + rng.uniform(4.0, 18.0)	Creates or updates `track_temp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	wind = rng.uniform(0.5, 7.0)	Creates or updates `wind`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
70	grip = rng.uniform(0.92, 1.06) * (1.0 - 0.35 * rainfall_base)	Creates or updates `grip`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	compound_id = int(rng.integers(0, 3))	Creates or updates `compound_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

		this point.
72	<code>tyre_deg_rate = [0.000035, 0.000022, 0.000015][compound_id]</code>	Creates or updates `tyre_deg_rate`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	<code>speed = rng.uniform(45.0, 60.0)</code>	Creates or updates `speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	<code>distance_total = 0.0</code>	Creates or updates `distance_total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>previous_speed = speed</code>	Creates or updates `previous_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
77	<code>previous_heading = 0.0</code>	Creates or updates `previous_heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	<code>records: list[dict[str, float int str]] = []</code>	Creates or updates `records: list[dict[str, float   int   str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>for frame_idx in range(frames_per_session):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
81	<code>timestamp = frame_idx * dt</code>	Creates or updates `timestamp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>progress = (distance_total % config.data.track_length_m) / config.data.track_len</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>geo_idx = min(int(progress * len(geometry)), len(geometry) - 1)</code>	Creates or updates `geo_idx`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>point = geometry.iloc[geo_idx]</code>	Creates or updates `point`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>curvature = float(point.curvature)</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>tyre_age_laps = distance_total / config.data.track_length_m</code>	Creates or updates `tyre_age_laps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	<code>lap_number = int(tyre_age_laps) + 1</code>	Creates or updates `lap_number`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 89-110

```

0089         # Slowly changing rain lets the model learn weather-conditioned dynamics.
0090         rainfall = float(np.clip(rainfall_base + 0.08 * math.sin(timestamp / 38.0 + phase), 0.0, 1.0))
0091         effective_grip = float(np.clip(grip - tyre_deg_rate * distance_total - 0.3 * rainfall, 0.45, 1.1))
0092         max_corner_speed = 92.0 * effective_grip / (1.0 + 4.8 * curvature)

```

Source → contract → runtime evidence → failure test

```

0093     straight_speed = 93.0 * driver_aggression
0094     target_speed = min(straight_speed, max_corner_speed)
0095     error = target_speed - speed
0096
0097     throttle = float(np.clip(0.5 + 0.045 * error, 0.0, 1.0))
0098     brake = float(np.clip(-0.06 * error - 0.12, 0.0, 1.0))
0099     # Smooth drivers avoid violent simultaneous input changes.
0100     control_noise = rng.normal(0.0, 0.025 * (1.1 - driver_smoothness))
0101     throttle = float(np.clip(throttle + control_noise, 0.0, 1.0))
0102     brake = float(np.clip(brake - control_noise, 0.0, 1.0))
0103     drs = int(curvature < 0.07 and throttle > 0.82 and rainfall < 0.12)
0104
0105     engine_accel = 13.5 * throttle * (1.0 + 0.05 * drs)
0106     brake_accel = 21.5 * brake
0107     aero_drag = 0.00145 * speed * speed
0108     corner_drag = 7.0 * curvature * speed / 80.0
0109     wet_drag = 2.0 * rainfall
0110     stochastic_force = rng.normal(0.0, 0.20)

```

Line	Code signal	What it is doing / what to verify
89	# Slowly changing rain lets the model learn weather-conditioned dynamics.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
90	rainfall = float(np.clip(rainfall_base + 0.08 * math.sin(timestamp / 38.0 + phas	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
91	effective_grip = float(np.clip(grip - tyre_deg_rate * distance_total - 0.3 * rai	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
92	max_corner_speed = 92.0 * effective_grip / (1.0 + 4.8 * curvature)	Creates or updates `max_corner_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
93	straight_speed = 93.0 * driver_aggression	Creates or updates `straight_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	target_speed = min(straight_speed, max_corner_speed)	Creates or updates `target_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	error = target_speed - speed	Creates or updates `error`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	throttle = float(np.clip(0.5 + 0.045 * error, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
98	brake = float(np.clip(-0.06 * error - 0.12, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
99	# Smooth drivers avoid violent simultaneous input changes.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
100	control_noise = rng.normal(0.0, 0.025 * (1.1 - driver_smoothness))	Creates or updates `control_noise`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
101	throttle = float(np.clip(throttle + control_noise, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
102	brake = float(np.clip(brake - control_noise, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
103	drs = int(curvature < 0.07 and throttle > 0.82	Creates or updates `drs`. Check its unit,

	and rainfall < 0.12)	shape, lifetime and whether it is derived only from information available at this point.
105	engine_accel = 13.5 * throttle * (1.0 + 0.05 * drs)	Creates or updates `engine_accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	brake_accel = 21.5 * brake	Creates or updates `brake_accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
107	aero_drag = 0.00145 * speed * speed	Creates or updates `aero_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
108	corner_drag = 7.0 * curvature * speed / 80.0	Creates or updates `corner_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
109	wet_drag = 2.0 * rainfall	Creates or updates `wet_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
110	stochastic_force = rng.normal(0.0, 0.20)	Creates or updates `stochastic_force`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 111-132

```

0111 acceleration = engine_accel - brake_accel - aero_drag - corner_drag - wet_drag + stochastic_force
0112 speed = float(np.clip(speed + acceleration * dt, 8.0, 101.0))
0113 distance_total += speed * dt
0114
0115 heading = float(point.heading)
0116 heading_change = (heading - previous_heading + np.pi) % (2 * np.pi) - np.pi
0117 steering_proxy = float(np.clip(heading_change / max(dt, 1e-6) / 1.8, -1.0, 1.0))
0118 previous_heading = heading
0119 gear = _gear_from_speed(speed)
0120 rpm = float(np.clip(4500 + 1100 * gear + 65 * speed + rng.normal(0, 180), 4000, 15000))
0121 acceleration_measured = (speed - previous_speed) / dt
0122 previous_speed = speed
0123
0124 records.append(
0125     {
0126         "session_id": session_id,
0127         "source": source,
0128         "track_id": track_id,
0129         "driver_id": driver_id,
0130         "timestamp_s": timestamp,
0131         "lap_number": lap_number,
0132         "speed_mps": speed,

```

Line	Code signal	What it is doing / what to verify
111	acceleration = engine_accel - brake_accel - aero_drag - corner_drag - wet_drag +	Creates or updates `acceleration`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	speed = float(np.clip(speed + acceleration * dt, 8.0, 101.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
113	distance_total += speed * dt	Creates or updates `distance_total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	heading = float(point.heading)	Creates or updates `heading`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
116	<code>heading_change = (heading - previous_heading + np.pi) % (2 * np.pi) - np.pi</code>	Creates or updates `heading_change`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
117	<code>steering_proxy = float(np.clip(heading_change / max(dt, 1e-6) / 1.8, -1.0, 1.0))</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
118	<code>previous_heading = heading</code>	Creates or updates `previous_heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	<code>gear = _gear_from_speed(speed)</code>	Creates or updates `gear`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	<code>rpm = float(np.clip(4500 + 1100 * gear + 65 * speed + rng.normal(0, 180), 4000,</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
121	<code>acceleration_measured = (speed - previous_speed) / dt</code>	Creates or updates `acceleration_measured`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	<code>previous_speed = speed</code>	Creates or updates `previous_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
124	<code>records.append(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
125	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
126	<code>"session_id": session_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
127	<code>"source": source,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
128	<code>"track_id": track_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
129	<code>"driver_id": driver_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
130	<code>"timestamp_s": timestamp,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
131	<code>"lap_number": lap_number,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
132	<code>"speed_mps": speed,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 133-154

```

0133     "acceleration_mps2": acceleration_measured,
0134     "track_progress_sin": math.sin(2 * math.pi * progress),
0135     "track_progress_cos": math.cos(2 * math.pi * progress),

```

Source → contract → runtime evidence → failure test

```

0136     "tyre_age_laps": tyre_age_laps,
0137     "throttle": throttle,
0138     "brake": brake,
0139     "gear_norm": gear / 8.0,
0140     "drs": float(drs),
0141     "steering_proxy": steering_proxy,
0142     "curvature": curvature,
0143     "air_temp_c": air_temp,
0144     "track_temp_c": track_temp,
0145     "rainfall": rainfall,
0146     "wind_speed_mps": wind,
0147     "grip_level": effective_grip,
0148     "lap_distance_m": distance_total % config.data.track_length_m,
0149     "x_m": float(point.x_m),
0150     "y_m": float(point.y_m),
0151     "rpm": rpm,
0152     "gear": gear,
0153     "compound_id": compound_id,
0154     "is_pit": 0,

```

Line	Code signal	What it is doing / what to verify
133	"acceleration_mps2": acceleration_measured,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
134	"track_progress_sin": math.sin(2 * math.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
135	"track_progress_cos": math.cos(2 * math.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
136	"tyre_age_laps": tyre_age_laps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
137	"throttle": throttle,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
138	"brake": brake,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
139	"gear_norm": gear / 8.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
140	"drs": float(drs),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
141	"steering_proxy": steering_proxy,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
142	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
143	"air_temp_c": air_temp,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
144	"track_temp_c": track_temp,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
145	"rainfall": rainfall,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
146	"wind_speed_mps": wind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
147	"grip_level": effective_grip,	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
148	"lap_distance_m": distance_total % config.data.track_length_m,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
149	"x_m": float(point.x_m),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
150	"y_m": float(point.y_m),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
151	"rpm": rpm,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
152	"gear": gear,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
153	"compound_id": compound_id,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
154	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 155-165

```

0155         "safety_car": 0,
0156     }
0157 )
0158
0159     output_frames.append(pd.DataFrame.from_records(records))
0160
0161     result = pd.concat(output_frames, ignore_index=True)
0162     output = Path(output_path)
0163     output.parent.mkdir(parents=True, exist_ok=True)
0164     result.to_csv(output, index=False)
0165     return result

```

Line	Code signal	What it is doing / what to verify
155	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
156	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
157	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
159	output_frames.append(pd.DataFrame.from_records(records))	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
161	result = pd.concat(output_frames, ignore_index=True)	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
162	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
163	output.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `output.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

164	<code>result.to_csv(output, index=False)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
165	<code>return result</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/validate.py

Implements schema, range, time and consistency quality gates.

The earliest cheap check should catch a defect before it reaches windows, normalization or training.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import numpy as np
0004 import pandas as pd
0005
0006 from apexsim.contracts import ContractReport, REQUIRED_COLUMNS, assert_canonical_frame
0007
0008
0009 def validate_canonical_frame(frame: pd.DataFrame, strict: bool = True) -> ContractReport:
0010     assert_canonical_frame(frame)
0011     null_cells = int(frame[REQUIRED_COLUMNS].isna().sum().sum())
0012     duplicate_frames = int(
0013         frame.duplicated(subset=["session_id", "driver_id", "timestamp_s"]).sum()
0014     )
0015
0016     violations = np.zeros(len(frame), dtype=bool)
0017     range_checks = {
0018         "speed_mps": (0.0, 120.0),
0019         "throttle": (0.0, 1.0),
0020         "brake": (0.0, 1.0),
0021         "gear_norm": (0.0, 1.0),
0022         "drs": (0.0, 1.0),
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from apexsim.contracts import ContractReport, REQUIRED_COLUMNS, assert_canonical	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def validate_canonical_frame(frame: pd.DataFrame, strict: bool = True) -> Contra	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	assert_canonical_frame(frame)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
11	null_cells = int(frame[REQUIRED_COLUMNS].isna().sum().sum())	Creates or updates `null_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
12	duplicate_frames = int(	Creates or updates `duplicate_frames`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
13	<code>frame.duplicated(subset=["session_id", "driver_id", "timestamp_s"]).sum()</code>	Creates or updates `frame.duplicated(subset)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
14	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>violations = np.zeros(len(frame), dtype=bool)</code>	Creates or updates `violations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>range_checks = {</code>	Creates or updates `range_checks`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>"speed_mps": (0.0, 120.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	<code>"throttle": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	<code>"brake": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	<code>"gear_norm": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	<code>"drs": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     "steering_proxy": (-1.0, 1.0),
0024     "rainfall": (0.0, 1.0),
0025     "grip_level": (0.2, 1.5),
0026 }
0027 for column, (minimum, maximum) in range_checks.items():
0028     values = pd.to_numeric(frame[column], errors="coerce")
0029     violations |= values.lt(minimum).to_numpy() | values.gt(maximum).to_numpy()
0030 out_of_range_cells = int(violations.sum())
0031
0032 passed = null_cells == 0 and duplicate_frames == 0 and out_of_range_cells == 0
0033 report = ContractReport(
0034     rows=len(frame),
0035     sessions=int(frame.session_id.nunique()),
0036     drivers=int(frame.driver_id.nunique()),
0037     tracks=int(frame.track_id.nunique()),
0038     null_cells=null_cells,
0039     duplicate_frames=duplicate_frames,
0040     out_of_range_cells=out_of_range_cells,
0041     passed=passed,
0042 )
0043 if strict and not passed:
0044     raise ValueError(f"Canonical data contract failed: {report.to_dict()}")

```

Line	Code signal	What it is doing / what to verify
23	<code>"steering_proxy": (-1.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>"rainfall": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

25	"grip_level": (0.2, 1.5),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	for column, (minimum, maximum) in range_checks.items():	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
28	values = pd.to_numeric(frame[column], errors="coerce")	Creates or updates `values`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	violations = values.lt(minimum).to_numpy() values.gt(maximum).to_numpy()	Creates or updates `violations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	out_of_range_cells = int(violations.sum())	Creates or updates `out_of_range_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	passed = null_cells == 0 and duplicate_frames == 0 and out_of_range_cells == 0	Provides an intentionally empty block; verify that this is a deliberate extension point.
33	report = ContractReport(	Creates or updates `report`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	rows=len(frame),	Creates or updates `rows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	sessions=int(frame.session_id.nunique()),	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	drivers=int(frame.driver_id.nunique()),	Creates or updates `drivers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	tracks=int(frame.track_id.nunique()),	Creates or updates `tracks`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	null_cells=null_cells,	Creates or updates `null_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	duplicate_frames=duplicate_frames,	Creates or updates `duplicate_frames`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	out_of_range_cells=out_of_range_cells,	Creates or updates `out_of_range_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	passed=passed,	Provides an intentionally empty block; verify that this is a deliberate extension point.
42	)	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
43	if strict and not passed:	Branches only when this condition is true; verify both the true and false paths in tests.
44	raise ValueError(f"Canonical data contract failed: {report.to_dict()}")	Stops execution because an invariant or supported-condition check failed.

Lines 45-45

0045 return report

Line	Code signal	What it is doing / what to verify
45	return report	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/windows.py

Builds group-safe sequence windows and normalization artifacts.

An off-by-one or cross-session window can make a model appear excellent while invalidating the experiment.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import pandas as pd
0007 import torch
0008 from torch.utils.data import Dataset
0009
0010 from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS
0011 from apexsim.data.features import Standardizer
0012
0013
0014 @dataclass(frozen=True)
0015 class WindowIndex:
0016     session_id: str
0017     start: int
0018     stop: int
0019
0020
0021 def split_sessions(
0022     frame: pd.DataFrame,
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from torch.utils.data import Dataset	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

14	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
15	class WindowIndex:	Defines a type that owns related state and behaviour behind a named interface.
16	session_id: str	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	start: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	stop: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	def split_sessions(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	frame: pd.DataFrame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     train_fraction: float,
0024     val_fraction: float,
0025     seed: int,
0026 ) -> dict[str, list[str]]:
0027     sessions = sorted(frame.session_id.unique().tolist())
0028     rng = np.random.default_rng(seed)
0029     rng.shuffle(sessions)
0030     n = len(sessions)
0031     n_train = max(1, int(round(n * train_fraction)))
0032     n_val = max(1, int(round(n * val_fraction)))
0033     if n_train + n_val >= n:
0034         n_val = 1
0035         n_train = max(1, n - 2)
0036     return {
0037         "train": sessions[:n_train],
0038         "val": sessions[n_train : n_train + n_val],
0039         "test": sessions[n_train + n_val :],
0040     }
0041
0042
0043 class TelemetryWindowDataset(Dataset):
0044     """Returns histories and next-state sequences without crossing session boundaries."""

```

Line	Code signal	What it is doing / what to verify
23	train_fraction: float,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	val_fraction: float,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
25	seed: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	) -> dict[str, list[str]]:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	sessions = sorted(frame.session_id.unique().tolist())	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	rng = np.random.default_rng(seed)	Creates or updates `rng`. Check its unit, shape, lifetime and whether it is derived

		only from information available at this point.
29	<code>rng.shuffle(sessions)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
30	<code>n = len(sessions)</code>	Creates or updates <code>`n`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>n_train = max(1, int(round(n * train_fraction)))</code>	Creates or updates <code>`n_train`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>n_val = max(1, int(round(n * val_fraction)))</code>	Creates or updates <code>`n_val`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>if n_train + n_val >= n:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
34	<code>n_val = 1</code>	Creates or updates <code>`n_val`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>n_train = max(1, n - 2)</code>	Creates or updates <code>`n_train`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>return {</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
37	<code>"train": sessions[:n_train],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	<code>"val": sessions[n_train : n_train + n_val],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>"test": sessions[n_train + n_val :],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	<code>}</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	<code>class TelemetryWindowDataset(Dataset):</code>	Defines a type that owns related state and behaviour behind a named interface.
44	<code>""Returns histories and next-state sequences without crossing session boundarie</code>	Begins or ends documentation describing the module, class or function contract.

Lines 45-66

```

0045
0046     def __init__(
0047         self,
0048         frame: pd.DataFrame,
0049         session_ids: list[str],
0050         sequence_length: int,
0051         prediction_horizon: int,
0052         standardizer: Standardizer,
0053         max_windows: int | None = None,
0054     ) -> None:
0055         self.sequence_length = sequence_length
0056         self.prediction_horizon = prediction_horizon
0057         self.standardizer = standardizer
0058         self.sessions: dict[str, pd.DataFrame] = {}

```

```

0059     self.indices: list[WindowIndex] = []
0060     total = sequence_length + prediction_horizon
0061     for session_id in session_ids:
0062         session = (
0063             frame[frame.session_id == session_id]
0064             .sort_values("timestamp_s")
0065             .reset_index(drop=True)
0066         )

```

Line	Code signal	What it is doing / what to verify
46	def __init__(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	self,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	frame: pd.DataFrame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	session_ids: list[str],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	sequence_length: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	prediction_horizon: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	max_windows: int None = None,	Creates or updates `max_windows: int   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
55	self.sequence_length = sequence_length	Creates or updates `self.sequence_length`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	self.prediction_horizon = prediction_horizon	Runs model inference using learned parameters without fitting on the requested examples.
57	self.standardizer = standardizer	Creates or updates `self.standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	self.sessions: dict[str, pd.DataFrame] = {}	Creates or updates `self.sessions: dict[str, pd.DataFrame]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	self.indices: list[WindowIndex] = []	Creates or updates `self.indices: list[WindowIndex]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	total = sequence_length + prediction_horizon	Creates or updates `total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	for session_id in session_ids:	Iterates over a collection or time dimension. Confirm order, boundaries and whether

		state should persist between iterations.
62	session = (	Creates or updates `session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	.sort_values("timestamp_s")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
65	.reset_index(drop=True)	Creates or updates `.reset_index(drop)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 67-88

```

0067         self.sessions[session_id] = session
0068         for start in range(0, len(session) - total + 1):
0069             self.indices.append(WindowIndex(session_id, start, start + total))
0070         if max_windows is not None and len(self.indices) > max_windows:
0071             positions = np.linspace(0, len(self.indices) - 1, max_windows).astype(int)
0072             self.indices = [self.indices[i] for i in positions]
0073
0074     def __len__(self) -> int:
0075         return len(self.indices)
0076
0077     def __getitem__(self, index: int) -> dict[str, torch.Tensor | str]:
0078         item = self.indices[index]
0079         window = self.sessions[item.session_id].iloc[item.start : item.stop]
0080         history = window.iloc[: self.sequence_length]
0081         future = window.iloc[self.sequence_length :]
0082         x_history = history[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0083         x_future = future[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0084         y_future = future[TARGET_COLUMNS].to_numpy(np.float32)
0085         state_history = history[STATE_COLUMNS].to_numpy(np.float32)
0086         return {
0087             "history": torch.from_numpy(self.standardizer.transform_inputs(x_history).astype(np.float32)),
0088             "future_inputs": torch.from_numpy(self.standardizer.transform_inputs(x_future).astype(np.float32)),

```

Line	Code signal	What it is doing / what to verify
67	self.sessions[session_id] = session	Creates or updates `self.sessions[session_id]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	for start in range(0, len(session) - total + 1):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
69	self.indices.append(WindowIndex(session_id, start, start + total))	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
70	if max_windows is not None and len(self.indices) > max_windows:	Branches only when this condition is true; verify both the true and false paths in tests.
71	positions = np.linspace(0, len(self.indices) - 1, max_windows).astype(int)	Creates or updates `positions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	self.indices = [self.indices[i] for i in positions]	Creates or updates `self.indices`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

74	<code>def __len__(self) -> int:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
75	<code>return len(self.indices)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
77	<code>def __getitem__(self, index: int) -> dict[str, torch.Tensor str]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
78	<code>item = self.indices[index]</code>	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
79	<code>window = self.sessions[item.session_id].iloc[item.start : item.stop]</code>	Creates or updates `window`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>history = window.iloc[: self.sequence_length]</code>	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>future = window.iloc[self.sequence_length :]</code>	Creates or updates `future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>x_history = history[MODEL_INPUT_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `x_history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>x_future = future[MODEL_INPUT_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `x_future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>y_future = future[TARGET_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `y_future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>state_history = history[STATE_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `state_history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>return {</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
87	<code>"history": torch.from_numpy(self.standardizer.transform_inputs(x_history).astype</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	<code>"future_inputs": torch.from_numpy(self.standardizer.transform_inputs(x_future).a</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-92

```

0089         "future_targets": torch.from_numpy(self.standardizer.transform_targets(y_future).astype(np.float32)),
0090         "state_history":
torch.from_numpy(self.standardizer.transform_targets(state_history).astype(np.float32)),
0091         "session_id": item.session_id,
0092     }

```

Line	Code signal	What it is doing / what to verify
89	<code>"future_targets": torch.from_numpy(self.standardizer.transform_targets(y_future)</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

90	"state_history": torch.from_numpy(self.standardizer.transfo rm_targets(state_hist	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"session_id": item.session_id,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

src/apexsim/evaluation.py

Converts standardized model output into physical and horizon-aware evidence.

Evaluation must use held-out groups, inverse transformations and physical guardrails; teacher-forced loss alone is not simulation quality.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import numpy as np
0007 import torch
0008 from torch import nn
0009 from torch.utils.data import DataLoader
0010
0011 from apexsim.contracts import TARGET_COLUMNS
0012 from apexsim.data.features import Standardizer
0013 from apexsim.models.rssm import RSSMWorldModel
0014
0015
0016 def evaluate_world_model(
0017     model: nn.Module,
0018     loader: DataLoader,
0019     standardizer: Standardizer,
0020     output_path: str | Path | None = None,
0021     device: str = "cpu",
0022 ) -> dict:
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.contracts import TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
12	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	def evaluate_world_model(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
17	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	output_path: str Path None = None,	Creates or updates `output_path: str   Path   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	device: str = "cpu",	Creates or updates `device: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	) -> dict:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     model.eval()
0024     model.to(device)
0025     predictions: list[np.ndarray] = []
0026     targets: list[np.ndarray] = []
0027     with torch.no_grad():
0028         for batch in loader:
0029             history = batch["history"].to(device)
0030             future_inputs = batch["future_inputs"].to(device)
0031             if isinstance(model, RSSMWorldModel):
0032                 predicted, _ = model(history, future_inputs, future_targets=None)
0033             else:
0034                 predicted = model(history, future_inputs)
0035             predictions.append(predicted.cpu().numpy())
0036             targets.append(batch["future_targets"].numpy())
0037     pred_z = np.concatenate(predictions, axis=0)
0038     target_z = np.concatenate(targets, axis=0)
0039     pred = standardizer.inverse_targets(pred_z)
0040     target = standardizer.inverse_targets(target_z)
0041     error = pred - target
0042     horizon_rmse = np.sqrt(np.mean(error**2, axis=(0, 2)))
0043     horizon_mae = np.mean(np.abs(error), axis=(0, 2))
0044     per_feature = {}

```

Line	Code signal	What it is doing / what to verify
23	model.eval()	Enables deterministic evaluation behaviour for layers that differ between training and inference.
24	model.to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
25	predictions: list[np.ndarray] = []	Creates or updates `predictions: list[np.ndarray]`. Check its unit, shape,

		lifetime and whether it is derived only from information available at this point.
26	targets: list[np.ndarray] = []	Creates or updates `targets: list[np.ndarray]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	with torch.no_grad():	Opens a managed resource or context and guarantees cleanup when the block exits.
28	for batch in loader:	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
29	history = batch["history"].to(device)	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	future_inputs = batch["future_inputs"].to(device)	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	if isinstance(model, RSSMWorldModel):	Branches only when this condition is true; verify both the true and false paths in tests.
32	predicted, _ = model(history, future_inputs, future_targets=None)	Creates or updates `predicted, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	else:	Handles all remaining cases; inspect whether this fallback is intentionally broad.
34	predicted = model(history, future_inputs)	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	predictions.append(predicted.cpu().numpy())	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	targets.append(batch["future_targets"].numpy())	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
37	pred_z = np.concatenate(predictions, axis=0)	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
38	target_z = np.concatenate(targets, axis=0)	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
39	pred = standardizer.inverse_targets(pred_z)	Creates or updates `pred`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	target = standardizer.inverse_targets(target_z)	Creates or updates `target`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	error = pred - target	Creates or updates `error`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	horizon_rmse = np.sqrt(np.mean(error**2, axis=(0, 2)))	Creates or updates `horizon_rmse`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	horizon_mae = np.mean(np.abs(error), axis=(0, 2))	Creates or updates `horizon_mae`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
44	per_feature = {}	Creates or updates `per_feature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     for index, feature in enumerate(TARGET_COLUMNS):
0046         per_feature[feature] = {
0047             "mae": float(np.mean(np.abs(error[... , index]))),
0048             "rmse": float(np.sqrt(np.mean(error[... , index] ** 2))),
0049         }
0050     physical = {
0051         "negative_speed_rate": float(np.mean(pred[... , 0] < 0)),
0052         "extreme_speed_rate": float(np.mean(pred[... , 0] > 120)),
0053         "progress_unit_circle_error": float(
0054             np.mean(np.abs(pred[... , 2] ** 2 + pred[... , 3] ** 2 - 1.0))
0055         ),
0056     }
0057     metrics = {
0058         "overall_mae": float(np.mean(np.abs(error))),
0059         "overall_rmse": float(np.sqrt(np.mean(error**2))),
0060         "speed_mae_mps": per_feature["speed_mps"]["mae"],
0061         "speed_rmse_mps": per_feature["speed_mps"]["rmse"],
0062         "per_feature": per_feature,
0063         "horizon_mae": horizon_mae.tolist(),
0064         "horizon_rmse": horizon_rmse.tolist(),
0065         "physical_violations": physical,
0066         "samples": int(pred.shape[0]),

```

Line	Code signal	What it is doing / what to verify
45	for index, feature in enumerate(TARGET_COLUMNS):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
46	per_feature[feature] = {	Creates or updates `per_feature[feature]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	"mae": float(np.mean(np.abs(error[... , index]))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	"rmse": float(np.sqrt(np.mean(error[... , index] ** 2))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	physical = {	Creates or updates `physical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	"negative_speed_rate": float(np.mean(pred[... , 0] < 0)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	"extreme_speed_rate": float(np.mean(pred[... , 0] > 120)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	"progress_unit_circle_error": float(np.mean(np.abs(pred[... , 2] ** 2 + pred[... , 3]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
54	** 2 - 1.0))	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

55	),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
56	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	metrics = {	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	"overall_mae": float(np.mean(np.abs(error))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
59	"overall_rmse": float(np.sqrt(np.mean(error**2))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
60	"speed_mae_mps": per_feature["speed_mps"] ["mae"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
61	"speed_rmse_mps": per_feature["speed_mps"]["rmse"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
62	"per_feature": per_feature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
63	"horizon_mae": horizon_mae.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	"horizon_rmse": horizon_rmse.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
65	"physical_violations": physical,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
66	"samples": int(pred.shape[0]),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 67-72

```

0067     }
0068     if output_path is not None:
0069         path = Path(output_path)
0070         path.parent.mkdir(parents=True, exist_ok=True)
0071         path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
0072     return metrics

```

Line	Code signal	What it is doing / what to verify
67	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
68	if output_path is not None:	Branches only when this condition is true; verify both the true and false paths in tests.
69	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
70	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

71	<code>path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
72	<code>return metrics</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """World-model and baseline architectures."""
```

Line	Code signal	What it is doing / what to verify
1	"""World-model and baseline architectures."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/models/baselines.py

Provides simple controls that expose leakage and unnecessary complexity.

A world model should not be promoted until it beats persistence and linear dynamics on the exact deployment protocol.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import numpy as np
0004 from sklearn.linear_model import Ridge
0005 from sklearn.multioutput import MultiOutputRegressor
0006
0007
0008 class PersistenceBaseline:
0009     """Predict that the next state is identical to the current state."""
0010
0011     def predict(self, current_state: np.ndarray, horizon: int) -> np.ndarray:
0012         return np.repeat(current_state[None, :], horizon, axis=0)
0013
0014
0015 class LinearTransitionBaseline:
0016     """Ridge regression for one-step state transitions."""
0017
0018     def __init__(self, alpha: float = 1.0) -> None:
0019         self.model = MultiOutputRegressor(Ridge(alpha=alpha))
0020
0021     def fit(self, inputs: np.ndarray, next_states: np.ndarray) -> "LinearTransitionBaseline":
0022         self.model.fit(inputs, next_states)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from sklearn.linear_model import Ridge	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from sklearn.multioutput import MultiOutputRegressor	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	class PersistenceBaseline:	Defines a type that owns related state and behaviour behind a named interface.
9	"""Predict that the next state is identical to the current state."""	Begins or ends documentation describing the module, class or function contract.
11	def predict(self, current_state: np.ndarray, horizon: int) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	return np.repeat(current_state[None, :], horizon, axis=0)	Returns the result to the caller; this is the function output boundary that tests should assert.

15	class LinearTransitionBaseline:	Defines a type that owns related state and behaviour behind a named interface.
16	"""Ridge regression for one-step state transitions."""	Begins or ends documentation describing the module, class or function contract.
18	def __init__(self, alpha: float = 1.0) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
19	self.model = MultiOutputRegressor(Ridge(alpha=alpha))	Creates or updates `self.model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	def fit(self, inputs: np.ndarray, next_states: np.ndarray) -> "LinearTransitionB	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	self.model.fit(inputs, next_states)	Learns parameters or preprocessing statistics from the supplied training evidence only.

Lines 23-26

```

0023         return self
0024
0025     def predict(self, inputs: np.ndarray) -> np.ndarray:
0026         return self.model.predict(inputs)

```

Line	Code signal	What it is doing / what to verify
23	return self	Returns the result to the caller; this is the function output boundary that tests should assert.
25	def predict(self, inputs: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	return self.model.predict(inputs)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/factory.py

Creates the configured model behind one common interface.

The factory isolates model selection so pipeline and evaluation logic do not fork by architecture.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-18

```
0001 from __future__ import annotations
0002
0003 from torch import nn
0004
0005 from apexsim.config import ModelConfig
0006 from apexsim.models.gru_world_model import GRUWorldModel
0007 from apexsim.models.rssm import RSSMWorldModel
0008 from apexsim.models.ssm_world_model import SSMWorldModel
0009
0010
0011 def build_model(config: ModelConfig, input_dim: int, state_dim: int) -> nn.Module:
0012     if config.kind == "gru":
0013         return GRUWorldModel(input_dim, state_dim, config.hidden_dim, config.layers, config.dropout)
0014     if config.kind == "ssm":
0015         return SSMWorldModel(input_dim, state_dim, config.hidden_dim, config.layers)
0016     if config.kind == "rssm":
0017         return RSSMWorldModel(input_dim, state_dim, config.hidden_dim, config.latent_dim)
0018     raise ValueError(f"Model kind {config.kind!r} is not a neural world model")
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from apexsim.config import ModelConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from apexsim.models.gru_world_model import GRUWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.models.ssm_world_model import SSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	def build_model(config: ModelConfig, input_dim: int, state_dim: int) -> nn.Module	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	if config.kind == "gru":	Branches only when this condition is true; verify both the true and false paths in tests.
13	return GRUWorldModel(input_dim, state_dim, config.hidden_dim, config.layers, con	Returns the result to the caller; this is the function output boundary that tests should assert.
14	if config.kind == "ssm":	Branches only when this condition is true;

Source → contract → runtime evidence → failure test

		verify both the true and false paths in tests.
15	return SSMWorldModel(input_dim, state_dim, config.hidden_dim, config.layers)	Returns the result to the caller; this is the function output boundary that tests should assert.
16	if config.kind == "rssm":	Branches only when this condition is true; verify both the true and false paths in tests.
17	return RSSMWorldModel(input_dim, state_dim, config.hidden_dim, config.latent_dim	Returns the result to the caller; this is the function output boundary that tests should assert.
18	raise ValueError(f"Model kind {config.kind! r} is not a neural world model")	Stops execution because an invariant or supported-condition check failed.

src/apexsim/models/gru_world_model.py

Implements deterministic recurrent history encoding and autoregressive rollout.

At each future step, known controls/context enter the transition while predicted state replaces the unknown future state.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005
0006
0007 class GRUWorldModel(nn.Module):
0008     """Deterministic sequence model that predicts future telemetry states.
0009
0010     The history encoder compresses the observed past into a hidden state. During rollout, the model
0011     consumes the planned future actions/context and its own previous predicted state.
0012     """
0013
0014     def __init__(
0015         self,
0016         input_dim: int,
0017         state_dim: int,
0018         hidden_dim: int = 64,
0019         layers: int = 1,
0020         dropout: float = 0.0,
0021     ) -> None:
0022         super().__init__()

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	class GRUWorldModel(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
8	"""Deterministic sequence model that predicts future telemetry states.	Begins or ends documentation describing the module, class or function contract.
10	The history encoder compresses the observed past into a hidden state. During rol	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	consumes the planned future actions/context and its own previous predicted state	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	"""	Begins or ends documentation describing the module, class or function contract.
14	def __init__(	Defines a callable boundary. Its parameters

		form the input contract and its return value or side effects form the output contract.
15	self,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	input_dim: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	state_dim: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	hidden_dim: int = 64,	Creates or updates `hidden_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	layers: int = 1,	Creates or updates `layers: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	dropout: float = 0.0,	Creates or updates `dropout: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	super().__init__()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 23-44

```

0023     self.state_dim = state_dim
0024     self.input_dim = input_dim
0025     self.encoder = nn.GRU(
0026         input_dim,
0027         hidden_dim,
0028         num_layers=layers,
0029         batch_first=True,
0030         dropout=dropout if layers > 1 else 0.0,
0031     )
0032     self.transition = nn.GRUCell(input_dim, hidden_dim)
0033     self.decoder = nn.Sequential(
0034         nn.Linear(hidden_dim, hidden_dim),
0035         nn.SiLU(),
0036         nn.Linear(hidden_dim, state_dim),
0037     )
0038
0039     def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.Tensor:
0040         _, hidden = self.encoder(history)
0041         hidden_t = hidden[-1]
0042         predictions = []
0043         future_t = future_inputs.clone()
0044         previous_state = history[:, -1, : self.state_dim]

```

Line	Code signal	What it is doing / what to verify
23	self.state_dim = state_dim	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	self.input_dim = input_dim	Creates or updates `self.input_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
25	self.encoder = nn.GRU(	Creates or updates `self.encoder`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
26	input_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	hidden_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	num_layers=layers,	Creates or updates `num_layers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	batch_first=True,	Creates or updates `batch_first`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	dropout=dropout if layers > 1 else 0.0,	Creates or updates `dropout`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
32	self.transition = nn.GRUCell(input_dim, hidden_dim)	Creates or updates `self.transition`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	self.decoder = nn.Sequential(	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	nn.Linear(hidden_dim, hidden_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	nn.SiLU(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	nn.Linear(hidden_dim, state_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.T	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	_, hidden = self.encoder(history)	Creates or updates `_, hidden`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	hidden_t = hidden[-1]	Creates or updates `hidden_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	predictions = []	Creates or updates `predictions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	future_t = future_inputs.clone()	Creates or updates `future_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
44	<code>previous_state = history[:, -1, : self.state_dim]</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-51

```

0045     for step in range(future_t.shape[1]):
0046         step_input = future_t[:, step].clone()
0047         step_input[:, : self.state_dim] = previous_state
0048         hidden_t = self.transition(step_input, hidden_t)
0049         previous_state = self.decoder(hidden_t)
0050         predictions.append(previous_state)
0051     return torch.stack(predictions, dim=1)

```

Line	Code signal	What it is doing / what to verify
45	<code>for step in range(future_t.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
46	<code>step_input = future_t[:, step].clone()</code>	Creates or updates `step_input`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	<code>step_input[:, : self.state_dim] = previous_state</code>	Creates or updates `step_input[:, : self.state_dim]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	<code>hidden_t = self.transition(step_input, hidden_t)</code>	Creates or updates `hidden_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>previous_state = self.decoder(hidden_t)</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>predictions.append(previous_state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
51	<code>return torch.stack(predictions, dim=1)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/rssm.py

Implements deterministic memory plus stochastic prior/posterior latent state.

Training may use observations through the posterior; imagination must proceed through the prior without future evidence.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005 from torch.distributions import Normal, kl_divergence
0006
0007
0008 class RSSMWorldModel(nn.Module):
0009     """Compact Dreamer-style recurrent state-space model for telemetry.
0010
0011     h_t is deterministic memory. z_t is a stochastic latent state. The posterior sees the observed
0012     state during training; the prior is used for imagination when future observations are unavailable.
0013     """
0014
0015     def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, latent_dim: int = 12) -> None:
0016         super().__init__()
0017         self.input_dim = input_dim
0018         self.state_dim = state_dim
0019         self.hidden_dim = hidden_dim
0020         self.latent_dim = latent_dim
0021         self.action_context_dim = input_dim - state_dim
0022         self.gru = nn.GRUCell(latent_dim + self.action_context_dim, hidden_dim)

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from torch.distributions import Normal, kl_divergence	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	class RSSMWorldModel(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
9	"""Compact Dreamer-style recurrent state-space model for telemetry.	Begins or ends documentation describing the module, class or function contract.
11	h_t is deterministic memory. z_t is a stochastic latent state. The posterior see	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	state during training; the prior is used for imagination when future observation	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

13	"""	Begins or ends documentation describing the module, class or function contract.
15	def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, latent_dim: int = 64, gru: nn.GRUCell = None):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
16	super().__init__()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	self.input_dim = input_dim	Creates or updates `self.input_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	self.state_dim = state_dim	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	self.hidden_dim = hidden_dim	Creates or updates `self.hidden_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	self.latent_dim = latent_dim	Creates or updates `self.latent_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	self.action_context_dim = input_dim - state_dim	Creates or updates `self.action_context_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	self.gru = nn.GRUCell(latent_dim + self.action_context_dim, hidden_dim)	Creates or updates `self.gru`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     self.prior = nn.Linear(hidden_dim, 2 * latent_dim)
0024     self.posterior = nn.Linear(hidden_dim + state_dim, 2 * latent_dim)
0025     self.decoder = nn.Sequential(
0026         nn.Linear(hidden_dim + latent_dim, hidden_dim),
0027         nn.SiLU(),
0028         nn.Linear(hidden_dim, state_dim),
0029     )
0030
0031     @staticmethod
0032     def _distribution(parameters: torch.Tensor) -> Normal:
0033         mean, raw_std = parameters.chunk(2, dim=-1)
0034         std = torch.nn.functional.softplus(raw_std) + 0.1
0035         return Normal(mean, std)
0036
0037     def forward(
0038         self,
0039         history: torch.Tensor,
0040         future_inputs: torch.Tensor,
0041         future_targets: torch.Tensor | None = None,
0042     ) -> tuple[torch.Tensor, torch.Tensor]:
0043         batch = history.shape[0]
0044         h = history.new_zeros((batch, self.hidden_dim))

```

Line	Code signal	What it is doing / what to verify
23	self.prior = nn.Linear(hidden_dim, 2 * latent_dim)	Creates or updates `self.prior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	self.posterior = nn.Linear(hidden_dim + state_dim, 2 * latent_dim)	Creates or updates `self.posterior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

		this point.
25	<code>self.decoder = nn.Sequential(</code>	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	<code>nn.Linear(hidden_dim + latent_dim, hidden_dim),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	<code>nn.SiLU(),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	<code>nn.Linear(hidden_dim, state_dim),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	<code>@staticmethod</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
32	<code>def _distribution(parameters: torch.Tensor) -> Normal:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
33	<code>mean, raw_std = parameters.chunk(2, dim=-1)</code>	Creates or updates `mean, raw_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>std = torch.nn.functional.softplus(raw_std) + 0.1</code>	Creates or updates `std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>return Normal(mean, std)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
37	<code>def forward(</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
38	<code>self,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>history: torch.Tensor,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	<code>future_inputs: torch.Tensor,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	<code>future_targets: torch.Tensor None = None,</code>	Creates or updates `future_targets: torch.Tensor   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>) -> tuple[torch.Tensor, torch.Tensor]:</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	<code>batch = history.shape[0]</code>	Creates or updates `batch`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	<code>h = history.new_zeros((batch, self.hidden_dim))</code>	Creates or updates `h`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     z = history.new_zeros((batch, self.latent_dim))
0046     # Observe history to initialize belief state.
0047     for step in range(history.shape[1]):
0048         state = history[:, step, : self.state_dim]
0049         action_context = history[:, step, self.state_dim :]
0050         h = self.gru(torch.cat([z, action_context], dim=-1), h)
0051         posterior = self._distribution(self.posterior(torch.cat([h, state], dim=-1)))
0052         z = posterior.rsample()
0053
0054     outputs = []
0055     kl_terms = []
0056     previous_state = history[:, -1, : self.state_dim]
0057     for step in range(future_inputs.shape[1]):
0058         action_context = future_inputs[:, step, self.state_dim :]
0059         h = self.gru(torch.cat([z, action_context], dim=-1), h)
0060         prior = self._distribution(self.prior(h))
0061         if future_targets is not None:
0062             posterior = self._distribution(
0063                 self.posterior(torch.cat([h, future_targets[:, step]], dim=-1))
0064             )
0065         z = posterior.rsample()
0066         kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1))

```

Line	Code signal	What it is doing / what to verify
45	<code>z = history.new_zeros((batch, self.latent_dim))</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	<code># Observe history to initialize belief state.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
47	<code>for step in range(history.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
48	<code>state = history[:, step, : self.state_dim]</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>action_context = history[:, step, self.state_dim :]</code>	Creates or updates `action_context`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>h = self.gru(torch.cat([z, action_context], dim=-1), h)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
51	<code>posterior = self._distribution(self.posterior(torch.cat([h, state], dim=-1)))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
52	<code>z = posterior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	<code>outputs = []</code>	Creates or updates `outputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	<code>kl_terms = []</code>	Creates or updates `kl_terms`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	<code>previous_state = history[:, -1, : self.state_dim]</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	<code>for step in range(future_inputs.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether

		state should persist between iterations.
58	<code>action_context = future_inputs[:, step, self.state_dim :]</code>	Creates or updates `action_context`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	<code>h = self.gru(torch.cat([z, action_context], dim=-1), h)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
60	<code>prior = self_distribution(self.prior(h))</code>	Creates or updates `prior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>if future_targets is not None:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
62	<code>posterior = self_distribution(</code>	Creates or updates `posterior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	<code>self.posterior(torch.cat([h, future_targets[:, step]], dim=-1))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
64	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
65	<code>z = posterior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	<code>kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1))</code>	Creates or updates `kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-73

```

0067         else:
0068             z = prior.rsample()
0069             previous_state = self.decoder(torch.cat([h, z], dim=-1))
0070             outputs.append(previous_state)
0071         prediction = torch.stack(outputs, dim=1)
0072         kl = torch.stack(kl_terms, dim=1).mean() if kl_terms else prediction.new_tensor(0.0)
0073         return prediction, kl

```

Line	Code signal	What it is doing / what to verify
67	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
68	<code>z = prior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	<code>previous_state = self.decoder(torch.cat([h, z], dim=-1))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
70	<code>outputs.append(previous_state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
71	<code>prediction = torch.stack(outputs, dim=1)</code>	Creates a new axis by stacking aligned values; document what the new axis represents.
72	<code>kl = torch.stack(kl_terms, dim=1).mean() if kl_terms else prediction.new_tensor(</code>	Creates a new axis by stacking aligned values; document what the new axis represents.
73	<code>return prediction, kl</code>	Returns the result to the caller; this is the

		function output boundary that tests should assert.
--	--	--

src/apexsim/models/ssm_world_model.py

Implements an educational stable selective state-space challenger.

It demonstrates content-dependent memory but does not claim full optimized Mamba fidelity.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005
0006
0007 class SelectiveSSMCell(nn.Module):
0008     """Small educational selective state-space cell.
0009
0010     This is intentionally not a full Mamba kernel. It demonstrates the central idea: a persistent
0011     state with input-dependent write/forget gates and stable diagonal decay.
0012     """
0013
0014     def __init__(self, input_dim: int, hidden_dim: int) -> None:
0015         super().__init__()
0016         self.logit_decay = nn.Parameter(torch.zeros(hidden_dim))
0017         self.input_projection = nn.Linear(input_dim, hidden_dim)
0018         self.gate_projection = nn.Linear(input_dim, hidden_dim)
0019
0020     def forward(self, x: torch.Tensor, state: torch.Tensor) -> torch.Tensor:
0021         decay = torch.sigmoid(self.logit_decay).unsqueeze(0)
0022         candidate = torch.tanh(self.gate_projection(x))
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	class SelectiveSSMCell(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
8	"""Small educational selective state-space cell.	Begins or ends documentation describing the module, class or function contract.
10	This is intentionally not a full Mamba kernel. It demonstrates the central idea:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	state with input-dependent write/forget gates and stable diagonal decay.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	"""	Begins or ends documentation describing the module, class or function contract.
14	def __init__(self, input_dim: int, hidden_dim: int) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value

		or side effects form the output contract.
15	<code>super().__init__()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>self.logit_decay = nn.Parameter(torch.zeros(hidden_dim))</code>	Creates or updates `self.logit_decay`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>self.input_projection = nn.Linear(input_dim, hidden_dim)</code>	Creates or updates `self.input_projection`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>self.gate_projection = nn.Linear(input_dim, hidden_dim)</code>	Creates or updates `self.gate_projection`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>def forward(self, x: torch.Tensor, state: torch.Tensor) -> torch.Tensor:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
21	<code>decay = torch.sigmoid(self.logit_decay).unsqueeze(0)</code>	Maps logits to values between zero and one; interpret them as probabilities only after calibration evidence.
22	<code>candidate = torch.tanh(self.input_projection(x))</code>	Creates or updates `candidate`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     gate = torch.sigmoid(self.gate_projection(x))
0024     return decay * state + (1.0 - decay) * gate * candidate
0025
0026
0027 class SSMWorldModel(nn.Module):
0028     def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, layers: int = 2) -> None:
0029         super().__init__()
0030         self.state_dim = state_dim
0031         self.cells = nn.ModuleList(
0032             [SelectiveSSMCell(input_dim if i == 0 else hidden_dim, hidden_dim) for i in range(layers)]
0033         )
0034         self.decoder = nn.Sequential(nn.LayerNorm(hidden_dim), nn.Linear(hidden_dim, state_dim))
0035
0036     def _step(self, x: torch.Tensor, states: list[torch.Tensor]) -> tuple[torch.Tensor, list[torch.Tensor]]:
0037         updated: list[torch.Tensor] = []
0038         value = x
0039         for cell, state in zip(self.cells, states):
0040             state = cell(value, state)
0041             updated.append(state)
0042             value = state
0043         return self.decoder(value), updated
0044

```

Line	Code signal	What it is doing / what to verify
23	<code>gate = torch.sigmoid(self.gate_projection(x))</code>	Maps logits to values between zero and one; interpret them as probabilities only after calibration evidence.
24	<code>return decay * state + (1.0 - decay) * gate * candidate</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
27	<code>class SSMWorldModel(nn.Module):</code>	Defines a type that owns related state and behaviour behind a named interface.
28	<code>def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, layers:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
29	<code>super().__init__()</code>	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
30	<code>self.state_dim = state_dim</code>	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>self.cells = nn.ModuleList(</code>	Creates or updates `self.cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>[SelectiveSSMCell(input_dim if i == 0 else hidden_dim, hidden_dim) for i in rang</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
34	<code>self.decoder = nn.Sequential(nn.LayerNorm(hidden_dim), nn.Linear(hidden_dim, sta</code>	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>def _step(self, x: torch.Tensor, states: list[torch.Tensor]) -> tuple[torch.Tens</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
37	<code>updated: list[torch.Tensor] = []</code>	Creates or updates `updated: list[torch.Tensor]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	<code>value = x</code>	Creates or updates `value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	<code>for cell, state in zip(self.cells, states):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
40	<code>state = cell(value, state)</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	<code>updated.append(state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
42	<code>value = state</code>	Creates or updates `value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	<code>return self.decoder(value), updated</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-58

```

0045     def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.Tensor:
0046         batch = history.shape[0]
0047         hidden_dim = self.cells[0].logit_decay.shape[0]
0048         states = [history.new_zeros((batch, hidden_dim)) for _ in self.cells]
0049         for step in range(history.shape[1]):
0050             _, states = self._step(history[:, step], states)
0051             previous_state = history[:, -1, : self.state_dim]
0052             outputs = []
0053             for step in range(future_inputs.shape[1]):
0054                 x = future_inputs[:, step].clone()
0055                 x[:, : self.state_dim] = previous_state
0056                 previous_state, states = self._step(x, states)
0057                 outputs.append(previous_state)

```

Source → contract → runtime evidence → failure test

0058 return torch.stack(outputs, dim=1)

Line	Code signal	What it is doing / what to verify
45	def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.T	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
46	batch = history.shape[0]	Creates or updates `batch`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	hidden_dim = self.cells[0].logit_decay.shape[0]	Creates or updates `hidden_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	states = [history.new_zeros((batch, hidden_dim)) for _ in self.cells]	Creates or updates `states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	for step in range(history.shape[1]):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
50	_, states = self._step(history[:, step], states)	Creates or updates `_, states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	previous_state = history[:, -1, : self.state_dim]	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	outputs = []	Creates or updates `outputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	for step in range(future_inputs.shape[1]):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
54	x = future_inputs[:, step].clone()	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	x[:, : self.state_dim] = previous_state	Creates or updates `x[:, : self.state_dim]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	previous_state, states = self._step(x, states)	Creates or updates `previous_state, states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	outputs.append(previous_state)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
58	return torch.stack(outputs, dim=1)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/pipeline/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """Reusable idempotent pipeline stages and local orchestration."""
```

Line	Code signal	What it is doing / what to verify
1	"""Reusable idempotent pipeline stages and local orchestration."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/pipeline/runner.py

Coordinates run lifecycle and stage transitions.

The runner owns ordering, failure state and summary publication while stage functions own domain logic.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from datetime import datetime, timezone
0005 from pathlib import Path
0006
0007 from apexsim.config import ProjectConfig
0008 from apexsim.pipeline.stages import (
0009     ablation_stage,
0010     dataset_stage,
0011     evaluate_stage,
0012     ingest_stage,
0013     publish_stage,
0014     quality_stage,
0015     train_stage,
0016 )
0017 from apexsim.registry import RunRegistry
0018
0019
0020 def run_pipeline(config: ProjectConfig, run_id: str) -> dict:
0021     run_dir = config.artifacts_dir / run_id
0022     run_dir.mkdir(parents=True, exist_ok=True)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from datetime import datetime, timezone	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.pipeline.stages import (	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	ablation_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
10	dataset_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

11	evaluate_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	ingest_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
13	publish_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	quality_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	train_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	from apexsim.registry import RunRegistry	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	def run_pipeline(config: ProjectConfig, run_id: str) -> dict:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
21	run_dir = config.artifacts_dir / run_id	Creates or updates `run_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	run_dir.mkdir(parents=True, exist_ok=True)	Creates or updates `run_dir.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023 registry = RunRegistry(config.artifacts_dir / "runs.sqlite")
0024 registry.start(run_id, config.model.kind, str(run_dir))
0025 started = datetime.now(timezone.utc).isoformat()
0026 try:
0027     canonical = ingest_stage(config, run_dir)
0028     quality = quality_stage(canonical, run_dir)
0029     dataset = dataset_stage(config, canonical, run_dir)
0030     model = train_stage(config, canonical, run_dir)
0031     metrics = evaluate_stage(config, canonical, run_dir)
0032     ablations = ablation_stage(config, canonical, run_dir)
0033     publication = publish_stage(config, canonical, run_dir)
0034     summary = {
0035         "run_id": run_id,
0036         "status": "succeeded",
0037         "started_at": started,
0038         "finished_at": datetime.now(timezone.utc).isoformat(),
0039         "canonical_data": str(canonical),
0040         "quality": quality,
0041         "splits": dataset["splits"],
0042         "model": model,
0043         "metrics": metrics,
0044         "best_ablation": ablations[0],

```

Line	Code signal	What it is doing / what to verify
23	registry = RunRegistry(config.artifacts_dir / "runs.sqlite")	Creates or updates `registry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	registry.start(run_id, config.model.kind, str(run_dir))	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
25	<code>started = datetime.now(timezone.utc).isoformat()</code>	Creates or updates `started`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	<code>try:</code>	Begins an operation that may fail for an expected reason.
27	<code>canonical = ingest_stage(config, run_dir)</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	<code>quality = quality_stage(canonical, run_dir)</code>	Creates or updates `quality`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	<code>dataset = dataset_stage(config, canonical, run_dir)</code>	Creates or updates `dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	<code>model = train_stage(config, canonical, run_dir)</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>metrics = evaluate_stage(config, canonical, run_dir)</code>	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>ablations = ablation_stage(config, canonical, run_dir)</code>	Creates or updates `ablations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>publication = publish_stage(config, canonical, run_dir)</code>	Creates or updates `publication`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>summary = {</code>	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>"run_id": run_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	<code>"status": "succeeded",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>"started_at": started,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	<code>"finished_at": datetime.now(timezone.utc).isoformat(),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>"canonical_data": str(canonical),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	<code>"quality": quality,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	<code>"splits": dataset["splits"],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	<code>"model": model,</code>	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
43	"metrics": metrics,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	"best_ablation": ablations[0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-52

```

0045     "publication": publication,
0046     }
0047     (run_dir / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
0048     registry.finish(run_id, "succeeded", metrics)
0049     return summary
0050 except Exception:
0051     registry.finish(run_id, "failed")
0052     raise

```

Line	Code signal	What it is doing / what to verify
45	"publication": publication,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	(run_dir / "summary.json").write_text(json.dumps(summary, indent=2), encoding="u	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
48	registry.finish(run_id, "succeeded", metrics)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
49	return summary	Returns the result to the caller; this is the function output boundary that tests should assert.
50	except Exception:	Handles a specific failure mode. The handler should preserve useful diagnostic evidence.
51	registry.finish(run_id, "failed")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
52	raise	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

src/apexsim/pipeline/stages.py

Defines reusable idempotent pipeline operations.

Each stage consumes paths/contracts and writes immutable evidence that can be retried independently.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import shutil
0005 from pathlib import Path
0006
0007 import joblib
0008 import numpy as np
0009 import pandas as pd
0010 import torch
0011 from torch.utils.data import DataLoader
0012
0013 from apexsim.config import ProjectConfig
0014 from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS
0015 from apexsim.data.features import Standardizer
0016 from apexsim.data.synthetic import generate_synthetic_sessions
0017 from apexsim.data.validate import validate_canonical_frame
0018 from apexsim.data.windows import TelemetryWindowDataset, split_sessions
0019 from apexsim.evaluation import evaluate_world_model
0020 from apexsim.models.factory import build_model
0021 from apexsim.training import train_world_model
0022
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import shutil	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import joblib	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Source → contract → runtime evidence → failure test

11	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	from apexsim.data.synthetic import generate_synthetic_sessions	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	from apexsim.data.validate import validate_canonical_frame	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
18	from apexsim.data.windows import TelemetryWindowDataset, split_sessions	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
19	from apexsim.evaluation import evaluate_world_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	from apexsim.models.factory import build_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
21	from apexsim.training import train_world_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Lines 23-44

```

0023
0024 def ingest_stage(config: ProjectConfig, run_dir: Path) -> Path:
0025     """Materialize one canonical telemetry file inside the immutable run directory.
0026
0027     Synthetic runs generate the file directly. Real-data runs first use a source
0028     adapter (FastF1 or OpenF1) to create the canonical contract, then provide that
0029     file through ``canonical_input_path``. Copying it into the run directory makes
0030     the exact training input part of run lineage and keeps downstream stages
0031     source-independent.
0032     """
0033     output = run_dir / "canonical_telemetry.csv"
0034     if output.exists():
0035         return output
0036
0037     if config.data.canonical_input_path is not None:
0038         source = Path(config.data.canonical_input_path)
0039         if not source.exists():
0040             raise FileNotFoundError(f"Canonical input does not exist: {source}")
0041         if source.resolve() != output.resolve():
0042             shutil.copy2(source, output)
0043         return output
0044

```

Line	Code signal	What it is doing / what to verify
24	def ingest_stage(config: ProjectConfig, run_dir: Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
25	"""Materialize one canonical telemetry file inside the immutable run directory.	Begins or ends documentation describing the module, class or function contract.
27	Synthetic runs generate the file directly. Real-data runs first use a source	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

28	adapter (FastF1 or OpenF1) to create the canonical contract, then provide that	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	file through ``canonical_input_path``. Copying it into the run directory makes	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	the exact training input part of run lineage and keeps downstream stages	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	source-independent.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	"""	Begins or ends documentation describing the module, class or function contract.
33	output = run_dir / "canonical_telemetry.csv"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
35	return output	Returns the result to the caller; this is the function output boundary that tests should assert.
37	if config.data.canonical_input_path is not None:	Branches only when this condition is true; verify both the true and false paths in tests.
38	source = Path(config.data.canonical_input_path)	Creates or updates `source`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	if not source.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
40	raise FileNotFoundError(f"Canonical input does not exist: {source}")	Stops execution because an invariant or supported-condition check failed.
41	if source.resolve() != output.resolve():	Branches only when this condition is true; verify both the true and false paths in tests.
42	shutil.copy2(source, output)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
43	return output	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```

0045     if config.data.source == "synthetic":
0046         generate_synthetic_sessions(config, output)
0047         return output
0048
0049     raise ValueError(
0050         "FastF1/OpenF1 sources require a canonical input file. Run the matching "
0051         "ingestion command, then use `apexsim run-canonical --input-path ...`."
0052     )
0053
0054
0055 def quality_stage(canonical_path: Path, run_dir: Path) -> dict:
0056     output = run_dir / "quality_report.json"
0057     if output.exists():
0058         return json.loads(output.read_text(encoding="utf-8"))
0059     frame = pd.read_csv(canonical_path)
0060     report = validate_canonical_frame(frame, strict=True).to_dict()
0061     output.write_text(json.dumps(report, indent=2), encoding="utf-8")
0062     return report
0063
0064

```

```

0065 def dataset_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0066     split_path = run_dir / "splits.json"

```

Line	Code signal	What it is doing / what to verify
45	if config.data.source == "synthetic":	Branches only when this condition is true; verify both the true and false paths in tests.
46	generate_synthetic_sessions(config, output)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
47	return output	Returns the result to the caller; this is the function output boundary that tests should assert.
49	raise ValueError(	Stops execution because an invariant or supported-condition check failed.
50	"FastF1/OpenF1 sources require a canonical input file. Run the matching "	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	"ingestion command, then use `apexsim run-canonical -input-path ...`."	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
55	def quality_stage(canonical_path: Path, run_dir: Path) -> dict:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
56	output = run_dir / "quality_report.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
58	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
59	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
60	report = validate_canonical_frame(frame, strict=True).to_dict()	Creates or updates `report`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	output.write_text(json.dumps(report, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
62	return report	Returns the result to the caller; this is the function output boundary that tests should assert.
65	def dataset_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) ->	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
66	split_path = run_dir / "splits.json"	Creates or updates `split_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     scaler_path = run_dir / "standardizer.json"
0068     if split_path.exists() and scaler_path.exists():
0069         return {
0070             "splits": json.loads(split_path.read_text(encoding="utf-8")),

```

Source → contract → runtime evidence → failure test

```

0071         "standardizer": json.loads(scaler_path.read_text(encoding="utf-8")),
0072     }
0073     frame = pd.read_csv(canonical_path)
0074     splits = split_sessions(
0075         frame,
0076         config.data.train_fraction,
0077         config.data.val_fraction,
0078         config.seed,
0079     )
0080     train_frame = frame[frame.session_id.isin(splits["train"])]
0081     standardizer = Standardizer.fit(train_frame)
0082     split_path.write_text(json.dumps(splits, indent=2), encoding="utf-8")
0083     scaler_path.write_text(json.dumps(standardizer.to_dict(), indent=2), encoding="utf-8")
0084     return {"splits": splits, "standardizer": standardizer.to_dict()}
0085
0086
0087 def _load_datasets(config: ProjectConfig, canonical_path: Path, run_dir: Path):
0088     frame = pd.read_csv(canonical_path)

```

Line	Code signal	What it is doing / what to verify
67	scaler_path = run_dir / "standardizer.json"	Creates or updates `scaler_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	if split_path.exists() and scaler_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
69	return {	Returns the result to the caller; this is the function output boundary that tests should assert.
70	"splits": json.loads(split_path.read_text(encoding="utf-8")),	Creates or updates ``splits``: json.loads(split_path.read_text(encoding=`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	"standardizer": json.loads(scaler_path.read_text(encoding="utf-8")),	Creates or updates ``standardizer``: json.loads(scaler_path.read_text(encoding=`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
74	splits = split_sessions(	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	frame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	config.data.train_fraction,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	config.data.val_fraction,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	config.seed,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
80	train_frame =	Creates or updates `train_frame`. Check its

	<code>frame[frame.session_id.isin(splits["train"])]</code>	unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>standardizer = Standardizer.fit(train_frame)</code>	Learns parameters or preprocessing statistics from the supplied training evidence only.
82	<code>split_path.write_text(json.dumps(splits, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
83	<code>scaler_path.write_text(json.dumps(standardizer.to_dict(), indent=2), encoding="u</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
84	<code>return {"splits": splits, "standardizer": standardizer.to_dict()}</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
87	<code>def load_datasets(config: ProjectConfig, canonical_path: Path, run_dir: Path):</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
88	<code>frame = pd.read_csv(canonical_path)</code>	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.

Lines 89-110

```

0089     splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))
0090     standardizer = Standardizer.from_dict(
0091         json.loads((run_dir / "standardizer.json").read_text(encoding="utf-8"))
0092     )
0093     datasets = {
0094         name: TelemetryWindowDataset(
0095             frame,
0096             session_ids,
0097             config.data.sequence_length,
0098             config.data.prediction_horizon,
0099             standardizer,
0100             max_windows=config.training.max_train_windows if name == "train" else 1200,
0101         )
0102         for name, session_ids in splits.items()
0103     }
0104     return frame, standardizer, datasets
0105
0106
0107 def train_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0108     model_dir = run_dir / "model"
0109     model_dir.mkdir(parents=True, exist_ok=True)
0110     checkpoint = model_dir / "world_model.pt"

```

Line	Code signal	What it is doing / what to verify
89	<code>splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))</code>	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
90	<code>standardizer = Standardizer.from_dict(</code>	Creates or updates `standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
91	<code>json.loads((run_dir / "standardizer.json").read_text(encoding="utf-8"))</code>	Creates or updates `json.loads((run_dir / "standardizer.json").read_text(encoding='`'. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
93	<code>datasets = {</code>	Creates or updates `datasets`. Check its unit, shape, lifetime and whether it is derived

		only from information available at this point.
94	name: TelemetryWindowDataset(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	frame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	session_ids,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	config.data.sequence_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	config.data.prediction_horizon,	Runs model inference using learned parameters without fitting on the requested examples.
99	standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	max_windows=config.training.max_train_windows if name == "train" else 1200,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
102	for name, session_ids in splits.items()	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
103	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
104	return frame, standardizer, datasets	Returns the result to the caller; this is the function output boundary that tests should assert.
107	def train_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> d	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
108	model_dir = run_dir / "model"	Creates or updates `model_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
109	model_dir.mkdir(parents=True, exist_ok=True)	Creates or updates `model_dir.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
110	checkpoint = model_dir / "world_model.pt"	Creates or updates `checkpoint`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 111-132

```

0111     meta_path = model_dir / "model_meta.json"
0112     if checkpoint.exists() and meta_path.exists():
0113         return json.loads(meta_path.read_text(encoding="utf-8"))
0114     _, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0115     if len(datasets["train"]) == 0 or len(datasets["val"]) == 0:
0116         raise RuntimeError("Not enough windows after session split; increase session duration or count.")
0117     train_loader = DataLoader(datasets["train"], batch_size=config.training.batch_size, shuffle=True)
0118     val_loader = DataLoader(datasets["val"], batch_size=config.training.batch_size, shuffle=False)
0119     model = build_model(config.model, len(MODEL_INPUT_COLUMNS), len(TARGET_COLUMNS))

```

Source → contract → runtime evidence → failure test


```

0120     result = train_world_model(model, train_loader, val_loader, config, model_dir)
0121     meta = {
0122         "kind": config.model.kind,
0123         "input_dim": len(MODEL_INPUT_COLUMNS),
0124         "state_dim": len(TARGET_COLUMNS),
0125         "hidden_dim": config.model.hidden_dim,
0126         "latent_dim": config.model.latent_dim,
0127         "layers": config.model.layers,
0128         "dropout": config.model.dropout,
0129         "best_val_loss": result.best_val_loss,
0130         "epochs_completed": result.epochs_completed,
0131     }
0132     meta_path.write_text(json.dumps(meta, indent=2), encoding="utf-8")

```

Line	Code signal	What it is doing / what to verify
111	meta_path = model_dir / "model_meta.json"	Creates or updates `meta_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	if checkpoint.exists() and meta_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
113	return json.loads(meta_path.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
114	_, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)	Creates or updates `_, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	if len(datasets["train"]) == 0 or len(datasets["val"]) == 0:	Branches only when this condition is true; verify both the true and false paths in tests.
116	raise RuntimeError("Not enough windows after session split; increase session dur	Stops execution because an invariant or supported-condition check failed.
117	train_loader = DataLoader(datasets["train"], batch_size=config.training.batch_si	Creates or updates `train_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
118	val_loader = DataLoader(datasets["val"], batch_size=config.training.batch_size,	Creates or updates `val_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	model = build_model(config.model, len(MODEL_INPUT_COLUMNS), len(TARGET_COLUMNS))	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	result = train_world_model(model, train_loader, val_loader, config, model_dir)	Creates or updates `result`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
121	meta = {	Creates or updates `meta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	"kind": config.model.kind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
123	"input_dim": len(MODEL_INPUT_COLUMNS),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
124	"state_dim": len(TARGET_COLUMNS),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
125	"hidden_dim": config.model.hidden_dim,	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
126	"latent_dim": config.model.latent_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
127	"layers": config.model.layers,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
128	"dropout": config.model.dropout,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
129	"best_val_loss": result.best_val_loss,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
130	"epochs_completed": result.epochs_completed,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
131	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
132	meta_path.write_text(json.dumps(meta, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.

Lines 133-154

```

0133     return meta
0134
0135
0136 def load_trained_model(config: ProjectConfig, run_dir: Path):
0137     meta = json.loads((run_dir / "model" / "model_meta.json").read_text(encoding="utf-8"))
0138     config.model.kind = meta["kind"]
0139     config.model.hidden_dim = meta["hidden_dim"]
0140     config.model.latent_dim = meta["latent_dim"]
0141     config.model.layers = meta["layers"]
0142     config.model.dropout = meta["dropout"]
0143     model = build_model(config.model, meta["input_dim"], meta["state_dim"])
0144     state = torch.load(run_dir / "model" / "world_model.pt", map_location="cpu", weights_only=True)
0145     model.load_state_dict(state)
0146     return model
0147
0148
0149 def evaluate_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0150     output = run_dir / "metrics.json"
0151     if output.exists():
0152         return json.loads(output.read_text(encoding="utf-8"))
0153     _, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0154     test_loader = DataLoader(datasets["test"], batch_size=config.training.batch_size, shuffle=False)

```

Line	Code signal	What it is doing / what to verify
133	return meta	Returns the result to the caller; this is the function output boundary that tests should assert.
136	def load_trained_model(config: ProjectConfig, run_dir: Path):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
137	meta = json.loads((run_dir / "model" / "model_meta.json").read_text(encoding="ut	Creates or updates `meta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
138	config.model.kind = meta["kind"]	Creates or updates `config.model.kind`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
139	config.model.hidden_dim = meta["hidden_dim"]	Creates or updates `config.model.hidden_dim`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
140	<code>config.model.latent_dim = meta["latent_dim"]</code>	Creates or updates `config.model.latent_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
141	<code>config.model.layers = meta["layers"]</code>	Creates or updates `config.model.layers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
142	<code>config.model.dropout = meta["dropout"]</code>	Creates or updates `config.model.dropout`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
143	<code>model = build_model(config.model, meta["input_dim"], meta["state_dim"])</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
144	<code>state = torch.load(run_dir / "model" / "world_model.pt", map_location="cpu", wei</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
145	<code>model.load_state_dict(state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
146	<code>return model</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
149	<code>def evaluate_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
150	<code>output = run_dir / "metrics.json"</code>	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
151	<code>if output.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
152	<code>return json.loads(output.read_text(encoding="utf-8"))</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
153	<code>_, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)</code>	Creates or updates `_`, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
154	<code>test_loader = DataLoader(datasets["test"], batch_size=config.training.batch_size</code>	Creates or updates `test_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 155-176

```

0155     model = load_trained_model(config, run_dir)
0156     return evaluate_world_model(model, test_loader, standardizer, output, config.training.device)
0157
0158
0159 def ablation_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> list[dict]:
0160     output = run_dir / "ablations.json"
0161     if output.exists():
0162         return json.loads(output.read_text(encoding="utf-8"))
0163     frame = pd.read_csv(canonical_path)
0164     splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))
0165     train = frame[frame.session_id.isin(splits["train"])]
0166     test = frame[frame.session_id.isin(splits["test"])]

```

Source → contract → runtime evidence → failure test

```

0167     feature_sets = {
0168         "state_only": STATE_COLUMNS,
0169         "state_plus_actions": STATE_COLUMNS + ["throttle", "brake", "gear_norm", "drs"],
0170         "full_context": MODEL_INPUT_COLUMNS,
0171         "no_weather": [c for c in MODEL_INPUT_COLUMNS if c not in {"rainfall", "air_temp_c", "track_temp_c",
"wind_speed_mps"}],
0172         "no_track_geometry": [c for c in MODEL_INPUT_COLUMNS if c not in {"curvature", "steering_proxy",
"track_progress_sin", "track_progress_cos"}],
0173     }
0174     results = []
0175     for name, features in feature_sets.items():
0176         # One-step ridge is intentionally used as a cheap, interpretable ablation probe.

```

Line	Code signal	What it is doing / what to verify
155	model = load_trained_model(config, run_dir)	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
156	return evaluate_world_model(model, test_loader, standardizer, output, config.tra	Returns the result to the caller; this is the function output boundary that tests should assert.
159	def ablation_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
160	output = run_dir / "ablations.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
161	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
162	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
163	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
164	splits = json.loads(run_dir / "splits.json").read_text(encoding="utf-8"))	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
165	train = frame[frame.session_id.isin(splits["train"])] copy()	Creates or updates `train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
166	test = frame[frame.session_id.isin(splits["test"])] copy()	Creates or updates `test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
167	feature_sets = {	Creates or updates `feature_sets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
168	"state_only": STATE_COLUMNS,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
169	"state_plus_actions": STATE_COLUMNS + ["throttle", "brake", "gear_norm", "drs"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
170	"full_context": MODEL_INPUT_COLUMNS,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

171	"no_weather": [c for c in MODEL_INPUT_COLUMNS if c not in {"rainfall", "air_temp	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
172	"no_track_geometry": [c for c in MODEL_INPUT_COLUMNS if c not in {"curvature", "	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
173	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
174	results = []	Creates or updates `results`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
175	for name, features in feature_sets.items():	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
176	# One-step ridge is intentionally used as a cheap, interpretable ablation probe.	Documents an assumption, intent or constraint that should remain aligned with the implementation.

Lines 177-198

```

0177     x_train = train[features].iloc[:-1].to_numpy(np.float32)
0178     y_train = train[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)
0179     same_session = train.session_id.iloc[:-1].to_numpy() == train.session_id.iloc[1:].to_numpy()
0180     x_train, y_train = x_train[same_session], y_train[same_session]
0181     x_test = test[features].iloc[:-1].to_numpy(np.float32)
0182     y_test = test[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)
0183     same_test = test.session_id.iloc[:-1].to_numpy() == test.session_id.iloc[1:].to_numpy()
0184     x_test, y_test = x_test[same_test], y_test[same_test]
0185     from sklearn.linear_model import Ridge
0186     from sklearn.preprocessing import StandardScaler
0187     from sklearn.pipeline import make_pipeline
0188     from sklearn.multioutput import MultiOutputRegressor
0189
0190     model = make_pipeline(StandardScaler(), MultiOutputRegressor(Ridge(alpha=1.0)))
0191     model.fit(x_train, y_train)
0192     prediction = model.predict(x_test)
0193     results.append(
0194         {
0195             "ablation": name,
0196             "features": features,
0197             "speed_mae_mps": float(np.mean(np.abs(prediction[:, 0] - y_test[:, 0]))),
0198             "overall_mae": float(np.mean(np.abs(prediction - y_test))),

```

Line	Code signal	What it is doing / what to verify
177	x_train = train[features].iloc[:-1].to_numpy(np.float32)	Creates or updates `x_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
178	y_train = train[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)	Creates or updates `y_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
179	same_session = train.session_id.iloc[:-1].to_numpy() == train.session_id.iloc[1:]	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
180	x_train, y_train = x_train[same_session], y_train[same_session]	Creates or updates `x_train, y_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
181	x_test = test[features].iloc[:-1].to_numpy(np.float32)	Creates or updates `x_test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
182	y_test =	Creates or updates `y_test`. Check its unit,

	test[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)	shape, lifetime and whether it is derived only from information available at this point.
183	same_test = test.session_id.iloc[:-1].to_numpy() == test.session_id.iloc[1:].to_	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
184	x_test, y_test = x_test[same_test], y_test[same_test]	Creates or updates `x_test, y_test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
185	from sklearn.linear_model import Ridge	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
186	from sklearn.preprocessing import StandardScaler	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
187	from sklearn.pipeline import make_pipeline	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
188	from sklearn.multioutput import MultiOutputRegressor	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
190	model = make_pipeline(StandardScaler(), MultiOutputRegressor(Ridge(alpha=1.0)))	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
191	model.fit(x_train, y_train)	Learns parameters or preprocessing statistics from the supplied training evidence only.
192	prediction = model.predict(x_test)	Runs model inference using learned parameters without fitting on the requested examples.
193	results.append(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
194	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
195	"ablation": name,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
196	"features": features,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
197	"speed_mae_mps": float(np.mean(np.abs(prediction[:, 0] - y_test[:, 0]))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
198	"overall_mae": float(np.mean(np.abs(prediction - y_test))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 199-220

```

0199         }
0200     )
0201     results.sort(key=lambda item: item["speed_mae_mps"])
0202     output.write_text(json.dumps(results, indent=2), encoding="utf-8")
0203     return results
0204
0205
0206 def publish_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0207     output = run_dir / "publication.json"
0208     if output.exists():

```

Source → contract → runtime evidence → failure test

```

0209     return json.loads(output.read_text(encoding="utf-8"))
0210     frame, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0211     model = load_trained_model(config, run_dir)
0212     sample = datasets["test"][0]
0213     history = sample["history"].unsqueeze(0)
0214     future_inputs = sample["future_inputs"].unsqueeze(0)
0215     with torch.no_grad():
0216         if config.model.kind == "rssm":
0217             prediction, _ = model(history, future_inputs, future_targets=None)
0218         else:
0219             prediction = model(history, future_inputs)
0220     predicted = standardizer.inverse_targets(prediction.numpy()[0])

```

Line	Code signal	What it is doing / what to verify
199	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
200	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
201	results.sort(key=lambda item: item["speed_mae_mps"])	Creates or updates `results.sort(key)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
202	output.write_text(json.dumps(results, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
203	return results	Returns the result to the caller; this is the function output boundary that tests should assert.
206	def publish_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) ->	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
207	output = run_dir / "publication.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
208	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
209	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
210	frame, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)	Creates or updates `frame, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
211	model = load_trained_model(config, run_dir)	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
212	sample = datasets["test"][0]	Creates or updates `sample`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
213	history = sample["history"].unsqueeze(0)	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
214	future_inputs = sample["future_inputs"].unsqueeze(0)	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
215	with torch.no_grad():	Opens a managed resource or context and

		guarantees cleanup when the block exits.
216	if config.model.kind == "rssm":	Branches only when this condition is true; verify both the true and false paths in tests.
217	prediction, _ = model(history, future_inputs, future_targets=None)	Creates or updates `prediction, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
218	else:	Handles all remaining cases; inspect whether this fallback is intentionally broad.
219	prediction = model(history, future_inputs)	Creates or updates `prediction`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
220	predicted = standardizer.inverse_targets(prediction.numpy()[0])	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 221-241

```

0221     truth = standardizer.inverse_targets(sample["future_targets"].numpy())
0222     comparison = pd.DataFrame(
0223         {
0224             "step": np.arange(len(predicted)),
0225             "predicted_speed_mps": predicted[:, 0],
0226             "actual_speed_mps": truth[:, 0],
0227             "predicted_accel_mps2": predicted[:, 1],
0228             "actual_accel_mps2": truth[:, 1],
0229         }
0230     )
0231     comparison_path = run_dir / "rollout_preview.csv"
0232     comparison.to_csv(comparison_path, index=False)
0233     payload = {
0234         "run_dir": str(run_dir),
0235         "model_kind": config.model.kind,
0236         "rollout_preview": str(comparison_path),
0237         "test_sessions": json.loads((run_dir / "splits.json").read_text())["test"],
0238         "ui_command": f"apexsim ui --run-dir {run_dir}",
0239     }
0240     output.write_text(json.dumps(payload, indent=2), encoding="utf-8")
0241     return payload

```

Line	Code signal	What it is doing / what to verify
221	truth = standardizer.inverse_targets(sample["future_targets"].numpy())	Creates or updates `truth`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
222	comparison = pd.DataFrame(	Creates or updates `comparison`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
223	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
224	"step": np.arange(len(predicted)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
225	"predicted_speed_mps": predicted[:, 0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
226	"actual_speed_mps": truth[:, 0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
227	"predicted_accel_mps2": predicted[:, 1],	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
228	"actual_accel_mps2": truth[:, 1],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
229	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
230	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
231	comparison_path = run_dir / "rollout_preview.csv"	Creates or updates `comparison_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
232	comparison.to_csv(comparison_path, index=False)	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
233	payload = {	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
234	"run_dir": str(run_dir),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
235	"model_kind": config.model.kind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
236	"rollout_preview": str(comparison_path),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
237	"test_sessions": json.loads((run_dir / "splits.json").read_text())["test"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
238	"ui_command": f"apexsim ui --run-dir {run_dir}",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
239	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
240	output.write_text(json.dumps(payload, indent=2, encoding="utf-8"))	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
241	return payload	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/plotting.py

Turns telemetry and rollout evidence into diagnostic figures.

Plots should expose alignment, horizon drift and intervention effects rather than merely decorate the UI.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import matplotlib.pyplot as plt
0006 import pandas as pd
0007
0008
0009 def save_track_map(frame: pd.DataFrame, output_path: str | Path) -> Path:
0010     path = Path(output_path)
0011     path.parent.mkdir(parents=True, exist_ok=True)
0012     fig, ax = plt.subplots(figsize=(6, 4.5))
0013     session = frame[frame.session_id == frame.session_id.iloc[0]]
0014     ax.plot(session.x_m, session.y_m, linewidth=2)
0015     ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")
0016     ax.set_title("Synthetic track geometry")
0017     ax.set_xlabel("x (m)")
0018     ax.set_ylabel("y (m)")
0019     ax.set_aspect("equal", adjustable="box")
0020     ax.legend()
0021     fig.tight_layout()
0022     fig.savefig(path, dpi=160)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import matplotlib.pyplot as plt	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def save_track_map(frame: pd.DataFrame, output_path: str Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
11	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

12	<code>fig, ax = plt.subplots(figsize=(6, 4.5))</code>	Creates or updates `fig, ax`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	<code>session = frame[frame.session_id == frame.session_id.iloc[0]]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	<code>ax.plot(session.x_m, session.y_m, linewidth=2)</code>	Creates or updates `ax.plot(session.x_m, session.y_m, linewidth=2)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	<code>ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")</code>	Creates or updates `ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	<code>ax.set_title("Synthetic track geometry")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	<code>ax.set_xlabel("x (m)")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
18	<code>ax.set_ylabel("y (m)")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
19	<code>ax.set_aspect("equal", adjustable="box")</code>	Creates or updates `ax.set_aspect("equal", adjustable="box")`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>ax.legend()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
21	<code>fig.tight_layout()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
22	<code>fig.savefig(path, dpi=160)</code>	Creates or updates `fig.savefig(path, dpi)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-41

```

0023     plt.close(fig)
0024     return path
0025
0026
0027 def save_training_curve(history: list[dict], output_path: str | Path) -> Path:
0028     path = Path(output_path)
0029     path.parent.mkdir(parents=True, exist_ok=True)
0030     frame = pd.DataFrame(history)
0031     fig, ax = plt.subplots(figsize=(6, 4))
0032     ax.plot(frame.epoch, frame.train_loss, marker="o", label="Train")
0033     ax.plot(frame.epoch, frame.val_loss, marker="o", label="Validation")
0034     ax.set_xlabel("Epoch")
0035     ax.set_ylabel("Loss")
0036     ax.set_title("World-model training")
0037     ax.legend()
0038     fig.tight_layout()
0039     fig.savefig(path, dpi=160)
0040     plt.close(fig)
0041     return path

```

Line	Code signal	What it is doing / what to verify
23	<code>plt.close(fig)</code>	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
24	return path	Returns the result to the caller; this is the function output boundary that tests should assert.
27	def save_training_curve(history: list[dict], output_path: str Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
28	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	frame = pd.DataFrame(history)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	fig, ax = plt.subplots(figsize=(6, 4))	Creates or updates `fig, ax`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	ax.plot(frame.epoch, frame.train_loss, marker="o", label="Train")	Creates or updates `ax.plot(frame.epoch, frame.train_loss, marker)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	ax.plot(frame.epoch, frame.val_loss, marker="o", label="Validation")	Creates or updates `ax.plot(frame.epoch, frame.val_loss, marker)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	ax.set_xlabel("Epoch")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
35	ax.set_ylabel("Loss")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	ax.set_title("World-model training")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
37	ax.legend()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	fig.tight_layout()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	fig.savefig(path, dpi=160)	Creates or updates `fig.savefig(path, dpi)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	plt.close(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
41	return path	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/registry.py

Persists run identity, status, configuration, metrics and artifact locations.

A registry is evidence, not a leaderboard. It makes promotion and rollback auditable.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import sqlite3
0005 from datetime import datetime, timezone
0006 from pathlib import Path
0007
0008
0009 class RunRegistry:
0010     def __init__(self, database_path: str | Path) -> None:
0011         self.path = Path(database_path)
0012         self.path.parent.mkdir(parents=True, exist_ok=True)
0013         with self._connect() as connection:
0014             connection.execute(
0015                 """
0016                 CREATE TABLE IF NOT EXISTS runs (
0017                     run_id TEXT PRIMARY KEY,
0018                     status TEXT NOT NULL,
0019                     model_kind TEXT NOT NULL,
0020                     started_at TEXT NOT NULL,
0021                     finished_at TEXT,
0022                     run_dir TEXT NOT NULL,
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import sqlite3	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from datetime import datetime, timezone	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	class RunRegistry:	Defines a type that owns related state and behaviour behind a named interface.
10	def __init__(self, database_path: str Path) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
11	self.path = Path(database_path)	Creates or updates `self.path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Source → contract → runtime evidence → failure test

12	<code>self.path.parent.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>self.path.parent.mkdir(parents=True)</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	<code>with self._connect() as connection:</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
14	<code>connection.execute(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	<code>"""</code>	Begins or ends documentation describing the module, class or function contract.
16	<code>CREATE TABLE IF NOT EXISTS runs (</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	<code>run_id TEXT PRIMARY KEY,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	<code>status TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	<code>model_kind TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	<code>started_at TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	<code>finished_at TEXT,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	<code>run_dir TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023         metrics_json TEXT
0024     )
0025     """
0026 )
0027
0028 def _connect(self) -> sqlite3.Connection:
0029     return sqlite3.connect(self.path)
0030
0031 def start(self, run_id: str, model_kind: str, run_dir: str) -> None:
0032     now = datetime.now(timezone.utc).isoformat()
0033     with self._connect() as connection:
0034         connection.execute(
0035             "INSERT OR REPLACE INTO runs VALUES (?, ?, ?, ?, ?, ?, ?)",
0036             (run_id, "running", model_kind, now, None, run_dir, None),
0037         )
0038
0039 def finish(self, run_id: str, status: str, metrics: dict | None = None) -> None:
0040     now = datetime.now(timezone.utc).isoformat()
0041     with self._connect() as connection:
0042         connection.execute(
0043             "UPDATE runs SET status=?, finished_at=?, metrics_json=? WHERE run_id=?",
0044             (status, now, json.dumps(metrics) if metrics else None, run_id),

```

Line	Code signal	What it is doing / what to verify
23	<code>metrics_json TEXT</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
25	"""	Begins or ends documentation describing the module, class or function contract.
26	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
28	def _connect(self) -> sqlite3.Connection:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
29	return sqlite3.connect(self.path)	Returns the result to the caller; this is the function output boundary that tests should assert.
31	def start(self, run_id: str, model_kind: str, run_dir: str) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	now = datetime.now(timezone.utc).isoformat()	Creates or updates `now`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
34	connection.execute(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	"INSERT OR REPLACE INTO runs VALUES (?, ?, ?, ?, ?, ?)",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	(run_id, "running", model_kind, now, None, run_dir, None),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	def finish(self, run_id: str, status: str, metrics: dict None = None) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	now = datetime.now(timezone.utc).isoformat()	Creates or updates `now`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
42	connection.execute(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	"UPDATE runs SET status=?, finished_at=?, metrics_json=? WHERE run_id=?",	Creates or updates `UPDATE runs SET status`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	(status, now, json.dumps(metrics) if metrics else None, run_id),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-56

```

0045         )
0046
0047     def list_runs(self) -> list[dict]:
0048         with self._connect() as connection:
0049             connection.row_factory = sqlite3.Row
0050             rows = connection.execute("SELECT * FROM runs ORDER BY started_at DESC").fetchall()
0051             result = []

```

Source → contract → runtime evidence → failure test

```

0052     for row in rows:
0053         item = dict(row)
0054         item["metrics"] = json.loads(item.pop("metrics_json")) if item.get("metrics_json") else None
0055         result.append(item)
0056     return result

```

Line	Code signal	What it is doing / what to verify
45	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
47	def list_runs(self) -> list[dict]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
48	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
49	connection.row_factory = sqlite3.Row	Creates or updates `connection.row_factory`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	rows = connection.execute("SELECT * FROM runs ORDER BY started_at DESC").fetchall	Creates or updates `rows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	result = []	Creates or updates `result`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	for row in rows:	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
53	item = dict(row)	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	item["metrics"] = json.loads(item.pop("metrics_json")) if item.get("metrics_json"	Creates or updates `item["metrics"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	result.append(item)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	return result	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/serving.py

Creates typed API boundaries for health, runs and scenarios.

Serving should load compatible artifacts once, validate units and avoid exposing internal tensors as public contracts.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 from fastapi import FastAPI, HTTPException
0007
0008 from apexsim.registry import RunRegistry
0009
0010
0011 def create_api(artifacts_dir: str | Path = "artifacts/runs") -> FastAPI:
0012     root = Path(artifacts_dir)
0013     registry = RunRegistry(root / "runs.sqlite")
0014     app = FastAPI(title="Project APEX API", version="0.1.0")
0015
0016     @app.get("/health")
0017     def health() -> dict[str, str]:
0018         return {"status": "ok"}
0019
0020     @app.get("/runs")
0021     def runs() -> list[dict]:
0022         return registry.list_runs()
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from fastapi import FastAPI, HTTPException	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.registry import RunRegistry	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	def create_api(artifacts_dir: str Path = "artifacts/runs") -> FastAPI:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	root = Path(artifacts_dir)	Creates or updates `root`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	registry = RunRegistry(root / "runs.sqlite")	Creates or updates `registry`. Check its unit,

Source → contract → runtime evidence → failure test

		shape, lifetime and whether it is derived only from information available at this point.
14	<code>app = FastAPI(title="Project APEX API", version="0.1.0")</code>	Creates or updates `app`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	<code>@app.get("/health")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
17	<code>def health() -> dict[str, str]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
18	<code>return {"status": "ok"}</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
20	<code>@app.get("/runs")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
21	<code>def runs() -> list[dict]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	<code>return registry.list_runs()</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 23-40

```

0023
0024     @app.get("/runs/{run_id}")
0025     def run_details(run_id: str) -> dict:
0026         path = root / run_id / "summary.json"
0027         if not path.exists():
0028             raise HTTPException(status_code=404, detail="Run not found")
0029         return json.loads(path.read_text())
0030
0031     @app.get("/runs/{run_id}/rollout")
0032     def rollout_preview(run_id: str) -> list[dict]:
0033         path = root / run_id / "rollout_preview.csv"
0034         if not path.exists():
0035             raise HTTPException(status_code=404, detail="Rollout preview not found")
0036         import pandas as pd
0037
0038         return pd.read_csv(path).to_dict(orient="records")
0039
0040     return app

```

Line	Code signal	What it is doing / what to verify
24	<code>@app.get("/runs/{run_id}")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
25	<code>def run_details(run_id: str) -> dict:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	<code>path = root / run_id / "summary.json"</code>	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	<code>if not path.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
28	<code>raise HTTPException(status_code=404, detail="Run not found")</code>	Stops execution because an invariant or supported-condition check failed.
29	<code>return json.loads(path.read_text())</code>	Returns the result to the caller; this is the function output boundary that tests should

		assert.
31	@app.get("/runs/{run_id}/rollout")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
32	def rollout_preview(run_id: str) -> list[dict]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
33	path = root / run_id / "rollout_preview.csv"	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	if not path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
35	raise HTTPException(status_code=404, detail="Rollout preview not found")	Stops execution because an invariant or supported-condition check failed.
36	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
38	return pd.read_csv(path).to_dict(orient="records")	Returns the result to the caller; this is the function output boundary that tests should assert.
40	return app	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/simulation.py

Applies bounded counterfactual interventions and runs imagined futures.

Actions and context can be changed only within explicit support; the scenario engine must preserve the model contract and communicate extrapolation.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import torch
0007 from torch import nn
0008
0009 from apexsim.contracts import ACTION_COLUMNS, CONTEXT_COLUMNS, MODEL_INPUT_COLUMNS, STATE_COLUMNS
0010 from apexsim.data.features import Standardizer
0011 from apexsim.models.rssm import RSSMWorldModel
0012
0013
0014 @dataclass(frozen=True)
0015 class Scenario:
0016     throttle_scale: float = 1.0
0017     brake_scale: float = 1.0
0018     grip_multiplier: float = 1.0
0019     rain_delta: float = 0.0
0020     tyre_degradation_multiplier: float = 1.0
0021
0022

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.contracts import ACTION_COLUMNS, CONTEXT_COLUMNS, MODEL_INPUT_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
14	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
15	class Scenario:	Defines a type that owns related state and behaviour behind a named interface.
16	throttle_scale: float = 1.0	Creates or updates `throttle_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	brake_scale: float = 1.0	Creates or updates `brake_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	grip_multiplier: float = 1.0	Creates or updates `grip_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	rain_delta: float = 0.0	Creates or updates `rain_delta: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	tyre_degradation_multiplier: float = 1.0	Creates or updates `tyre_degradation_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023 def apply_scenario(future_inputs_raw: np.ndarray, scenario: Scenario) -> np.ndarray:
0024     modified = future_inputs_raw.copy()
0025     index = {name: i for i, name in enumerate(MODEL_INPUT_COLUMNS)}
0026     modified[:, index["throttle"]] = np.clip(
0027         modified[:, index["throttle"]] * scenario.throttle_scale, 0.0, 1.0
0028     )
0029     modified[:, index["brake"]] = np.clip(
0030         modified[:, index["brake"]] * scenario.brake_scale, 0.0, 1.0
0031     )
0032     modified[:, index["grip_level"]] = np.clip(
0033         modified[:, index["grip_level"]] * scenario.grip_multiplier, 0.2, 1.5
0034     )
0035     modified[:, index["rainfall"]] = np.clip(
0036         modified[:, index["rainfall"]] + scenario.rain_delta, 0.0, 1.0
0037     )
0038     tyre_index = index["tyre_age_laps"]
0039     base_age = modified[0, tyre_index]
0040     modified[:, tyre_index] = base_age + (
0041         modified[:, tyre_index] - base_age
0042     ) * scenario.tyre_degradation_multiplier
0043     return modified
0044

```

Line	Code signal	What it is doing / what to verify
23	def apply_scenario(future_inputs_raw: np.ndarray, scenario: Scenario) -> np.ndarray	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
24	modified = future_inputs_raw.copy()	Creates or updates `modified`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
25	index = {name: i for i, name in enumerate(MODEL_INPUT_COLUMNS)}	Creates or updates `index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	modified[:, index["throttle"]] = np.clip(	Bounds a value to a supported range.

		Monitor how often clipping occurs because it may hide distribution shift.
27	<code>modified[:, index["throttle"]] * scenario.throttle_scale, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
29	<code>modified[:, index["brake"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
30	<code>modified[:, index["brake"]] * scenario.brake_scale, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
32	<code>modified[:, index["grip_level"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
33	<code>modified[:, index["grip_level"]] * scenario.grip_multiplier, 0.2, 1.5</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
34	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
35	<code>modified[:, index["rainfall"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
36	<code>modified[:, index["rainfall"]] + scenario.rain_delta, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	<code>tyre_index = index["tyre_age_laps"]</code>	Creates or updates `tyre_index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	<code>base_age = modified[0, tyre_index]</code>	Creates or updates `base_age`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>modified[:, tyre_index] = base_age + (</code>	Creates or updates `modified[:, tyre_index]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	<code>modified[:, tyre_index] - base_age</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	<code>) * scenario.tyre_degradation_multiplier</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	<code>return modified</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```
0045
0046 def rollout(
```

Source → contract → runtime evidence → failure test

```

0047     model: nn.Module,
0048     history_raw: np.ndarray,
0049     future_inputs_raw: np.ndarray,
0050     standardizer: Standardizer,
0051     scenario: Scenario,
0052     device: str = "cpu",
0053 ) -> np.ndarray:
0054     model.eval().to(device)
0055     future_modified = apply_scenario(future_inputs_raw, scenario)
0056     history = torch.from_numpy(
0057         standardizer.transform_inputs(history_raw).astype(np.float32)[None, ...]
0058     ).to(device)
0059     future = torch.from_numpy(
0060         standardizer.transform_inputs(future_modified).astype(np.float32)[None, ...]
0061     ).to(device)
0062     with torch.no_grad():
0063         if isinstance(model, RSSMWorldModel):
0064             prediction_z, _ = model(history, future, future_targets=None)
0065         else:
0066             prediction_z = model(history, future)

```

Line	Code signal	What it is doing / what to verify
46	def rollout(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	history_raw: np.ndarray,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	future_inputs_raw: np.ndarray,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	scenario: Scenario,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	device: str = "cpu",	Creates or updates `device: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	) -> np.ndarray:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
54	model.eval().to(device)	Enables deterministic evaluation behaviour for layers that differ between training and inference.
55	future_modified = apply_scenario(future_inputs_raw, scenario)	Creates or updates `future_modified`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	history = torch.from_numpy(	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	standardizer.transform_inputs(history_raw). astype(np.float32)[None, ...]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
58	).to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
59	future = torch.from_numpy(	Creates or updates `future`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
60	<code>standardizer.transform_inputs(future_modified).astype(np.float32)[None, ...]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
61	<code>).to(device)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
62	<code>with torch.no_grad():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
63	<code>if isinstance(model, RSSMWorldModel):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
64	<code>prediction_z, _ = model(history, future, future_targets=None)</code>	Creates or updates `prediction_z, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
66	<code>prediction_z = model(history, future)</code>	Creates or updates `prediction_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-67

```
0067     return standardizer.inverse_targets(prediction_z.cpu().numpy()[0])
```

Line	Code signal	What it is doing / what to verify
67	<code>return standardizer.inverse_targets(prediction_z.cpu().numpy()[0])</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/training.py

Owns optimization, validation, checkpoints and model-specific loss paths.

The training loop is a state machine: clear gradients, forward, measure loss, backpropagate, update, validate, compare and persist.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import random
0005 from dataclasses import dataclass
0006 from pathlib import Path
0007
0008 import numpy as np
0009 import torch
0010 from torch import nn
0011 from torch.utils.data import DataLoader
0012
0013 from apexsim.config import ProjectConfig
0014 from apexsim.models.rssm import RSSMWorldModel
0015
0016
0017 @dataclass
0018 class TrainingResult:
0019     best_val_loss: float
0020     epochs_completed: int
0021     checkpoint_path: Path
0022     history_path: Path
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import random	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
11	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	@dataclass	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
18	class TrainingResult:	Defines a type that owns related state and behaviour behind a named interface.
19	best_val_loss: float	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	epochs_completed: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	checkpoint_path: Path	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	history_path: Path	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023
0024
0025 def seed_everything(seed: int) -> None:
0026     random.seed(seed)
0027     np.random.seed(seed)
0028     torch.manual_seed(seed)
0029
0030
0031 def _batch_loss(model: nn.Module, batch: dict, device: torch.device) -> tuple[torch.Tensor, torch.Tensor]:
0032     history = batch["history"].to(device)
0033     future_inputs = batch["future_inputs"].to(device)
0034     targets = batch["future_targets"].to(device)
0035     if isinstance(model, RSSMWorldModel):
0036         predictions, kl = model(history, future_inputs, future_targets=targets)
0037         reconstruction = torch.nn.functional.smooth_l1_loss(predictions, targets)
0038         # KL weight is intentionally modest for a small educational dataset.
0039         loss = reconstruction + 0.02 * kl
0040     else:
0041         predictions = model(history, future_inputs)
0042         loss = torch.nn.functional.smooth_l1_loss(predictions, targets)
0043     return loss, predictions
0044

```

Line	Code signal	What it is doing / what to verify
25	def seed_everything(seed: int) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	random.seed(seed)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
27	np.random.seed(seed)	Fixes a random seed to make the experiment easier to reproduce and compare.
28	torch.manual_seed(seed)	Fixes a random seed to make the experiment easier to reproduce and compare.

31	<code>def _batch_loss(model: nn.Module, batch: dict, device: torch.device) -> tuple[to</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	<code>history = batch["history"].to(device)</code>	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>future_inputs = batch["future_inputs"].to(device)</code>	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>targets = batch["future_targets"].to(device)</code>	Creates or updates `targets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>if isinstance(model, RSSMWorldModel):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
36	<code>predictions, kl = model(history, future_inputs, future_targets=targets)</code>	Creates or updates `predictions, kl`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	<code>reconstruction = torch.nn.functional.smooth_l1_loss(predictions, targets)</code>	Creates or updates `reconstruction`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	<code># KL weight is intentionally modest for a small educational dataset.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
39	<code>loss = reconstruction + 0.02 * kl</code>	Creates or updates `loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
41	<code>predictions = model(history, future_inputs)</code>	Creates or updates `predictions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>loss = torch.nn.functional.smooth_l1_loss(predictions, targets)</code>	Creates or updates `loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	<code>return loss, predictions</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```

0045
0046 def train_world_model(
0047     model: nn.Module,
0048     train_loader: DataLoader,
0049     val_loader: DataLoader,
0050     config: ProjectConfig,
0051     run_dir: str | Path,
0052 ) -> TrainingResult:
0053     seed_everything(config.seed)
0054     run_path = Path(run_dir)
0055     run_path.mkdir(parents=True, exist_ok=True)
0056     device = torch.device(config.training.device)
0057     model.to(device)
0058     optimizer = torch.optim.AdamW(
0059         model.parameters(),
0060         lr=config.training.learning_rate,

```

```

0061     weight_decay=config.training.weight_decay,
0062 )
0063 history: list[dict[str, float | int]] = []
0064 best_val = float("inf")
0065 stale_epochs = 0
0066 checkpoint = run_path / "world_model.pt"

```

Line	Code signal	What it is doing / what to verify
46	def train_world_model(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	train_loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	val_loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	config: ProjectConfig,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	run_dir: str Path,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	) -> TrainingResult:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	seed_everything(config.seed)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
54	run_path = Path(run_dir)	Creates or updates `run_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	run_path.mkdir(parents=True, exist_ok=True)	Creates or updates `run_path.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	device = torch.device(config.training.device)	Creates or updates `device`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	model.to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
58	optimizer = torch.optim.AdamW(	Creates or updates `optimizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	model.parameters(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
60	lr=config.training.learning_rate,	Creates or updates `lr`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	weight_decay=config.training.weight_decay,	Creates or updates `weight_decay`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	)	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
63	history: list[dict[str, float int]] = []	Creates or updates `history: list[dict[str, float   int]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	best_val = float("inf")	Creates or updates `best_val`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	stale_epochs = 0	Creates or updates `stale_epochs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	checkpoint = run_path / "world_model.pt"	Creates or updates `checkpoint`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067
0068     for epoch in range(1, config.training.epochs + 1):
0069         model.train()
0070         train_losses = []
0071         for batch in train_loader:
0072             optimizer.zero_grad(set_to_none=True)
0073             loss, _ = _batch_loss(model, batch, device)
0074             loss.backward()
0075             torch.nn.utils.clip_grad_norm_(model.parameters(), config.training.grad_clip)
0076             optimizer.step()
0077             train_losses.append(float(loss.detach().cpu()))
0078
0079         model.eval()
0080         val_losses = []
0081         with torch.no_grad():
0082             for batch in val_loader:
0083                 loss, _ = _batch_loss(model, batch, device)
0084                 val_losses.append(float(loss.detach().cpu()))
0085         train_loss = float(np.mean(train_losses))
0086         val_loss = float(np.mean(val_losses))
0087         history.append({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss})
0088

```

Line	Code signal	What it is doing / what to verify
68	for epoch in range(1, config.training.epochs + 1):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
69	model.train()	Enables training behaviour such as dropout and running-statistic updates.
70	train_losses = []	Creates or updates `train_losses`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	for batch in train_loader:	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
72	optimizer.zero_grad(set_to_none=True)	Clears gradients left from the previous optimization step so they are not accumulated unintentionally.
73	loss, _ = _batch_loss(model, batch, device)	Creates or updates `loss, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	loss.backward()	Runs reverse-mode automatic differentiation to populate gradients for

		trainable parameters.
75	<code>torch.nn.utils.clip_grad_norm_(model.parameters(), config.training.grad_clip)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
76	<code>optimizer.step()</code>	Updates trainable parameters using the optimizer state and current gradients.
77	<code>train_losses.append(float(loss.detach().cpu()))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
79	<code>model.eval()</code>	Enables deterministic evaluation behaviour for layers that differ between training and inference.
80	<code>val_losses = []</code>	Creates or updates `val_losses`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>with torch.no_grad():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
82	<code>for batch in val_loader:</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
83	<code>loss, _ = _batch_loss(model, batch, device)</code>	Creates or updates `loss, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>val_losses.append(float(loss.detach().cpu()))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
85	<code>train_loss = float(np.mean(train_losses))</code>	Creates or updates `train_loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>val_loss = float(np.mean(val_losses))</code>	Creates or updates `val_loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	<code>history.append({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss})</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 89-101

```

0089         if val_loss < best_val - 1e-6:
0090             best_val = val_loss
0091             stale_epochs = 0
0092             torch.save(model.state_dict(), checkpoint)
0093         else:
0094             stale_epochs += 1
0095             if stale_epochs >= config.training.patience:
0096                 break
0097
0098     model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
0099     history_path = run_path / "training_history.json"
0100     history_path.write_text(json.dumps(history, indent=2), encoding="utf-8")
0101     return TrainingResult(best_val, len(history), checkpoint, history_path)

```

Line	Code signal	What it is doing / what to verify
89	<code>if val_loss < best_val - 1e-6:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
90	<code>best_val = val_loss</code>	Creates or updates `best_val`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

91	<code>stale_epochs = 0</code>	Creates or updates `stale_epochs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	<code>torch.save(model.state_dict(), checkpoint)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
93	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
94	<code>stale_epochs += 1</code>	Creates or updates `stale_epochs +`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	<code>if stale_epochs >= config.training.patience:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
96	<code>break</code>	Stops the enclosing loop after the required condition is reached.
98	<code>model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=T</code>	Creates or updates `model.load_state_dict(torch.load(checkpoint, map_location=, weights_only=T`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
99	<code>history_path = run_path / "training_history.json"</code>	Creates or updates `history_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
100	<code>history_path.write_text(json.dumps(history, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
101	<code>return TrainingResult(best_val, len(history), checkpoint, history_path)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/ui.py

Builds the interactive race-engineering interface.

The UI is an interpretation layer: it must preserve units, loading/error states, uncertainty and the distinction between recorded and imagined futures.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import gradio as gr
0007 import numpy as np
0008 import pandas as pd
0009 import plotly.graph_objects as go
0010 import torch
0011
0012 from apexsim.config import load_config
0013 from apexsim.contracts import MODEL_INPUT_COLUMNS
0014 from apexsim.data.features import Standardizer
0015 from apexsim.data.windows import TelemetryWindowDataset
0016 from apexsim.pipeline.stages import load_trained_model
0017 from apexsim.simulation import Scenario, rollout
0018
0019
0020 CUSTOM_CSS = """
0021 .gradio-container {max-width: 1500px !important; background: radial-gradient(circle at 15% 5%, #191b22 0%,
0022 #090a0e 48%, #050506 100%);}
0022 #hero {border: 1px solid #30343d; border-radius: 22px; padding: 24px; background: linear-gradient(135deg,
0022 rgba(239,35,60,.18), rgba(255,255,255,.025));}
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import gradio as gr	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import plotly.graph_objects as go	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

10	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
12	from apexsim.config import load_config	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.contracts import MODEL_INPUT_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	from apexsim.data.windows import TelemetryWindowDataset	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	from apexsim.pipeline.stages import load_trained_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	from apexsim.simulation import Scenario, rollout	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	CUSTOM_CSS = """	Creates or updates `CUSTOM_CSS`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	.gradio-container {max-width: 1500px !important; background: radial-gradient(cir	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	#hero {border: 1px solid #30343d; border-radius: 22px; padding: 24px; background	Documents an assumption, intent or constraint that should remain aligned with the implementation.

Lines 23-44

```

0023 .metric-card {border: 1px solid #2e323b; border-radius: 16px; padding: 14px; background: rgba(255,255,255,.035);}
0024 ""
0025
0026
0027 def _load_bundle(run_dir: Path, config_path: Path):
0028     config = load_config(config_path)
0029     frame = pd.read_csv(run_dir / "canonical_telemetry.csv")
0030     splits = json.loads((run_dir / "splits.json").read_text())
0031     standardizer = Standardizer.from_dict(json.loads((run_dir / "standardizer.json").read_text()))
0032     model = load_trained_model(config, run_dir)
0033     dataset = TelemetryWindowDataset(
0034         frame,
0035         splits["test"],
0036         config.data.sequence_length,
0037         max(config.data.prediction_horizon, 80),
0038         standardizer,
0039         max_windows=200,
0040     )
0041     return config, frame, splits, standardizer, model, dataset
0042
0043
0044 def _track_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:

```

Line	Code signal	What it is doing / what to verify
23	.metric-card {border: 1px solid #2e323b; border-radius: 16px; padding: 14px; bac	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	""	Begins or ends documentation describing the module, class or function contract.
27	def _load_bundle(run_dir: Path, config_path: Path):	Defines a callable boundary. Its parameters form the input contract and its return value

		or side effects form the output contract.
28	<code>config = load_config(config_path)</code>	Creates or updates `config`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	<code>frame = pd.read_csv(run_dir / "canonical_telemetry.csv")</code>	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
30	<code>splits = json.loads((run_dir / "splits.json").read_text())</code>	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>standardizer = Standardizer.from_dict(json.loads((run_dir / "standardizer.json"))</code>	Creates or updates `standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>model = load_trained_model(config, run_dir)</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>dataset = TelemetryWindowDataset(</code>	Creates or updates `dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>frame,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	<code>splits["test"],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	<code>config.data.sequence_length,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>max(config.data.prediction_horizon, 80),</code>	Runs model inference using learned parameters without fitting on the requested examples.
38	<code>standardizer,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>max_windows=200,</code>	Creates or updates `max_windows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
41	<code>return config, frame, splits, standardizer, model, dataset</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
44	<code>def _track_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.

Lines 45-66

```

0045     session = frame[frame.session_id == session_id]
0046     fig = go.Figure()
0047     fig.add_trace(
0048         go.Scatter(
0049             x=session.x_m,
0050             y=session.y_m,
0051             mode="lines",
0052             line=dict(width=6, color="#ef233c"),

```

Source → contract → runtime evidence → failure test

```

0053     hovertemplate="x=%{x:.0f}m<br>y=%{y:.0f}m<extra></extra>",
0054     )
0055 )
0056 fig.update_layout(
0057     template="plotly_dark",
0058     title=f"Track trace - {session_id}",
0059     xaxis_visible=False,
0060     yaxis_visible=False,
0061     height=430,
0062     margin=dict(l=15, r=15, t=50, b=15),
0063     paper_bgcolor="rgba(0,0,0,0)",
0064     plot_bgcolor="rgba(0,0,0,0)",
0065     yaxis_scaleanchor="x",
0066 )

```

Line	Code signal	What it is doing / what to verify
45	session = frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	fig = go.Figure()	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	fig.add_trace(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	go.Scatter(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	x=session.x_m,	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	y=session.y_m,	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	mode="lines",	Creates or updates `mode`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	line=dict(width=6, color="#ef233c"),	Creates or updates `line`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	hovertemplate="x=%{x:.0f}m y=%{y:.0f}m<extra></extra>",	Creates or updates `hovertemplate`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
55	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	fig.update_layout(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	template="plotly_dark",	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	title=f"Track trace - {session_id}",	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	xaxis_visible=False,	Creates or updates `xaxis_visible`. Check its

		unit, shape, lifetime and whether it is derived only from information available at this point.
60	yaxis_visible=False,	Creates or updates `yaxis_visible`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	height=430,	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	margin=dict(l=15, r=15, t=50, b=15),	Creates or updates `margin`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	paper_bgcolor="rgba(0,0,0,0)",	Creates or updates `paper_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	plot_bgcolor="rgba(0,0,0,0)",	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	yaxis_scaleanchor="x",	Creates or updates `yaxis_scaleanchor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 67-88

```

0067     return fig
0068
0069
0070 def _telemetry_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:
0071     session = frame[frame.session_id == session_id].iloc[:1200]
0072     fig = go.Figure()
0073     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.speed_mps * 3.6, name="Speed km/h"))
0074     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.throttle * 100, name="Throttle %", yaxis="y2"))
0075     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.brake * 100, name="Brake %", yaxis="y2"))
0076     fig.update_layout(
0077         template="plotly_dark",
0078         title="Observed telemetry",
0079         xaxis_title="Time (s)",
0080         yaxis_title="Speed (km/h)",
0081         yaxis2=dict(title="Pedal (%)", overlaying="y", side="right", range=[0, 100]),
0082         height=430,
0083         legend=dict(orientation="h"),
0084         paper_bgcolor="rgba(0,0,0,0)",
0085         plot_bgcolor="rgba(255,255,255,.02)",
0086     )
0087     return fig
0088

```

Line	Code signal	What it is doing / what to verify
67	return fig	Returns the result to the caller; this is the function output boundary that tests should assert.
70	def _telemetry_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
71	session = frame[frame.session_id == session_id].iloc[:1200]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

72	<code>fig = go.Figure()</code>	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
73	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.speed_mps * 3.6, name=</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.throttle * 100, name=</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.brake * 100, name="Bra</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>fig.update_layout(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	<code>template="plotly_dark",</code>	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	<code>title="Observed telemetry",</code>	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
79	<code>xaxis_title="Time (s)",</code>	Creates or updates `xaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>yaxis_title="Speed (km/h)",</code>	Creates or updates `yaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>yaxis2=dict(title="Pedal (%)", overlaying="y", side="right", range=[0, 100]),</code>	Creates or updates `yaxis2`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>height=430,</code>	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>legend=dict(orientation="h"),</code>	Creates or updates `legend`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>paper_bgcolor="rgba(0,0,0,0)",</code>	Creates or updates `paper_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>plot_bgcolor="rgba(255,255,255,.02)",</code>	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
87	<code>return fig</code>	Returns the result to the caller; this is the function output boundary that tests should

Lines 89-110

```

0089
0090 def create_app(run_dir: str | Path, config_path: str | Path = "configs/fast.yaml") -> gr.Blocks:
0091     run_path = Path(run_dir)
0092     config, frame, splits, standardizer, model, dataset = _load_bundle(run_path, Path(config_path))
0093     metrics = json.loads((run_path / "metrics.json").read_text())
0094     ablations = pd.DataFrame(json.loads((run_path / "ablations.json").read_text()))
0095     sessions = sorted(frame.session_id.unique().tolist())
0096
0097     def update_session(session_id: str):
0098         session = frame[frame.session_id == session_id]
0099         cards = (
0100             f"### Session snapshot\n"
0101             f"- **Driver:** {session.driver_id.iloc[0]}\n"
0102             f"- **Track:** {session.track_id.iloc[0]}\n"
0103             f"- **Frames:** {len(session):,}\n"
0104             f"- **Peak speed:** {session.speed_mps.max() * 3.6:.1f} km/h\n"
0105             f"- **Rain peak:** {session.rainfall.max():.2f}"
0106         )
0107         return _track_figure(frame, session_id), _telemetry_figure(frame, session_id), cards
0108
0109     def simulate(index: int, throttle: float, brake: float, grip: float, rain: float, tyre: float):
0110         index = int(np.clip(index, 0, len(dataset) - 1))

```

Line	Code signal	What it is doing / what to verify
90	def create_app(run_dir: str Path, config_path: str Path = "configs/fast.yaml")	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
91	run_path = Path(run_dir)	Creates or updates `run_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	config, frame, splits, standardizer, model, dataset = _load_bundle(run_path, Pat	Creates or updates `config, frame, splits, standardizer, model, dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
93	metrics = json.loads((run_path / "metrics.json").read_text())	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	ablations = pd.DataFrame(json.loads((run_path / "ablations.json").read_text()))	Creates or updates `ablations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	sessions = sorted(frame.session_id.unique().tolist())	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	def update_session(session_id: str):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
98	session = frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	cards = (	Creates or updates `cards`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
100	f"### Session snapshot\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	f"- **Driver:** {session.driver_id.iloc[0]}\n"	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
102	f'- **Track:** {session.track_id.iloc[0]}\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
103	f'- **Frames:** {len(session);,}\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
104	f'- **Peak speed:** {session.speed_mps.max() * 3.6:.1f} km/h\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
105	f'- **Rain peak:** {session.rainfall.max():.2f}"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
106	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
107	return _track_figure(frame, session_id), _telemetry_figure(frame, session_id), c	Returns the result to the caller; this is the function output boundary that tests should assert.
109	def simulate(index: int, throttle: float, brake: float, grip: float, rain: float	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
110	index = int(np.clip(index, 0, len(dataset) - 1))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.

Lines 111-132

```

0111     item = dataset[index]
0112     history_z = item["history"].numpy()
0113     future_z = item["future_inputs"].numpy()
0114     history_raw = history_z * standardizer.input_std + standardizer.input_mean
0115     future_raw = future_z * standardizer.input_std + standardizer.input_mean
0116     scenario = Scenario(throttle, brake, grip, rain, tyre)
0117     predicted = rollout(model, history_raw, future_raw, standardizer, scenario)
0118     actual = standardizer.inverse_targets(item["future_targets"].numpy())
0119     seconds = np.arange(len(predicted)) / config.data.sample_hz
0120     fig = go.Figure()
0121     fig.add_trace(go.Scatter(x=seconds, y=actual[:, 0] * 3.6, name="Recorded future", line=dict(width=3)))
0122     fig.add_trace(go.Scatter(x=seconds, y=predicted[:, 0] * 3.6, name="Imagined future", line=dict(width=3,
dash="dash"))))
0123     fig.update_layout(
0124         template="plotly_dark",
0125         title="World-model rollout: recorded vs imagined speed",
0126         xaxis_title="Seconds into future",
0127         yaxis_title="Speed (km/h)",
0128         height=470,
0129         paper_bgcolor="rgba(0,0,0,0)",
0130         plot_bgcolor="rgba(255,255,255,.02)",
0131     )
0132     delta = float((predicted[-1, 0] - actual[-1, 0]) * 3.6)

```

Line	Code signal	What it is doing / what to verify
111	item = dataset[index]	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	history_z = item["history"].numpy()	Creates or updates `history_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
113	future_z = item["future_inputs"].numpy()	Creates or updates `future_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
114	history_raw = history_z * standardizer.input_std + standardizer.input_mean	Creates or updates `history_raw`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	future_raw = future_z * standardizer.input_std + standardizer.input_mean	Creates or updates `future_raw`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
116	scenario = Scenario(throttle, brake, grip, rain, tyre)	Creates or updates `scenario`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
117	predicted = rollout(model, history_raw, future_raw, standardizer, scenario)	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
118	actual = standardizer.inverse_targets(item["future_ta rgets"].numpy())	Creates or updates `actual`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	seconds = np.arange(len(predicted)) / config.data.sample_hz	Creates or updates `seconds`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	fig = go.Figure()	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
121	fig.add_trace(go.Scatter(x=seconds, y=actual[:, 0] * 3.6, name="Recorded future"	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	fig.add_trace(go.Scatter(x=seconds, y=predicted[:, 0] * 3.6, name="Imagined futu	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
123	fig.update_layout(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
124	template="plotly_dark",	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
125	title="World-model rollout: recorded vs imagined speed",	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
126	xaxis_title="Seconds into future",	Creates or updates `xaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
127	yaxis_title="Speed (km/h)",	Creates or updates `yaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
128	height=470,	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
129	paper_bgcolor="rgba(0,0,0,0)",	Creates or updates `paper_bgcolor`. Check its

		unit, shape, lifetime and whether it is derived only from information available at this point.
130	plot_bgcolor="rgba(255,255,255,0.2)",	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
131	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
132	delta = float((predicted[-1, 0] - actual[-1, 0]) * 3.6)	Creates or updates `delta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 133-154

```

0133     summary = (
0134         f"### Scenario result\n"
0135         f"- Rollout horizon: **{len(predicted) / config.data.sample_hz:.1f} s**\n"
0136         f"- Final imagined speed: **{predicted[-1, 0] * 3.6:.1f} km/h**\n"
0137         f"- Difference from recorded future: **{delta:+.1f} km/h**\n"
0138         f"- Model: **{config.model.kind.upper()} world model**"
0139     )
0140     return fig, summary
0141
0142     with gr.Blocks(title="Project APEX") as app:
0143         gr.Markdown(
0144             """
0145 <div id="hero">
0146 <h1>PROJECT APEX</h1>
0147 <h3>F1 Telemetry World-Simulation Engine</h3>
0148 <p>Explore real-shaped telemetry, inspect the learned world model, alter future conditions, and watch the engine
0149 imagine what happens next.</p>
0149 </div>
0150             """
0151         )
0152         with gr.Tabs():
0153             with gr.Tab("Race Engineering Desk"):
0154                 with gr.Row():

```

Line	Code signal	What it is doing / what to verify
133	summary = (	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
134	f"### Scenario result\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
135	f"- Rollout horizon: **{len(predicted) / config.data.sample_hz:.1f} s**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
136	f"- Final imagined speed: **{predicted[-1, 0] * 3.6:.1f} km/h**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
137	f"- Difference from recorded future: **{delta:+.1f} km/h**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
138	f"- Model: **{config.model.kind.upper()} world model**"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
139	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
140	return fig, summary	Returns the result to the caller; this is the function output boundary that tests should

		assert.
142	with gr.Blocks(title="Project APEX") as app:	Opens a managed resource or context and guarantees cleanup when the block exits.
143	gr.Markdown()	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
144	"""	Begins or ends documentation describing the module, class or function contract.
145	<div id="hero">	Creates or updates `<div id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
146	PROJECT APEX	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
147	F1 Telemetry World-Simulation Engine	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
148	<p>Explore real-shaped telemetry, inspect the learned world model, alter future	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
149	</div>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
150	"""	Begins or ends documentation describing the module, class or function contract.
151	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
152	with gr.Tabs():	Opens a managed resource or context and guarantees cleanup when the block exits.
153	with gr.Tab("Race Engineering Desk"):	Opens a managed resource or context and guarantees cleanup when the block exits.
154	with gr.Row():	Opens a managed resource or context and guarantees cleanup when the block exits.

Lines 155-176

```

0155         session_select = gr.Dropdown(sessions, value=sessions[0], label="Session")
0156         session_summary = gr.Markdown()
0157     with gr.Row():
0158         track_plot = gr.Plot(_track_figure(frame, sessions[0]))
0159         telemetry_plot = gr.Plot(_telemetry_figure(frame, sessions[0]))
0160         session_select.change(update_session, session_select, [track_plot, telemetry_plot,
session_summary])
0161         initial = frame[frame.session_id == sessions[0]]
0162         session_summary.value = f"Frames: {len(initial):,} | Peak speed:
{initial.speed_mps.max()*3.6:.1f} km/h"
0163
0164     with gr.Tab("World Rollout Simulator"):
0165         gr.Markdown("Change the future controls and environment. The model rolls forward from a real
test-window history.")
0166         with gr.Row():
0167             with gr.Column(scale=1):
0168                 sample_index = gr.Slider(0, max(0, len(dataset) - 1), value=0, step=1, label="Test
window")
0169                 throttle = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Throttle multiplier")
0170                 brake = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Brake multiplier")
0171                 grip = gr.Slider(0.6, 1.2, value=1.0, step=0.02, label="Grip multiplier")
0172                 rain = gr.Slider(-0.2, 0.5, value=0.0, step=0.02, label="Rain change")
0173                 tyre = gr.Slider(0.5, 2.0, value=1.0, step=0.05, label="Tyre degradation multiplier")
0174                 run_button = gr.Button("IMAGINE FUTURE", variant="primary")
0175             with gr.Column(scale=2):
0176                 rollout_plot = gr.Plot()

```

Line	Code signal	What it is doing / what to verify
155	<code>session_select = gr Dropdown(sessions, value=sessions[0], label="Session")</code>	Creates or updates `session_select`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
156	<code>session_summary = gr.Markdown()</code>	Creates or updates `session_summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
157	<code>with gr.Row():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
158	<code>track_plot = gr.Plot(_track_figure(frame, sessions[0]))</code>	Creates or updates `track_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
159	<code>telemetry_plot = gr.Plot(_telemetry_figure(frame, sessions[0]))</code>	Creates or updates `telemetry_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
160	<code>session_select.change(update_session, session_select, [track_plot, telemetry_plo</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
161	<code>initial = frame[frame.session_id == sessions[0]]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
162	<code>session_summary.value = f"***Frames:** {len(initial);} **Peak speed:** {initia</code>	Creates or updates `session_summary.value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
164	<code>with gr.Tab("World Rollout Simulator"):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
165	<code>gr.Markdown("Change the future controls and environment. The model rolls forward</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
166	<code>with gr.Row():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
167	<code>with gr.Column(scale=1):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
168	<code>sample_index = gr.Slider(0, max(0, len(dataset) - 1), value=0, step=1, label="Te</code>	Creates or updates `sample_index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
169	<code>throttle = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Throttle multiplier"</code>	Creates or updates `throttle`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
170	<code>brake = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Brake multiplier")</code>	Creates or updates `brake`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
171	<code>grip = gr.Slider(0.6, 1.2, value=1.0, step=0.02, label="Grip multiplier")</code>	Creates or updates `grip`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
172	<code>rain = gr.Slider(-0.2, 0.5, value=0.0, step=0.02, label="Rain change")</code>	Creates or updates `rain`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
173	<code>tyre = gr.Slider(0.5, 2.0, value=1.0, step=0.05, label="Tyre degradation multipl</code>	Creates or updates `tyre`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
174	<code>run_button = gr.Button("IMAGINE FUTURE", variant="primary")</code>	Creates or updates `run_button`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
175	<code>with gr.Column(scale=2):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
176	<code>rollout_plot = gr.Plot()</code>	Creates or updates `rollout_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 177-198

```

0177         rollout_summary = gr.Markdown()
0178         run_button.click(simulate, [sample_index, throttle, brake, grip, rain, tyre], [rollout_plot,
rollout_summary])
0179
0180         with gr.Tab("Model & Evaluation"):
0181             gr.Markdown(
0182                 f"""
0183     ### Test performance
0184     - **Overall MAE:** {metrics['overall_mae']:.4f}
0185     - **Speed MAE:** {metrics['speed_mae_mps'] * 3.6:.2f} km/h
0186     - **Speed RMSE:** {metrics['speed_rmse_mps'] * 3.6:.2f} km/h
0187     - **Test windows:** {metrics['samples']:,}
0188
0189     The horizon curve matters more than a single one-step number: simulation error compounds as the model consumes
its own predictions.
0190             """
0191             )
0192             horizon = pd.DataFrame({"step": np.arange(1, len(metrics["horizon_rmse"]) + 1), "rmse":
metrics["horizon_rmse"]})
0193             fig = go.Figure(go.Scatter(x=horizon.step, y=horizon.rmse, mode="lines+markers"))
0194             fig.update_layout(template="plotly_dark", title="Rollout RMSE by horizon",
xaxis_title="Prediction step", yaxis_title="Normalized RMSE")
0195             gr.Plot(fig)
0196
0197         with gr.Tab("Ablation Lab"):
0198             gr.Markdown("Ablations answer one causal engineering question at a time by removing information
while holding the evaluation protocol fixed.")

```

Line	Code signal	What it is doing / what to verify
177	<code>rollout_summary = gr.Markdown()</code>	Creates or updates `rollout_summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
178	<code>run_button.click(simulate, [sample_index, throttle, brake, grip, rain, tyre], [r</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
180	<code>with gr.Tab("Model & Evaluation"):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
181	<code>gr.Markdown(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
182	<code>f"""</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
183	<code>### Test performance</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
184	<code>- **Overall MAE:** {metrics['overall_mae']:.4f}</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
185	<code>- **Speed MAE:** {metrics['speed_mae_mps'] * 3.6:.2f} km/h</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
186	- Speed RMSE: {metrics['speed_rmse_mps'] * 3.6:.2f} km/h	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
187	- Test windows: {metrics['samples'];,}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
189	The horizon curve matters more than a single one-step number: simulation error c	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
190	"""	Begins or ends documentation describing the module, class or function contract.
191	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
192	horizon = pd.DataFrame({"step": np.arange(1, len(metrics["horizon_rmse"]) + 1),	Creates or updates `horizon`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
193	fig = go.Figure(go.Scatter(x=horizon.step, y=horizon.rmse, mode="lines+markers"))	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
194	fig.update_layout(template="plotly_dark", title="Rollout RMSE by horizon", xaxis	Creates or updates `fig.update_layout(template)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
195	gr.Plot(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
197	with gr.Tab("Ablation Lab"):	Opens a managed resource or context and guarantees cleanup when the block exits.
198	gr.Markdown("Ablations answer one causal engineering question at a time by remov	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 199-220

```

0199         gr.Dataframe(ablations[["ablation", "speed_mae_mps", "overall_mae"]], interactive=False)
0200         fig = go.Figure(go.Bar(x=ablations.ablation, y=ablations.speed_mae_mps * 3.6))
0201         fig.update_layout(template="plotly_dark", title="One-step speed MAE by feature ablation",
0202         yaxis_title="MAE (km/h)")
0202         gr.Plot(fig)
0203
0204         with gr.Tab("System Architecture"):
0205             gr.Markdown(
0206                 """
0207                 ```text
0208                 FastF1 / OpenF1 / Synthetic / Future F1 25 UDP
0209                     ↓
0210                 Source-specific adapters
0211                     ↓
0212                 Canonical telemetry contract
0213                     ↓
0214                 Validation → alignment → session splits
0215                     ↓
0216                 Sequence windows + train-only scaling
0217                     ↓
0218                 Persistence / Linear / GRU / RSSM / SSM models
0219                     ↓
0220                 Horizon evaluation + physical checks

```

Line	Code signal	What it is doing / what to verify
199	gr.Dataframe(ablations[["ablation", "speed_mae_mps", "overall_mae"]],	Creates or updates `gr.Dataframe(ablations[["ablation",

	interactiv	"speed_mae_mps", "overall_mae"]], interactive'. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
200	fig = go.Figure(go.Bar(x=ablations.ablation, y=ablations.speed_mae_mps * 3.6))	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
201	fig.update_layout(template="plotly_dark", title="One-step speed MAE by feature a	Creates or updates `fig.update_layout(template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
202	gr.Plot(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
204	with gr.Tab("System Architecture"):	Opens a managed resource or context and guarantees cleanup when the block exits.
205	gr.Markdown(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
206	"""	Begins or ends documentation describing the module, class or function contract.
207	```text	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
208	FastF1 / OpenF1 / Synthetic / Future F1 25 UDP	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
209	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
210	Source-specific adapters	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
211	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
212	Canonical telemetry contract	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
213	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
214	Validation → alignment → session splits	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
215	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
216	Sequence windows + train-only scaling	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
217	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
218	Persistence / Linear / GRU / RSSM / SSM models	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
219	↓	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
220	Horizon evaluation + physical checks	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 221-236

```

0221         ↓
0222     Scenario interventions + imagined rollouts
0223         ↓
0224     Registry / artifacts / API / interactive UI
0225     ```
0226         """
0227     )
0228     return app
0229
0230
0231 def launch(run_dir: str | Path, config_path: str | Path = "configs/fast.yaml", share: bool = False) -> None:
0232     create_app(run_dir, config_path).launch(share=share, css=CUSTOM_CSS)
0233
0234
0235 if __name__ == "__main__":
0236     launch("artifacts/runs/reference_gru")

```

Line	Code signal	What it is doing / what to verify
221	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
222	Scenario interventions + imagined rollouts	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
223	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
224	Registry / artifacts / API / interactive UI	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
225	```	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
226	"""	Begins or ends documentation describing the module, class or function contract.
227	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
228	return app	Returns the result to the caller; this is the function output boundary that tests should assert.
231	def launch(run_dir: str Path, config_path: str Path = "configs/fast.yaml", s	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
232	create_app(run_dir, config_path).launch(share=share, css=CUSTOM_CSS)	Creates or updates `create_app(run_dir, config_path).launch(share`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
235	if __name__ == "__main__":	Branches only when this condition is true; verify both the true and false paths in tests.
236	launch("artifacts/runs/reference_gru")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Educational Labs and Mini-Projects - Source Map

labs/01_units_and_shapes/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 speed_kmh = np.array([0.0, 72.0, 144.0])
0004 speed_mps = speed_kmh / 3.6
0005 telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
0006 print("speed_mps:", speed_mps)
0007 print("telemetry shape [time, features]:", telemetry.shape)
0008 assert telemetry.shape == (3, 2)
```

labs/02_vectorized_kinematics/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 dt = 0.2
0004 time = np.arange(0, 5, dt)
0005 speed = 20 + 4*np.sin(time)
0006 accel = np.gradient(speed, dt)
0007 reconstructed = speed[0] + np.cumsum(accel)*dt
0008 print("first accelerations:", accel[:5].round(3))
0009 print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

labs/03_force_model/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from education.core import CarState, step_car
0002
0003 state = CarState(0.0, 0.0, 40.0)
0004 for i in range(5):
0005     state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)
0006     print(i, state)
```

labs/04_sampling_aliasing/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 for hz in [5, 12, 40]:
0004     t=np.arange(0,2,1/hz)
0005     signal=np.sin(2*np.pi*8*t)
0006     print(hz, "Hz samples:", np.round(signal[:8],3))
```

labs/05_regression_baseline/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002 from sklearn.linear_model import Ridge
0003
0004 rng=np.random.default_rng(4)
0005 speed=rng.uniform(20,80,1000)
0006 throttle=rng.uniform(0,1,1000)
0007 brake=rng.uniform(0,1,1000)
```



```

0008 y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
0009 X=np.column_stack([speed,throttle,brake])
0010 model=Ridge(alpha=1.0).fit(X[:800],y[:800])
0011 print("coefficients:", model.coef_)
0012 print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))

```

labs/06_autograd_drag/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002
0003 speed=torch.linspace(5,70,200)
0004 true_drag=0.002
0005 target=-true_drag*speed**2
0006 drag=torch.tensor(0.01,requires_grad=True)
0007 opt=torch.optim.SGD([drag],lr=1e-7)
0008 for step in range(200):
0009     opt.zero_grad()
0010     pred=-drag*speed**2
0011     loss=((pred-target)**2).mean()
0012     loss.backward(); opt.step()
0013 print("learned drag:", drag.item(), "loss:", loss.item())

```

labs/07_pytorch_module/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class Transition(nn.Module):
0005     def __init__(self):
0006         super().__init__()
0007         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
0008     def forward(self,x): return self.net(x)
0009
0010 model=Transition(); x=torch.randn(4,5); y=model(x)
0011 print(model); print("input",x.shape,"output",y.shape)

```

labs/08_dataset_windows/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import numpy as np
0002 from education.core import sliding_windows
0003
0004 values=np.arange(30,dtype=float).reshape(10,3)
0005 x,y=sliding_windows(values,history=4,horizon=2)
0006 print("x",x.shape,"y",y.shape)
0007 print("first history\n",x[0]); print("first future\n",y[0])

```

labs/09_loss_comparison/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch.nn import functional as F
0003
0004 pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
0005 print("MSE",F.mse_loss(pred,target).item())
0006 print("MAE",F.l1_loss(pred,target).item())
0007 print("Huber",F.huber_loss(pred,target).item())

```

labs/10_training_loop/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
0005 model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
0006 for epoch in range(8):
0007     model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
0008     grad=float(model.weight.grad.norm()); opt.step()
0009     model.eval();
0010     with torch.no_grad(): val=((model(x)-y)**2).mean()
0011     print(epoch,float(loss),grad,float(val))
```

labs/11_rnn_from_scratch/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002
0003 torch.manual_seed(0); W=torch.randn(3,2); U=torch.randn(3,3); b=torch.zeros(3); h=torch.zeros(3)
0004 sequence=torch.tensor([[1.,0.],[0.,1.],[1.,1.]])
0005 for t,x in enumerate(sequence):
0006     h=torch.tanh(W@x+U@h+b); print(t,h.numpy().round(3))
```

labs/12_gru_gates/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002
0003 torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
0004 for x in [torch.tensor([[1.,0.]],torch.tensor([[0.,1.]],torch.tensor([[0.,1.]])]:
0005     h=cell(x,h); print(h.detach().numpy().round(3))
```

labs/13_linear_ssm/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002 A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
0003 for u in [1,1,0,0,0]:
0004     x=A@x+B[:,0]*u; print(x.round(3))
```

labs/14_selective_ssm/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 class SelectiveCell(nn.Module):
0005     def __init__(self,d):
0006         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
0007     def forward(self,x,h):
0008         a=torch.sigmoid(self.decay(x))*0.99
0009         b=torch.tanh(self.write(x))
0010         return a*h+(1-a)*b
0011 cell=SelectiveCell(3); h=torch.zeros(1,3)
0012 for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))
```

labs/15_autoencoder/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 torch.manual_seed(3); x=torch.randn(256,12)
0005 enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
0006 for _ in range(80):
0007     opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward(); opt.step()
0008 print("latent",z.shape,"loss",float(loss))
```

labs/16_vae_reparameterization/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002
0003 mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[ -1.0,0.5]])
0004 std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
0005 kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
0006 print("mu",mu,"std",std,"sample",z,"KL",kl)
```

labs/17_rssm_step/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
0005 h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
0006 h=cell(torch.cat([prev_z,action],-1),h)
0007 prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
0008 print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

labs/18_cem_planning/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
0004 for it in range(5):
0005     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
0006     speed=40+np.cumsum(2*actions-0.4,axis=1)
0007     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
0008     elite=actions[np.argsort(score)[-50:]]
0009     mean,std=elite.mean(0),elite.std(0)+1e-3
0010 print(it,mean.round(2),score.max().round(3))
```

labs/19_time_alignment/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import pandas as pd
0002
0003 car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
0004 weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
0005 aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
0006 print(aligned)
```

labs/20_pipeline_contract/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from pathlib import Path
0002 import json
0003
0004 run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
0005 stages=['ingest','validate','window','train','evaluate','publish']
0006 for i,name in enumerate(stages):
0007     path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
0008     print(path)
```

projects/01_car_integrator/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from education.core import CarState, step_car
0002
0003 def run():
0004     state=CarState(0,0,0)
0005     trace=[]
0006     for i in range(100):
0007         state=step_car(state,0.8 if i<60 else 0.0,0.0 if i<60 else 0.5,0.1)
0008         trace.append(state)
0009     assert trace[-1].speed_mps >= 0
0010     print(trace[-1])
0011 if __name__=='__main__': run()
```

projects/02_telemetry_quality/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import pandas as pd
0002
0003 REQUIRED={'time_s','speed_mps','throttle','brake'}
0004 def validate(df):
0005     errors=[]
0006     if not REQUIRED.issubset(df): errors.append('missing columns')
0007     if not df.time_s.is_monotonic_increasing: errors.append('time not monotonic')
0008     if (df.speed_mps<0).any(): errors.append('negative speed')
0009     if not df.throttle.between(0,1).all(): errors.append('throttle range')
0010     return errors
0011 def run():
0012     df=pd.DataFrame({'time_s':[0,0.1,0.2],'speed_mps':[0,3,4],'throttle':[0,1,0.5],'brake':[0,0,0]})
0013     assert validate(df)==[]; print('quality passed')
0014 if __name__=='__main__': run()
```

projects/03_lap_physics/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 def run(n=600):
0004     progress=np.linspace(0,1,n,endpoint=False); curvature=0.02+0.12*(np.sin(6*np.pi*progress)**2)
0005     target=np.clip(90/(1+10*curvature),20,90); speed=np.empty(n); speed[0]=30
0006     for i in range(1,n): speed[i]=speed[i-1]+0.08*np.clip(target[i]-speed[i-1],-8,3)
0007     print('speed range',float(speed.min()),float(speed.max()))
0008     return progress,curvature,speed
0009 if __name__=='__main__': run()
```

projects/04_mlp_transition/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class Transition(nn.Module):
0005     def __init__(self):
0006         super().__init__()
0007         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
0008     def forward(self,x): return self.net(x)
0009
0010 model=Transition(); x=torch.randn(4,5); y=model(x)
0011 print(model); print("input",x.shape,"output",y.shape)
0012
0013 print("Extend this module with the dataset from Project 3.")

```

projects/05_track_encoder/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class TrackEncoder(nn.Module):
0005     def __init__(self):
0006         super().__init__(); self.net=nn.Sequential(nn.Conv1d(3,8,3,padding=1),nn.ReLU(),nn.AdaptiveAvgPool1d(1))
0007     def forward(self,x): return self.net(x).squeeze(-1)
0008 if __name__=='__main__':
0009     x=torch.randn(4,3,64); print(TrackEncoder()(x).shape)

```

projects/06_gru_forecaster/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class GRUForecaster(nn.Module):
0005     def __init__(self,f=6,h=32,o=2):
0006         super().__init__(); self.gru=nn.GRU(f,h,batch_first=True); self.head=nn.Linear(h,o)
0007     def forward(self,x): return self.head(self.gru(x)[0][:,-1])
0008 if __name__=='__main__': print(GRUForecaster()(torch.randn(8,20,6)).shape)

```

projects/07_selective_ssm/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class SelectiveCell(nn.Module):
0005     def __init__(self,d):
0006         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
0007     def forward(self,x,h):
0008         a=torch.sigmoid(self.decay(x))*0.99
0009         b=torch.tanh(self.write(x))
0010         return a*h+(1-a)*b
0011 cell=SelectiveCell(3); h=torch.zeros(1,3)
0012 for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))

```

projects/08_rssm_imagination/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
0005 h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
0006 h=cell(torch.cat([prev_z,action],-1),h)
0007 prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
0008 print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)

```

projects/09_latent_mpc/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

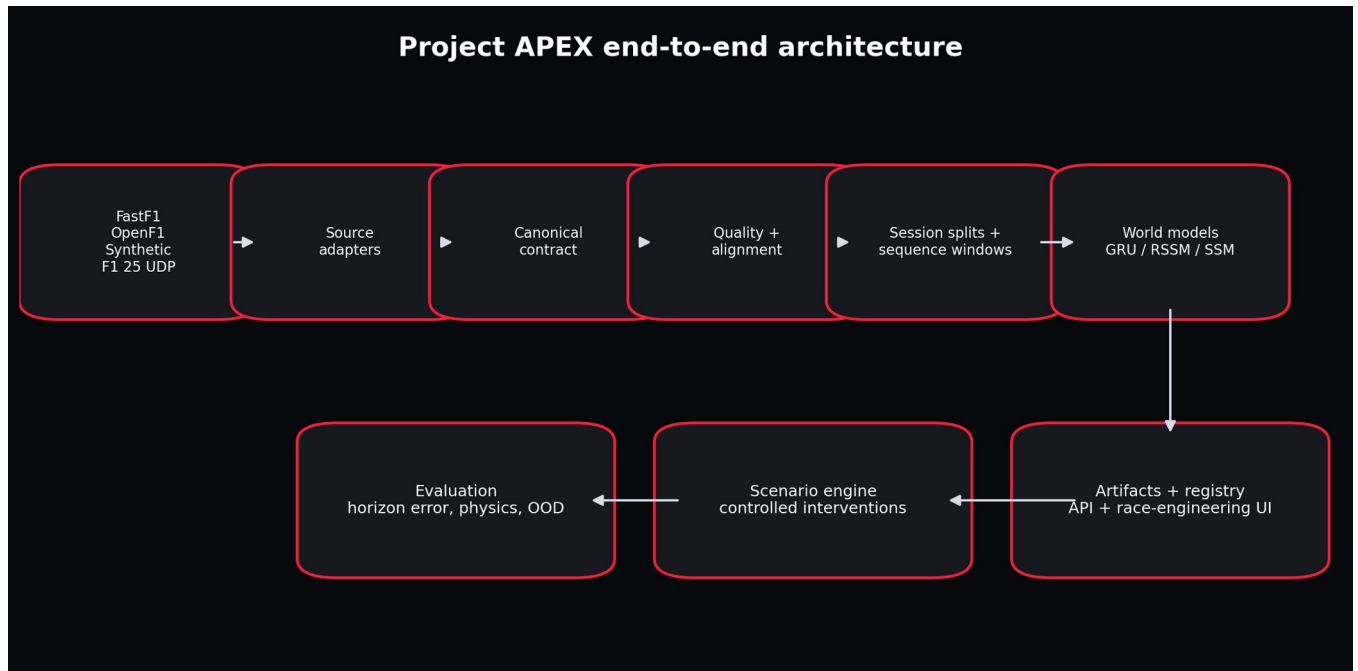
0001 import numpy as np
0002
0003 rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
0004 for it in range(5):
0005     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
0006     speed=40+np.cumsum(2*actions-0.4,axis=1)
0007     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
0008     elite=actions[np.argsort(score)[-50:]]
0009     mean,std=elite.mean(0),elite.std(0)+1e-3
0010     print(it,mean.round(2),score.max().round(3))

```

PROJECT APEX

THE ULTIMATE F1 WORLD-SIMULATION ENGINE TEXTBOOK

From first principles to FastF1/OpenF1 ingestion, GRU/RSSM/SSM world models, production pipelines, evaluation, fine-tuning, and a working interactive V1

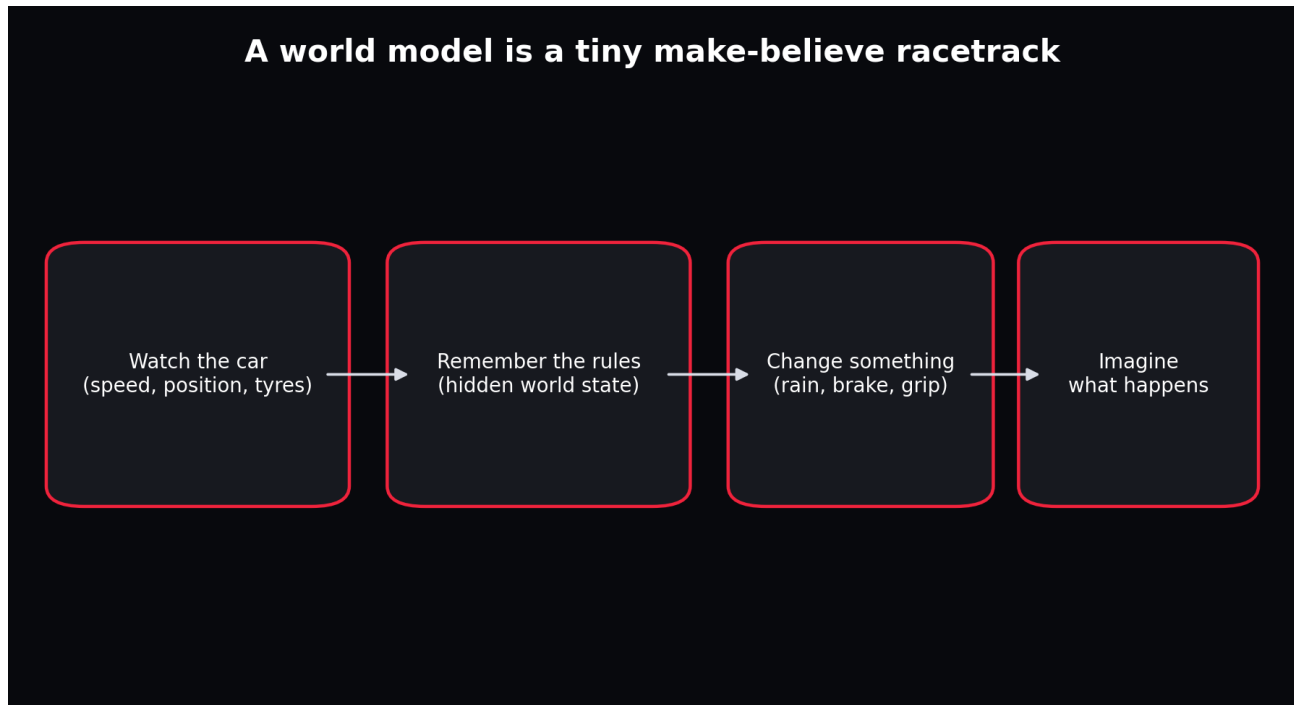


The complete learning path and production architecture.

Instructor Edition | Beginner to Production

How to Use This Book

This is not a book to skim and then declare understood. It is a build-along course, engineering reference, and debugging manual. The chapters first build the mental machinery required for the project. The later parts ask you to predict outputs, run code, inspect artifacts, break assumptions, compare implementations, and defend decisions.



The child-level explanation and the production system describe the same idea at different resolutions.

Mode	What you do	Evidence of learning
Read	Study the concept and redraw the visual from memory.	You can explain it without copying the wording.
Run	Execute the associated notebook or command.	You can predict shapes, files and approximate behaviour before execution.
Break	Introduce the listed defect.	You can identify which invariant was violated and where detection belongs.
Choose	Compare alternatives using the decision table.	You can state why one design is appropriate now and what future evidence would justify another.
Rebuild	Implement the component without viewing the solution.	Tests pass and your implementation preserves the same contract.

Table of Contents

Part	Theme	Coverage
Part I	First-Principles Foundations	Chapters 1-10
Part II	Sequence Models and State-Space Models	Chapters 11-18
Part III	World Models and Dreamer	Chapters 19-27
Part IV	Project Scope and Architecture	Chapters 28-33
Part V	Ingestion, Contracts and Data Flow	Chapters 34-42
Part VI	Guided Construction of V1	Chapters 43-55
Part VII	Evaluation, Ablations and Fine-Tuning	Chapters 56-63
Part VIII	Production, UI and Operations	Chapters 64-70
Part IX	F1 25 Migration and Long-Term Roadmap	Chapters 71-74
Part X	Independent Mastery and Reference	Chapters 75-80

Part I - First-Principles Foundations

The purpose of this part is not to turn you into a mechanical engineer before you write Python. It is to give every variable, loss and model output a physical or statistical meaning. Code becomes easy to reason about when you can tell a coherent story about the quantities moving through it.

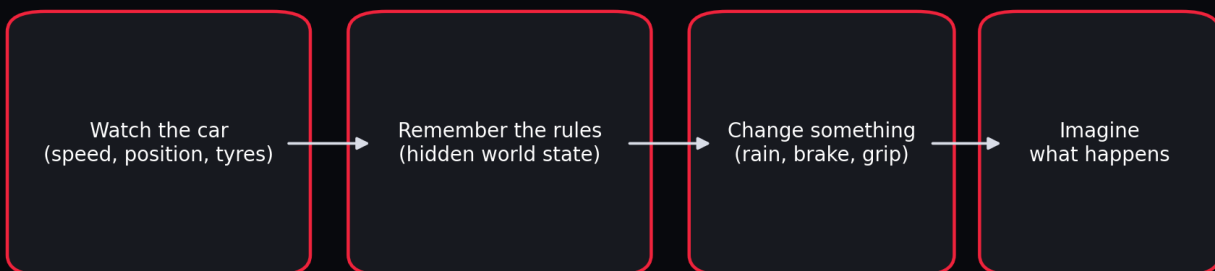
Chapter 1 - Explain the Entire Project to a Five-Year-Old

WHAT YOU ARE LEARNING: The simplest truthful explanation of a world model and the exact boundary of Project APEX.

WHY IT MATTERS: If the simple explanation and the technical architecture disagree, the technical system is probably confused.

YOU ARE FINISHED WHEN: You can explain the project using toys and then translate every toy into an engineering term.

A world model is a tiny make-believe racetrack



A world model observes, remembers, accepts a change, and imagines a future.

Imagine a toy car on a track. You watch it for a while. You notice that pressing the accelerator usually makes it faster, pressing the brake makes it slower, tight corners require lower speed, rain

makes the track slippery, and old tyres gradually lose grip. Now imagine building a little brain that remembers those patterns. When you ask, "What happens if it rains more?", the brain draws a possible next few seconds.

The toy car is the environment. What you can measure is the observation. The compact description of what matters now is the state. Driver controls are actions. Track and weather variables are context. The learned rule that moves the current state into the next state is the transition model. Repeatedly applying the transition is a rollout or imagination.

SCOPE BOUNDARY: V1 is a telemetry dynamics simulator. It is not a complete rigid-body physics engine, a licensed F1 game, or an autonomous racing agent. It learns short future telemetry trajectories from recorded-style data.

INSTRUCTOR CHECKPOINT: Close the book and explain the system to a child in four sentences. Then explain it to an ML engineer using state, action, context, transition and rollout.

Chapter 2 - What Is a Simulation Engine?

WHAT YOU ARE LEARNING: The difference among replay, forecasting, simulation, physics modelling and control.

WHY IT MATTERS: Many projects call any chart with a prediction a simulator. That produces exaggerated claims and poor evaluation.

YOU ARE FINISHED WHEN: You can classify a component correctly and state what evidence would upgrade it to the next class.

System	Input	Output	Can answer interventions?
Replay	Recorded data	The same recorded trajectory	No
Forecast	Past observations	Likely future observations	Only weakly
Data-driven simulator	Past state plus explicit future actions/context	A conditional future trajectory	Yes, within support
Physics simulator	Physical parameters and controls	Numerically integrated physical state	Yes, under model assumptions
Controller/planner	State, objective and simulator	Selected actions	Yes, by searching or learning

Project APEX begins as a data-driven simulator. It is conditioned on planned control/context trajectories. Its counterfactual claims are credible only near combinations represented in training.

A throttle multiplier of 1.05 is a modest intervention; inventing zero grip at 350 km/h is far outside the learned world.

Worked example

QUESTION: A dashboard predicts the next lap time from previous lap times but has no input for tyre choice or weather changes. Is it a simulator?

Step 1. Identify whether the model receives causes that a user can deliberately change.

Step 2. Notice that it predicts an outcome from history but cannot condition on a proposed intervention.

Step 3. Classify it as a forecast and describe how to make it simulation-like.

ANSWER: It is a forecasting model. Add explicit state, action and context variables, then evaluate controlled interventions against held-out examples or a trusted physical simulator.

Chapter 3 - Units, Sampling and Dimensional Thinking

WHAT YOU ARE LEARNING: SI units, conversions, rates, discrete sampling and dimensional checks.

WHY IT MATTERS: Most telemetry bugs are not neural-network bugs. They are km/h versus m/s, milliseconds versus seconds, percentages versus fractions, or mismatched sample rates.

YOU ARE FINISHED WHEN: You can detect a unit error from magnitude and verify an equation dimensionally.

The canonical contract uses metres, seconds, metres per second, metres per second squared, degrees Celsius, and unit fractions for controls. Source adapters perform conversion once. Model code should never ask whether Speed means km/h or m/s.

Quantity	Canonical unit	Common source representation	Conversion
Speed	m/s	km/h	divide by 3.6
Throttle	0 to 1	0 to 100 percent	divide by 100
Time	seconds	timedelta or ISO timestamp	convert to elapsed seconds
Gear	integer plus normalized fraction	integer 0-8	gear_norm = gear / 8
Rainfall	0 to 1 severity proxy	boolean or category	define and document adapter mapping

```
speed_kmh = 288.0
speed_mps = speed_kmh / 3.6
sample_hz = 5
dt = 1.0 / sample_hz
distance_one_frame = speed_mps * dt
print(speed_mps, distance_one_frame)
```

At 288 km/h, the car travels 16 metres during one 5 Hz frame.

Worked example

QUESTION: A car moves at 270 km/h. How far does it travel between samples at 4 Hz?

Step 1. Convert 270 km/h to m/s: $270 / 3.6 = 75$ m/s.

Step 2. At 4 Hz, one frame lasts $1/4 = 0.25$ s.

Step 3. Distance = speed x time = 75×0.25 .

ANSWER: 18.75 metres. This large spacing explains why independently sampled location and control streams require careful alignment.

Chapter 4 - Kinematics Without Fear

WHAT YOU ARE LEARNING: Position, velocity, acceleration, discrete differences and integration.

WHY IT MATTERS: The predicted channels must obey a coherent motion story.

YOU ARE FINISHED WHEN: You can compute acceleration from frames and explain why differencing amplifies noise.

Velocity describes how position changes. Acceleration describes how velocity changes. In sampled telemetry, acceleration is commonly estimated as $(v_t - v_{(t-1)}) / dt$. This estimate magnifies measurement noise because a small speed error is divided by a small time interval. Smooth only with a documented causal filter when the model will run online.

```
dt = timestamp_s.diff()
acceleration = speed_mps.diff() / dt
# Never allow zero or negative dt to pass silently.
```

COMMON FAILURE: Interpolating across a long data gap creates a smooth but imaginary acceleration profile. Mark large gaps and split the episode instead of fabricating continuity.

Chapter 5 - Forces and a Useful Toy Vehicle Model

WHAT YOU ARE LEARNING: Engine force, braking, aerodynamic drag, corner demand, wet grip and tyre degradation.

WHY IT MATTERS: The offline generator needs causal structure strong enough to test the learning system.

YOU ARE FINISHED WHEN: You can explain every term in the synthetic acceleration equation and identify what it omits.

The generator uses a deliberately simple longitudinal equation: acceleration equals engine contribution minus braking, aerodynamic drag, corner demand and wet penalty, plus small noise. It is not a CFD or tyre model. It is a controlled world where changing throttle, brake, curvature, rain and grip produces the correct directional effects.

```
acceleration = (  
    engine_accel  
    - brake_accel  
    - aero_drag  
    - corner_drag  
    - wet_drag  
    + stochastic_force  
)  
speed_next = clip(speed_now + acceleration * dt, 8.0, 101.0)
```

Term	Teaching role	Missing real-world detail
engine_accel	Throttle causes positive acceleration	Power curve, ERS, gear ratios, traction limits
brake_accel	Brake causes deceleration	Brake temperature, balance, lock-up
aero_drag	Penalty grows with speed squared	Ride height, wings, DRS maps, yaw
corner_drag	Curvature limits desired speed	Lateral tyre force and load transfer
wet_drag/grip	Weather changes dynamics	Water depth, tyre compound, aquaplaning

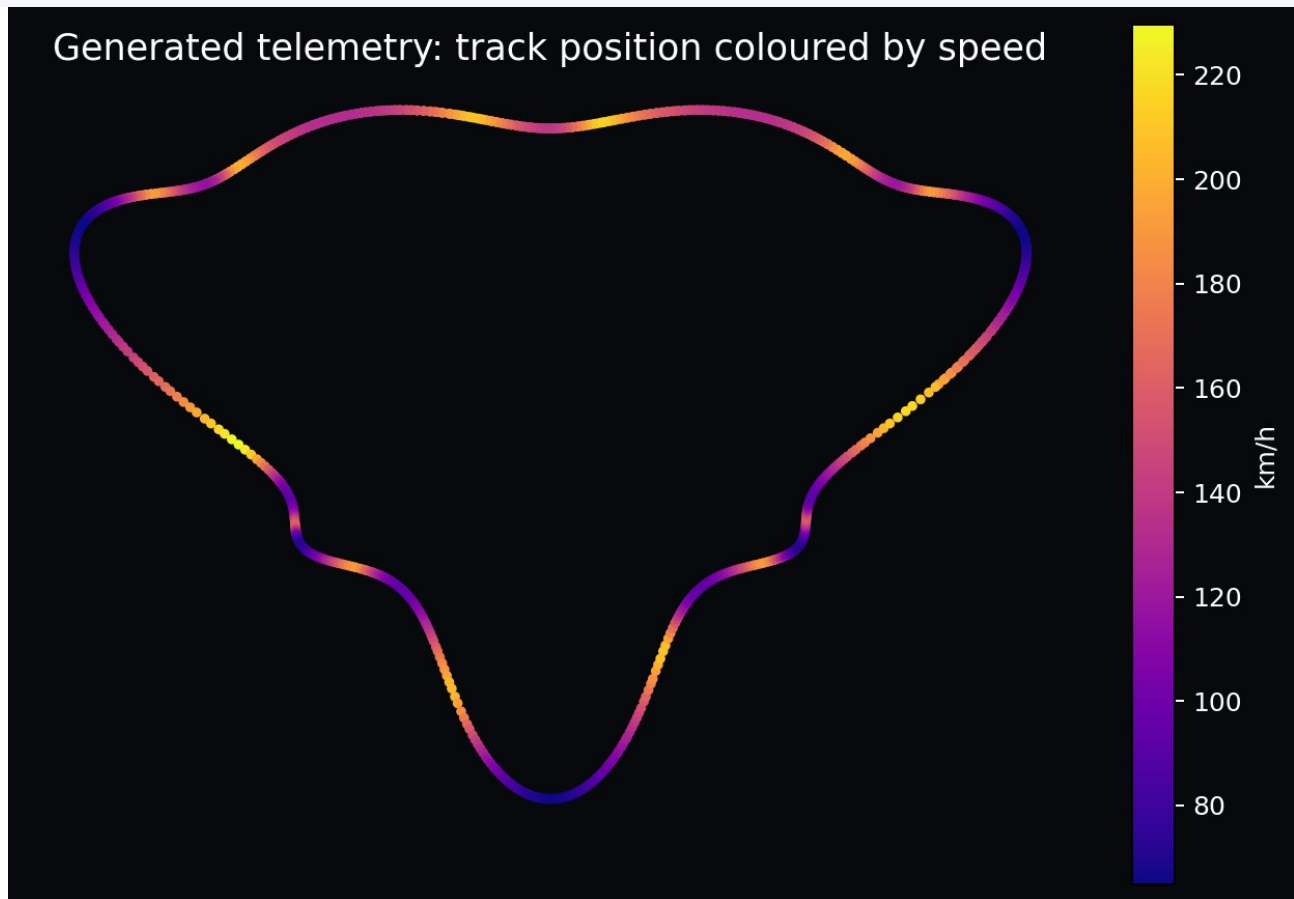
INSTRUCTOR CHECKPOINT: State two reasons a learned model can outperform this toy equation on its own generated data, and two reasons that would not prove better real-world physics.

Chapter 6 - Track Geometry, Progress and Curvature

WHAT YOU ARE LEARNING: Cartesian position, heading, curvature, cyclic progress and track maps.

WHY IT MATTERS: The same speed means different things on a straight and at a hairpin.

YOU ARE FINISHED WHEN: You can encode cyclic progress without a discontinuity at the finish line.



A generated closed track; colour reveals the causal relationship between geometry and speed.

Raw progress jumps from almost 1 back to 0 at the finish line. A neural model may interpret that as a violent discontinuity. Encode progress as $\sin(2\pi p)$ and $\cos(2\pi p)$. Points just before and after the line are then close on the unit circle.

```
progress_sin = np.sin(2 * np.pi * progress)
progress_cos = np.cos(2 * np.pi * progress)
# Recover angle with atan2(progress_sin, progress_cos).
```

Worked example

QUESTION: Progress changes from 0.99 to 0.01. Why is raw mean squared error misleading?

Step 1. Raw difference is 0.98 even though the car moved a tiny distance across the start line.

Step 2. The representation ignores circular topology.

Step 3. Map both values to nearby points on a unit circle.

ANSWER: Use sine/cosine encoding or a circular loss. Also monitor whether predicted sine/cosine remain near unit norm.

Chapter 7 - Probability as Multiple Possible Futures

WHAT YOU ARE LEARNING: Random variables, conditional distributions, expectation, variance and uncertainty.

WHY IT MATTERS: Traffic, driver decisions, grip and measurement noise create futures that are not perfectly deterministic.

YOU ARE FINISHED WHEN: You can distinguish noise, model uncertainty and scenario uncertainty.

A deterministic model returns one trajectory. A probabilistic world model represents a distribution $p(s_{\text{future}} \mid \text{history, actions, context})$. The mean is not automatically the most useful output. At a braking point with two plausible driver behaviours, the mean trajectory may resemble neither behaviour.

Uncertainty	Example	Treatment
Aleatoric	Small grip fluctuations or measurement noise	Predict a distribution or multiple samples
Epistemic	Unseen wet track or new driver	Ensembles, OOD detection, more data
Scenario	Different planned actions	Condition explicitly and compare rollouts
Data quality	Missing or misaligned packets	Quality flags; do not disguise as model uncertainty

Chapter 8 - Statistics for Telemetry

WHAT YOU ARE LEARNING: Mean, variance, covariance, correlation, conditional analysis and train-only normalization.

WHY IT MATTERS: Telemetry features have different scales and strong dependence.

YOU ARE FINISHED WHEN: You can calculate a standardized value and explain why global normalization leaks information.

Standardization transforms x into $(x - \text{mean_train}) / \text{std_train}$. Fit statistics on training sessions only. A mean computed using test weather or test-driver speed distribution transfers information from evaluation into training.

Worked example

QUESTION: Training speed has mean 62 m/s and standard deviation 12 m/s. What is the standardized value of 86 m/s?

Step 1. Subtract the training mean: $86 - 62 = 24$.

Step 2. Divide by training standard deviation: $24 / 12$.

ANSWER: The standardized speed is 2.0. It is two training standard deviations above the training mean.

CORRELATION WARNING: Throttle and speed may be positively correlated on straights and negatively related immediately before corners because drivers lift before speed falls. Time direction and context matter.

Chapter 9 - Vectors, Matrices and Tensors as Organized Measurements

WHAT YOU ARE LEARNING: Feature vectors, batches, sequence axes, matrix multiplication and tensor semantics.

WHY IT MATTERS: Deep-learning code becomes English when every axis has a sentence.

YOU ARE FINISHED WHEN: You can annotate [B,T,F] and catch an accidental axis swap.

For Project APEX, a history tensor has shape [batch, time, features]. Batch groups independent examples. Time preserves causal order. Features describe one instant. A model that receives [time, batch, features] when it expects batch first may still run and silently learn nonsense.

```
history.shape == [B, T_history, F_input]
future_inputs.shape == [B, T_future, F_input]
future_targets.shape == [B, T_future, F_state]
```

INSTRUCTOR CHECKPOINT: For a batch of 64 windows, 32 history frames, 16 input features and an 8-step future with 5 targets, write all three tensor shapes without looking.

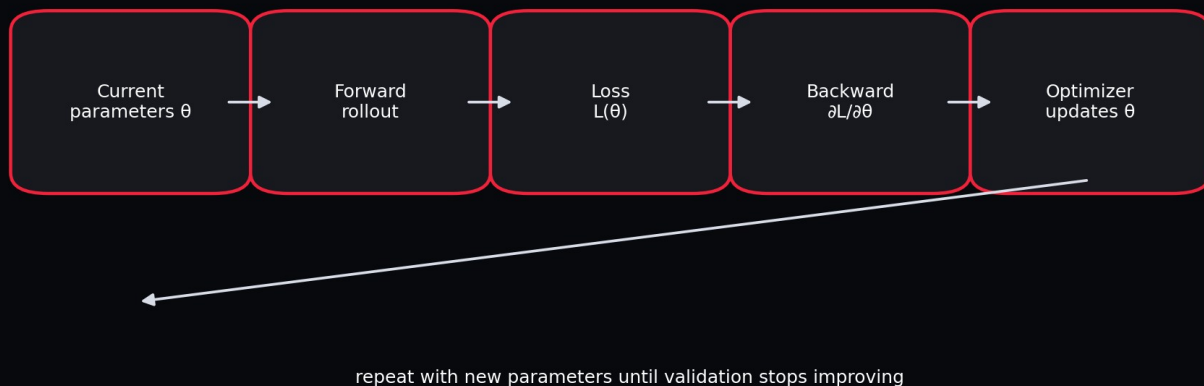
Chapter 10 - Gradients and Optimization

WHAT YOU ARE LEARNING: Loss functions, derivatives, backpropagation, learning rate, clipping and validation.

WHY IT MATTERS: Training is a controlled numerical procedure, not a magical call to fit.

YOU ARE FINISHED WHEN: You can narrate each line of the training loop and diagnose a loss that does not decrease.

The training loop as a state transition



Training repeatedly changes parameters using gradients computed from prediction error.

The gradient is a local sensitivity: if a parameter changes slightly, how does the loss change? Backpropagation applies the chain rule through every operation. The optimizer uses those

sensitivities to update parameters. A learning rate that is too large can overshoot; one that is too small can look frozen. Gradient clipping limits an unstable update but should not hide a broken loss or scale.

```
optimizer.zero_grad(set_to_none=True)
prediction = model(history, future_inputs)
loss = smooth_l1_loss(prediction, future_targets)
loss.backward()
clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

Part II - Sequence Models and State-Space Models

Chapter 11 - Time Series Are Not Shuffled Tables

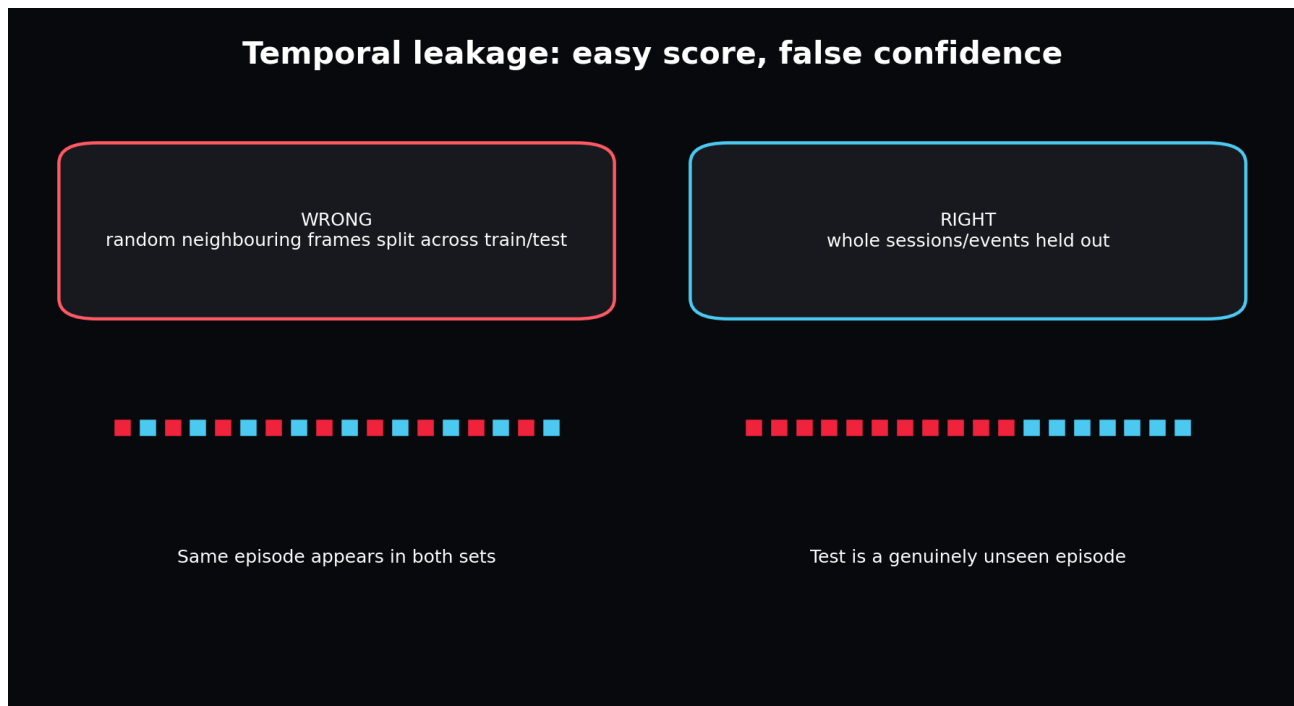
WHAT YOU ARE LEARNING: Autocorrelation, non-stationarity, episodes, gaps and causal order.

WHY IT MATTERS: Random tabular assumptions produce leakage and unrealistic validation.

YOU ARE FINISHED WHEN: You can design an episode-aware split and explain what future information is allowed.

Adjacent telemetry frames are highly similar. Weather and tyre state evolve slowly. A random row split creates test examples almost duplicated in training. Project APEX splits whole sessions, then creates windows that never cross session boundaries.

Temporal leakage: easy score, false confidence



Neighbour leakage creates flattering metrics that collapse on genuinely new sessions.

Chapter 12 - Markov State and Partial Observability

WHAT YOU ARE LEARNING: The Markov property, hidden variables and belief state.

WHY IT MATTERS: A single telemetry row rarely contains everything required to predict the future.

YOU ARE FINISHED WHEN: You can explain why history is needed even when current speed and controls are known.

A state is Markov if the next state depends on the present state and action, not the entire past. Real telemetry is partially observed: tyre temperature, driver intent, exact grip and energy state may be hidden. A sequence model uses history to build a belief state that summarizes those hidden causes.

Worked example

QUESTION: Two cars have identical speed, throttle and track position. One has overheated tyres and the other does not. Are the visible rows Markov?

Step 1. The future differs because an omitted variable changes available grip.

Step 2. Current visible features are therefore insufficient.

Step 3. History may reveal the hidden condition through recent slip, pace or temperature proxies.

ANSWER: No. The observation is partially observable. Add the missing sensor if possible or use recurrent history to estimate a belief state.

Chapter 13 - RNNs: A Memory That Updates One Step at a Time

WHAT YOU ARE LEARNING: Recurrent hidden states, unrolling and vanishing/exploding gradients.

WHY IT MATTERS: RNNs establish the basic pattern used by GRUs, RSSMs and recurrent SSM inference.

YOU ARE FINISHED WHEN: You can write the recurrence and draw an unrolled sequence.

A simple RNN updates $h_t = \tanh(W_x x_t + W_h h_{(t-1)} + b)$. The same parameters are reused at every time step. This weight sharing lets it process variable-length streams. Long chains of derivatives can vanish or explode, motivating gates and structured dynamics.

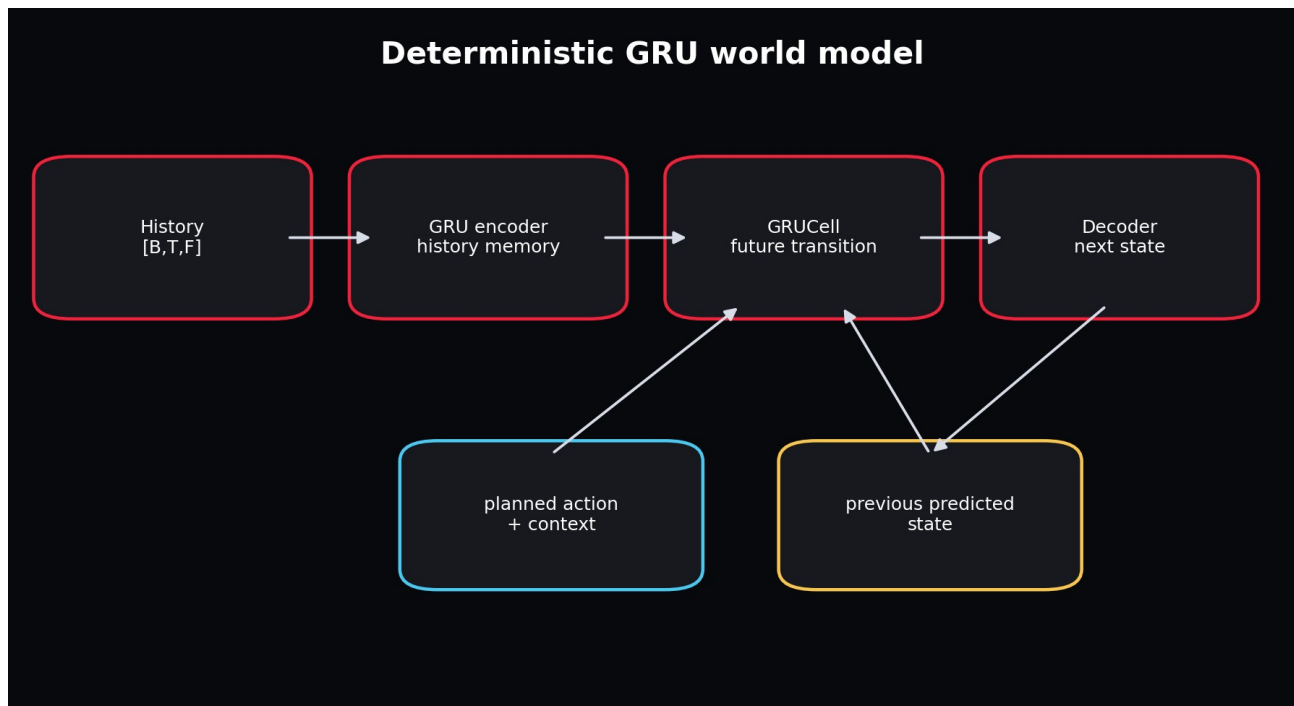
Chapter 14 - GRUs: Gated, Practical Memory

WHAT YOU ARE LEARNING: Update gates, reset gates and a deterministic sequence baseline.

WHY IT MATTERS: The GRU is the safest first neural world model for this project.

YOU ARE FINISHED WHEN: You can explain what the gate controls without memorizing every symbol.

The update gate chooses how much previous memory to keep. The reset gate controls how much old memory participates in a new candidate. In practice, PyTorch implements these equations efficiently. Your job is to understand the input/output contract and rollout semantics.



The history encoder creates memory; the transition cell rolls forward using planned controls and previous predictions.

WHY GRU FIRST: It is causal, compact, available in core PyTorch, easy to profile, and strong enough to reveal data-contract problems.

Chapter 15 - Classical State-Space Models

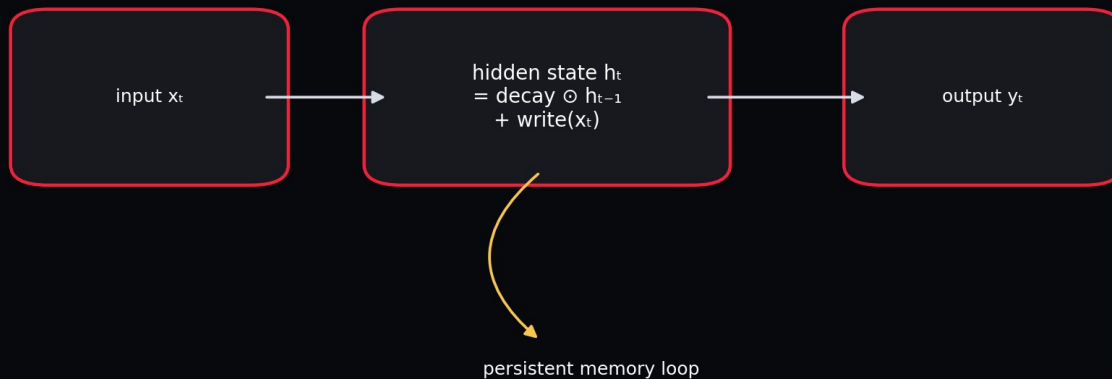
WHAT YOU ARE LEARNING: Hidden state, transition matrix, input matrix and observation matrix.

WHY IT MATTERS: Modern SSMs inherit a powerful continuous-time modelling perspective.

YOU ARE FINISHED WHEN: You can map A, B and C to memory, input writing and output reading.

A continuous linear state-space model can be written $\dot{h}(t) = A h(t) + B x(t)$, $y(t) = C h(t) + D x(t)$. Discretization turns it into a recurrence. A controls memory dynamics, B writes inputs, C reads the hidden state, and D provides a direct path. The hidden state can be much richer than the observed output.

State-space memory: retain, forget, write



Selective SSMs make forgetting and writing depend on the current input.

A state-space model keeps a persistent state rather than recomputing all pairwise interactions.

Chapter 16 - S4 and Structured Long-Range Memory

WHAT YOU ARE LEARNING: Why naive SSMs are hard to train and how structure enables efficient sequence modelling.

WHY IT MATTERS: S4 is the bridge from classical control models to modern deep sequence architectures.

YOU ARE FINISHED WHEN: You can state the engineering advantage without deriving the Cauchy kernel.

S4 parameterizes the state matrix so long-range dynamics remain stable and can be computed efficiently. During training, an SSM can often be represented as a convolution across a known sequence; during streaming inference, it can run as a recurrence with fixed-size memory. This duality is attractive for long telemetry histories.

DO NOT OVERCLAIM: The included APEX selective SSM cell teaches the recurrence and gating idea. It does not reproduce the full S4 mathematical parameterization or optimized kernels.

Chapter 17 - Mamba and Selective State Spaces

WHAT YOU ARE LEARNING: Input-dependent retention, linear sequence scaling and recurrent inference.

WHY IT MATTERS: Telemetry contains occasional events that deserve selective memory: braking zones, pit entry, rain onset, damage and safety cars.

YOU ARE FINISHED WHEN: You can explain why input-dependent parameters are more expressive than a time-invariant recurrence.

Traditional linear time-invariant SSM dynamics treat every token with the same transition rule. Mamba makes selected state-space parameters functions of the input, allowing the model to retain or forget information based on current content. The published architecture also uses a hardware-aware scan for efficient training and inference.

Choose architecture from the bottleneck



Do not choose the most fashionable model. Choose the simplest model that solves the measured failure.

Architecture should be selected from the measured bottleneck, not prestige.

Use case	Prefer	Reason
Short, medium telemetry windows and first implementation	GRU	Lowest debugging and dependency cost
Long streaming histories with bounded recurrent state	SSM/Mamba	Linear sequence scaling and efficient recurrent inference
Multiple plausible futures and	RSSM	Stochastic latent imagination

uncertainty		
Flexible pairwise interactions in moderate sequences	Transformer	Direct attention across positions

Chapter 18 - Transformers in Context

WHAT YOU ARE LEARNING: Self-attention, causal masks, positional information and computational trade-offs.

WHY IT MATTERS: Transformers are a useful comparator, not the automatic default.

YOU ARE FINISHED WHEN: You can identify when attention adds value and when it adds avoidable cost.

Self-attention lets every step compute a weighted combination of other steps. This is powerful when distant events interact in content-dependent ways. Standard attention costs roughly quadratically with sequence length, while recurrent GRU/SSM inference keeps a fixed state. For an 8-second, 5 Hz V1 window, a Transformer is feasible; it is simply not necessary before the data and evaluation system are trusted.

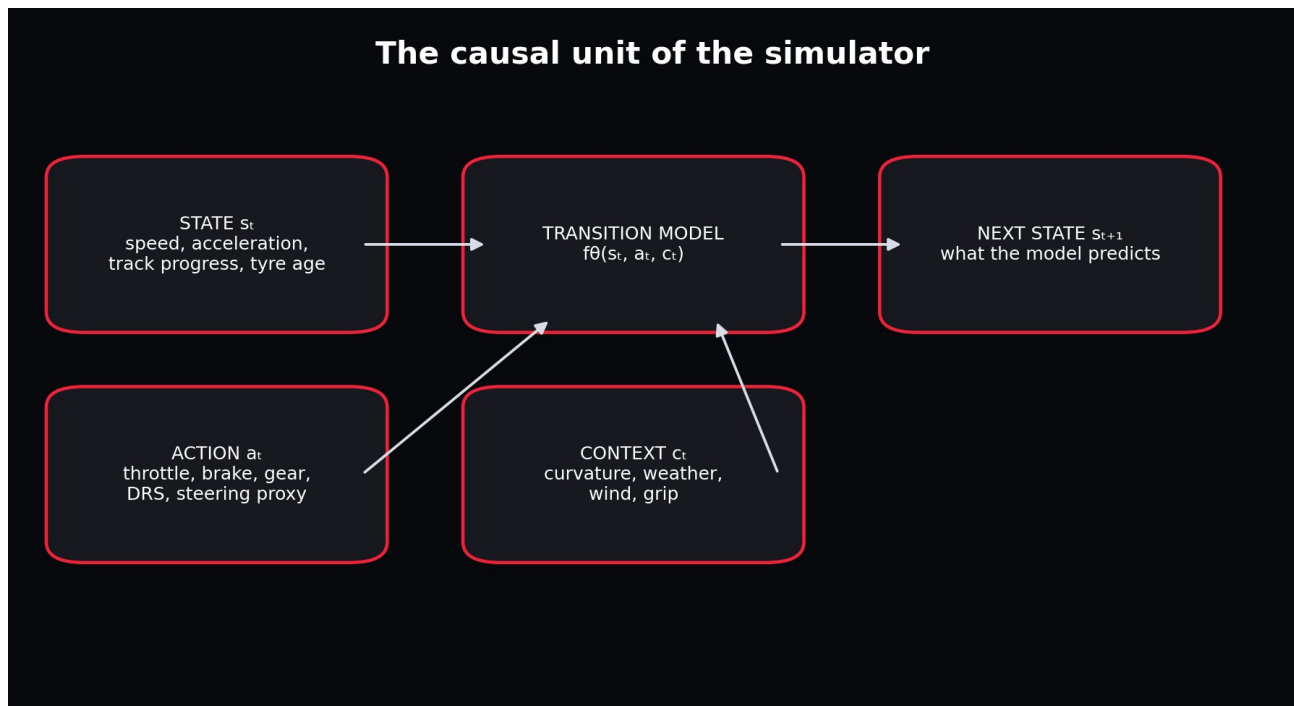
Part III - World Models and Dreamer

Chapter 19 - What Makes a Model a World Model?

WHAT YOU ARE LEARNING: Learned dynamics, observations, actions, rewards and imagination.

WHY IT MATTERS: A sequence predictor becomes useful for planning only when it represents action-conditioned consequences.

YOU ARE FINISHED WHEN: You can distinguish an unconditional forecast from a controllable learned world.



The project models the conditional transition from present state and causes to next state.

A world model is a learned representation of environment dynamics. It may predict observations directly or in a latent space. For control, it must respond to actions. Dreamer adds reward and continuation predictions, then trains an actor and critic inside imagined latent trajectories. Project APEX V1 stops before actor-critic control and concentrates on validating dynamics.

Chapter 20 - Observation, State, Action, Context, Reward and Continuation

WHAT YOU ARE LEARNING: The semantic roles in a control-oriented model.

WHY IT MATTERS: Mixing action and state makes interventions invalid.

YOU ARE FINISHED WHEN: You can classify every canonical feature and defend ambiguous cases.

Role	APEX V1 examples	Future F1 25 examples
Observation/state target	speed, acceleration, cyclic progress, tyre age	wheel speeds, temperatures, suspension, yaw, damage
Action	throttle, brake, gear, DRS, steering proxy	actual steering, clutch, overtake/ERS controls
Context	curvature, weather, wind, grip	setup, surface type, session state

Reward/cost	not trained in V1	lap-time progress, penalties, tyre/energy costs
Continuation	session/window availability	terminal crash, retirement, session end

AMBIGUITY: Gear can be viewed as an action or a state depending on whether the controller chooses it. The contract must state the intended causal interpretation.

Chapter 21 - Latent Models and Autoencoders

WHAT YOU ARE LEARNING: Encoders, latent representations and decoders.

WHY IT MATTERS: High-dimensional observations can be compressed into dynamics-relevant state.

YOU ARE FINISHED WHEN: You can explain why reconstruction alone may preserve visually dominant but control-irrelevant detail.

An encoder maps observation o_t to latent z_t . A decoder reconstructs or predicts observations from z_t . In low-dimensional telemetry, an encoder is optional because the input is already compact. In F1 25, a latent encoder becomes more useful as wheel-level, motion, setup, damage and multi-car features expand.

Chapter 22 - Variational Autoencoders and Stochastic Latents

WHAT YOU ARE LEARNING: Distributions, reparameterization and KL divergence.

WHY IT MATTERS: An RSSM needs a trainable stochastic latent, not a random-number black box.

YOU ARE FINISHED WHEN: You can explain the posterior, prior and why their KL is optimized.

The encoder outputs mean and standard deviation. A sample is written $z = \text{mean} + \text{std} * \text{epsilon}$, with epsilon drawn from a standard normal. This reparameterization lets gradients flow through mean and std. KL divergence penalizes disagreement between the observation-informed posterior and the predictive prior.

Worked example

QUESTION: What happens if KL weight is zero?

Step 1. The posterior can encode observations freely.

Step 2. The prior receives no pressure to match those latent states.

Step 3. At imagination time, only the prior is available, so rollouts can collapse.

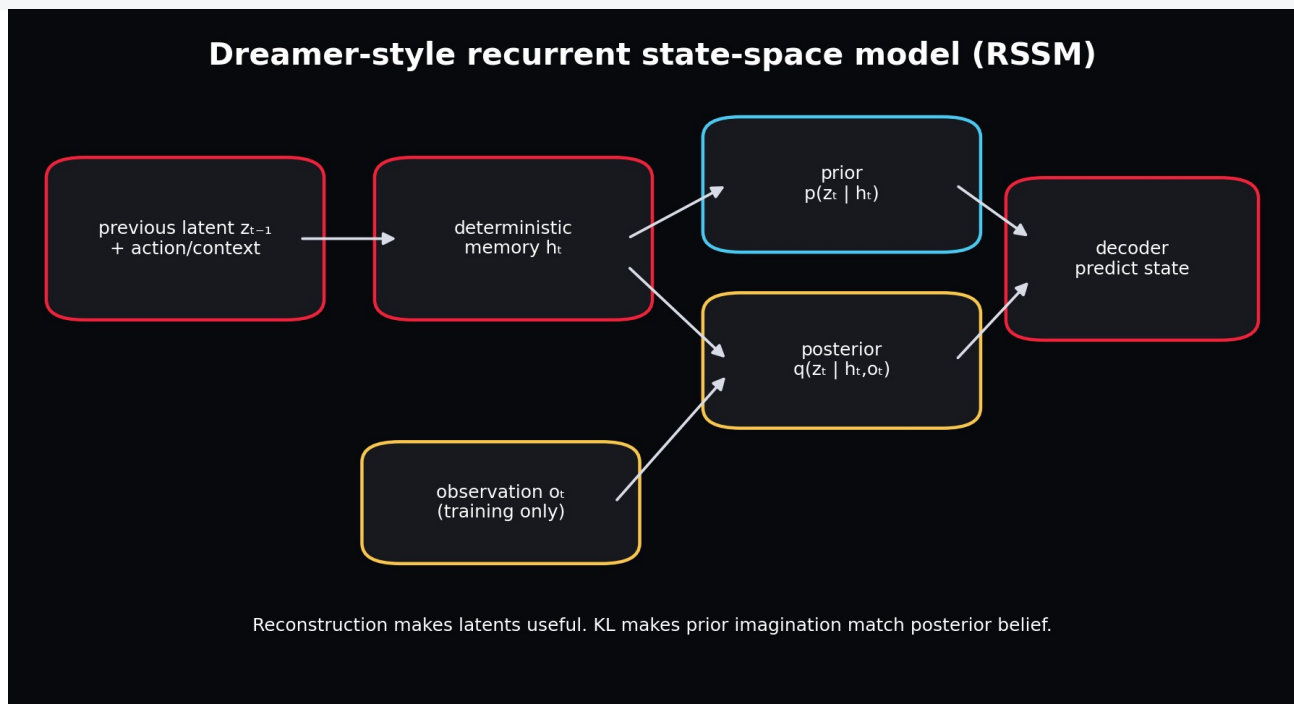
ANSWER: Reconstruction may look excellent during teacher-forced training while observation-free imagination fails.

Chapter 23 - Recurrent State-Space Models (RSSMs)

WHAT YOU ARE LEARNING: Deterministic recurrent memory plus stochastic latent state.

WHY IT MATTERS: RSSMs combine persistent history with uncertainty and are central to Dreamer.

YOU ARE FINISHED WHEN: You can trace posterior training and prior imagination step by step.



During training the posterior sees the observation; during imagination the model samples from the prior.

The deterministic state h_t stores long-lived context. The stochastic state z_t represents uncertainty or details that can branch. The transition first updates h_t from the previous latent and current

action/context. It then predicts a prior distribution over z_t . During training, a posterior also sees the observed state. The decoder predicts telemetry from h_t and z_t .

```
h = GRUCell(concat(z_previous, action_context), h_previous)
prior = p(z | h)
posterior = q(z | h, observed_state) # training only
z = sample(posterior)                # training
prediction = decoder(concat(h, z))
```

Chapter 24 - Dreamer: Learning and Acting in Imagination

WHAT YOU ARE LEARNING: World-model learning, imagined trajectories, actor, critic and returns.

WHY IT MATTERS: This is the long-term conceptual target for an F1 25 agent.

YOU ARE FINISHED WHEN: You can explain why a good dynamics model must precede imagined policy optimization.

Dreamer learns the world model from experience, starts imagined trajectories from encoded states, trains a critic to estimate return, and trains an actor to choose actions that improve imagined return. The policy does not need to render every future observation; it acts in compact latent space. DreamerV3 introduced robust normalization and transformations that work across many domains with a common configuration.

CRITICAL SEQUENCING: Do not add actor-critic optimization to a dynamics model whose rollouts violate speed, position or intervention logic. The actor will exploit model errors.

Chapter 25 - Rewards for an F1 Simulation Agent

WHAT YOU ARE LEARNING: Progress, lap time, track limits, tyre wear, energy, damage and multi-objective costs.

WHY IT MATTERS: A reward defines what the future controller will optimize.

YOU ARE FINISHED WHEN: You can identify reward hacking and propose guardrails.

A naive reward of speed encourages driving straight through corners. Progress per time is better but may still cut the track. A realistic objective combines progress, track validity, collision/damage penalties, tyre and energy costs, and terminal outcomes. Reward components should be logged separately so an agent cannot improve the total by exploiting one hidden term.

Reward component	Desired behaviour	Possible exploit
Track progress	Complete distance quickly	Cut track or teleport if state errors permit
Lap time	Fast complete laps	Ignore tyre or fuel sustainability
Track-limit penalty	Stay within circuit	Drive unrealistically slowly
Damage penalty	Preserve car	Avoid racing or overtaking
Tyre/energy cost	Manage resources	Sacrifice too much pace

Chapter 26 - Planning: MPC and CEM Before Full RL

WHAT YOU ARE LEARNING: Model predictive control, candidate action sequences and cross-entropy method search.

WHY IT MATTERS: Planning can use a learned world model without immediately training an actor.

YOU ARE FINISHED WHEN: You can describe receding-horizon control and its compute trade-off.

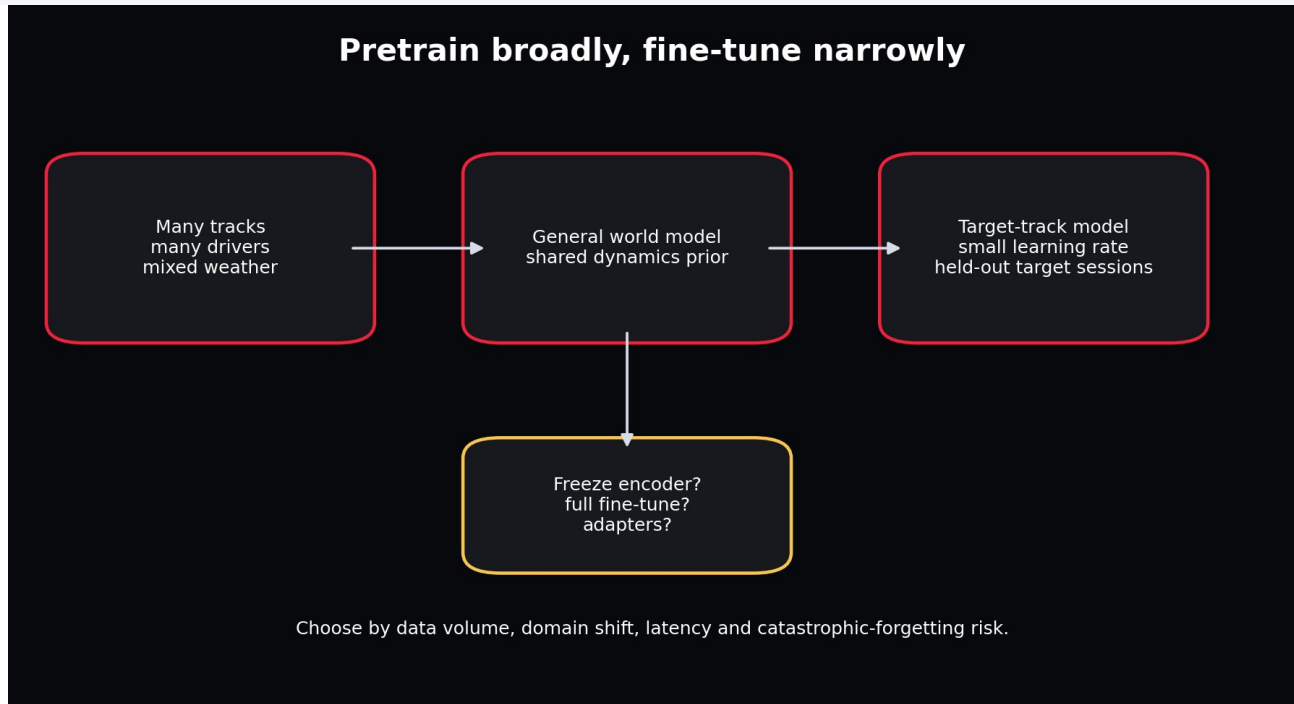
MPC repeatedly samples candidate action sequences, rolls each through the world model, scores predicted outcomes, executes only the first action, then replans from the new real observation. CEM iteratively refits the candidate distribution to high-scoring sequences. This is easier to audit than a learned actor but can be expensive at high frequency.

Chapter 27 - Fine-Tuning Existing Models Versus Building Your Own

WHAT YOU ARE LEARNING: Domain adaptation paths for world models and pretrained time-series backbones.

WHY IT MATTERS: World models are not as plug-and-play as language models because observation/action contracts differ.

YOU ARE FINISHED WHEN: You can choose among warm-starting, frozen features, adapters and full fine-tuning.



General pretraining provides a dynamics prior; target-track fine-tuning adapts with limited data.

There are three realistic reuse paths. First, pretrain the APEX architecture across many tracks and fine-tune on a target track or driver. Second, use a pretrained time-series backbone such as PatchTST or Chronos for representations/forecasting, then add action-conditioned heads; this is useful but not automatically a control world model. Third, adapt an existing Dreamer implementation, which requires defining observation/action spaces, replay data, reward and continuation heads, and often rewriting environment interfaces.

Path	Strength	Main risk
APEX cross-track pretraining	Exact contract and causal inputs	Needs enough diverse F1 data
Frozen time-series foundation features	Fast low-data benchmark	Backbone may ignore actions or domain physics
Full/flexible fine-tune	Maximum adaptation	Overfitting and catastrophic forgetting
Dreamer codebase adaptation	Complete control algorithm	High complexity; environment mismatch; difficult debugging

Part IV - Project Scope and Architecture

Chapter 28 - V1 Product Definition

WHAT YOU ARE LEARNING: A measurable first release and explicit non-goals.

WHY IT MATTERS: A world-model project can expand without limit unless the first operational contract is fixed.

YOU ARE FINISHED WHEN: You can state V1 inputs, outputs, horizon, users and acceptance tests.

V1 takes a history of canonical telemetry plus a proposed future action/context sequence and predicts speed, acceleration, cyclic track progress and tyre age over a short horizon. It supports historical exploration, model comparison, intervention sliders and artifact inspection through a UI. It does not optimize driving actions or claim race-strategy accuracy.

Dimension	V1 decision
Prediction unit	One driver in one session window
History	32 frames at 5 Hz = 6.4 seconds
Default horizon	8 frames = 1.6 seconds
Data	Offline synthetic plus optional FastF1/OpenF1
Primary model	Deterministic GRU
Challengers	Selective SSM and RSSM
Primary metric	Speed MAE plus horizon curve
Guardrails	Physical violation checks and session holdout
Interface	Gradio UI and FastAPI inspection service

Chapter 29 - Data Limitations Before F1 25

WHAT YOU ARE LEARNING: What public historical sources contain and what they do not.

WHY IT MATTERS: The model cannot infer reliable causal control dynamics from signals that do not exist.

YOU ARE FINISHED WHEN: You can distinguish measured action, proxy, context and unavailable variable.

FastF1 exposes timing, car telemetry, position, tyre and weather information through convenient Pandas-based objects. OpenF1 exposes historical JSON/CSV endpoints for car data, location, laps, weather, stints and more. Public data lacks the complete high-rate player control, wheel and setup state available in the game UDP feed. Steering is therefore approximated from heading change in V1.

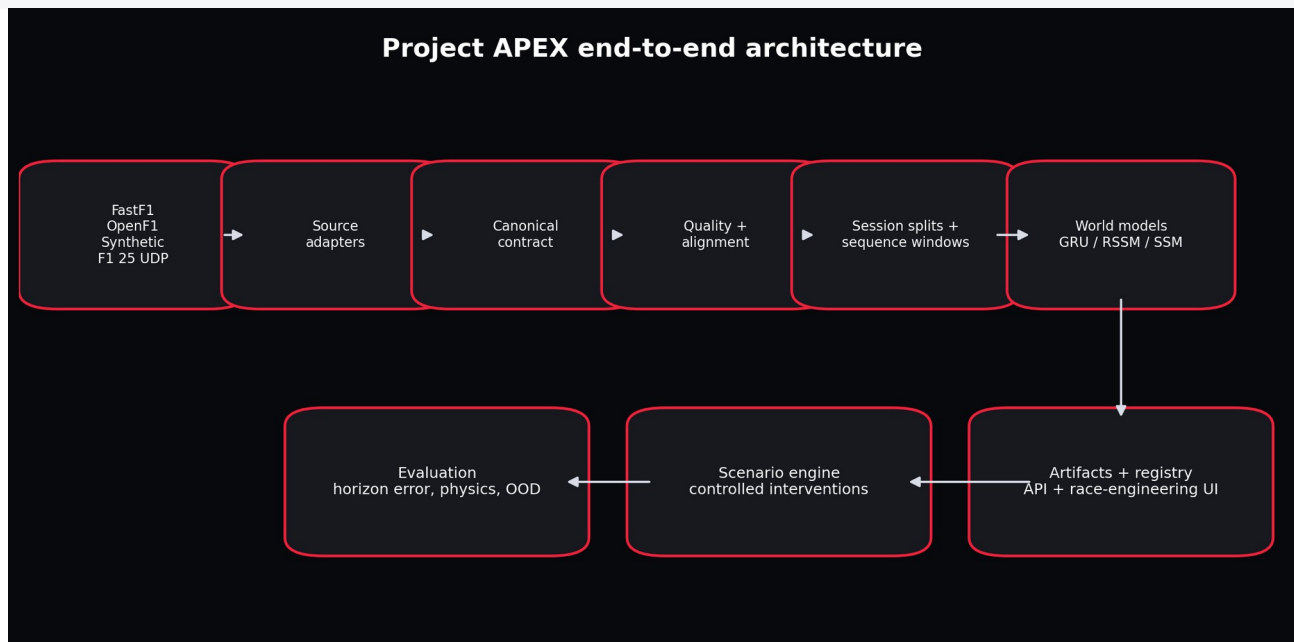
CAUSAL CAUTION: A steering proxy derived from future path geometry is not equivalent to a driver steering command. Treat it as context/derived action and document the limitation.

Chapter 30 - Full System Architecture

WHAT YOU ARE LEARNING: Every component from source data to user interface.

WHY IT MATTERS: A clear architecture makes the project replaceable and testable instead of becoming one notebook.

YOU ARE FINISHED WHEN: You can trace a frame from source endpoint to UI chart and name every persisted artifact.



The full system separates unstable external sources from a stable internal contract and reusable modelling core.

The source adapter is the anti-corruption layer. The canonical contract is the stable boundary. Quality and alignment stages protect downstream assumptions. Session splits and train-only

normalization protect evaluation. Model modules share a common history/future contract. Evaluation, simulation and publication consume model artifacts. The registry and interfaces never need to know whether the source was synthetic, FastF1 or F1 25.

Chapter 31 - Repository Architecture

WHAT YOU ARE LEARNING: How folders mirror responsibilities.

WHY IT MATTERS: Folder structure should reveal system boundaries before code is opened.

YOU ARE FINISHED WHEN: You can find any behaviour by following the responsibility map.

```
src/apexsim/  
  data/      # adapters, validation, features, windows  
  models/    # GRU, RSSM, selective SSM, baselines  
  pipeline/  # idempotent stages and orchestration  
  config.py  # validated experiment intent  
  contracts.py # source-independent schema  
  training.py # parameter optimization  
  evaluation.py # horizon and physical metrics  
  simulation.py # scenario interventions and rollout  
  registry.py # durable run state  
  serving.py # REST inspection interface  
  ui.py      # interactive client
```

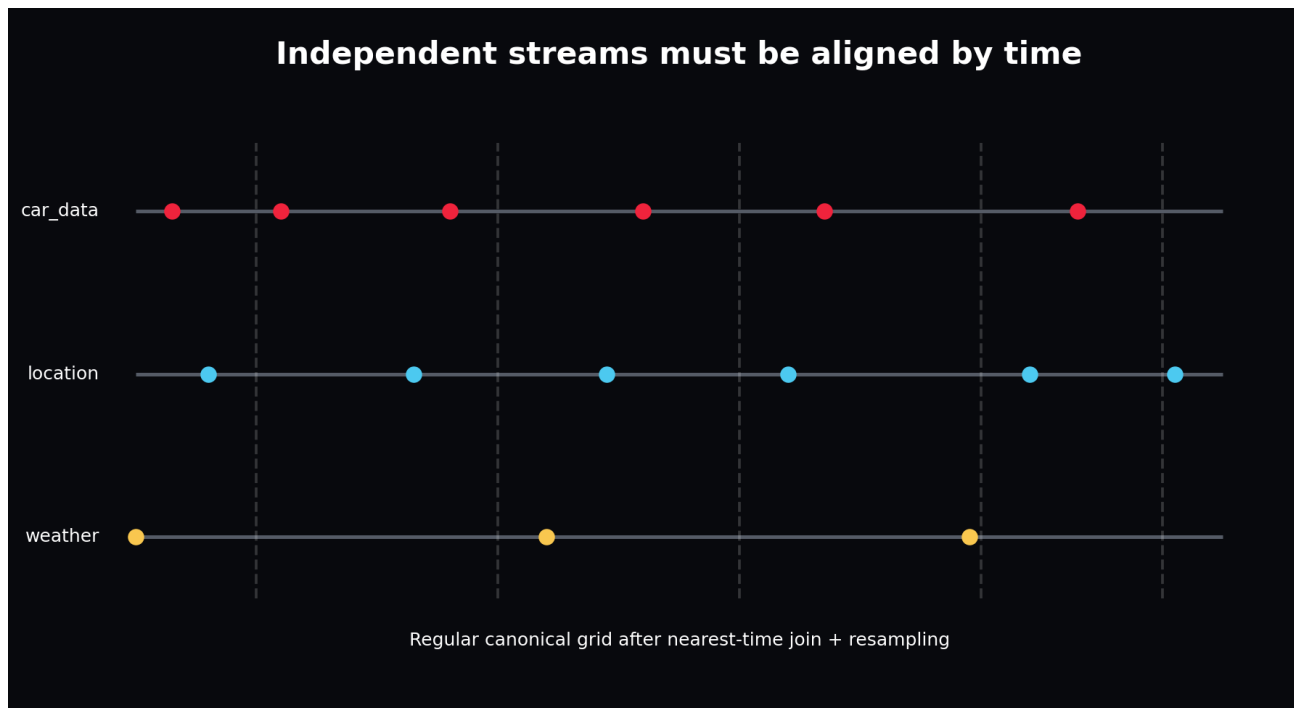
DESIGN TEST: If replacing Gradio with React requires editing the model or ingestion code, the boundary is wrong.

Chapter 32 - Data Flow From API to Tensor

WHAT YOU ARE LEARNING: Raw payload, normalized table, validated frame, episode split, window and batch.

WHY IT MATTERS: Every transformation can introduce leakage or semantic corruption.

YOU ARE FINISHED WHEN: You can describe the representation and invariants at each stage.



Asynchronous source events become a regular canonical time grid only after timestamp-aware alignment.

Stage	Representation	Key invariant
Raw landing	Original JSON/cache/dataframe	Preserve source timestamps and identifiers
Adapter output	Canonical dataframe	Units and column meanings are stable
Validated data	Admitted dataframe + report	Ranges, keys and missingness meet contract
Split data	Session ID lists	No session appears in multiple sets
Window dataset	History/future tensors	No window crosses an episode
Batch	[B,T,F] tensors	Axis semantics and normalization match model

Chapter 33 - Decision Records: Why This Implementation?

WHAT YOU ARE LEARNING: Architecture decision records and reversible choices.

WHY IT MATTERS: The project should teach how to choose, not only what was chosen.

YOU ARE FINISHED WHEN: You can write a one-page decision with context, options, evidence, consequence and revisit trigger.

Decision	Chosen now	Alternative	Revisit when
UI	Gradio	React/Next.js	Multiple users, auth, richer client state
V1 dynamics	GRU	Transformer/Mamba	Long histories or attention bottleneck measured
Storage	CSV + JSON artifacts	Parquet/object store	Data scale or query cost increases
Registry	SQLite	Postgres/MLflow	Concurrent teams and remote deployment
Orchestration	Local runner + thin Airflow DAG	Distributed workflow system	Scheduled multi-session scale requires it

Part V - Ingestion, Contracts and Data Flow

Chapter 34 - FastF1 Ingestion

WHAT YOU ARE LEARNING: Session loading, caching, laps, telemetry and conversion.

WHY IT MATTERS: FastF1 is the most convenient offline historical starting point.

YOU ARE FINISHED WHEN: You can ingest one driver session and identify where source-specific assumptions end.

```
session = fastf1.get_session(year, event, session_code)
session.load(telemetry=True, laps=True, weather=True)
driver_laps = session.laps.pick_drivers(driver)
for _, lap in driver_laps.iterlaps():
    telemetry = lap.get_telemetry()
```

FastF1 provides extended Pandas objects and caching. The adapter resamples each lap, converts speed to m/s, estimates acceleration, creates cyclic progress, maps controls to fractions, and writes the canonical schema. Keep the raw cache because future adapter corrections should not require another external download.

VERSION AWARENESS: External APIs and library fields can change. Pin and record dependency versions, preserve raw source data, and validate every adapter run.

Chapter 35 - OpenF1 Ingestion

WHAT YOU ARE LEARNING: HTTP endpoint queries, JSON dataframes and asynchronous stream joins.

WHY IT MATTERS: OpenF1 provides broad historical endpoints but does not guarantee row-by-row alignment across them.

YOU ARE FINISHED WHEN: You can merge car, location and weather streams by time with an explicit tolerance.

OpenF1 historical data begins in 2023 and is accessible without authentication, while real-time access has separate requirements. Car data is sampled at only a few hertz, and other endpoints have their own cadence. The adapter parses UTC timestamps, sorts each stream, uses `merge_asof` with tolerances, then resamples onto a regular grid.

```
merged = pd.merge_asof(
    car.sort_values("date"),
    location.sort_values("date"),
    on="date",
    direction="nearest",
    tolerance=pd.Timedelta("1s"),
)
```

Worked example

QUESTION: Why is concatenating the `nth` `car_data` row with the `nth` `location` row invalid?

Step 1. The endpoints can start at different times and sample at different rates.

Step 2. Dropped or delayed records shift row indices.

Step 3. The resulting row may combine speed from one instant with position from another.

ANSWER: Join by timestamp with a documented tolerance, inspect unmatched rates, and quarantine sessions with excessive alignment gaps.

Run either adapter through the same downstream pipeline

```
apexsim ingest-fastf1 --year 2025 --event Monza --session R --driver VER --output data/raw/monza_ver.csv

apexsim run-canonical --input-path data/raw/monza_ver.csv --config configs/fast.yaml --run-id monza_ver_gru
```

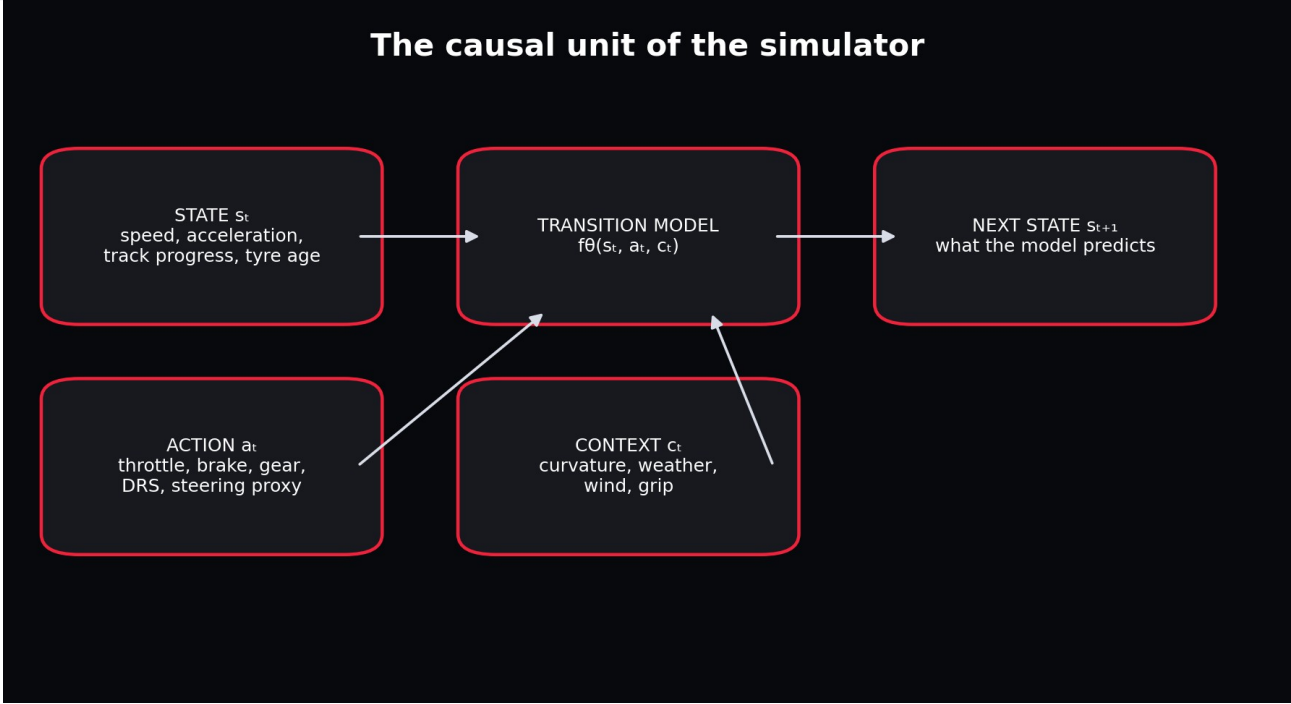
The adapter owns source-specific retrieval, units and alignment. `run-canonical` copies the exact adapter output into the immutable run directory, then uses the identical validation, session splitting, train-only normalization, training, evaluation, ablation and publication stages used by synthetic data. This prevents the real-data path from becoming a second untested application.

Chapter 36 - The Canonical Telemetry Contract

WHAT YOU ARE LEARNING: Stable identity, state, action, context and auxiliary columns.

WHY IT MATTERS: The contract lets the entire downstream system ignore source-specific naming.

YOU ARE FINISHED WHEN: You can read one row as a causal sentence.



The contract preserves causal roles rather than presenting one undifferentiated feature list.

Group	Columns
Identity	session_id, source, track_id, driver_id, timestamp_s, lap_number
State targets	speed_mps, acceleration_mps2, track_progress_sin/cos, tyre_age_laps
Actions	throttle, brake, gear_norm, drs, steering_proxy
Context	curvature, air_temp_c, track_temp_c, rainfall, wind_speed_mps, grip_level
Auxiliary	lap_distance_m, x_m, y_m, rpm, gear, compound_id, is_pit, safety_car

CONTRACT RULE: Adding a column does not make it causally valid. State whether it is observable at inference time and whether the user/controller can intervene on it.

Chapter 37 - Validation and Quality Gates

WHAT YOU ARE LEARNING: Schema, nulls, duplicate frame keys, ranges, gaps and warnings.

WHY IT MATTERS: Bad data should fail near ingestion, not become mysterious model error.

YOU ARE FINISHED WHEN: You can classify each condition as error, warning, quarantine or monitor.

```
report = validate_canonical_frame(frame, strict=True)
assert report.null_cells == 0
assert report.duplicate_frames == 0
assert report.out_of_range_cells == 0
```

Condition	Default treatment	Reason
Missing required column	Error	Downstream semantics undefined
Throttle = 1.5	Error	Impossible under unit-fraction contract
Unusually wet session	Warning/monitor	Valid but distribution-shifting
Long timestamp gap	Split/quarantine	Interpolation would invent trajectory
Rare compound	Keep + monitor	Operationally important tail case
Duplicate session/driver/time key	Error	Ambiguous truth

INSTRUCTOR CHECKPOINT: Deliberately corrupt one row, predict which report count changes, then run the test.

Chapter 38 - Resampling and Temporal Alignment

WHAT YOU ARE LEARNING: Regular grids, interpolation policies, forward fill and gap handling.

WHY IT MATTERS: Sequence models expect consistent temporal meaning.

YOU ARE FINISHED WHEN: You can choose interpolation by variable type and refuse unsafe gaps.

Variable type	Example	Reasonable policy
Continuous sensor	speed, temperature	Linear interpolation over short gaps
Piecewise control	gear, DRS, safety car	Forward fill or nearest
Event	pit entry, penalty	Preserve timestamp; derive active state carefully
Circular angle	heading	Unwrap, interpolate, wrap
Large gap	missing 5 seconds	Split episode; do not interpolate

Resampling does not create information. It creates a regular representation of available information under explicit assumptions. Store source timestamps and alignment diagnostics so later analysis can distinguish model error from synchronization error.

Chapter 39 - Feature Engineering

WHAT YOU ARE LEARNING: Derived acceleration, steering proxy, curvature, cyclic progress and normalized controls.

WHY IT MATTERS: Useful inductive bias can reduce data requirements but can also leak future information.

YOU ARE FINISHED WHEN: You can audit each feature for causality and inference availability.

Curvature can be derived from static track geometry and is valid future context. Steering proxy derived from heading changes is more ambiguous: when computed from the future trajectory it partially encodes what the car actually did. In V1 it is a documented approximation; in F1 25 it should be replaced by actual steering control.

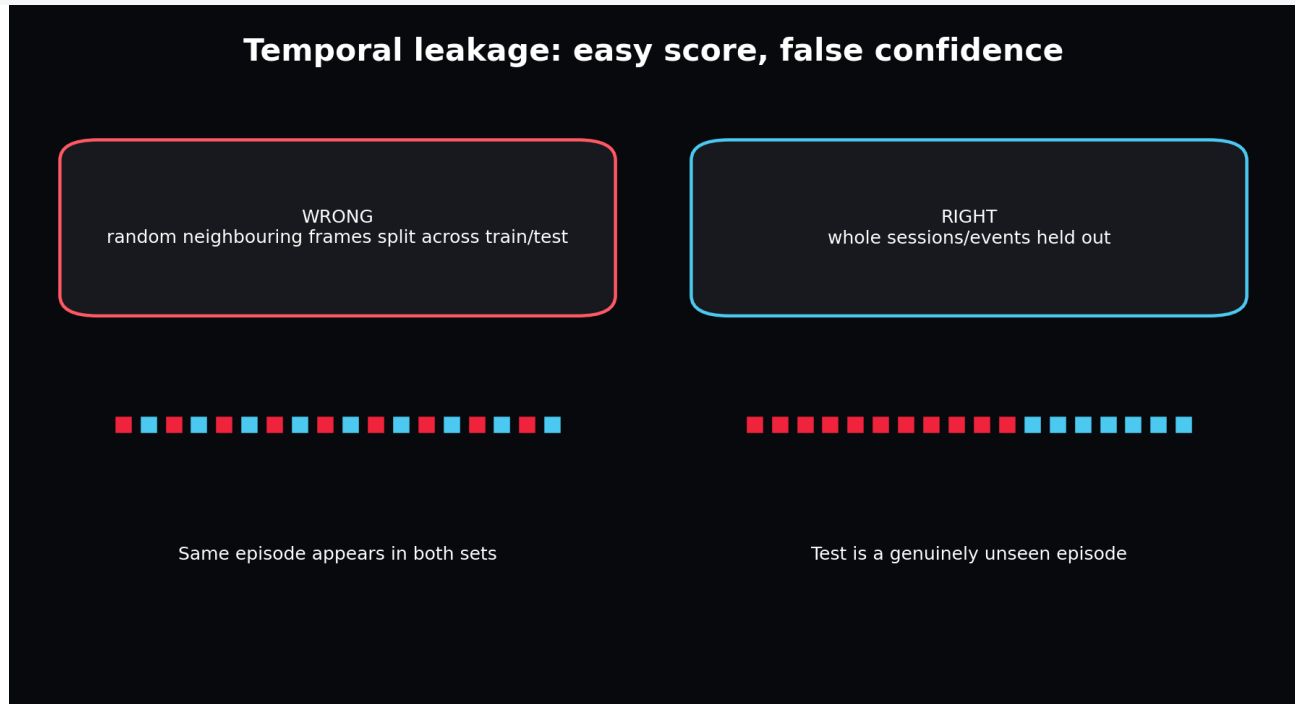
LEAKAGE AUDIT: For each feature, ask: could this exact value be known at the moment the prediction starts? If not, it belongs in the target, an oracle experiment, or nowhere.

Chapter 40 - Session-Level Splitting

WHAT YOU ARE LEARNING: Train, validation and test partitions by independent episode.

WHY IT MATTERS: Evaluation must measure generalization rather than memorization of neighbouring frames.

YOU ARE FINISHED WHEN: You can implement deterministic splits and prove disjointness.



Session holdouts are harder and more honest than random frame splits.

```
assert set(splits["train"]).isdisjoint(splits["val"])
assert set(splits["train"]).isdisjoint(splits["test"])
assert set(splits["val"]).isdisjoint(splits["test"])
```

For stronger evaluation, hold out complete tracks, drivers, weather regimes or seasons. Validation should resemble the deployment question. A random session holdout does not prove generalization to a new circuit.

Chapter 41 - Train-Only Standardization

WHAT YOU ARE LEARNING: Fitting and persisting normalization statistics.

WHY IT MATTERS: A model checkpoint is unusable without the exact feature order and scaling transform.

YOU ARE FINISHED WHEN: You can serialize and invert predictions correctly.

```
standardizer = Standardizer.fit(train_frame)
x_z = standardizer.transform_inputs(x_raw)
y_z = standardizer.transform_targets(y_raw)
y_raw_again = standardizer.inverse_targets(y_z)
```

PRODUCTION ARTIFACT: The standardizer is part of the model. Version it with feature order, units and checkpoint.

Chapter 42 - Sequence Windows and DataLoader

WHAT YOU ARE LEARNING: History, known future causes, future target, episode safety and batching.

WHY IT MATTERS: Window semantics determine what the model is allowed to know.

YOU ARE FINISHED WHEN: You can trace one manifest/index into tensors and verify all shapes.

History window, future controls, and future target



The model observes history, receives future scenario causes, and predicts future states.

```
return {
    "history": [T_history, F_input],
    "future_inputs": [T_future, F_input],
    "future_targets": [T_future, F_state],
    "session_id": session_id,
}
```

The future input tensor includes placeholders for state because the common feature order is convenient. During autoregressive rollout, the model overwrites the state portion with its own previous prediction while preserving the planned action/context portion.

Part VI - Guided Construction of V1

Chapter 43 - Environment Setup and First Proof

WHAT YOU ARE LEARNING: Install, test, compile and run the fast configuration.

WHY IT MATTERS: You need a known-good reference before changing anything.

YOU ARE FINISHED WHEN: Six tests pass and a named run produces a complete artifact tree.

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
pytest -q
python -m py_compile dags/apexsim_dag.py
apexsim run --config configs/fast.yaml --run-id my_first_run
```

PREDICT BEFORE RUNNING: Write the expected folders and files. After execution, explain why each exists.

Chapter 44 - Generate and Inspect the Offline World

WHAT YOU ARE LEARNING: Track geometry, driver policy, weather, tyre wear and causal dynamics.

WHY IT MATTERS: Offline reproducibility lets you debug before external data or the game exists.

YOU ARE FINISHED WHEN: You can alter one physical factor and predict the directional change.

```
config = load_config("configs/fast.yaml")
frame = generate_synthetic_sessions(config, "data/raw/synthetic.csv")
print(frame.shape)
print(frame.groupby("session_id").speed_mps.max())
```

Open `data/synthetic.py`. Follow distance to progress, progress to geometry, curvature to target speed, driver error to throttle/brake, and forces to next speed. This is the first complete simulator in the project. The world model later learns an approximation of these hidden rules from outputs.

Chapter 45 - Baseline Models

WHAT YOU ARE LEARNING: Persistence and linear one-step transitions.

WHY IT MATTERS: Deep learning earns its complexity by beating correct controls.

YOU ARE FINISHED WHEN: You can state when a baseline is stronger operationally despite lower peak accuracy.

Persistence predicts no change. A ridge transition predicts next state from current features. Linear models train quickly, expose coefficient direction, and power cheap ablation probes. They are especially useful for detecting target alignment mistakes: a suspiciously perfect linear result may indicate leakage.

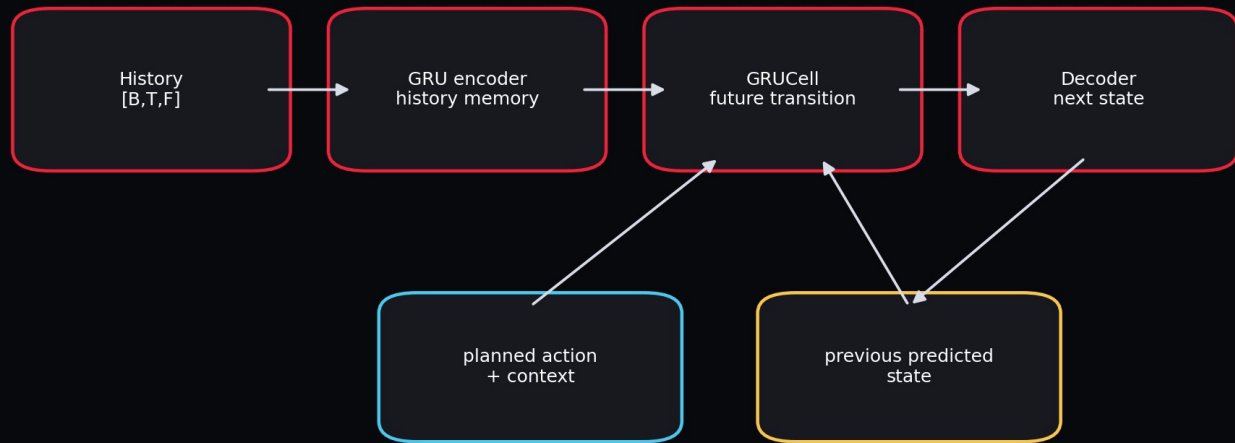
Chapter 46 - Build the GRU World Model

WHAT YOU ARE LEARNING: History encoding, recurrent transition, decoding and autoregressive state replacement.

WHY IT MATTERS: This is the primary V1 implementation.

YOU ARE FINISHED WHEN: You can trace every tensor and explain why future targets never enter forward inference.

Deterministic GRU world model



GRU rollout reuses the predicted state at the next step, so errors compound honestly.

```
_, hidden = self.encoder(history)
hidden_t = hidden[-1]
previous_state = history[:, -1, :state_dim]
for step in range(future_inputs.shape[1]):
    step_input = future_inputs[:, step].clone()
    step_input[:, :state_dim] = previous_state
    hidden_t = self.transition(step_input, hidden_t)
    previous_state = self.decoder(hidden_t)
    predictions.append(previous_state)
```

Line 1 compresses the observed past. Line 2 selects the top recurrent layer. Line 3 extracts the latest normalized state. The loop copies planned future features, replaces the state fields with model belief, updates memory, decodes a next state, and stores it. The model is therefore exposed to its own mistakes during every multi-step forward pass.

Chapter 47 - Loss Functions for Telemetry

WHAT YOU ARE LEARNING: MSE, MAE, Smooth L1, per-channel weighting and geometric constraints.

WHY IT MATTERS: A scalar loss decides which errors the model prioritizes.

YOU ARE FINISHED WHEN: You can choose a loss based on outliers, units and operational cost.

Project APEX uses Smooth L1 in normalized space. Small errors behave approximately quadratically, encouraging precision; large errors behave linearly, reducing domination by outliers. Normalization prevents speed magnitude from completely overwhelming cyclic progress.

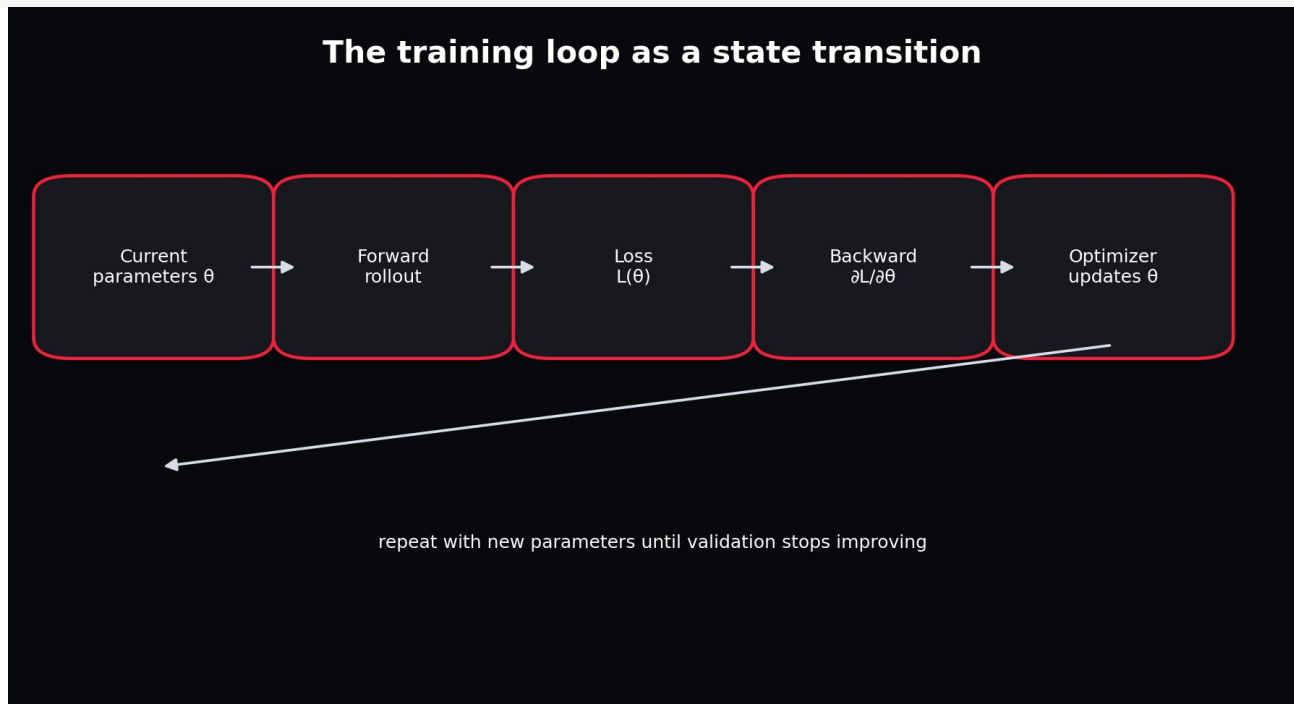
FUTURE IMPROVEMENT: Add a unit-circle penalty for progress sine/cosine and a non-negative speed transform if physical violations become material.

Chapter 48 - The Training Loop, Line by Line

WHAT YOU ARE LEARNING: Batches, forward pass, loss, backward pass, clipping, update, validation and checkpoint.

WHY IT MATTERS: Debugging requires knowing exactly when model state changes.

YOU ARE FINISHED WHEN: You can annotate the loop without saying "PyTorch handles it".



Each batch computes a local direction for changing the parameter state.

```
for batch in train_loader:
    optimizer.zero_grad(set_to_none=True)
    loss, prediction = batch_loss(model, batch)
    loss.backward()
    clip_grad_norm_(model.parameters(), grad_clip)
    optimizer.step()
```

Validation runs with `model.eval()` and `torch.no_grad()`. Eval mode changes dropout/batch normalization behaviour. No-grad prevents graph construction. The best validation checkpoint is restored after training, rather than keeping the last epoch by habit.

Chapter 49 - Overfit-One-Batch Debugging

WHAT YOU ARE LEARNING: A minimal training sanity test.

WHY IT MATTERS: Long training cannot repair a broken target, disconnected gradient or wrong axis.

YOU ARE FINISHED WHEN: A tiny dataset loss falls dramatically and predictions move toward targets.

1. Choose 16-64 windows from one session.
2. Disable regularization and augmentation.
3. Use a moderately large model.
4. Train for many updates on exactly those windows.
5. Inspect prediction and target arrays directly.
6. If loss does not collapse, debug code and data before collecting more data.

Chapter 50 - Build the Selective SSM Challenger

WHAT YOU ARE LEARNING: Stable decay, input projection, write gate and stacked recurrent state.

WHY IT MATTERS: The challenger teaches long-memory design without hidden dependencies.

YOU ARE FINISHED WHEN: You can explain which part corresponds to retention and which part writes new information.

```
decay = sigmoid(logit_decay)
candidate = tanh(input_projection(x))
gate = sigmoid(gate_projection(x))
state = decay * state + (1 - decay) * gate * candidate
```

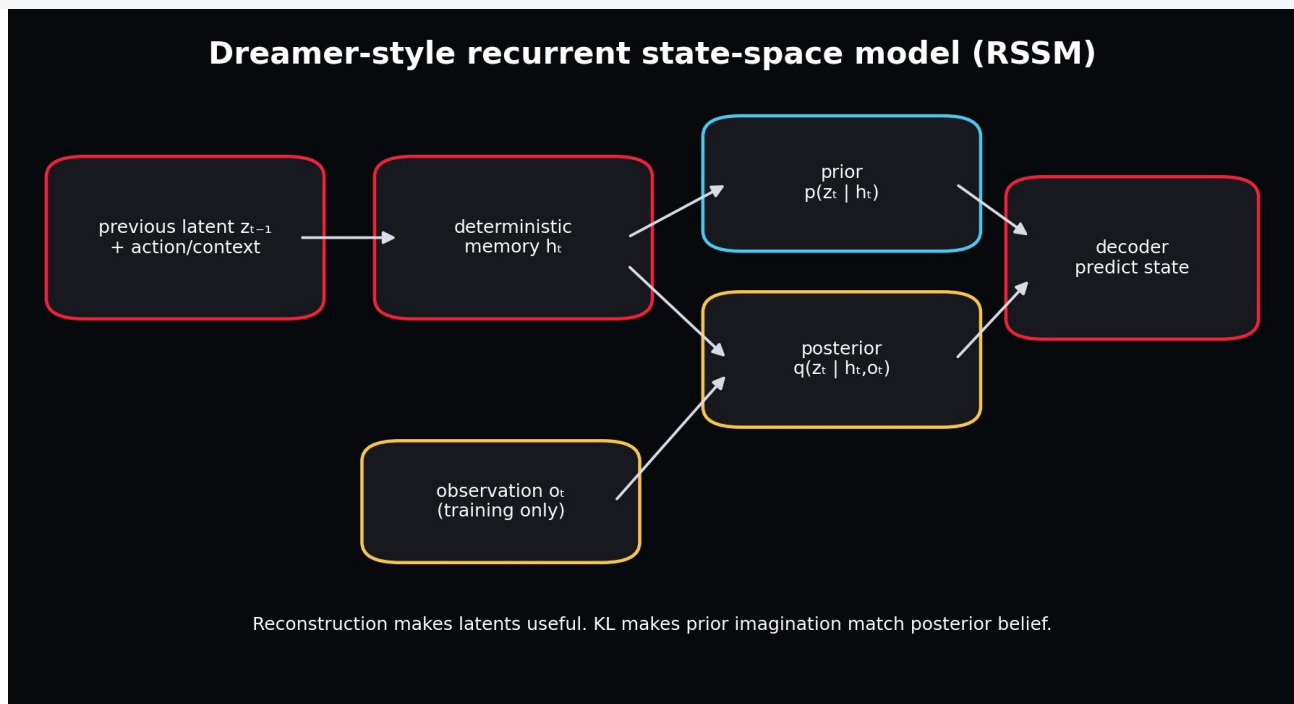
Decay near one retains memory. The gate controls input-dependent writing. Multiple cells stack state transformations. A full Mamba layer adds a more sophisticated parameterization, local convolution/gating components and highly optimized selective scan. Use the simple cell to validate the need for SSM behaviour before adding that complexity.

Chapter 51 - Build the Dreamer-Style RSSM

WHAT YOU ARE LEARNING: Deterministic memory, prior, posterior, sampling, decoder and KL.

WHY IT MATTERS: This is the conceptual bridge to future Dreamer control.

YOU ARE FINISHED WHEN: You can run training and imagination modes and explain their difference.



Posterior training and prior imagination must converge toward compatible latent states.

```
h = gru(concat(z, action_context), h)
prior = distribution(prior_head(h))
posterior = distribution(posterior_head(concat(h, target_state)))
z = posterior.rsample()
kl = KL(posterior || prior)
prediction = decoder(concat(h, z))
```

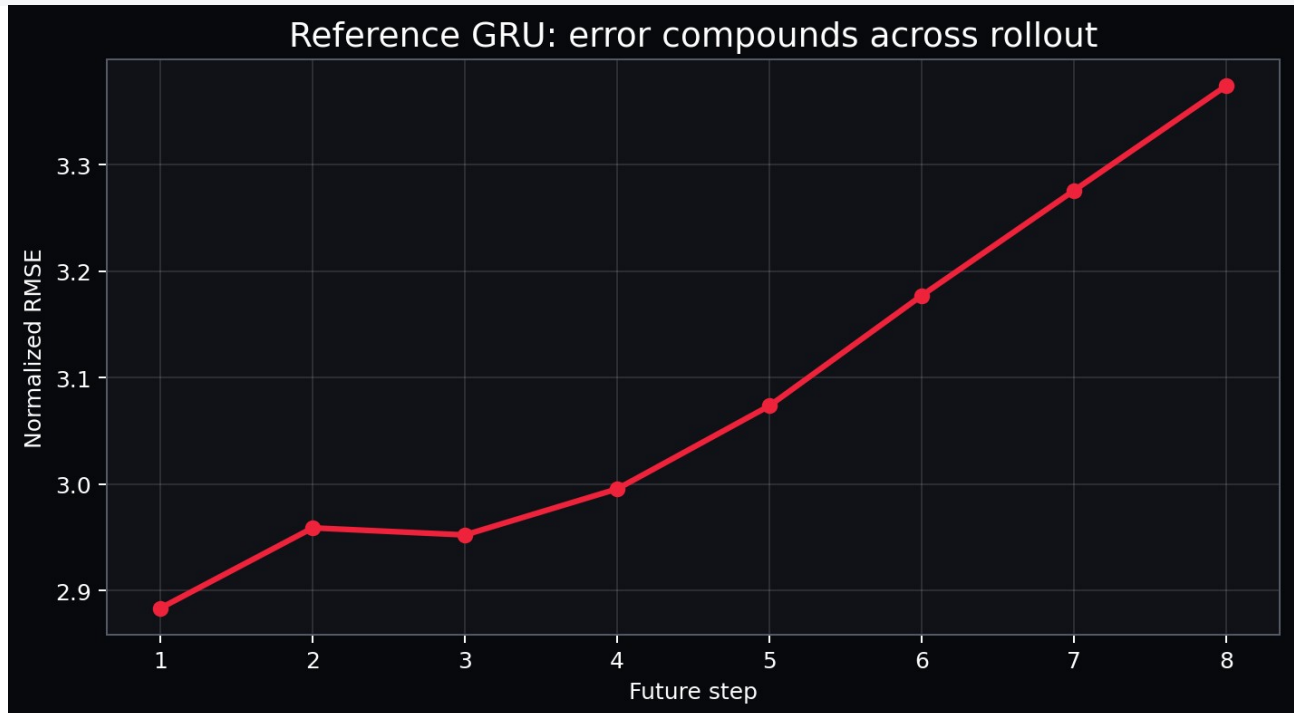
REFERENCE RESULT: Under the intentionally tiny fast budget, the packaged RSSM has worse speed error than the GRU. This is a lesson in optimization and evidence, not a defect to hide.

Chapter 52 - Evaluate the Held-Out Rollout

WHAT YOU ARE LEARNING: Inverse transforms, per-feature errors, horizon curves and physical checks.

WHY IT MATTERS: A normalized scalar loss is not interpretable to a race engineer.

YOU ARE FINISHED WHEN: You can report km/h error and identify where error compounds.



The reference rollout error rises as predictions feed the next transition.

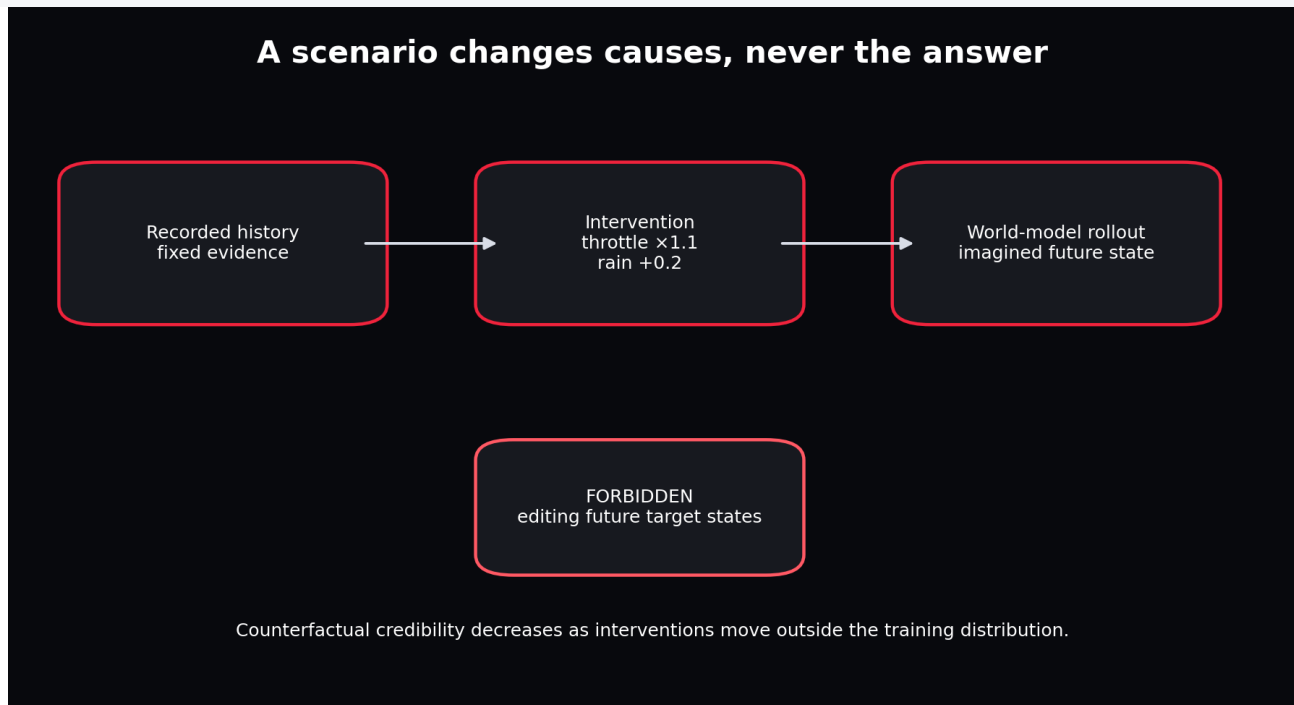
```
pred_raw = standardizer.inverse_targets(prediction_z)
error = pred_raw - target_raw
speed_mae = mean(abs(error[..., speed_index]))
horizon_rmse = sqrt(mean(error**2, axis=(batch, feature)))
```

Chapter 53 - Scenario Interventions

WHAT YOU ARE LEARNING: Changing future actions/context while holding history fixed.

WHY IT MATTERS: Simulation requires changing causes, not editing outcomes.

YOU ARE FINISHED WHEN: You can create an intervention and audit exactly which columns changed.



A valid scenario edits causes and lets the world model produce the consequence.

```
scenario = Scenario(
    throttle_scale=1.10,
    brake_scale=0.95,
    grip_multiplier=0.90,
    rain_delta=0.20,
)
future_changed = apply_scenario(future_inputs_raw, scenario)
prediction = rollout(model, history_raw, future_changed, standardizer)
```

Intervention distance matters. The model may behave plausibly under small control changes and fail under extreme combinations. Compute feature-space distance from training data and flag extrapolative scenarios in future versions.

Chapter 54 - Publish the First Simulation Artifact

WHAT YOU ARE LEARNING: Rollout CSV, metrics JSON, model metadata and summary.

WHY IT MATTERS: A prediction that exists only in memory cannot be reproduced or reviewed.

YOU ARE FINISHED WHEN: Every result can be traced to run ID, config, checkpoint, scaler and split.

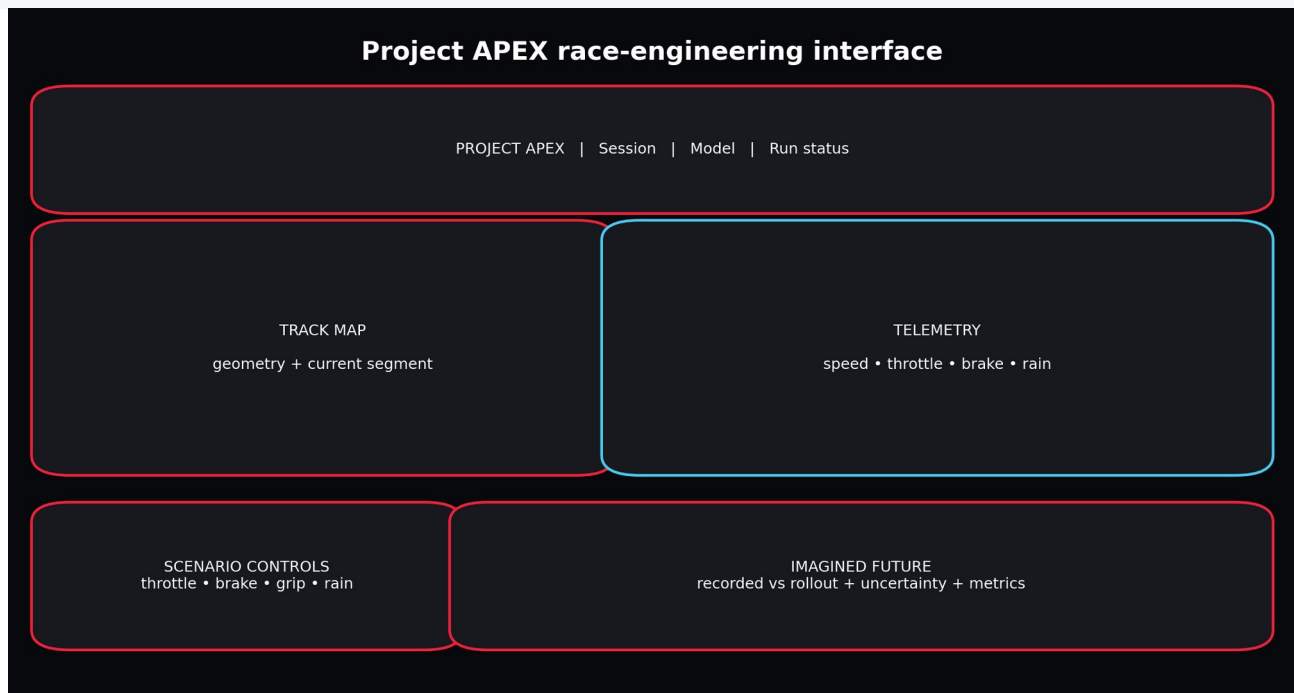
```
artifacts/runs/<run_id>/
canonical_telemetry.csv
quality_report.json
splits.json
standardizer.json
model/world_model.pt
model/model_meta.json
training_history.json
metrics.json
ablations.json
rollout_preview.csv
summary.json
```

Chapter 55 - Launch the Working UI

WHAT YOU ARE LEARNING: Race desk, telemetry explorer, rollout simulator, evaluation and ablation tabs.

WHY IT MATTERS: A usable interface turns model artefacts into an inspectable product.

YOU ARE FINISHED WHEN: You can launch the UI and explain why it does not contain training business logic.



The interface is a replaceable client over stable artifacts and simulation functions.

```
apexsim ui --run-dir artifacts/runs/reference_gru --config configs/fast.yaml
```

The Race Engineering Desk visualizes a session and track trace. The World Rollout Simulator changes controls and context. Model Evaluation shows horizon behaviour. Ablation Lab exposes evidence for feature choice. The architecture tab makes the data/model boundary visible.

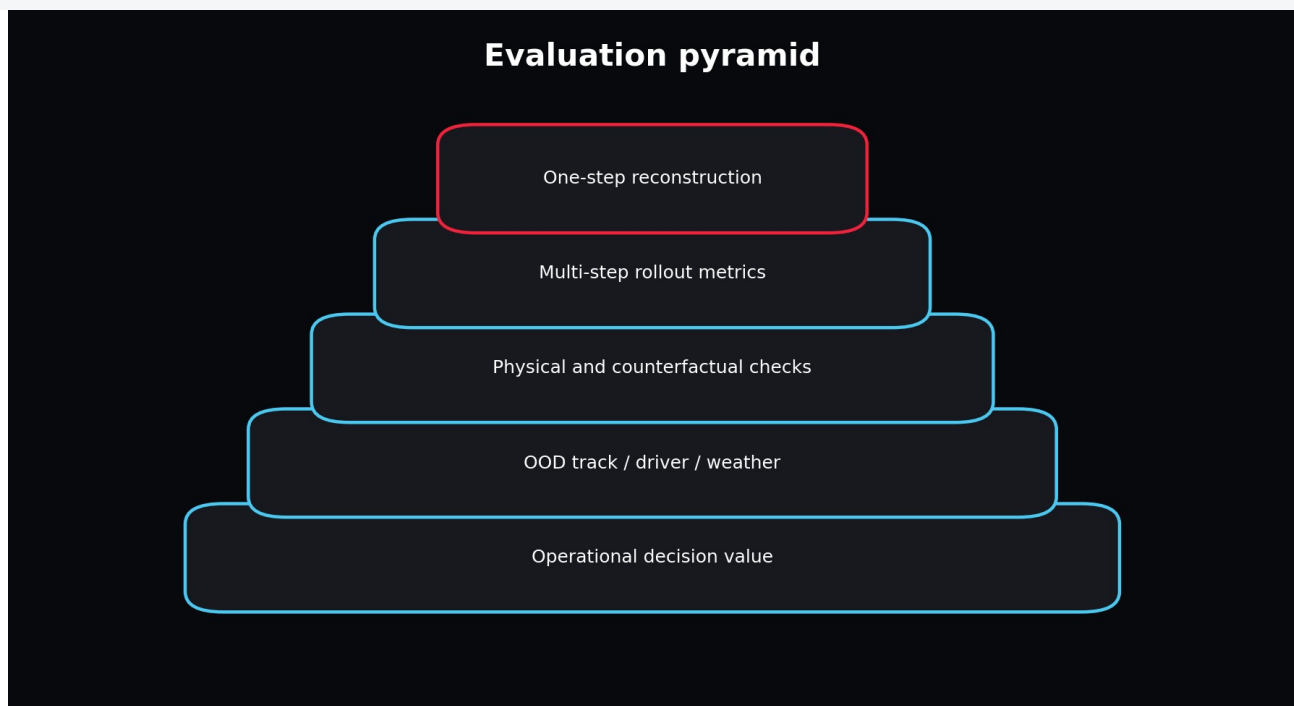
Part VII - Evaluation, Ablations and Fine-Tuning

Chapter 56 - The Evaluation Pyramid

WHAT YOU ARE LEARNING: From one-step reconstruction to operational decision value.

WHY IT MATTERS: Good one-step error is necessary but insufficient.

YOU ARE FINISHED WHEN: You can design a layered evaluation plan.



Evaluation becomes more realistic and expensive as it moves upward.

Start with feature error to debug. Add rollout curves to measure compounding. Add physical and counterfactual checks to reject impossible behaviour. Add out-of-distribution track/driver/weather holdouts. Finally ask whether the simulator improves a decision or controller.

Chapter 57 - Metrics by Feature and Horizon

WHAT YOU ARE LEARNING: MAE, RMSE, normalized error and horizon aggregation.

WHY IT MATTERS: Different channels and time horizons fail differently.

YOU ARE FINISHED WHEN: You can choose metrics that reflect operational consequences.

Metric	Use	Caution
MAE	Typical absolute error in real units	Treats all errors linearly
RMSE	Penalizes large misses	Sensitive to outliers
Normalized loss	Balanced training across channels	Hard to interpret operationally
Horizon curve	Shows compounding and useful range	Needs consistent sampling rate
Circular error	Track-progress angle	Must respect wraparound

Chapter 58 - Physical and Semantic Validity

WHAT YOU ARE LEARNING: Range checks, conservation-like constraints and representation validity.

WHY IT MATTERS: A low average error can coexist with impossible individual trajectories.

YOU ARE FINISHED WHEN: You can define model-specific invariants.

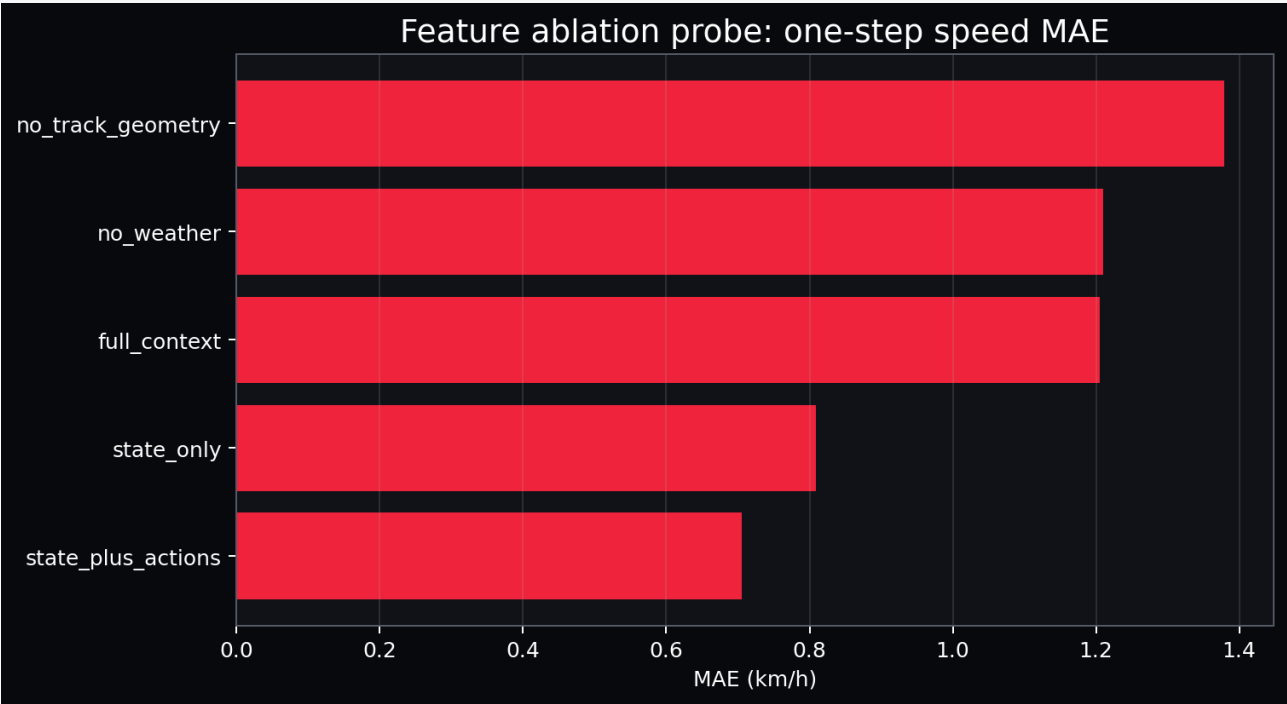
- Speed should not be negative or far beyond plausible bounds.
- Throttle/brake scenario inputs remain in $[0,1]$.
- Progress sine/cosine should remain near unit norm.
- Tyre age should not decrease during an uninterrupted stint.
- Position/progress should move consistently with positive speed.
- Predictions should respond in the expected direction to small controlled interventions.

Chapter 59 - Ablation Science

WHAT YOU ARE LEARNING: Control, treatment, held-fixed variables and decision criteria.

WHY IT MATTERS: Random hyperparameter changes do not reveal which information or mechanism matters.

YOU ARE FINISHED WHEN: You can write a complete experiment card before running code.



The cheap reference probe compares information sets under one fixed linear model.

Ablation	Question
state_only	Can recent state carry the short transition without controls?
state_plus_actions	Do driver controls improve next-state prediction?
full_context	Does weather/geometry add measurable value?
no_weather	How much does environmental context contribute?
no_track_geometry	Does the model need where/what corner it is approaching?

INTERPRETATION: An ablation result is conditional on model, horizon, split and data. A feature useless for one-step ridge prediction may be essential for long stochastic rollouts.

Chapter 60 - Architecture Comparisons

WHAT YOU ARE LEARNING: Fair GRU, SSM and RSSM experiments.

WHY IT MATTERS: Different models should be compared under equivalent information and budgets.

YOU ARE FINISHED WHEN: You can prevent a misleading benchmark.

7. Use identical train/validation/test sessions.
8. Use the same feature order and scaler.
9. Match or report parameter counts.
10. Use the same maximum epochs and early-stopping protocol.
11. Report training/inference time and memory.
12. Evaluate deterministic and stochastic models appropriately.
13. Repeat across seeds if the decision matters.

Chapter 61 - Pretraining Across Tracks

WHAT YOU ARE LEARNING: Self-supervised next-state learning over diverse sessions.

WHY IT MATTERS: A general dynamics prior can reduce data needed for a new circuit.

YOU ARE FINISHED WHEN: You can create pretrain and target-track split policies without leakage.

Pretrain on many track/session episodes. Hold the target circuit completely out. Then fine-tune using only target-track training sessions and evaluate on separate target-track sessions. Never pretrain on the target test session and call the result zero-shot.

Chapter 62 - Fine-Tuning Strategies

WHAT YOU ARE LEARNING: Frozen backbone, head-only, adapters, partial unfreezing and full fine-tune.

WHY IT MATTERS: Target data volume and domain shift determine how much of the model should move.

YOU ARE FINISHED WHEN: You can choose a strategy and monitor forgetting.

Strategy	Use when	Risk
Head-only	Backbone features transfer and target data is tiny	Insufficient adaptation

Last layer/block	Moderate shift and limited data	Representation remains partly mismatched
Adapters/LoRA-like modules	Need parameter efficiency	Implementation complexity for custom recurrent models
Full fine-tune	Substantial target data or strong shift	Overfit and forget broad dynamics
Joint rehearsal	Need target adaptation without forgetting	Requires source replay data

```
# Conceptual target-track fine-tune
model.load_state_dict(pretrained_checkpoint)
optimizer = AdamW(model.parameters(), lr=1e-4)
train_on(target_track_train_sessions)
evaluate_on(target_track_test_sessions)
```

Chapter 63 - Adapting Existing Time-Series and Dreamer Models

WHAT YOU ARE LEARNING: Patch-based forecasters, time-series foundation models and Dreamer repositories.

WHY IT MATTERS: Reusing weights saves work only if input semantics and objective align.

YOU ARE FINISHED WHEN: You can write a compatibility checklist before downloading a large model.

Compatibility question	Why it matters
Does the model accept multivariate covariates/actions?	Unconditional forecasts cannot represent control interventions
Is sampling frequency compatible?	Temporal scale changes dynamics
Can it predict multiple state channels jointly?	Independent forecasts may violate cross-feature physics
Can features be normalized exactly as expected?	Pretraining assumptions affect transfer
Is recurrent online inference available?	Simulation may require low-latency step-by-step rollout
Can uncertainty be calibrated?	Samples are not useful if coverage is wrong

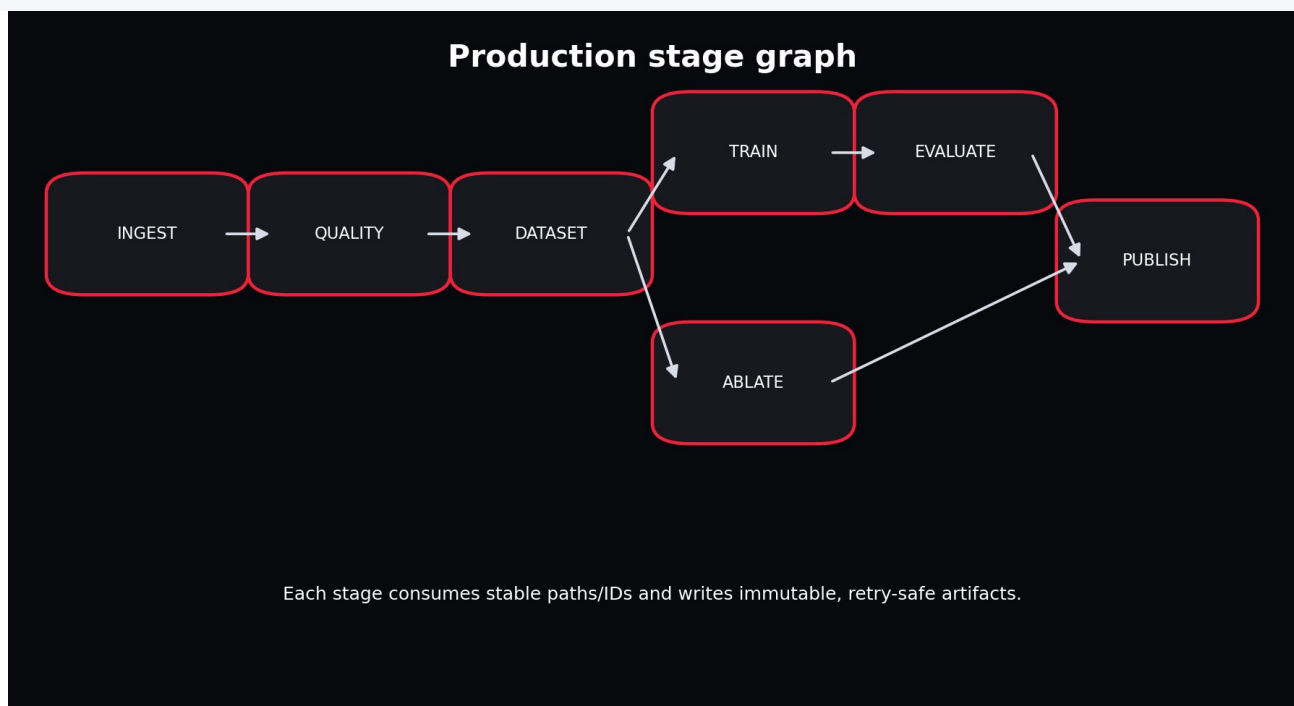
Part VIII - Production, UI and Operations

Chapter 64 - Idempotent Pipeline Stages

WHAT YOU ARE LEARNING: Path-based inputs, immutable outputs and safe retries.

WHY IT MATTERS: Production pipelines fail and rerun; duplicate side effects are unacceptable.

YOU ARE FINISHED WHEN: You can rerun a completed stage without changing its output.



The local runner and Airflow wrapper call the same stage functions.

Each stage checks for its expected artifact. Ingestion produces canonical data. Quality produces a report. Dataset stage produces split and scaler artifacts. Training writes a checkpoint and metadata. Evaluation and ablation write results. Publication writes rollout artifacts and summary. In a larger system, use content-addressed or versioned paths instead of silently overwriting.

Chapter 65 - Run Registry and Lineage

WHAT YOU ARE LEARNING: Run IDs, states, timestamps, metrics and artifact locations.

WHY IT MATTERS: Without lineage, a good result cannot be trusted or reproduced.

YOU ARE FINISHED WHEN: You can answer which code/config/data/model produced a displayed rollout.

SQLite is appropriate for a single-user local V1. The registry records running, succeeded or failed status, model type, start/finish time, directory and metrics. A multi-user production system would move to a server database or experiment platform with transactions and access control.

Chapter 66 - Airflow Orchestration

WHAT YOU ARE LEARNING: DAGs, tasks, retries, scheduling and thin orchestration.

WHY IT MATTERS: Scheduled ingestion and retraining should reuse tested business logic.

YOU ARE FINISHED WHEN: The DAG imports safely and task bodies call project stages rather than reimplementing them.

```
@dag(schedule=None, catchup=False)
def project_apex_world_model():
    @task(retries=2)
    def run_pipeline_task(config_path, run_id):
        return run_pipeline(load_config(config_path), run_id)
    run_pipeline_task("configs/fast.yaml", "airflow_scheduled_run")
```

SCALE-UP DESIGN: For real scale, split stages into separate tasks, pass only paths/IDs through XCom, use pools for API rate limits and GPU resources, and make backfills explicit.

Chapter 67 - Testing Strategy

WHAT YOU ARE LEARNING: Unit, contract, model-shape, scenario and end-to-end tests.

WHY IT MATTERS: ML systems fail in ordinary software layers as often as in model mathematics.

YOU ARE FINISHED WHEN: You can state which regression each included test prevents.

Test	Protects
Synthetic contract	Generator and schema compatibility

Window shapes/splits	Axis semantics and episode leakage
Model shapes	Architecture interface compatibility
Scenario edit	Counterfactual changes only intended features
End-to-end pipeline	Stages, artifacts, registry and training integration

Chapter 68 - Monitoring and Drift

WHAT YOU ARE LEARNING: Data, feature, model, physical and product health.

WHY IT MATTERS: A model can remain online while its assumptions decay.

YOU ARE FINISHED WHEN: You can separate drift detection from automatic retraining.

Layer	Signals
Pipeline	freshness, failures, runtime, retry count
Data	schema, missingness, timestamp gaps, source changes
Feature	mean/std/quantile shift, unseen tracks/weather
Model	horizon error, calibration, physical violations
Product	UI/API latency, failed scenarios, user feedback

RETRAINING CAUTION: Drift is a diagnosis signal. Automatic retraining without labels, evaluation and promotion gates can deploy a worse model faster.

Chapter 69 - FastAPI Service Boundary

WHAT YOU ARE LEARNING: Health, runs, run details and rollout artifacts.

WHY IT MATTERS: Interfaces should expose stable contracts instead of internal Python objects.

YOU ARE FINISHED WHEN: You can replace the UI without touching model code.

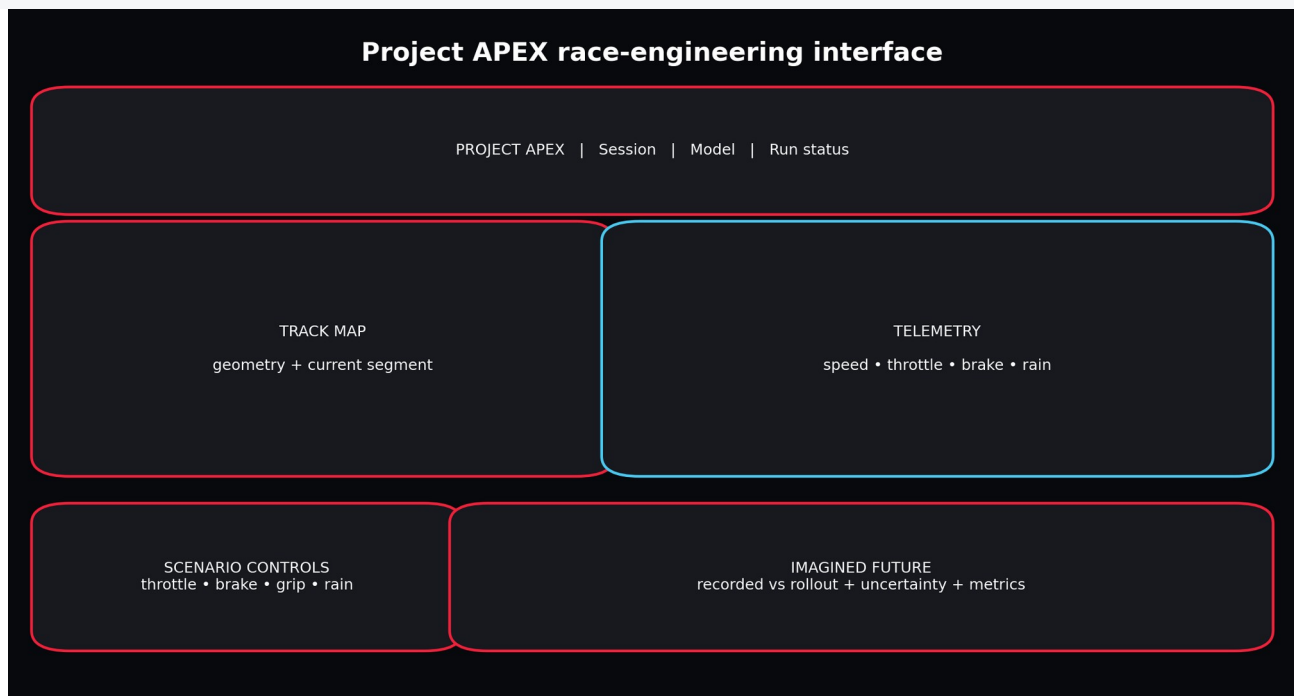
```
GET /health
GET /runs
GET /runs/{run_id}
GET /runs/{run_id}/rollout
```

Chapter 70 - Designing a Beautiful, Useful UI

WHAT YOU ARE LEARNING: Information hierarchy, race-engineering workflows and progressive disclosure.

WHY IT MATTERS: A beautiful interface is not decoration; it helps users distinguish observation, model belief and scenario choice.

YOU ARE FINISHED WHEN: You can identify what belongs on each tab and why.



The layout separates evidence, controls and imagined output.

- Use one dominant question per tab.
- Label recorded data and imagined data differently.
- Display units everywhere.
- Make model/run identity visible.
- Show horizon and uncertainty, not only a final number.
- Prevent extreme unsupported interventions or warn about them.
- Keep raw diagnostic artifacts available for engineers.

Part IX - F1 25 Migration and Long-Term Roadmap

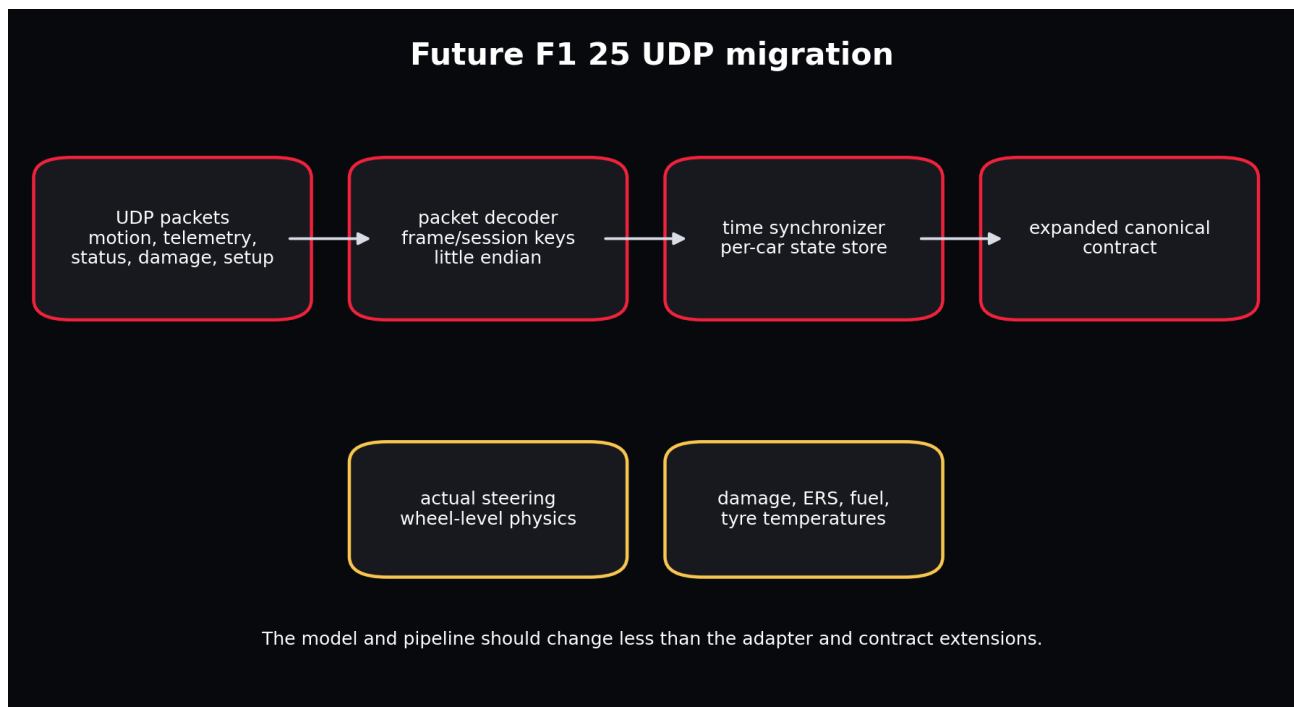
Chapter 71 - Understanding the F1 25 UDP Feed

WHAT YOU ARE LEARNING: Packets, frame/session identifiers, send rate, coordinates, privacy restrictions and packed binary structures.

WHY IT MATTERS: The game feed transforms APEX from action-poor historical modelling into richer closed-loop telemetry.

YOU ARE FINISHED WHEN: You can outline a robust listener and synchronization layer.

The official F1 25 specification describes packet types for motion, session, lap data, participants, car setups, telemetry, status, damage, tyre sets, extended motion, time trial and lap positions. UDP output can be configured in game, commonly on port 20777 with a selectable send rate. Structures are packed and little-endian; vehicle indices remain stable within a session. Some fields can be restricted in multiplayer privacy settings.



The adapter expands while the downstream model/pipeline interfaces remain largely stable.

NETWORK REALITY: UDP is unordered and unreliable. Use packet sequence/frame identifiers, detect drops and reordering, and never assume arrival order is perfect.

Chapter 72 - Expanded F1 25 Contract

WHAT YOU ARE LEARNING: Motion, controls, wheel state, damage, setup, energy and race state.

WHY IT MATTERS: Richer sensors enable a more Markov and physically meaningful world state.

YOU ARE FINISHED WHEN: You can prioritize features by causal value and availability.

Packet family	Potential APEX features
Motion	world position/velocity, direction vectors, G-forces, yaw/pitch/roll
Car telemetry	speed, throttle, steer, brake, clutch, gear, RPM, temperatures, pressures
Car status	fuel, ERS, compounds, DRS permission, traction/ABS settings
Damage	wing/floor/diffuser/sidepod/engine/gearbox/tyre wear and damage
Setup	wing, differential, suspension, brakes, pressures
Session/lap	track, weather, safety car, sector, pit and race state

Chapter 73 - From Offline World Model to Closed-Loop Simulator

WHAT YOU ARE LEARNING: UDP recording, replay, online inference, shadow mode and controller integration.

WHY IT MATTERS: Closed-loop use introduces latency, distribution shift and safety concerns.

YOU ARE FINISHED WHEN: You can design a staged deployment rather than connecting an untested policy directly.

14. Record raw packets with timestamps and packet identifiers.
15. Build deterministic replay and compare decoded state with in-game displays.
16. Run world-model inference in shadow mode without controlling anything.
17. Measure real-time latency and packet-loss resilience.

18. Validate short rollouts against subsequent real frames.
19. Add MPC suggestions while a human remains in control.
20. Only then test limited automated control in a safe environment.

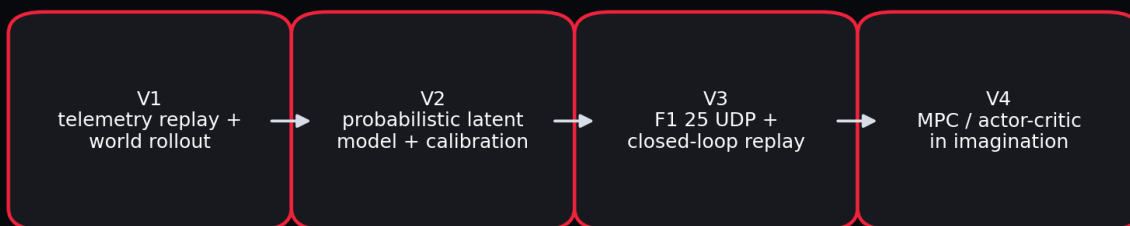
Chapter 74 - Roadmap to Dreamer for F1 25

WHAT YOU ARE LEARNING: Progressive milestones from telemetry replay to imagined control.

WHY IT MATTERS: The ultimate vision needs ordered evidence gates.

YOU ARE FINISHED WHEN: You can explain why each milestone must pass before the next.

Roadmap: data replay to Dreamer-style control



Do not add a planner until the world model is accurate, stable, uncertainty-aware, and validated under interventions.

The path to actor-critic control begins with trustworthy replay and dynamics.

V1 establishes data contracts and deterministic rollout. V2 adds calibrated stochastic latent dynamics. V3 integrates real game actions and richer state through UDP and validates online replay. V4 adds MPC or Dreamer-style actor/critic learning in imagination. At every step, retain a simple baseline and closed-loop reality check.

Part X - Independent Mastery and Reference

Chapter 75 - The Complete Instructor-Led Build Sequence

WHAT YOU ARE LEARNING: A literal order for completing the project.

WHY IT MATTERS: The sequence prevents model excitement from skipping data and evaluation foundations.

YOU ARE FINISHED WHEN: Every phase has an artifact, a deliberate failure and a decision record.

21. Write V1 scope and non-goals.
22. Set up environment and run tests.
23. Generate synthetic telemetry.
24. Inspect units and one causal trajectory.
25. Validate the canonical contract.
26. Break ranges and duplicate keys.
27. Create deterministic session splits.
28. Fit and persist train-only normalization.
29. Build sequence windows.
30. Implement persistence and ridge baselines.
31. Trace GRU shapes.
32. Overfit one batch.
33. Train full fast GRU.
34. Evaluate by feature and horizon.
35. Run scenario interventions.
36. Run feature ablations.
37. Train SSM challenger.
38. Train RSSM challenger and inspect KL.
39. Register and publish artifacts.
40. Launch UI and API.
41. Add a real FastF1/OpenF1 adapter run.
42. Pretrain across tracks and fine-tune a held-out target.
43. Design F1 25 packet adapter.
44. Rebuild core components without viewing source.

Chapter 76 - Common Failure Atlas

WHAT YOU ARE LEARNING: Symptoms, likely causes and minimum diagnostic steps.

WHY IT MATTERS: Knowing where to look is more valuable than trying random hyperparameters.

YOU ARE FINISHED WHEN: You can isolate a failure to data, optimization, model, evaluation or product layer.

Symptom	Likely cause	First check
Near-perfect test score	Temporal leakage or future feature	Print session IDs and audit feature availability
Loss is NaN	bad values, exploding gradients, invalid std	Inspect batch min/max/finite and loss components
Training improves, rollout fails	exposure bias/compounding	Plot horizon curve and overfit autoregressive objective
Scenario has no effect	action not used or overwritten	Compare modified input columns and gradients
RSSM reconstructs but imagines badly	prior/posterior mismatch	Track KL and prior-only rollout
UI crashes after retrain	artifact/schema mismatch	Validate model metadata and scaler feature order
Real data much worse	domain shift/adaptor error	Compare units, cadence and feature distributions

Chapter 77 - Exam-Style Questions

45. Why is a random telemetry-frame split usually invalid?
46. Explain the difference between state, action and context using five APEX columns.
47. Why encode track progress with sine and cosine?
48. What does `loss.backward()` compute conceptually?
49. Why can an RSSM posterior not be used during future imagination?
50. What benefit does a selective SSM offer over a fixed linear recurrence?
51. Why is a future steering proxy potentially leaky?
52. Design an ablation to test whether weather matters at a 5-second horizon.
53. What artifacts are required to reproduce a checkpoint prediction?
54. Why should an actor not be trained against an inaccurate world model?
55. How would you fine-tune a cross-track model on Monza without leaking the Monza test session?
56. Which F1 25 packets would most improve the Markov state and why?

Chapter 78 - Solutions to Exam-Style Questions

1. Adjacent frames are highly correlated and often nearly identical; random splitting puts the same episode in train and test. Hold out complete sessions/events/tracks.
2. State: speed, acceleration, tyre age. Action: throttle, brake. Context: curvature, rain. The distinction is whether it is world condition, controller choice or predicted condition.
3. Raw progress has a discontinuity from 1 to 0 at the finish line. Sine/cosine preserves circular proximity.
4. It applies the chain rule through the computation graph to calculate the gradient of loss with respect to trainable parameters.
5. The posterior requires the actual future observation. At imagination time that observation is unknown, so the predictive prior must generate the latent.
6. Input-dependent write/forget behaviour can retain rare important events while maintaining a compact recurrent state and linear sequence scaling.
7. If derived from the realized future path, it contains information about what actually happened rather than only what was known/planned at prediction time.
8. Train identical models/splits/budgets with full context and with weather columns removed; compare horizon-specific error and wet-session subsets across seeds.
9. Code/model architecture, checkpoint, feature order, units, standardizer, config, data version, split, run ID and inference/scenario parameters.
10. The actor can discover and exploit inaccuracies, maximizing imagined reward while failing in the real environment.
11. Pretrain without any Monza sessions, fine-tune on a Monza training subset, choose checkpoints on separate Monza validation, and report on sealed Monza test sessions.
12. Actual steering and motion improve action/state; telemetry adds pedal/wheel temperatures; status/setup/damage add hidden causes affecting dynamics and long-term outcomes.

Chapter 79 - Code Atlas

File	Responsibility
config.py	Validate experiment and runtime intent
contracts.py	Define stable source-independent semantics
data/synthetic.py	Generate causal offline telemetry
data/fastf1_adapter.py	Translate FastF1 session data
data/openf1_adapter.py	Join and translate OpenF1 streams
data/validate.py	Enforce quality invariants
data/features.py	Fit and persist train-only scaling

data/windows.py	Create episode-safe sequence examples
models/gru_world_model.py	Primary deterministic dynamics
models/rssm.py	Stochastic Dreamer-style dynamics
models/ssm_world_model.py	Selective state-space challenger
training.py	Optimize and checkpoint parameters
evaluation.py	Compute horizon and physical metrics
simulation.py	Apply controlled future interventions
pipeline/stages.py	Idempotent business stages
pipeline/runner.py	Local state machine and registry updates
ui.py	Interactive client
serving.py	REST inspection service

Chapter 80 - References and Further Study

57. FastF1 official documentation and repository: access to F1 timing, telemetry, position, tyre, weather and results data; documentation version observed during preparation: 3.8.1.
58. OpenF1 official API documentation: historical telemetry/timing endpoints from 2023 onward, including car data, location, laps, weather, stints and session data.
59. EA SPORTS, Data Output from F1 25 Game, UDP specification version 3: packet structures, telemetry configuration, coordinate conventions and packet families.
60. Hafner, Pasukonis, Ba, Lillicrap. Mastering Diverse Domains through World Models (DreamerV3), arXiv:2301.04104 and official repository.
61. Gu, Goel, Re. Efficiently Modeling Long Sequences with Structured State Spaces (S4), arXiv:2111.00396.
62. Gu and Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces, arXiv:2312.00752.
63. Nie et al. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers (PatchTST).
64. Ansari et al. Chronos: Learning the Language of Time Series, TMLR 2024 and official Amazon Science repository.

FINAL STANDARD: You have mastered Project APEX when you can build a different action-conditioned telemetry simulator from scratch, explain every axis and artifact, design an honest evaluation, and know exactly what evidence is missing before making stronger claims.

Appendix A - Key Commands

```
# Verify
pytest -q
python -m py_compile dags/apexsim_dag.py

# Build a complete offline run
apexsim run --config configs/fast.yaml --run-id reference_gru

# Compare architectures
apexsim run --config configs/ssm_fast.yaml --run-id reference_ssm
apexsim run --config configs/rssm_fast.yaml --run-id reference_rssm

# Launch product surfaces
apexsim ui --run-dir artifacts/runs/reference_gru --config configs/fast.yaml
apexsim api --artifacts-dir artifacts/runs

# Optional historical sources
apexsim ingest-fastf1 --year 2025 --event Monza --session R --driver VER
apexsim ingest-openf1 --session-key 9159 --driver-number 55
```

Appendix B - Verified Reference Run

Item	Verified value
Canonical rows	8,400
Sessions	8
Model	GRU
Input features	16
Predicted state features	5
Held-out windows	1200
Speed MAE	13.71 km/h
Speed RMSE	20.42 km/h
Negative-speed rate	0.0
Best cheap ablation	state_plus_actions

INTERPRETATION: This is a small synthetic educational benchmark. It proves that the system works end to end; it does not establish real F1 predictive accuracy.